Árboles de Decisión y Algoritmos de Clasificación

Fundamentos teóricos, modelos geométricos, criterios de división y algoritmos de construcción

---

Índice de Contenidos

(El índice de contenidos se actualizará automáticamente al abrir el documento en Microsoft Word o Google Docs).

1. Introducción a los Árboles de Decisión

Un **árbol de decisión** es un algoritmo de aprendizaje automático supervisado ampliamente utilizado tanto para tareas de **clasificación** como de **regresión**. Su estructura jerárquica emula un diagrama de flujo donde las decisiones se organizan de forma intuitiva:

- **Nodos internos:** Representan atributos o características de los datos sobre los que se realiza una prueba.
- **Ramas:** Representan el resultado de la regla de decisión aplicada en el nodo superior.
- **Nodos hoja (o terminales):** Representan el resultado final, la etiqueta de clase o el valor de predicción asignado.

El algoritmo opera particionando los datos de manera recursiva en subconjuntos cada vez más homogéneos. Este procedimiento de selección de la mejor característica y división de los datos se conoce como el **proceso de división (splitting process)**, el cual busca maximizar la ganancia de información en cada etapa.

> **Ventajas y Mitigación del Sobreajuste:** Una de las principales ventajas de los árboles de decisión es su elevada interpretabilidad humana. Sin embargo, tienden a sufrir de sobreajuste (overfitting). Para mitigarlo, se aplican técnicas de poda (pruning) o métodos de ensamble como Random Forest y Gradient Boosting, los cuales combinan múltiples árboles para mayor robustez.

2. Motivación y Comparación con la Regresión Logística

Para comprender la necesidad de los árboles de decisión, es útil revisar el modelo de clasificación más sencillo y directo: la **regresión logística**. La regresión logística ofrece un desempeño óptimo bajo dos condiciones fundamentales:

- **Separabilidad clara en el espacio de características:** Las clases están claramente distanciadas geográficamente o numéricamente (por ejemplo, puntos de terreno agrícola frente a terreno seco según latitud y longitud).
- **Geometría favorable de la frontera de decisión:** La frontera donde las probabilidades de pertenecer a la clase 1 y a la clase 0 son iguales (P(Y=1) = P(Y=0) = 0.5) se puede definir mediante una ecuación lineal simple o un hiperplano.

Ecuación de la frontera de decisión lineal:

$$x \cdot \beta = 0 \quad \Rightarrow \quad latitud = 0.8 \cdot longitud + 0.1$$

Limitaciones de los Modelos Lineales y Deseos del Modelo Ideal

Cuando las fronteras de decisión se vuelven complejas o no lineales, la regresión logística requiere introducir términos polinomiales de orden superior (por ejemplo, $x1^2 + x2^2 - 0.25 = 0$). Aunque estos términos logran moldear fronteras curvas, **la interpretación del impacto de cada variable sobre los log-odds se vuelve sumamente difícil**. Esto nos lleva a plantear nuestra lista de deseos para el modelo ideal:

1. **Permitir fronteras de decisión complejas y no lineales.**
2. **Mantener una interpretación sencilla y directa para las personas.**

3. Intuición y Modelos Basados en Diagramas de Flujo

Los seres humanos utilizan de forma natural esquemas secuenciales para diferenciar objetos y tomar decisiones en la vida cotidiana. Un ejemplo clásico es el diagrama de flujo de ingeniería:

- **¿El objeto se mueve?** Si no debería moverse pero se mueve → Solución: Cinta americana (Duct Tape).
- **¿El objeto no se mueve?** Si debería moverse pero no lo hace → Solución: Golpear con un martillo.

Este principio se traduce matemáticamente en el modelo de árbol de decisión. Las fronteras resultantes son **localmente lineales** (segmentos paralelos a los ejes de coordenadas), lo que permite construir fronteras globales complejas a partir de componentes matemáticos simples.

Ejemplo Práctico: Clasificación de Frutas (Limones vs. Naranjas)

Consideremos la clasificación de frutas según sus dimensiones de **altura (height)** y **ancho (width)**:

- **Nodo Raíz:** ¿Altura > 6.5?
    - **Sí (Rama derecha):** ¿Ancho > 9.5? → Sí: **Limón** | No: **Naranja**

- **No (Rama izquierda):** ¿Ancho > 6.0? → Sí: **Limón** | No: **Naranja**

Cada árbol corresponde exactamente a una **partición del espacio de características** mediante hiperplanos paralelos a los ejes. La etiqueta asignada a cada región corresponde a la clase mayoritaria en ese subconjunto.

Predicción mediante Recorrido del Árbol (Traversing)

Para clasificar una nueva muestra con **altura = 5.9** y **ancho = 5.8**:

3. En el nodo raíz, evaluamos si la altura es mayor a 6.5. Como 5.9 ≤ 6.5, avanzamos hacia la **izquierda**.
4. En el nodo interno, evaluamos si el ancho es mayor a 6.0. Como 5.8 ≤ 6.0, avanzamos hacia la **izquierda**.
5. Llegamos a un nodo hoja cuya clase mayoritaria es **Naranja**. Esa es la predicción final.

4. Criterios de División (Splitting Criteria)

Para construir el árbol, debemos seleccionar la característica **p** y el umbral de corte **t_p** óptimos. Aunque existen múltiples opciones, todo criterio razonable debe seguir tres pautas fundamentales:

- **Aumento progresivo de la pureza:** A medida que el árbol se divide, las regiones deben volverse cada vez más puras (especializándose hacia una sola clase).
- **Evitar regiones vacías:** Las divisiones no deben generar subconjuntos sin puntos de entrenamiento.
- **Formulación diferenciable:** Es deseable que la métrica sea diferenciable para facilitar procesos de optimización numérica.

A. Error de Clasificación (Classification Error)

El error de clasificación mide la proporción de observaciones que pertenecen a la clase minoritaria en una región determinada:

$$\mathbf{Error(R) = 1 - max_k\ (\Psi_k)}$$

Donde **Ψ_k** representa la proporción de puntos de entrenamiento en la región que pertenecen a la clase **k**. Para evaluar la calidad global de una división entre dos regiones R1 y R2, se calcula el **promedio ponderado** por el número de puntos de cada región:

$$Error_Ponderado = (n1 / N) \cdot Error(R1) + (n2 / N) \cdot Error(R2)$$

B. Índice Gini (Gini Index)

El índice Gini es una medida de impureza alternativa que reemplaza la función de máximo por la suma de los cuadrados de las proporciones:

$$\mathbf{Gini(R) = 1 - \sum_{k=1}^{K} (\Psi_k)^2}$$

> **Propiedad Matemática del Índice Gini:** ¿Por qué elevar al cuadrado en lugar de tomar el máximo?

> Sumar los cuadrados actúa como una aproximación suave (soft max) del valor máximo, ya que el término de mayor valor domina la suma. Además, a diferencia de la función máximo, la suma de cuadrados es totalmente diferenciable.

C. Entropía y Teoría de la Información (Entropy)

Basado en la teoría de la información, la entropía cuantifica el grado de desorden o la fuerza de la señal de una distribución de clases:

$$\text{Entropía}(R) = -\sum_{k=1}^{K} \Psi_k \cdot \log_2(\Psi_k)$$

Cuando una región es totalmente pura (todos los puntos de una clase), $\Psi = 1$ y la entropía es **0**. Cuando la región está equitativamente dividida entre clases ($\Psi = 0.5$), la entropía alcanza su valor máximo.

Comparación Resumida de los Tres Criterios

Criterio	Fórmula Matemática	Diferenciable	Penalización Impureza	Rango (2 Clases)
Error de Clasificación	$1 - \max_k(\Psi_k)$	No	Mínima	[0.0 , 0.5]
Índice Gini	$1 - \sum(\Psi_k)^2$	Sí	Intermedia	[0.0 , 0.5]
Entropía (Escalada)	$-0.5 \cdot \sum \Psi_k \log_2(\Psi_k)$	Sí	MÁXIMA	[0.0 , 0.5]

5. Construcción del Árbol: Algoritmo Voraz y Condiciones de Parada

Encontrar el árbol de decisión teóricamente óptimo para un conjunto de datos es un problema **NP-completo (computacionalmente intratable)**. Por esta razón, la construcción de árboles de decisión en la práctica se realiza mediante un **algoritmo voraz (greedy algorithm)** paso a paso:

6. **Paso Inicial:** Comenzar con un árbol vacío y el espacio de características completo sin dividir.
7. **Selección Óptima Local:** En cada nodo, evaluar todas las variables p y umbrales tp para elegir la división que minimice la impureza ponderada (Gini o Entropía).
8. **Recursión:** Repetir el proceso recursivamente en los nodos hijos generados.
9. **Asignación de Hoja:** Cuando se alcance una condición de parada, asignar como predicción la clase mayoritaria de los puntos en esa región.

Condiciones de Parada (Stopping Conditions)

Para evitar que el árbol crezca indefinidamente hasta memorizar el ruido de los datos, se aplican criterios de parada:

- **Pureza absoluta del nodo:** Todos los puntos pertenecientes al nodo corresponden a la misma clase.
- **Profundidad máxima alcanzada (max_depth):** Se restringe el número máximo de niveles del árbol.
- **Número mínimo de muestras por nodo (min_samples_split / min_samples_leaf):** Se detiene el split si un nodo contiene menos observaciones del umbral configurado.
- **Ganancia de información insuficiente:** La disminución de la impureza producida por la división es inferior a un valor ε prefijado.

> **Conclusión Clave:** El algoritmo voraz no garantiza encontrar el óptimo global, pero proporciona una solución altamente efectiva en tiempo polinómico, equilibrando flexibilidad geométrica, rapidez de cómputo e interpretabilidad.

Apuntes: Motivación para Árboles de Decisión

Límites de Decisión, Regresión Logística y la Necesidad de Modelos Complejos e Interpretables

1. Eficacia de la Regresión Logística en Geometrías Simples

La **regresión logística** es uno de los algoritmos fundamentales para problemas de clasificación. Sin embargo, su máxima eficacia operacional se alcanza bajo dos condiciones específicas en el espacio de características:

- **Separabilidad de clases:** Las clases se encuentran claramente diferenciadas y bien separadas en el espacio de atributos.
- **Geometría simple de la frontera:** La frontera o límite de clasificación posee una estructura geométrica regular (como una línea, plano u hiperplano).

2. Formulación Matemática de la Frontera de Decisión

En la regresión logística, la **frontera de decisión** corresponde al conjunto de puntos donde la probabilidad condicional de pertenecer a la clase 1 es igual a la probabilidad de pertenecer a la clase 0. Es decir, donde la certidumbre de clasificación es neutral:

> **FORMULACIÓN DE LA FRONTERA DE DECISIÓN**
>
> $P(Y = 1) = 1 - P(Y = 1) \Rightarrow P(Y = 1) = 0.5$
>
> En términos de los log-odds (logaritmo de la razón de probabilidades):
>
> $\log [P(Y = 1) / (1 - P(Y = 1))] = X\beta$
>
> Por lo tanto, la frontera de decisión queda definida por la ecuación lineal: $X\beta = 0$

Esta ecuación representa un **hiperplano** en el espacio de características. Si bien este modelo puede generalizarse introduciendo términos polinomiales de orden superior para describir hiperplanos o superficies no lineales, dicha extensión complica drásticamente la interpretación del modelo.

Ejemplo Numérico: Separación de Terrenos Agricolas y Secos

Considere el ejemplo práctico de clasificación geográfica entre **Terreno Agrícola (Agricultural Land)** y **Terreno Seco (Dry Land)** según su Latitud y Longitud. La frontera lineal dibujada sobre los datos se describe formalmente como:

- **Forma Explícita:** Latitud = 0.8 · Longitud + 0.1
- **Forma Implícita (Ecuación del Hiperplano):** -0.8 · Longitud + Latitud - 0.1 = 0

3. Desafíos con Distribuciones Geométricas Complejas

Cuando los datos presentan estructuras no lineales en el espacio de atributos, la regresión logística lineal queda severamente limitada:

- **Cajas delimitadoras circulares o múltiples regiones:** En patrones donde una clase está rodeada por otra (forma circular) o repartida en múltiples regiones disjuntas (bloques en esquinas opuestas), se requieren fronteras complejas.
- **Imposibilidad de descripción lineal única:** Aunque las clases se mantengan bien separadas en el espacio de características, los límites de decisión no se pueden describir fácilmente mediante una sola ecuación lineal.
- **Pérdida de interpretabilidad:** Si bien la regresión logística lineal es muy intuitiva (cada coeficiente cuantifica el impacto directo en las probabilidades logarítmicas), al añadir términos polinomiales de mayor orden resulta sumamente difícil interpretar el efecto de las variables sobre la clasificación resultante.

4. Objetivos y Motivación de los Árboles de Decisión

Ante las limitaciones descritas, surge la necesidad de desarrollar modelos de aprendizaje automático alternativos. La **motivación principal** para el uso de árboles de decisión se fundamenta en crear modelos que cumplan simultáneamente dos requisitos fundamentales:

> **REQUISITOS IDEALES DEL MODELO**
>
> 1. Flexibilidad geométrica: Permitir fronteras y límites de decisión complejos y no lineales.
> 2. Alta interpretabilidad: Mantener un modelo fácil de comprender, explicar e interpretar por seres humanos.

Tabla Comparativa de Modelos de Clasificación

Modelo	Límite de Decisión	Interpretabilidad	Adecuación Geométricas
Regresión Logística Lineal	Hiperplano lineal ($X\beta = 0$)	Alta (coeficientes en log-odds)	Solo clases linealmente separables
Regresión Logística Polinomial	Superficie no lineal compleja	Baja (difícil interpretar términos)	Geometrías moderadamente complejas
Árboles de Decisión	Fronteras complejas (por tramos)	Alta (reglas si/entonces interpretables)	Excelente para geometrías complejas

Estructura, Geometría e Inferencia en Árboles de Decisión

Guía de Apuntes Técnicos Basada en el Modelo de Clasificación de Frutas (Limones y Naranjas)

Índice de Contenidos

(Nota: El índice de contenidos se actualizará automáticamente al abrir el documento en Microsoft Word o Google Docs).

1. Modelos Interpretables e Intuición Cotidiana

Los seres humanos emplean de manera natural modelos interpretables para estructurar la toma de decisiones y diferenciar clases de objetos o fenómenos en diversos contextos de la vida diaria. Un ejemplo clásico e intuitivo es el diagrama de flujo de ingeniería (*Engineering Flowchart*), en el cual se evalúan condiciones secuenciales simples para determinar una acción correctora.

> **Símil Práctico: El Diagrama de Flujo de Ingeniería**
> El diagrama de ingeniería evalúa dos preguntas fundamentales:
> 1. ¿Se mueve el objeto? (Does it move?)
> 2. ¿Debería moverse? (Should it?)
>
> • Si se mueve y NO debería: Se aplica cinta aislante (Duct Tape).
> • Si NO se mueve y SÍ debería: Se aplica un golpe con martillo (Hammer).
> • En los casos donde el estado coincide con lo deseado: No hay problema (No problem).
>
> Este esquema lógico demuestra cómo reglas condicionales encadenadas resuelven problemas complejos mediante pasos binarios simples e interpretables.

Esta lógica secuencial puede formularse matemáticamente como un modelo de clasificación que posee dos propiedades ideales:

- **Fronteras de decisión suficientemente complejas:** Capaces de adaptar la separación entre clases en espacios no lineales.

- **Alta interpretabilidad humana:** Las reglas pueden inspeccionarse y comprenderse directamente sin recurrir a 'cajas negras'.

Además, las fronteras de decisión generadas por estos modelos son **localmente lineales**. Esto significa que cada segmento individual del límite de separación se describe sencillamente mediante una ecuación matemática lineal perpendicular a uno de los ejes de las características.

2. Anatomía y Estructura de un Árbol de Decisión

Un árbol de decisión organiza el flujo de inferencia mediante una estructura jerárquica compuesta por tres tipos fundamentales de nodos:

Tipo de Nodo	Ubicación y Función	Representación Visual
Nodo Raíz (Root Node)	Posicionado en la parte superior del árbol. Inicia todo el proceso de toma de decisiones evaluando la primera característica clave.	Cuadro rectangular en la cima
Nodos Internos (Internal Nodes)	Todos los nodos intermedios subsecuentes. Evalúan condiciones adicionales sobre las características para subdividir los datos.	Cuadros rectangulares intermedios
Nodos Hoja (Leaf Nodes)	Puntos terminales del árbol donde no ocurren divisiones adicionales. Representan el resultado final o la clase predicha.	Frutas / Círculos en la base

3. Modelo Práctico: Clasificación de Limones y Naranjas

El modelo de ejemplo tiene como objetivo clasificar frutas en dos clases continuas: **Limones** (representados simbólicamente por triángulos amarillos) y **Naranjas** (representadas por círculos naranjas).

Geometría del espacio de características:
Cada nodo del árbol corresponde a una partición del espacio bidimensional definido por dos variables continuas de la fruta: la altura (*Height*) y el ancho (*Width*). Las particiones dividen el plano mediante líneas paralelas a los ejes coordinados.

Estructura exacta de reglas de división del árbol:

- **Evaluación Raíz:** ¿Height > 6.5?
  - **Rama 'SÍ' (Derecha):** Se evalúa el nodo interno: ¿Width > 9.5?
    - **SÍ → Hoja:** Limón

    - **NO → Hoja:** Naranja
- **Rama 'NO' (Izquierda):** Se evalúa el nodo interno: ¿Width > 6.0?
    - **SÍ → Hoja:** Limón
    - **NO → Hoja:** Naranja

4. Inferencia y Predicción Paso a Paso (Travesía del Árbol)

Cuando se ingresa un nuevo punto de datos no visto, el modelo realiza la predicción mediante la navegación o recorrido del árbol (*traversing the tree*). Este proceso compara secuencialmente los valores del punto contra las condiciones de cada nodo hasta alcanzar un nodo hoja terminal.

> **Caso Práctico de Evaluación de Predicción**
> Se presenta una fruta no clasificada con las siguientes mediciones:
> • Altura (Height) = 5.9
> • Ancho (Width) = 5.8
>
> Desglose de la travesía del árbol:
> 1. Nodo Raíz (¿Height > 6.5?):
> - Dado que 5.9 NO es mayor que 6.5, el algoritmo toma la rama 'NO' (hacia la izquierda).
>
> 2. Nodo Interno Izquierdo (¿Width > 6.0?):
> - Dado que 5.8 NO es mayor que 6.0, el algoritmo toma la rama 'NO' (hacia la izquierda).
>
> 3. Nodo Hoja Terminal:
> - Se alcanza el nodo hoja correspondiente a NARANJA.
>
> Conclusión del modelo: La fruta es clasificada como NARANJA debido a que cae en la región del espacio donde la mayoría de los puntos de entrenamiento corresponden a naranjas.

5. Resumen de Reglas de Clasificación e Intervalos

A partir del árbol construido, el espacio 2D queda particionado en cuatro regiones ortogonales claramente delimitadas. La siguiente tabla sintetiza las condiciones geométricas y el resultado de clasificación asignado:

Región	Condición de Altura (Height)	Condición de Ancho (Width)	Clase Predicha
Región 1	Height ≤ 6.5	Width ≤ 6.0	**Naranja**
Región 2	Height ≤ 6.5	Width > 6.0	**Limón**
Región 3	Height > 6.5	Width ≤ 9.5	**Naranja**

Región 4	Height > 6.5	Width > 9.5	**Limón**

Árboles de Decisión: Criterios de División y Error de Clasificación

Guía de Apuntes Técnicos: Formulación Matemática, Evaluación de Pureza en Regiones y Optimización por Promedio Ponderado

---

Índice de Contenidos

1. Fundamentos Teóricos del Error de Clasificación

En la construcción de árboles de decisión para tareas de clasificación, uno de los objetivos centrales durante el proceso de partición de datos (*splitting process*) es evaluar cuán pura resulta cada una de las regiones generadas en el espacio de características. El **Error de Clasificación** es una métrica directa de la impureza que mide la proporción de observaciones que serían clasificadas erróneamente si asignáramos a toda la región la etiqueta de la clase mayoritaria.

> **Definición Matemática Básica:**
>
> Error de Clasificación = (Número de puntos de datos minoritarios) / (Número total de puntos de datos)
>
> O bien, en términos de la clase mayoritaria:
>
> Error de Clasificación = 1 - (Número de puntos de datos mayoritarios) / (Número total de puntos de datos)

Formalizando para un escenario general con **K** clases y un conjunto de particiones resultantes, denotamos por **Ψ(k | R_r)** a la proporción de puntos de entrenamiento dentro de la región *R_r* que pertenecen a la clase *k*. Bajo esta notación, la fórmula general del error para la región *R_r* se expresa como:

> **Formulación General con la Proporción Máxima (Ψ):**
>
> Error(R_r) = 1 - máx_k (Ψ(k | R_r))

> Donde Ψ(k | R_r) es la fracción de observaciones etiquetadas con la clase k en la región R_r, y el operador máx_k selecciona la proporción correspondiente a la clase dominante.

2. Caso Práctico 1: Evaluación de la División Inicial en $X_1 = 5.5$

Consideremos un espacio bidimensional compuesto por dos variables continuas ($\mathbf{X_1}$ y $\mathbf{X_2}$) y dos clases diferenciadas: **Clase 1** (representada por círculos azules) y **Clase 2** (representada por triángulos naranjas). En la primera prueba de partición, se establece un umbral vertical en $\mathbf{X_1 = 5.5}$, lo que divide el plano en dos regiones disjuntas: $\mathbf{R_1}$ (a la izquierda, $X_1 \leq 5.5$) y $\mathbf{R_2}$ (a la derecha, $X_1 > 5.5$).

- **Análisis de la Región R_1:** Contiene exactamente 6 puntos de datos, todos pertenecientes a la Clase 1 (azules) y 0 puntos de la Clase 2 (naranjas). La región es perfectamente pura.
- **Análisis de la Región R_2:** Contiene un total de 13 puntos de datos, distribuidos en 5 círculos azules (Clase 1) y 8 triángulos naranjas (Clase 2). En esta región coexisten ambas clases, por lo que presenta cierta impureza.

Tabla 1: Desglose del Cálculo del Error para el Ejercicio 1 (Umbral $X_1 = 5.5$)

Región	Clase 1 (●)	Clase 2 (▲)	Cálculo del Error de Clasificación
R_1	6	0	1 - máx(6/6, 0/6) = 1 - 1 = 0.00 (Pureza total)
R_2	5	8	1 - máx(5/13, 8/13) = 1 - 8/13 = 5/13 ≈ 0.38

Como se observa en la Tabla 1, la región $\mathbf{R_1}$ alcanza un error nulo (0.00) debido a la homogeneidad total de sus datos, mientras que en $\mathbf{R_2}$ la clase dominante son los triángulos (8 de 13), arrojando un error del 38.46% (5/13) representado por las 5 observaciones azules minoritarias.

3. Caso Práctico 2: División Alternativa e Impacto en la Pureza

Para analizar la sensibilidad de la métrica frente a distintas distribuciones de los datos, examinamos un segundo escenario de prueba donde la frontera se mantiene en el umbral $\mathbf{X_1 = 5.5}$, pero sobre un conjunto de observaciones con diferente densidad y dispersión.

- **Configuración de R_1:** Contiene únicamente 1 punto de datos (1 círculo azul y 0 triángulos naranjas). Total $N_1 = 1$.
- **Configuración de R_2:** Contiene un total de 17 puntos de datos, de los cuales 3 son círculos azules y 14 son triángulos naranjas. Total $N_2 = 17$.

Tabla 2: Desglose del Cálculo del Error para el Ejercicio 2 (División Alternativa)

Región	Clase 1 (●)	Clase 2 (▲)	Cálculo del Error de Clasificación
R_1	1	0	1 - máx(1/1, 0/1) = 1 - 1 = 0.00 (Pureza total)
R_2	3	14	1 - máx(3/17, 14/17) = 1 - 14/17 = 3/17 ≈ 0.18

En este segundo ejercicio, aunque la región **R_1** vuelve a tener un error de 0.00, la región **R_2** ha incrementado sustancialmente su pureza en comparación con el primer caso, reduciendo el error individual de la región del 0.38 al 0.18 (17.65%).

4. Minimización del Error Mediante el Promedio Ponderado

Calcular el error de clasificación de forma aislada en cada región no es suficiente para tomar la decisión de división óptima. Una región extremadamente pequeña (por ejemplo, con 1 solo punto como en el Ejercicio 2) puede dar un error de 0, pero aporta poco al volumen total de datos. Por ello, el algoritmo de construcción de árboles requiere calcular el **promedio ponderado** del error a través de todas las regiones creadas, ponderando según la cantidad de observaciones *N_r* que caen en cada región respecto al total *N*.

Ecuación de Optimización del Promedio Ponderado:

$$\text{mín}_{p, t_p} [(N_1 / N) * \text{Error}(R_1 \mid p, t_p) + (N_2 / N) * \text{Error}(R_2 \mid p, t_p)]$$

Donde:

- N_1 y N_2 representan el número de puntos de entrenamiento dentro de las regiones R_1 y R_2.
- $N = N_1 + N_2$ representa la cantidad total de puntos involucrados en la división.
- p es la variable/predictor seleccionada (ej. X_1 o X_2).
- t_p es el umbral numérico escogido para la partición.

A continuación, comparamos explícitamente el Promedio Ponderado Global para ambos ejercicios presentados en las fuentes:

- **Promedio Ponderado del Ejercicio 1 ($N_1 = 6$, $N_2 = 13$, $N = 19$):**
  Error Ponderado = (6 / 19) * 0.00 + (13 / 19) * (5 / 13) = 0 + (5 / 19) ≈ 0.2632 (26.32%)
- **Promedio Ponderado del Ejercicio 2 ($N_1 = 1$, $N_2 = 17$, $N = 18$):**
  Error Ponderado = (1 / 18) * 0.00 + (17 / 18) * (3 / 17) = 0 + (3 / 18) = 1 / 6 ≈ 0.1667 (16.67%)

Conclusión de Optimización: El algoritmo voraz (*greedy algorithm*) selecciona la combinación de variable *p* y umbral *t_p* que minimice este valor global. En este caso, la partición del Ejercicio 2 resulta superior desde el punto de vista del criterio de error ponderado (16.67% vs 26.32%), ya que logra consolidar una masa dominante de la Clase 2 en la región $\mathbf{R_2}$ con menor tasa de interferencia minoritaria.

ÁRBOLES DE DECISIÓN: CRITERIOS AVANZADOS DE DIVISIÓN Y ALGORITMO VORAZ

Análisis detallado del Índice Gini, Entropía, Comparación de Criterios y Algoritmo Voraz (Greedy Search)

ÍNDICE DE CONTENIDOS (Actualizable al abrir en Word / Google Docs):

1. El Índice Gini como Medida Suave y Diferenciable de Impureza

El **Índice Gini (Gini Index)** representa un criterio alternativo al error de clasificación para medir el grado de impureza de un nodo o región R. A diferencia del error de clasificación simple, que toma únicamente el valor máximo de la proporción de clase (max_k), el Índice Gini evalúa la suma de las proporciones al cuadrado de todas las clases presentes.

> **Formulación Matemática del Índice Gini:**
> Para un espacio con K clases, dado un nodo o región R con una proporción de observaciones $\Psi(k|R)$ pertenecientes a la clase k, el Índice Gini se define como:
>
> $$Gini(R) = 1 - \sum [\Psi(k|R)]^2 \quad (\text{para } k = 1, ..., K)$$
>
> Donde $\Psi(k|R)$ representa la proporción observada de puntos etiquetados como clase k dentro de la región R.

1.1. Fundamentación Teórica: ¿Por qué Elevar al Cuadrado?

La cualidad de sumar las proporciones al cuadrado en lugar de utilizar el operador máximo responde a dos ventajas computacionales esenciales:

- **Aproximación Suave al Máximo (Soft Maximum):** Al elevar las proporciones al cuadrado, la clase con mayor presencia (el término máximo) domina cuantitativamente el valor de la suma. Esto genera una penalización progresiva para nodos impuros y premia con valores cercanos a cero a las regiones homogéneas.
- **Diferenciabilidad Matemática:** La función máximo (max) presenta un punto anguloso no diferenciable en el punto de máxima incertidumbre ($\Psi = 0.5$). Por el contrario, el Índice Gini es una función cuadrática continua y diferenciable en todo su dominio, lo que habilita el uso de algoritmos analíticos de optimización.

1.2. Caso Práctico de Cálculo del Índice Gini

Tomando el ejemplo con observaciones divididas por la frontera $X_1 = 5.5$ en dos regiones (R_1 y R_2), evaluamos el Índice Gini exacto para cada región:

Región	Composición de Clases	Fórmula Aplicada	Índice Gini Resultante
Región 1 (R_1)	6 Círculos Azules 0 Triángulos Naranjas (Total $N_1 = 6$)	$1 - [(6/6)^2 + (0/6)^2]$ $= 1 - [1^2 + 0]$	0.00 (Pura)
Región 2 (R_2)	5 Círculos Azules 8 Triángulos Naranjas (Total $N_2 = 13$)	$1 - [(5/13)^2 + (8/13)^2]$ $= 1 - [25/169 + 64/169]$	0.4734 (~0.47)
Promedio Ponderado	Total N = 19 puntos $N_1 = 6$, $N_2 = 13$	$(6/19) \times 0.00 + (13/19) \times 0.4734$ $= 0 + 6.154 / 19$	0.3239 (~0.32)

2. Entropía y Ganancia de Información (Teoría de la Información)

El tercer criterio principal se fundamenta en la **Teoría de la Información**. En este enfoque, se evalúa la intensidad o fuerza de la señal (signal strength) cuantificando la incertidumbre o desorden de la distribución de clases en cada región mediante el cálculo de la **Entropía**.

> **Formulación Matemática de la Entropía:**
> Para una variable aleatoria discreta con proporción de clases $\Psi(k|R)$ en la región R, la entropía se define mediante la suma ponderada del logaritmo en base 2 de dichas proporciones:
>
> $$\text{Entropía}(R) = - \sum [\ \Psi(k|R) \times \log_2(\Psi(k|R))\ ] \quad (\text{para } k = 1, ..., K)$$
>
> Nota: Por convención, cuando $\Psi(k|R) = 0$, el término $0 \times \log_2(0)$ se define como 0.

2.1. Caso Práctico de Cálculo de la Entropía

Aplicando la fórmula de entropía a las regiones R_1 y R_2 de nuestro dataset de ejemplo:

Región	Distribución de Clases	Desarrollo del Logaritmo (Base 2)	Entropía Resultante
Región 1 (R_1)	6 Azules, 0 Naranjas ($N_1 = 6$, Pura)	$- [\ (6/6) \log_2(6/6) + 0\ ]$ $= - [\ 1 \times 0\ ]$	0.00 (Incertidumbre nula)
Región 2 (R_2)	5 Azules, 8 Naranjas ($N_2 = 13$)	$- [\ (5/13) \log_2(5/13) + (8/13) \log_2(8/13)\ ]$ $= - [\ 0.3846 \times (-1.3785) + 0.6154 \times (-0.7004)\ ]$	0.9612 (~0.96)
Promedio	$N_1 = 6$, $N_2 = 13$	$(6/19) \times 0.00 + (13/19) \times 0.9612$	0.6576

Ponderado	Total N = 19	= 0 + 12.495 / 19	(~0.66)

3. Comparativa de Criterios y Curvas de Impureza

Para realizar una comparación rigurosa entre los tres criterios de división en un problema binario (Clase 0 vs Clase 1), se escala la Entropía multiplicándola por el factor 0.5 (0.5 × Entropía). De esta forma, el valor máximo de los tres criterios queda normalizado a 0.5 en el punto de máxima impureza:

3.1. Normalización y Comportamiento Funcional

Criterio de División	Fórmula para 2 Clases (Ψ = P(Clase 1))	Valor en Pureza (Ψ=0 o Ψ=1)	Valor en Mezcla Máxima (Ψ=0.5)
Error de Clasificación	$1 - \max(\Psi, 1 - \Psi)$	0.00	0.50
Índice Gini	$1 - [\Psi^2 + (1 - \Psi)^2] = 2\Psi(1 - \Psi)$	0.00	0.50
Entropía Escalada (0.5 × H)	$-0.5 \times [\Psi \log_2\Psi + (1-\Psi) \log_2(1-\Psi)]$	0.00	0.50

3.2. Conclusiones Fundamentales de la Comparación de Criterios

- **Coincidencia Cualitativa:** Los tres criterios capturan cualitativamente el mismo fenómeno de impureza. Todos alcanzan su mínimo (0.0) cuando la región es completamente pura ($\Psi = 0$ o $\Psi = 1$) y su máximo en el punto de máxima mezcla ($\Psi = 0.5$).
- **Jerarquía de Penalización:** Para valores intermedios de impureza, la Entropía es la función que penaliza más severamente la presencia de clases mezcladas. El Índice Gini ocupa una posición intermedia, mientras que el Error de Clasificación es el criterio menos severo.
- **Preferencia Práctica en Implementaciones:** Debido a que el Error de Clasificación carece de derivadas continuas en $\Psi = 0.5$ y es menos sensible a cambios en la pureza de los nodos hijo, casi todas las librerías de Machine Learning (como Scikit-Learn) utilizan exclusivamente el Índice Gini o la Entropía para construir los árboles.

4. Algoritmo Voraz (Greedy Algorithm) y Complejidad Computacional

Determinar la estructura globalmente óptima de un árbol de decisión para un conjunto de datos dado es un problema de clase **NP-Completo (NP-Complete)**, lo que significa que es computacionalmente intratable explorar todas las combinaciones posibles de particiones.

> **Solución Práctica: La Estrategia Voraz (Greedy Search):**
> Dado que la búsqueda del óptimo global es inviable, los algoritmos de construcción de árboles emplean un enfoque voraz (top-down, greedy approach). En cada paso, el algoritmo selecciona la mejor división local sin considerar el impacto en divisiones futuras ni reconsiderar decisiones previas.

4.1. Procedimiento Paso a Paso del Algoritmo Voraz

Paso 1: Inicialización del Espacio: Comenzar con un árbol vacío y el espacio completo de características conteniendo todas las observaciones de entrenamiento.

Paso 2: Selección de la División Óptima Local: Para cada predictor disponible p y para cada posible umbral de división t_p, calcular la métrica de impureza ponderada (Gini o Entropía). Seleccionar el par (p, t_p) que minimice dicha impureza.

Paso 3: Partición Recursiva: Dividir la región actual en dos subregiones hijo y aplicar de manera recursiva el procedimiento de división a cada nuevo nodo generado.

Paso 4: Evaluación de Condiciones de Parada: Verificar en cada paso si se cumple alguna condición de parada (ej. alcanzar la profundidad máxima, número mínimo de puntos por nodo o pureza total). Detener la división en ese nodo si la condición se satisface.

Paso 5: Asignación de Etiquetas de Predicción: Para los nodos hoja resultantes, asignar como etiqueta de predicción la clase mayoritaria (majority class) entre los puntos de entrenamiento contenidos en esa región.

1.1.5 - Teoría de la Información y Entropía

Evaluación de la Calidad de División mediante Métricas de Incertidumbre y Fuerza de Señal

Tabla de Contenidos (Actualizable en Word / Google Docs)

1. Fundamentación Teórica: Evaluación basada en Incertidumbre

La evaluación de la calidad de una división (split) en un árbol de decisión puede abordarse desde distintas perspectivas métricas. La tercera gran métrica analítica —tras el error de clasificación y el Índice Gini— está motivada por los principios fundamentales de la **Teoría de la Información**, introducida originalmente por Claude Shannon. En este contexto, la calidad de una partición del espacio de características se mide en función de la **reducción de incertidumbre** (o ganancia de información) que se logra al dividir los datos.

> **Principio Clave de la Teoría de la Información:** Una división de alta calidad es aquella que transforma un conjunto heterogéneo de datos (alta incertidumbre) en subregiones donde una de las clases predomine de forma clara, generando una 'señal' inequívoca de clasificación.

2. Análisis de Pureza e Intensidad de Señal (Caso Práctico)

Para ilustrar la necesidad de medir la intensidad de la señal más allá del simple conteo de errores, considérese la partición del espacio bidimensional definido por las variables **X1** y **X2** mediante un umbral vertical ubicado en **X1 = 5.5**. Esta división genera dos subregiones distintas: la región izquierda **R1** (donde X1 ≤ 5.5) y la región derecha **R2** (donde X1 > 5.5).

- **Región R1 (X1 ≤ 5.5):** Esta región contiene casi exclusivamente elementos pertenecientes a la Clase 2 (triángulos naranjas), con una presencia marginal de la Clase 1 (un solo círculo azul). Aunque técnicamente no es una región 100% pura, **R1 envía una 'señal' muy fuerte y clara a favor de la Clase 2**.
- **Región R2 (X1 > 5.5):** Esta región presenta una distribución altamente mezclada entre círculos azules (Clase 1) y triángulos naranjas (Clase 2). En consecuencia, la 'señal' transmitida por R2 es débil e imprecisa, lo que refleja un estado de elevada incertidumbre.

3. Formulacion Matemática de la Entropía de Shannon

Para matematizar el concepto intuitivo de 'fuerza de señal' e 'incertidumbre', la Teoría de la Información calcula la **entropía** de la distribución de clases dentro de una región R con K clases posibles mediante la siguiente ecuación formal:

> **Ecuación de la Entropía de una Región R:**
> $H(R) = - \Sigma_{k=1}^{K} \Psi_k \cdot \log_2(\Psi_k)$
>
> donde Ψ_k representa la proporción de puntos de entrenamiento dentro de la región R que pertenecen a la clase k.

Propiedades matemáticas clave de la Entropía:

- **Mínimo Absoluto (H = 0):** Se alcanza cuando todos los datos de la región pertenecen a una sola clase ($\Psi_k = 1$ para alguna clase k, y 0 para las demás). Corresponde a una región perfectamente pura donde no existe incertidumbre.
- **Máximo Absoluto (H = 1 en clasificación binaria):** Se alcanza cuando las clases están distribuidas de forma perfectamente equitativa ($\Psi_1 = 0.5$, $\Psi_2 = 0.5$). Representa la máxima incertidumbre y la ausencia total de señal preferencial.

4. Tabla Comparativa de Fuerza de Señal e Impureza

A continuación se sintetizan las propiedades observadas en las subregiones R1 y R2 tras aplicar la partición en X1 = 5.5:

Región	Composición de Clases	Intensidad de Señal	Nivel de Entropía	Diagnóstico de Región
R1 (X1 ≤ 5.5)	Mayoría Clase 2 (Triángulos naranjas)	Señal Muy Fuerte	Entropía Baja (Próxima a 0)	Alta Pureza Dominante Clase 2
R2 (X1 > 5.5)	Mezcla Heterogénea Clase 1 y Clase 2	Señal Débil / Incierta	Entropía Alta (Próxima a 1)	Baja Pureza Alta Incertidumbre

Conclusión del Análisis: La región R1 demuestra que la teoría de la información permite identificar divisiones de calidad superior incluso cuando las subregiones resultantes contienen ligeras mezclas de datos. Al cuantificar el descenso de la entropía, los algoritmos de árboles de decisión privilegian divisiones como R1 que concentran la densidad de una clase y clarifican la regla de decisión.

Cálculo de Entropía y Optimización de Cortes en Árboles de Decisión

Apuntes Académicos sobre la Sección 1.1.6: Entropía de Shannon, Evaluación Numérica y Entropía Media Ponderada

---

1. Formulación Teórica de la Entropía de Shannon

La **Entropía** es una métrica proveniente de la teoría de la información que cuantifica el nivel de incertidumbre o desorden en la distribución de clases de una región dada. En el contexto de los algoritmos de clasificación, supongamos un espacio de características definido por **P predictores** y **K clases distintas**. Cuando elegimos dividir una región mediante el k-ésimo predictor a lo largo de un umbral específico $t_p \in \mathbb{R}$, evaluamos la calidad de la partición calculando la entropía en cada subregión resultante.

$$\text{Entropía}(R_r \mid p, t_p) = -\sum [\, \Psi(k \mid R_r) \cdot \log_2 \Psi(k \mid R_r) \,]$$

> **Q NOTA PARA EL APRENDIZ:**
> *El cálculo de la entropía representa, en realidad, la entropía condicional de la distribución de los puntos de entrenamiento entre las K clases, dado que el punto seleccionado se encuentra confinado dentro de la región r.*

2. Evaluación Numérica de la División en $X_1 = 5.5$

Al aplicar un corte vertical sobre la variable **X_1 en el umbral 5.5**, el espacio bidimensional de características queda dividido en dos subregiones: **Región R_1** (lado izquierdo, $X_1 \leq 5.5$) y **Región R_2** (lado derecho, $X_1 > 5.5$). La siguiente tabla detalla la composición de observaciones y el desarrollo explícito del cálculo de entropía:

Región	Clase 1 (Círculos)	Clase 2 (Triángulos)	Cálculo Explícito de Entropía
Región R_1	6	0	$-\,[\,(6/6)\cdot\log_2(6/6) + (0/6)\cdot\log_2(0/6)\,] = 0.00$
Región R_2	5	8	$-\,[\,(5/13)\cdot\log_2(5/13) + (8/13)\cdot\log_2(8/13)\,] \approx 1.38$

3. Comparativa Multicriterio en la Región R_2

Al analizar la impureza de la región **Región R_2** (compuesta por 5 observaciones de Clase 1 y 8 de Clase 2), los tres criterios de división tradicionales ofrecen métricas numéricas distintas debido a sus formulaciones matemáticas:

Criterio de División	Valor Obtenido en R_2	Sensibilidad y Comportamiento Analítico
Tasa de Error de Clasificación	0.38	Mide directamente la proporción de la clase minoritaria (5/13). Es insensible a variaciones finas en la mezcla.
Índice Gini	0.47	Suma de proporciones al cuadrado (1 - (5/13)² - (8/13)²). Ofrece diferenciabilidad continua y penalización suave.
Entropía de Shannon	1.38	Aplica escala logarítmica en base 2. Mide la incertidumbre condicional y penaliza con la mayor severidad la falta de pureza.

4. Minimización de la Entropía Media Ponderada

El objetivo global del algoritmo durante el proceso de aprendizaje consiste en seleccionar la variable predictor *p* y el umbral de corte óptimo *t_p* que minimicen la **Entropía Media Ponderada** sobre las dos regiones hijas creadas:

$$\min_{p,\ t_p} \left[\frac{N_1}{N} \cdot \text{Entropía}(R_1 \mid p,\ t_p) + \frac{N_2}{N} \cdot \text{Entropía}(R_2 \mid p,\ t_p) \right]$$

Donde *N_r* representa el número total de puntos de entrenamiento contenidos en la región *R_r*, y *N* es el tamaño total de la muestra ($N = N_1 + N_2 = 6 + 13 = 19$).

Sustituyendo los valores de nuestro ejemplo obtenemos:
• Entropía Ponderada = $(6/19) \cdot 0.00 + (13/19) \cdot 1.38 \approx$ **0.9433**

Esta función objetivo garantiza que el algoritmo no solo busque regiones puras, sino que premie aquellas particiones que logren aislar una gran cantidad de observaciones en nodos altamente homogéneos.

Comparación de Criterios de Impureza y Algoritmo Voraz

Apuntes Técnicos de Aprendizaje Automático: Sección 1.1.7 - Comparación y Construcción de Árboles

1. Comparación Analítica y Geométrica de Criterios de Impureza

En la construcción de árboles de decisión para clasificación binaria (Clase 0 vs. Clase 1), la calidad de una región o nodo se evalúa mediante la métrica de **pureza (Ψ)**, definida como la proporción de observaciones pertenecientes a la clase positiva en dicha región ($0 \leq \Psi \leq 1$). Las tres métricas estándar —Entropía, Índice Gini y Error de Clasificación— miden el grado de incertidumbre del nodo pero presentan propiedades curvas y comportamientos analíticos diferenciados.

1.1 Comportamiento Límite y Puntos Críticos

Todas las funciones de impureza comparten dos puntos de coincidencia fundamentales basados en la teoría de la información y la decisión:

- **Nodos Puros ($\Psi = 0.0$ o $\Psi = 1.0$):** Cuando una región contiene únicamente elementos de la Clase 0 ($\Psi = 0$) o de la Clase 1 ($\Psi = 1$), la incertidumbre es nula. En ambos extremos, la Entropía, el Índice Gini y el Error de Clasificación alcanzan exactamente el valor 0.0.
- **Máxima Impureza ($\Psi = 0.5$):** Cuando la región contiene una mezcla equitativa (50% Clase 0 y 50% Clase 1), la incertidumbre es máxima. En este punto central, las tres métricas alcanzan su valor pico relativo: la Entropía es 1.0 (o 0.5 si se escala por 0.5), el Índice Gini es 0.5 y el Error de Clasificación es 0.5.

1.2 Geometría Funcional, Curvatura y Diferenciabilidad

Aunque las tres medidas alcanzan su máximo en $\Psi = 0.5$, la trayectoria geométrica que siguen para llegar allí condiciona de forma determinante su rendimiento en los algoritmos de optimización:

- **Entropía e Impureza Gini (Curvas Cóncavas):** Son funciones estrictamente cóncavas hacia abajo ($f''(\Psi) < 0$) y continuas en todo el intervalo (0, 1). Esta curvatura suave garantiza que sean diferenciables, lo que proporciona un gradiente continuo que penaliza fuertemente los nodos impuros y facilita la búsqueda de divisiones que incrementen sustancialmente la pureza.
- **Error de Clasificación (Línea Recta a Trozos):** Es una función lineal por partes en forma de 'tienda' con un pico no diferenciable en $\Psi = 0.5$. Al ser una línea recta en los tramos (0, 0.5) y (0.5, 1), el error de clasificación carece de sensibilidad a cambios en la pureza que no alteren la clase mayoritaria, lo que la hace poco apta como criterio de división durante el crecimiento del árbol.

Tabla 1: Comparativa Matemática de los Criterios de Impureza (Problema Binario)

Criterio	Fórmula Matemática	Valor Máx. (Ψ=0.5)	Geometría	Diferenciable
Entropía (Shannon)	$H(\Psi) = -\Psi \log_2\Psi - (1-\Psi) \log_2(1-\Psi)$	1.0 (o 0.5 escalado)	Curva cóncava suave	Sí (en $0 < \Psi < 1$)
Índice Gini	$Gini(\Psi) = 1 - [\Psi^2 + (1-\Psi)^2]$	0.5	Curva cóncava suave	Sí (en $0 < \Psi < 1$)
Error Clasificación	$E(\Psi) = 1 - \max(\Psi, 1-\Psi)$	0.5	Línea recta a trozos	No (pico en $\Psi=0.5$)

CONCLUSIÓN SOBRE MÉTRICAS DE IMPUREZA

Ninguna métrica de impureza es universalmente 'superior' a las demás; simplemente asignan distintos pesos a diferentes valores de pureza Ψ.

Para la construcción del árbol, la Entropía y el Índice Gini son ampliamente preferidos sobre el Error de Clasificación debido a su estricta concavidad y diferenciabilidad, que permiten detectar mejoras sutiles en la pureza de los nodos.

2. Construcción de Árboles mediante el Algoritmo Voraz (Greedy Search)

2.1 Complejidad Computacional e Intratabilidad NP-Completo

Aprender el árbol de decisión 'óptimo' a nivel global para un conjunto de datos dado es un problema **NP-completo (intratable)** para la mayoría de las definiciones simples de 'óptimo'. La búsqueda exhaustiva de todas las posibles combinaciones de divisiones y estructuras de árbol requeriría un tiempo exponencial. Para solucionar esta limitación, se emplea un algoritmo heurístico voraz (Greedy Algorithm).

2.2 Flujo Operativo del Algoritmo Voraz en 4 Pasos

PASO 1	**Paso 1: Inicialización de la Raíz** Comience con un árbol de decisión vacío, donde todo el conjunto de entrenamiento reside en el nodo raíz y el espacio de características no está dividido.
PASO 2	**Paso 2: Selección Local Óptima del Punto de Corte** Elija el predictor óptimo p* y el valor umbral óptimo t_p* que maximicen la reducción de impureza (o minimicen la impureza ponderada de las regiones resultantes R_1 y R_2).
PASO 3	**Paso 3: Inducción Recursiva** Realice el proceso de partición de forma recursiva en cada nuevo nodo hijo resultante hasta que se cumpla una condición de parada predefinida (nodo puro, profundidad máxima o mínimo de muestras por hoja).

PASO 4	**Paso 4: Asignación de Etiquetas (Clasificador de Bayes)** En el caso de clasificación, asigne a cada región hoja una etiqueta de clase basada en la clase mayoritaria de los puntos de entrenamiento contenidos en dicha región (Regla de Decisión de Bayes).

2.3 Compromisos Teóricos del Enfoque Voraz

El algoritmo voraz no evalúa el impacto de una división actual en decisiones futuras; toma la mejor decisión inmediata en cada paso. Aunque esto no garantiza formalmente encontrar el árbol óptimo global, en la práctica proporciona modelos de muy alta precisión con una complejidad computacional escalable $O(P \cdot N \log N)$.

> **SÍNTESIS DE LA CONSTRUCCIÓN VORAZ**
>
> El enfoque voraz (Top-Down Induction) sustituye la búsqueda global NP-completa por una serie de elecciones óptimas locales.
>
> Cada división divide recursivamente el espacio hasta que se alcanza la condición de parada, momento en el cual la predicción en la hoja queda determinada por la regla de mayoría bayesiana.

Condiciones de Detención, Modos de Crecimiento y Equilibrio Sesgo-Varianza en Árboles de Decisión

Guía completa de teoría, hiperparámetros en Sklearn y optimización de clasificadores de aprendizaje supervisado

---

1. Control del Sobreajuste: La Necesidad de Detener el Crecimiento del Árbol

Un árbol de decisión no restringido que crece de forma indefinida dividirá el espacio de características hasta que cada región hoja contenga exactamente un único punto del conjunto de entrenamiento. En este escenario extremo, el modelo alcanza una precisión de entrenamiento del 100%, memorizando por completo el ruido y las particularidades de la muestra de datos.

Este fenómeno se conoce como **sobreajuste (overfitting)**. Para evitar que el árbol genere fronteras de decisión excesivamente complejas e incapaces de generalizar ante nuevos datos, es imprescindible aplicar **condiciones de detención (stopping conditions)** que interrumpan manualmente el proceso recursivo de división.

> **Principio Clave del Sobreajuste:** Si no se especifica ninguna condición de parada, cada nodo hoja se convierte en una región unitaria. El modelo logra un ajuste perfecto en entrenamiento, pero su varianza se dispara y su capacidad predictiva en validación decae drásticamente.

2. Criterios de Detención e Hiperparámetros en Sklearn

Existen diversas reglas para restringir la expansión del árbol. En la librería **scikit-learn (sklearn)**, estas reglas están implementadas a través de hiperparámetros específicos que controlan la profundidad, el tamaño de las hojas o la ganancia mínima de impureza.

Condición de Detención	Hiperparámetro Sklearn	Mecanismo de Funcionamiento
Profundidad Máxima	`max_depth`	Limita el número máximo de niveles de división desde la raíz. Por ejemplo, max_depth=1 permite sólo 1 división (árbol 'stump'); max_depth=2 permite 2 niveles de ramas.
Pureza del Nodo	**Automático (sin**	Interrumpe la división en una región si todas las

	ganancia)	instancias contenidas pertenecen a la misma clase. Dividir hojas puras no genera ninguna ganancia de información.
Muestras Mínimas en Hoja	min_samples_leaf	Impide la división de un nodo si alguna de las subregiones resultantes caería por debajo de un umbral mínimo de datos (ej. min_samples_leaf=4).
Nodos Hoja Máximos	max_leaf_nodes	Establece un límite superior al número total de hojas en el árbol. Al alcanzar este tope, el algoritmo detiene cualquier división futura.
Disminución Mínima de Impureza	min_impurity_decrease	Exige que la ganancia de pureza alcanzada al dividir la región R en R1 y R2 supere un valor mínimo predefinido para autorizar el corte.

3. Modos de Crecimiento del Árbol: Orden de Nivel vs. Mejor Primero

El orden en el que se seleccionan y agregan las divisiones al árbol depende del tipo de condición de detención utilizada. En **scikit-learn**, existen dos modos principales de construcción:

- **Crecimiento por Orden de Nivel (Level-Wise / Breadth-First):** Funciona rellenando de forma exhaustiva cada nivel del árbol horizontalmente antes de pasar al nivel inferior. Es el comportamiento predeterminado cuando se fijan parámetros como max_depth o min_samples_leaf.
- **Crecimiento de Mejor Primero (Best-First / Leaf-Wise):** Evalúa la disminución de impureza en todas las divisiones potenciales disponibles en el árbol y selecciona la división que ofrece la mayor ganancia, sin importar el nivel en el que se encuentre. Este modo se activa automáticamente en sklearn al especificar max_leaf_nodes.

Caso de Estudio: Predicción de Enfermedad Cardíaca (270 muestras)

Consideremos un conjunto de datos con 270 muestras para predecir enfermedades cardíacas según edad, sexo, presión arterial y colesterol con max_depth=2:

1. En Modo Orden de Nivel: Primero se divide el nodo raíz por sexo (nivel 1), generando dos ramas. A continuación, se evalúan simultáneamente los dos nodos restantes: la rama izquierda se divide por presión arterial y la rama derecha por edad (nivel 2), completando totalmente ambos niveles.

2. En Modo Mejor Primero: Se calcula la reducción de impureza en la rama izquierda (ej. ΔImpureza = 0.019) y en la rama derecha (ej. ΔImpureza = 0.35). El algoritmo prioriza la división de la rama derecha por aportar la mayor reducción de impureza, antes de proceder con las divisiones secundarias.

> **Comparación Estructural:** Aunque los árboles generados por 'Orden de Nivel' y 'Mejor Primero' pueden diferir temporalmente en su estructura intermedia durante la fase de crecimiento, al alcanzar la misma

> condición de parada final terminan produciendo resultados predictivos prácticamente idénticos.

4. El Compromiso Sesgo-Varianza en Función de la Profundidad

La elección de la profundidad máxima del árbol ilustra de manera directa el dilema clásico entre sesgo y varianza (bias-variance tradeoff) en aprendizaje automático. A medida que incrementamos max_depth, reducimos el sesgo pero aumentamos progresivamente la varianza.

Profundidad del Árbol	Nivel de Sesgo	Nivel de Varianza	Comportamiento del Modelo y Frontera de Decisión
Baja Profundidad (ej. max_depth = 4)	Alto Sesgo (Underfitting)	Baja Varianza (Alta Estabilidad)	El árbol es demasiado rígido y no captura fronteras no lineales complejas. No obstante, es muy robusto a pequeñas variaciones en los datos de entrenamiento; el límite trazado es estable.
Alta Profundidad (ej. max_depth = 100)	Bajo Sesgo (Memoria Total)	Alta Varianza (Sensible al Ruido)	El árbol clasifica correctamente todos los datos haciendo 'zigzag' alrededor de cada punto. Es extremadamente sensible a pequeñas perturbaciones en la frontera, sobreajusta y es costoso computacionalmente.

5. Selección Óptima de Hiperparámetros Mediante Validación Cruzada

Dado que existen múltiples reglas de detención (profundidad máxima, número de muestras por hoja, nodos máximos, ganancia mínima de impureza) y cada una requiere ajustar uno o varios hiperparámetros, surge la interrogante de cómo determinar la combinación óptima.

La estrategia estándar e imprescindible en el aprendizaje automático es el uso de **Validación Cruzada (Cross-Validation)**. Mediante validación cruzada (por ejemplo, K-Fold CV), se evalúa empíricamente el rendimiento del clasificador sobre conjuntos de validación independientes para diferentes combinaciones de hiperparámetros, seleccionando aquella configuración que maximice la capacidad de generalización y mantenga el equilibrio óptimo entre sesgo y varianza.

> **Regla de Oro Práctica:** Nunca evalúe el hiperparámetro óptimo utilizando únicamente el conjunto de entrenamiento. La precisión en entrenamiento crecerá monótonamente con la profundidad, mientras que la precisión en validación cruzada revelará el punto óptimo antes de que comience el sobreajuste.

Condiciones de Parada y Control del Sobreajuste en Árboles de Decisión

Guía Técnica sobre Hiperparámetros, Crecimiento Estratégico y Formulación de Ganancia en Scikit-Learn

---

Tabla de Contenidos

(Nota: La tabla de contenidos se actualizará automáticamente al abrir el archivo en Microsoft Word o Google Docs).

1. El Problema del Crecimiento Irrestricto y el Sobreajuste

Un árbol de decisión no restringido continúa su proceso recursivo de partición hasta que cada subregión del espacio de características alcanza la pureza absoluta. En este escenario extremo, cada nodo hoja final acaba encerrando **exactamente un punto de entrenamiento** ($N_{hoja} = 1$). Este fenómeno garantiza una exactitud del 100% sobre los datos de entrenamiento, pero constituye un caso severo de **sobreajuste (overfitting)**, ya que el modelo memoriza el ruido específico de la muestra en lugar de capturar la estructura subyacente del problema.

> **Pregunta Fundamental de Diseño:**
>
> ¿Qué ocurre si no detenemos manualmente el algoritmo del árbol de decisión?
>
> Respuesta: El árbol dividirá el espacio hasta que cada hoja contenga un único punto de datos. Aunque esto alcanza un 100% de precisión en entrenamiento, destruye la capacidad de generalización del modelo ante datos no observados.

Para evitar que el árbol se ramifique de forma indefinida y memorice los datos, se aplican mecanismos de **parada anticipada (early stopping o pre-pruning)**. Estas condiciones imponen límites estructurales o de ganancia de información durante la fase de construcción del árbol.

2. Condiciones de Parada Principales e Hiperparámetros

Existen cinco criterios o condiciones de parada fundamentales en la construcción de árboles de decisión de clasificación. Cada condición actúa como una regla lógica que desautoriza divisiones adicionales sobre un nodo determinado:

2.1. Homogeneidad de Clase (Nodos Hoja Puros)

No se divide una región si todas las observaciones presentes en ella pertenecen exactamente a la misma clase. Cuando la proporción de la clase dominante alcanza el 100% ($\Psi_k = 1$), la impureza del nodo es exactamente cero ($M(R) = 0$). En este punto, cualquier división adicional resultaría en subnodos redundantes sin ninguna ganancia de pureza.

2.2. Restricción de Profundidad Máxima (max_depth)

Constituye el método de parada más extendido. Limita la distancia máxima medida en número de bordes desde la raíz hasta la hoja más lejana. Por ejemplo, una profundidad máxima de 1 (a menudo llamada **stump de decisión**) permite únicamente un corte horizontal o vertical, produciendo un máximo de dos nodos hoja. Una profundidad de 2 permite hasta cuatro hojas, equilibrando la complejidad espacial con la varianza del modelo.

2.3. Muestras Mínimas por Hoja (min_samples_leaf)

Impide cualquier división que dé lugar a una subregión hija cuyo número total de instancias sea inferior a un umbral prefijado. Por ejemplo, si se establece **min_samples_leaf = 4**, el algoritmo evaluará si la propuesta de partición generaría algún hijo con 3 o menos puntos. De ser así, se cancela el corte y el nodo actual se convierte automáticamente en hoja terminal, previniendo la creación de reglas ajustadas a observaciones aisladas.

2.4. Número Máximo de Nodos Hoja (max_leaf_nodes)

Fija un límite global sobre el número total de terminales permitidos en toda la estructura del árbol. Por ejemplo, al especificar **max_leaf_nodes = 3**, el proceso de crecimiento finaliza en el momento exacto en que la estructura alcanza 3 hojas, impidiendo la expansión de nodos adicionales aunque estos no sean puros.

2.5. Umbral de Ganancia Mínima de Impureza (min_impurity_decrease)

Exige que la ganancia neta de pureza conseguida tras una partición potencial supere un valor prefijado. Si la reducción de impureza ponderada es inferior a dicho umbral, el corte se considera insignificante y el nodo no se divide.

Tabla 1: Matriz de Hiperparámetros de Parada y Criterios de Evaluación en Scikit-Learn

Criterio de Parada	Hiperparámetro (Sklearn)	Descripción y Regla Operativa
Nodos Hoja Puros	N/A (Regla intrínseca)	Detiene la división si todas las instancias en la región corresponden a la misma clase.
Profundidad Máxima	max_depth	Restringe el número máximo de niveles verticales desde el nodo raíz hasta las hojas.
Muestras por Hoja	min_samples_leaf	Exige un número mínimo de observaciones en

		cada subnodo hijo para autorizar el corte.
Nodos Hoja Máximos	max_leaf_nodes	Limita el número total de hojas finales permitidas en el árbol completo.
Ganancia de Impureza	min_impurity_decrease	Exige una reducción ponderada mínima de impureza para llevar a cabo la división.

3. Formulación Matemática de la Ganancia en Pureza

Para decidir si una región R con N observaciones se divide en dos subregiones R_1 (con N_1 observaciones) y R_2 (con N_2 observaciones), se calcula analíticamente la ganancia de pureza $Gain(R)$ mediante la diferencia entre la impureza original y el promedio ponderado de la impureza de sus subregiones:

> **Ecuación Formal de Ganancia en Pureza:**
>
> $Gain(R) = \Delta(R) = M(R) - (N_1 / N) \cdot M(R_1) - (N_2 / N) \cdot M(R_2)$
>
> Donde M(R) representa la métrica de impureza seleccionada (Error de Clasificación, Índice Gini o Entropía de Shannon), N es el número total de instancias en la región padre, y N_1, N_2 son los tamaños de las subregiones hijas.

Si el valor calculado $Gain(R)$ es menor que el umbral estipulado en **min_impurity_decrease**, se desestima la división y el nodo R se consolida como una hoja terminal del modelo.

4. Algoritmos de Crecimiento en Scikit-Learn: Primero en Profundidad vs. Mejor Primero

El orden en el que se seleccionan y expanden los nodos durante el entrenamiento altera la arquitectura intermedia del árbol y depende de los hiperparámetros especificados:

- **Modo Primero en Profundidad (Depth-First / Level-Wise):** Es la estrategia por defecto en Scikit-learn cuando se utilizan parámetros como **max_depth**. Expande todos los nodos nivel a nivel a lo largo de toda la anchura del árbol antes de descender al siguiente nivel. Se le conoce también como búsqueda en amplitud (breadth-first).
- **Modo Mejor Primero (Best-First / Leaf-Wise):** Se activa automáticamente en Scikit-learn cuando se define el hiperparámetro **max_leaf_nodes**. En lugar de expandir por niveles, evalúa de forma global todas las divisiones potenciales entre todos los nodos hoja existentes y expande siempre aquél que ofrece la mayor reducción absoluta de impureza ($Gain(R)$). Esto produce árboles asimétricos orientados a la máxima eficiencia de clasificación.

5. Conclusiones y Recomendaciones Prácticas

El ajuste correcto de las condiciones de parada determina el equilibrio adecuado entre el sesgo y la varianza del clasificador:

- **Valores restrictivos** (como depths muy bajas o min_samples_leaf elevados) previenen el sobreajuste pero pueden provocar desajuste (underfitting) si la frontera de decisión subyacente es compleja.
- **Valores permisivos** permiten aislar fronteras complejas pero incrementan sustancialmente la varianza, produciendo modelos inestables ante pequeñas perturbaciones en los datos.
- **Validación Cruzada:** La selección óptima de la combinación de estos hiperparámetros siempre debe llevarse a cabo mediante validación cruzada (cross-validation) sobre el conjunto de entrenamiento.

Estrategias de Crecimiento en Árboles de Decisión: Orden de Nivel vs. Mejor Primero

Análisis comparativo de algoritmos de expansión, formulación analítica de la disminución de impureza y topología estructural en Scikit-Learn

1. Modos de Expansión de Nodos en Algoritmos de Clasificación

La construcción de un árbol de decisión presupone la selección secuencial de variables y puntos de corte. Sin embargo, el orden en el que se seleccionan y expanden los nodos de la estructura arbórea no es único y depende directamente de la condición de parada e hiperparámetro especificado. En librerías estándar como **Scikit-Learn**, existen dos estrategias fundamentales para hacer crecer un árbol: la búsqueda por **orden de nivel (Level-Wise / Depth-First)** y la búsqueda de **mejor primero (Best-First / Leaf-Wise)**.

2. Crecimiento por Orden de Nivel (Level-Wise / Depth-First)

El método de orden de nivel opera completando de forma simétrica e integral cada nivel de profundidad del árbol antes de proceder a dividir los nodos pertenecientes al siguiente nivel. Esta estrategia se activa por defecto cuando la condición de parada principal está definida por la profundidad máxima (**max_depth**).

> **Ejemplo Práctico: Predicción de Enfermedad Cardíaca (N = 270 muestras)**
>
> • Nodo Raíz: Sexo <= 0.5 | Gini = 0.494 | N = 270 | Clase Predicha = No Enfermedad
>
> • Nivel 1 (Rama Izquierda): Presión Arterial (BP) <= 143.5 | Gini = 0.354 | N = 87 | Clase = No Enfermedad
>
> • Nivel 1 (Rama Derecha): Edad <= 53.5 | Gini = 0.496 | N = 183 | Clase = Enfermedad
>
> • Nivel 2 (Hojas de la Izquierda): N11 (Gini = 0.233, N = 67, No Enfermedad) y N12 (Gini = 0.495, N = 20, Enfermedad)
>
> • Nivel 2 (Hojas de la Derecha): N21 (Gini = 0.474, N = 83, No Enfermedad) y N22 (Gini = 0.435, N = 100, Enfermedad)

Bajo este esquema, se garantiza que todas las ramas alcancen la misma profundidad de forma balanceada, rellenando horizontalmente la estructura antes de descender.

3. Crecimiento de Mejor Primero (Best-First / Leaf-Wise)

El crecimiento de mejor primero sustituye la expansión por niveles por una evaluación codiciosa (*greedy*) entre todos los nodos hoja disponibles en todo el árbol. En cada iteración, el algoritmo calcula la **disminución de impureza ponderada** que produciría dividir cada hoja candidata, y ejecuta prioritariamente la división que maximice dicha reducción. Este modo se activa en Scikit-Learn al especificar el parámetro **max_leaf_nodes**.

Disminución de Impureza = (N_nodo / N_total) * [Gini(nodo) - (N_izq / N_nodo)*Gini(izq) - (N_der / N_nodo)*Gini(der)]

Para ilustrar el funcionamiento paso a paso sobre el conjunto de datos de enfermedad cardíaca:

Candidato	División Eval	Cálculo Numérico de Impureza	Resultado
Rama Izquierda	BP <= 143.5 (N = 87, Gini = 0.354)	(87/270) * [0.354 - (67/87)*0.233 - (20/87)*0.495]	≈ 0.019
Rama Derecha	Edad <= 53.5 (N = 183, Gini = 0.496)	(183/270) * [0.496 - (83/183)*0.474 - (100/183)*0.435]	≈ 0.350
Decisión	**Selección prioritarias**	**0.350 >> 0.019 => Se divide PRIMERO la Rama Derecha**	**Rama Derecha**

Como la disminución de impureza en la rama derecha (**0.350**) es significativamente mayor que en la rama izquierda (**0.019**), el algoritmo de mejor primero ejecuta inmediatamente la división de la rama derecha. En las iteraciones subsecuentes, el algoritmo evalúa continuamente las tres hojas resultantes (BP <= 143.5, Hoja Edad <= 53.5 Izq y Hoja Edad <= 53.5 Der) y añade divisiones focalizadas hasta alcanzar el límite fijado por **max_leaf_nodes**.

4. Comparación Topológica y Ventajas Operativas

La elección entre el crecimiento por orden de nivel y el crecimiento de mejor primero genera diferencias estructurales sustanciales en la topología final del árbol de decisión:

Dimensión de Análisis	Orden de Nivel (Level-Wise)	Mejor Primero (Best-First)
Hiperparámetro de Control	max_depth	max_leaf_nodes
Topología del Árbol	Simétrica y equilibrada (se expanden todos los nodos del nivel)	Asimétrica y sesgada (focalizada en regiones impuras)
Estrategia de Selección	Expansión uniforme por niveles horizontales	Selección codiciosa (greedy) por disminución de impureza
Eficiencia en Subgrupos	Puede gastar capacidad dividiendo nodos ya puros o poco informativos	Concentra los cortes en las subregiones con mayor densidad de error

En conclusión, mientras que el crecimiento por orden de nivel con **max_depth** produce estructuras simétricas fáciles de inspeccionar, el enfoque de mejor primero con **max_leaf_nodes** maximiza la ganancia de pureza por cada hoja añadida, optimizando el rendimiento predictivo en presencia de patrones complejos e impuros.

Equilibrio entre Varianza y Sesgo en Árboles de Decisión

Sección 1.2.3 — Análisis del Sobreajuste, Complejidad Topológica y Determinación de Hiperparámetros mediante Validación Cruzada

---

Índice del Documento

(Nota: El índice se actualizará automáticamente al abrir este archivo en Microsoft Word o Google Docs).

1. El Dilema entre Sesgo y Varianza en Modelos Basados en Árboles

La selección de la condición de parada adecuada en un árbol de decisión representa la decisión central para controlar el equilibrio entre **sesgo (bias)** y **varianza (variance)**. Los árboles de decisión altamente complejos no solo son más difíciles de interpretar para los especialistas, sino que también conllevan un coste computacional significativamente mayor durante el entrenamiento y la inferencia.

> **Pregunta Fundamental de Diseño:** ¿Cómo determinamos la condición de parada óptima para evitar tanto el subajuste (underfitting) como el sobreajuste (overfitting)?
>
> El objetivo fundamental consiste en restringir el crecimiento de la estructura jerárquica para capturar únicamente el patrón subyacente de la población sin memorizar el ruido estocástico del conjunto de entrenamiento.

1.1 Caracterización Formal del Sesgo y la Varianza

Los conceptos de sesgo y varianza definen dos regímenes de comportamiento opuestos en la modelización mediante árboles de decisión:

- **Alto Sesgo (Subajuste / Underfitting):** Se manifiesta en árboles de muy baja profundidad (p. ej., max_depth = 1 o 4). Estos modelos simplificados no logran capturar la frontera no lineal compleja que separa las clases en el espacio de características. En el caso de una frontera circular, un árbol poco profundo la aproxima mediante un cuadrado ortogonal basto, cometiendo un alto número de errores de clasificación estructurales.
- **Baja Varianza:** Los árboles de baja profundidad son extremadamente robustos frente a pequeñas perturbaciones o ruido en los datos de entrenamiento. Si se alteran ligeramente las posiciones de

los puntos límite, el hiperplano o región rectangular definida por el modelo permanece prácticamente inalterada.

- **Bajo Sesgo (Sobreajuste / Overfitting):** Se alcanza cuando se permite una profundidad arbitrariamente elevada (p. ej., max_depth = 100). El árbol crea micro-regiones rectangulares alrededor de cada observación, logrando clasificar correctamente la totalidad de las muestras de entrenamiento mediante límites en forma de 'zigzag'.
- **Alta Varianza:** Los árboles profundos son hiper-sensibles a las variaciones del conjunto de entrenamiento. La modificación de un solo punto en la frontera altera drásticamente la topología completa del árbol, generando múltiples islotes rectangulares fútiles (falsos positivos) que destruyen la capacidad de generalización en datos no observados.

1.2 Matriz Comparativa de Comportamiento según la Profundidad del Árbol

Profundidad (max_depth)	Nivel de Sesgo (Bias)	Nivel de Varianza	Geometría de la Frontera
max_depth = 4 (Baja profundidad)	**Alto Sesgo (Subajuste)**	**Baja Varianza (Modelo Estable)**	Regiones rectangulares amplias. Intenta encajar una región circular en un hueco cuadrado.
max_depth = 6 (Profundidad intermedia)	**Sesgo Moderado (Buen Ajuste)**	**Varianza Moderada (Equilibrio)**	Frontera escalonada regular. Captura la curvatura general con aislamiento progresivo de ruido.
max_depth = 100 (Alta profundidad)	**Bajo Sesgo (Memorización 100%)**	**Alta Varianza (Sobreajuste Severo)**	Límites fragmentados en 'zigzag'. Generación masiva de islotes rectangulares y falsos positivos.

La tabla demuestra el dilema fundamental: al aumentar la profundidad máxima, se reduce el sesgo pero se incrementa drásticamente la varianza del clasificador.

2. Análisis del Experimento Visual: Frontera Circular en el Espacio de Características

Para ilustrar el dilema entre sesgo y varianza, se analiza un conjunto de datos sintético constituido por una distribución circular central de puntos de datos verdes (terrenos agrícolas) sobre un fondo disperso (terrenos secos) en un plano bidimensional (Latitud vs. Longitud):

- **Caso max_depth = 4 (Analogía de la Clavija Redonda en Hueco Cuadrado):** Dado que los árboles de decisión únicamente generan cortes rectilíneos paralelos a los ejes ortogonales, una profundidad de 4 intenta encajar un área circular dentro de una estructura de cajas cuadradas gruesas. La

aproximación resulta deficiente debido a la imposibilidad de modelar la curvatura con tan pocos cortes.

- **Caso max_depth = 6 (Transición hacia el Ajuste Óptimo):** A medida que la profundidad aumenta a 6, los hiperplanos ortogonales se subdividen, afinando el contorno del círculo central. Sin embargo, comienzan a emerger pequeñas regiones aisladas en los márgenes exteriores donde el modelo predice erróneamente puntos verdes sin justificación estructural.
- **Caso max_depth = 100 (Degeneración por Sobreajuste):** Alcanzar una profundidad de 100 resulta en una frontera caótica y altamente desordenada. El clasificador genera islotes rectangulares microscópicos para encerrar cada punto de datos verde en la periferia, produciendo falsos positivos masivos. Este sobreajuste destruye el valor predictivo del modelo para datos futuros no observados.

> **Regla Direcciónal del Dilema Sesgo-Varianza:**
> • **Incremento de Profundidad (max_depth ↑):** Disminuye el Sesgo (Bias decreases), pero aumenta el riesgo de sobreajuste (can overfit).
> • **Reducción de Profundidad (max_depth ↓):** Disminuye la Varianza (Variance decreases), pero aumenta el riesgo de subajuste (can underfit).

3. Determinación de Hiperparámetros Mediante Validación Cruzada

Dado que no es posible conocer a priori la profundidad ni la complejidad óptima de un árbol para un conjunto de datos específico, la **validación cruzada (cross-validation)** constituye la metodología estándar e imprescindible para seleccionar la combinación óptima de hiperparámetros.

- **Evaluación Out-of-Sample:** La validación cruzada (p. ej., K-Fold Cross Validation) divide repetidamente los datos en pliegues de entrenamiento y validación. Esto permite estimar la tasa de error fuera de muestra (out-of-sample error) para diferentes profundidades máximas.
- **Curva de Error de Validación:** Al graficar el error en función de la profundidad, la curva de entrenamiento disminuye monótonamente hacia 0, mientras que la curva de validación muestra una forma de 'U'. El punto mínimo de la curva de validación identifica el valor óptimo de max_depth, donde el modelo equilibra perfectamente el sesgo y la varianza.

Optimización Conjunta de Hiperparámetros: Mediante técnicas como Grid Search o Random Search sobre la validación cruzada, se ajustan de manera simultánea múltiples parámetros de parada (max_depth, min_samples_leaf, max_leaf_nodes, min_impurity_decrease) para garantizar la máxima robustez predictiva.

ÁRBOLES DE DECISIÓN PARA REGRESIÓN Y TRATAMIENTO DE VARIABLES CATEGÓRICAS

Fundamentos Teóricos, Minimización del MSE, Codificación One-Hot e Implementación en Scikit-Learn

Tabla de Contenidos

1. Árboles de Decisión Aplicados a la Regresión (Resultados Continuos)

Los árboles de decisión, ampliamente conocidos por su aplicación en problemas de clasificación categórica, constituyen también un algoritmo potente para tareas de **regresión**, donde el objetivo principal es predecir un resultado cuantitativo o continuo ($Y \in \mathbb{R}$). Para extender la estructura jerárquica de árbol a un entorno continuo, es necesario responder formalmente a tres interrogantes analíticos:

> **Cuestiones Fundamentales de la Regresión con Árboles:**
>
> **1. Criterio de División:** ¿Qué métrica matemática permite seleccionar el predictor X_j y el umbral t_j óptimos para partir los datos continuos?
>
> **2. Función Objetivo:** ¿Cuál es la función de pérdida adecuada para ajustar el modelo a lo largo de las particiones?
>
> **3. Estimación en Hojas:** ¿Cómo se calcula el valor predicho $\hat{y}$ para cada instancia que cae dentro de un nodo terminal?

1.1. Predicción en las Hojas Mediante la Media Muestral

En un árbol de regresión, el espacio de características se particiona recursivamente en M regiones disjuntas $R_1, R_2, \dots, R_M$. A diferencia de la clasificación (donde se asigna la clase mayoritaria por votación), la predicción $\hat{y}_{R_m}$ para cualquier punto x contenido en la región R_m se define como la **media aritmética de la variable de respuesta y** correspondiente a las observaciones del conjunto de entrenamiento en esa hoja:

$$\hat{y}_{R_m} = \frac{1}{N_m} \sum_{i \in R_m} y_i$$

Donde N_m es el número total de puntos de entrenamiento que pertenecen a la región R_m. Por ejemplo, si una partición evalúa si $x \le 6.5$, el árbol predecirá la media de las observaciones con $x <$

6.5$ para la rama izquierda (ej. $\hat{y} = -0.008$) y la media de las observaciones con $x \ge 6.5$ para la rama derecha (ej. $\hat{y} = 0.697$).

1.2. Criterio de División: Minimización del Error Cuadrático Medio Ponderado (Weighted MSE)

Para elegir la división óptima en datos continuos, la función objetivo fundamental es el **Error Cuadrático Medio (MSE - Mean Squared Error)**. Sin embargo, una suma simple de MSEs ignora la densidad muestral de cada subdivisión. Por lo tanto, el algoritmo selecciona la variable p y el umbral t_p que minimizan el **MSE ponderado por el número de observaciones** presentes en cada subregión resultante (R_1 y R_2):

$$\text{MSE}_{ponderado} = \frac{N_1}{N} \text{MSE}(R_1) + \frac{N_2}{N} \text{MSE}(R_2)$$

donde $\text{MSE}(R_m) = \frac{1}{N_m} \sum_{i \in R_m} (y_i - \hat{y}_{R_m})^2$. A medida que el árbol profundiza e introduce más divisiones (por ejemplo, particionando en $x < 3.5$, $3.5 \le x < 6.5$, $6.5 \le x < 7.5$ y $x \ge 7.5$), la capacidad de ajuste a los datos aumenta progresivamente. No obstante, divisiones excesivas conducen a sobreajuste, por lo que la construcción se detiene mediante criterios de parada cuantitativos.

1.3. Condiciones de Parada y Ganancia de Reducción de MSE

En sustitución de las métricas de pureza (Gini o Entropía), los árboles de regresión evalúan la **reducción de MSE (Ganancia de Precisión)** al dividir una región R en R_1 y R_2:

$$\Delta \text{MSE} = \text{MSE}(R) - \left(\frac{N_1}{N_R} \text{MSE}(R_1) + \frac{N_2}{N_R} \text{MSE}(R_2) \right)$$

El algoritmo detiene el crecimiento del árbol cuando ΔMSE cae por debajo de un umbral predefinido (`min_impurity_decrease`), o bien cuando se alcanzan restricciones estructurales como `max_depth` o `min_samples_leaf`.

2. Tratamiento de Variables Categóricas en Árboles de Decisión

En problemas del mundo real, los conjuntos de datos combinan con frecuencia características numéricas continuas (ej. ancho del sépalo, presión arterial) y características cualitativas o categóricas (ej. color de la flor: amarillo, rojo, morado). Mientras que las comparaciones del tipo $X_j < \text{umbral}$ operan de forma directa en variables numéricas, no poseen un significado intuitivo o matemático inmediato al aplicarse a categorías desordenadas.

2.1. Inadecuación y Riesgos de la Codificación Ordinal Ingenua (Ordinal Encoding)

Una estrategia intuitiva pero problemática consiste en mapear arbitrariamente cada categoría a un número entero. Por ejemplo, asignar *Amarillo = 0, Rojo = 1, Morado = 2*. Bajo este esquema, una regla numérica $X_{\text{color}} < 1.5$ agrupa a **Amarillo y Rojo** en un lado, y a **Morado** en el otro.

El inconveniente fundamental de la codificación ordinal sobre variables nominales es que **introduce un orden implícito ficticio**. Si reordenamos las etiquetas como *Rojo = 0, Morado = 1, Amarillo = 2*, las particiones posibles permitidas por el umbral numérico cambian drásticamente. Por tanto, la optimización del árbol pasa a depender del mapeo arbitrario elegido, generando divisiones numéricas carentes de sentido semántico.

2.2. Solución Formal: Codificación One-Hot (Dummy Encoding)

La solución estándar para preservar la independencia cualitativa es la **Codificación One-Hot (Variables Indicadoras / Dummy)**. Se crea una columna binaria $C_k \in \{0, 1\}$ para cada categoría. Si una flor tiene color violeta, se establece $C_1 = 1$, $C_2 = 0$ y $C_3 = 0$.

> **Eficiencia y Eliminación de Redundancia en One-Hot Encoding:**
>
> **Propiedad de Cómputo:** Para una variable categórica con K niveles, incluir K columnas indicadoras es matemáticamente redundante. Dado que la suma de todas las columnas es igual a 1, conocer $K-1$ columnas determina exactamente el valor de la última.
>
> **Ejemplo Práctico:** Si una flor no es Violeta ($C_1 = 0$) ni Roja ($C_2 = 0$), se deduce inequívocamente que es Amarilla ($C_3 = 1$). Por consiguiente, la tercera columna (C_3) puede ser omitida en la matriz de diseño sin pérdida de información.

2.3. Implementación Técnica en Scikit-Learn (CART y Preprocesamiento)

Actualmente, la implementación oficial de árboles de decisión en **scikit-learn** se basa en una versión altamente optimizada del algoritmo CART (*Classification and Regression Trees*). Sin embargo, **scikit-learn no admite internamente variables categóricas directas (strings/objetos)**, por lo que se exige transformar obligatoriamente los datos cualitativos mediante `OneHotEncoder` antes del modelado.

A pesar de que la codificación manual añade columnas a la matriz de datos, en la práctica el impacto sobre el rendimiento predictivo del modelo suele ser insignificante, obteniéndose resultados perfectamente comparables a algoritmos nativos categóricos.

3. Tablas Comparativas Resumen

Tabla 1: Comparativa entre Árboles de Clasificación y Árboles de Regresión

Dimension de Análisis	Árboles de Clasificación	Árboles de Regresión
Variable Objetivo (Y)	Categórica / Discreta (Clases)	Continua / Cuantitativa ($Y \in \mathbb{R}$)
Estimación en la Hoja	Clase mayoritaria por votación	Media aritmética $\hat{y}_{R_m} = \frac{1}{N_m} \sum y_i$

Criterio de Selección de Corte	Gini, Entropía o Error de Clasificación	Error Cuadrático Medio Ponderado (Weighted MSE)
Métrica de Ganancia de Parada	Reducción de Impureza / Pureza de Nodo	Reducción de MSE ($\Delta \text{MSE} > \text{Umbral}$)

Tabla 2: Comparativa de Métodos de Codificación Categórica

Característica	Codificación Ordinal Ingenua	Codificación One-Hot (Dummy)
Mapeo de Datos	Asigna números enteros (0, 1, 2, ...)	Crea K (o $K-1$) columnas binarias {0, 1}
Supuesto de Orden	Asume un orden implícito $0 < 1 < 2$	Preserva la independencia cualitativa pura
Efecto en el Árbol	Divisiones sensibles al orden arbitrario asignado	Cortes ortogonales limpios por categoría
Soporte en Scikit-Learn	No recomendado para variables nominales	Requisito estándar antes de entrenar en Scikit-Learn

Criterios de División y Optimización mediante ECM en Árboles de Regresión

Sección 2.1.1: Evaluación de ajuste continuo, formulación analítica del Error Cuadrático Medio ponderado y particionado jerárquico

---

1. Extensión de los Árboles de Decisión a Variables de Respuesta Continuas

Los árboles de decisión adaptan su arquitectura jerárquica para resolver problemas de regresión donde la variable objetivo es continua. A diferencia de los modelos de clasificación categórica, que asignan etiquetas basadas en la clase mayoritaria de una región, un árbol de regresión estima el valor de salida calculando la **media aritmética muestral** de las observaciones pertenecientes a cada nodo hoja.

Para ilustrar este mecanismo, considérese un conjunto de datos donde la variable independiente continua *x* oscila entre 0 y 8, y la variable de respuesta *y* presenta variaciones no lineales entre -1.5 y 1.5. En la configuración más elemental con una única condición de división en el umbral **x > 6.5**, el espacio unidimensional se particiona en dos subregiones rectangulares:

Configuración del Corte Inicial (Umbral x > 6.5):

- **Rama No (x ≤ 6.5):** Se calcula el promedio de las observaciones en este intervalo, obteniendo la predicción constante **y_pred = -0.008**.
- **Rama Yes (x > 6.5):** Se promedian las observaciones del intervalo superior, generando la predicción constante **y_pred = 0.697**.

Al examinar la representación gráfica de este ajuste inicial, se observa que una única función escalonada con dos tramos no captura adecuadamente la estructura sinusoidal o con múltiples valles del conjunto de datos. Para mejorar la aproximación, es necesario cuantificar con rigurosidad matemática qué constituye una 'buena' división.

2. Evaluación de Calidad de Corte mediante Error Cuadrático Medio (ECM)

La calidad de una partición en un árbol de regresión se evalúa mediante la métrica del **Error Cuadrático Medio (ECM / MSE)**. Para una subregión específica R_r que contiene n observaciones, el ECM mide la dispersión de los valores observados y_i con respecto a la media regional $\bar{y}_{R_r}$:

$$MSE(R_r) = (1 / n) \cdot \Sigma_{i \in R_r} (y_i - \bar{y}_{R_r})^2$$

El cálculo del ECM bruto para cada subregión por separado es insuficiente, ya que no contempla la proporción de datos que cae en cada lado del corte. Una división que cree una región extremadamente pura con solo 2 puntos no debe ponderarse igual que una región con 80 puntos. Por esta razón, el algoritmo utiliza el **Error Cuadrático Medio Ponderado** entre las regiones resultantes *R_1* y *R_2*:

$$MSE(R_1, R_2) = (N_1 / N) \cdot MSE(R_1) + (N_2 / N) \cdot MSE(R_2)$$

Donde *N* representa el número total de observaciones en el nodo padre, mientras que *N_1* y *N_2* indican la cantidad de puntos que se desplazan hacia las subregiones izquierda y derecha, respectivamente.

La búsqueda de la división óptima se formaliza como un problema de minimización donde se selecciona el predictor *p* y el umbral numérico *t_p* que reducen al mínimo el ECM ponderado combinado:

$$\min_{p, t_p} [(N_1 / N) \cdot MSE(R_1) + (N_2 / N) \cdot MSE(R_2)]$$

3. Refinamiento mediante Particionado Recurso de 3 Umbrales

Al permitir que el árbol de regresión continúe subdividiendo las regiones iniciales mediante una mayor profundidad, se incorporan nuevos puntos de corte que aproximan la función objetivo con notable precisión. En el ejemplo avanzado con tres umbrales de división (**x = 3.5, x = 6.5, x = 7.5**), el espacio de características queda particionado en 4 regiones continuas:

Región Continuada	Intervalo de x	Condición en el Árbol	Predicción (Media $\bar{y}$)
Región 1 (R_1)	$x \leq 3.5$	$x \leq 6.5$ AND $x \leq 3.5$	$\bar{y} \approx 0.55$ (Nivel alto inicial)
Región 2 (R_2)	$3.5 < x \leq 6.5$	$x \leq 6.5$ AND $x > 3.5$	$\bar{y} \approx -0.65$ (Valle central)
Región 3 (R_3)	$6.5 < x \leq 7.5$	$x > 6.5$ AND $x \leq 7.5$	$\bar{y} \approx 0.50$ (Recuperación)
Región 4 (R_4)	$x > 7.5$	$x > 6.5$ AND $x > 7.5$	$\bar{y} \approx 1.00$ (Pico superior)

Como se aprecia en la tabla, el árbol de 3 cortes genera predicciones regionales significativamente más cercanas a las observaciones reales en cada tramo. La varianza residual interna de cada nodo hoja disminuye sustancialmente, logrando un **ECM ponderado global mucho más bajo** que el del modelo inicial de corte único.

4. Evaluación de la Capacidad de Ajuste vs. Sobreajuste

A medida que se añaden más cortes y el árbol gana profundidad, la capacidad de ajuste sobre el conjunto de entrenamiento aumenta progresivamente. Sin embargo, este proceso debe controlarse rigurosamente para evitar el sobreajuste (*overfitting*).

Principios Clave en la Construcción de Árboles de Regresión:

1. Criterio de Predicción en Hoja: En regresión, la predicción de cualquier nodo hoja siempre es la media aritmética de la variable de respuesta $\bar{y}$ de los datos de entrenamiento en esa región.

2. Criterio de Evaluación de Corte: Se evalúa la reducción del Error Cuadrático Medio ponderado [MSE(R_1, R_2)]. La división elegida es aquella que maximiza la disminución del MSE con respecto al nodo padre.

3. Necesidad de Condiciones de Parada: Al igual que en clasificación, los árboles de regresión requieren hiperparámetros de detención (profundidad máxima, número mínimo de muestras por hoja o ganancia mínima de MSE) para prevenir la memorización del ruido de los datos.

Condiciones de Parada y Ganancia de Información en Árboles de Regresión

Apuntes Técnicos | Sección 2.1.2: Criterios de Detención, Reducción de Varianza y Ganancia de ECM

1. Extensión de los Criterios de Parada a Árboles de Regresión

Los criterios de detención estructurales utilizados habitualmente en los árboles de clasificación son directamente aplicables a los modelos de regresión en scikit-learn. El objetivo primario de estas restricciones es prevenir la memorización del ruido de entrenamiento y acotar la complejidad del árbol, asegurando estimaciones estables en la variable respuesta continua.

> 📌 **EQUIVALENCIA DE HIPERPARÁMETROS:** Hiperparámetros de control estructural en Scikit-Learn: La profundidad máxima (max_depth), el número mínimo de muestras por hoja (min_samples_leaf) y el número máximo de nodos hoja (max_leaf_nodes) funcionan bajo la misma lógica geométrica tanto en problemas de clasificación como de regresión.

Hiperparámetro	Parámetro en Scikit-Learn	Función en Árboles de Regresión
Profundidad Máxima	**max_depth**	Limita el número máximo de niveles verticales de división, evitando la fragmentación excesiva del espacio continuo.
Muestras Mínimas por Hoja	**min_samples_leaf**	Exige un número mínimo de observaciones continuas en cada región terminal para garantizar promedios representativos.
Nodos Hoja Máximos	**max_leaf_nodes**	Restringe el número total de hojas en el modelo, forzando un crecimiento de tipo 'mejor primero' (best-first).

2. Ganancia de Información mediante Reducción de Varianza / ECM

De manera análoga a los árboles de clasificación que evalúan la calidad de un corte mediante la **ganancia de pureza** (restringiendo la impureza Gini o Entropía), los árboles de regresión utilizan la **ganancia de información** —entendida formalmente como la **reducción en la varianza**— para medir la adecuación de dividir una región determinada.

Esta métrica cuantifica exactamente la disminución del Error Cuadrático Medio (ECM) que se logra al realizar una partición. El proceso de división recurrente se detiene cuando la ganancia de información calculada cae por debajo de un umbral predefinido (ej. min_impurity_decrease).

3. Formulación Analítica de la Ganancia en Regresión

Ecuación Fundamental de Ganancia de Información (Reducción de ECM)

$$\text{Ganancia}(R) = \Delta(R) = ECM(R) - (N_1 / N) \cdot ECM(R_1) - (N_2 / N) \cdot ECM(R_2)$$

Desglose de Términos de la Ecuación:

- **Ganancia(R) o Δ(R):** Incremento en la precisión del modelo logrado al dividir la región R en dos subregiones R_1 y R_2.
- **ECM(R):** Error cuadrático medio de las observaciones contenidas en la región padre R respecto a su media aritmética.
- **N_1 / N y N_2 / N:** Pesos relativos (proporción de observaciones) de las subregiones hijas R_1 y R_2 respecto al total de observaciones N en R.
- **ECM(R_1) y ECM(R_2):** Errores cuadráticos medios individuales de las dos subregiones resultantes tras el corte.

4. Criterio de Parada y Control del Crecimiento

El algoritmo evalúa continuamente todas las posibles divisiones sobre todas las características continuas. Si para una región R dada, la máxima ganancia conseguible Δ(R) es inferior al umbral predefinido de tolerancia, la división se cancela inmediatamente y la región R se consolida como un nodo hoja final.

Árboles de Regresión: Predicción, Criterios de División y Condiciones de Parada

Guía Teórico-Práctica sobre la Inferencia Continua, el Error Cuadrático Medio (MSE) y la Reducción de Varianza

---

1. Mecanismo de Predicción e Inferencia (Sección 2.1.3)

Para realizar una predicción sobre un resultado continuo y cuantitativo mediante un árbol de regresión, se sigue un procedimiento sistemático de dos etapas basado en el recorrido de la estructura jerárquica y la estimación promediada en el nodo hoja final.

- **Paso 1: Recorrido del Árbol (Tree Traversal).** Se evalúa el punto de datos objetivo x_i comenzando desde el nodo raíz y descendiendo a través de las ramas según las condiciones lógicas de los atributos (ej. x > 6.5), hasta alcanzar una hoja terminal.
- **Paso 2: Estimación por Promedio en el Nodo Hoja (y_bar_t).** Una vez alcanzado el nodo hoja terminal R_t, el valor predicho para la variable respuesta continua y es exactamente el promedio aritmético de las observaciones de entrenamiento contenidas en dicho nodo:

> 📌 **FORMULACIÓN DE LA PREDICCIÓN EN HOJA:** La predicción para cualquier punto x en la región R_t viene dada por la media muestral de entrenamiento:
>
> y_pred = y_bar_t = (1 / N_t) * Σ_{i ∈ R_t} y_i
>
> Donde N_t es el número de observaciones de entrenamiento que cayeron dentro de la región del nodo hoja R_t.

2. Criterios de División y Error Cuadrático Medio (Sección 2.1.1)

A diferencia de los árboles de clasificación (que emplean métricas de pureza como el Índice Gini o la Entropía), los árboles de regresión miden la calidad de una división evaluando la dispersión del resultado continuo mediante el Error Cuadrático Medio (ECM o MSE).

El MSE de una región individual R_r se calcula como la media de las desviaciones al cuadrado respecto al promedio estimado:

$$MSE(R_r) = (1 / n) * \Sigma_{i \in R_r} (y_i - y_bar_{R_r})^2$$

Dado que el MSE bruto no considera la proporción de datos contenida en cada subregión, se define el **MSE Ponderado Combinado** para evaluar la partición completa de una región en dos subregiones R_1 y R_2:

$$MSE(R_1, R_2) = (N_1 / N) * MSE(R_1) + (N_2 / N) * MSE(R_2)$$

Para encontrar la partición óptima, el algoritmo escoge el predictor p y el umbral de corte t_p que minimizan de forma codiciosa (greedy) el MSE ponderado:

$$\min_{p, t_p} [(N_1 / N) * MSE(R_1) + (N_2 / N) * MSE(R_2)]$$

3. Comparativa de Modelos: Corte Único vs. Múltiples Divisiones

La capacidad de ajuste de un árbol de regresión aumenta con la adición de puntos de corte jerárquicos. En los ejemplos presentados en el material de origen, se contrapone una división ruda frente a un modelo refinado:

Modelo	Puntos de Corte	Número de Regiones	Desempeño / Predicciones
Corte Único Inicial	x = 6.5	2 Regiones	Lado Izq (x <= 6.5): y_pred = -0.008 Lado Der (x > 6.5): y_pred = 0.697 Alta imprecisión regional.
Árbol Ampliado (3 Cortes)	x = 3.5, 6.5, 7.5	4 Regiones	R1 (x <= 3.5): Media local alta (~0.55) R2 (3.5 < x <= 6.5): Media negativa (~ -0.68) R3 (6.5 < x <= 7.5): Media intermedia (~ 0.50) R4 (x > 7.5): Media superior (~ 1.00)
Efecto en la Función	Partición binaria simple	Función constante a trozos con 2 escalones	Corta variabilidad compleja y subajusta los datos.
Efecto en la Función	Partición jerárquica con 2 niveles de profundidad	Función constante a trozos con 4 escalones	Reducción sustancial del MSE; ajuste óptimo a la tendencia.

4. Condiciones de Parada y Ganancia de Información (Sección 2.1.2)

A medida que un árbol de regresión se profundiza, la capacidad de ajuste sobre el conjunto de entrenamiento aumenta, pudiendo llegar a memorizar cada observación individual si no se aplican restricciones.

Para evaluar si una división es justificable, se utiliza la **Ganancia de Información** (equivalente a la **Reducción de Varianza**), la cual mide la disminución neta del MSE tras la división:

$$Ganancia(R) = \Delta(R) = ECM(R) - (N_1 / N) * ECM(R_1) - (N_2 / N) * ECM(R_2)$$

📌 **REGLAS DE DETENCIÓN PARA PREVENIR EL SOBREAJUSTE:** Principales Hiperparámetros de Detención en Regresión (Scikit-Learn):

• max_depth: Limita la profundidad máxima del árbol para evitar ramificaciones excesivas.

• min_samples_leaf: Garantiza un número mínimo de muestras en cada hoja antes de permitir un corte.

• max_leaf_nodes: Restringe el número total de hojas finales utilizando crecimiento de mejor primero (best-first).

• min_impurity_decrease: Detiene el crecimiento cuando la ganancia Δ(R) cae por debajo de un umbral prefijado.

2.1.4 Manejo de Atributos Numéricos y Categóricos

Apuntes Académicos sobre Codificación Ordinal, One-Hot Encoding e Implementación en Scikit-Learn

1. El Desafío de los Datos Mixtos en Árboles de Decisión

Los algoritmos de árbol de decisión basan sus divisiones en el mecanismo de **comparación y ramificación (compare and branch)**. Mientras que para las variables numéricas continuas o discretas las condiciones del tipo **característica < umbral** están perfectamente definidas, el tratamiento de variables categóricas nominales (con más de dos valores sin orden inherente) plantea una limitación analítica directa.

> **Planteamiento del Problema:**
> Para un atributo categórico nominal como 'Color' con valores {'Yellow', 'Red', 'Purple'}, expresiones matemáticas del tipo 'Color < 1.5' carecen de significado semántico natural a menos que se defina un mapa explícito de transformación.

Tabla 1: Conjunto de datos de prueba con atributos mixtos (Numérico y Categórico)

Ancho del Sépalo (Sepal Width)	Color (Color)	Especie de Flor (Flower)
3.0 mm	Yellow (Amarillo)	Sunflower (Girasol)
3.5 mm	Red (Rojo)	Rose (Rosa)
4.5 mm	Purple (Púrpura)	Orchid (Orquídea)
3.7 mm	Purple (Púrpura)	Tulip (Tulipán)

2. Solución 1: Codificación Ordinal (Ordinal Encoding)

La estrategia más directa consiste en mapear cada categoría discreta a un número entero y tratar la variable como si fuera una magnitud escalar continua. Este procedimiento se conoce como **codificación ordinal**.

Ejemplo de mapeo ordinal: Asignando los valores Yellow = 0, Red = 1 y Purple = 2, el árbol de decisión evalúa primero el atributo numérico continuo y posteriormente el atributo codificado:

- **Nivel 1 (Nodo Raíz):** Ancho del Sépalo > 4.0 mm. Si es SÍ, predice directamente 'Orquídea' (Orchid).
- **Nivel 2 (Rama No):** Se evalúa el atributo Color codificado. ¿Color <= 1.5? (Equivale a separar Purple de {Yellow, Red}).

- **Nivel 3a (Color <= 1.5):** Si Color <= 0.5, predice 'Girasol' (Yellow=0). En caso contrario, predice 'Rosa' (Red=1).
- **Nivel 3b (Color > 1.5):** Predice 'Tulipán' (Purple=2).

3. Sensibilidad del Árbol a la Asignación Numérica Ordinal

Un aspecto crítico de la codificación ordinal sobre datos puramente nominales es que la asignación numérica impone un orden topológico artificial. Dado que los umbrales del árbol evalúan cortes en una escala unidireccional (por ejemplo, **Color < 1.5**), las particiones binarias accesibles en un solo paso varían según el orden elegido.

Tabla 2: Comparativa de particiones evaluables según el esquema de codificación ordinal

Esquema de Codificación	Mapeo Numérico	Particiones Binarias Posibles en 1 Corte
Codificación A	Yellow = 0, Red = 1, Purple = 2	{{Yellow}, {Red, Purple}} y {{Yellow, Red}, {Purple}}
Codificación B	Yellow = 2, Red = 0, Purple = 1	{{Red}, {Yellow, Purple}} y {{Red, Purple}, {Yellow}}

> **Conclusión Práctica sobre Codificación Ordinal Ingenua:**
> Aunque en teoría cambiar la codificación altera el espacio de cortes evaluados en cada nivel, en la práctica empírica el impacto de una codificación ordinal ingenua sobre variables categóricas suele ser insignificante, obteniendo modelos con un rendimiento global perfectamente comparable.

4. Solución 2: Codificación One-Hot (One-Hot Encoding / Dummy)

Para evitar la imposición de un orden artificial en atributos puramente nominales, se utiliza la **codificación One-Hot (OHE) o variables ficticias (dummy encoding)**. Este método expande la variable categórica de K niveles en K columnas binarias independientes (donde cada columna representa la presencia '1' o ausencia '0' de una categoría).

Tabla 3: Transformación One-Hot de la variable Color (C1=Purple, C2=Red, C3=Yellow)

Ancho Sépalo	C1 (Purple)	C2 (Red)	C3 (Yellow)	Especie (Flower)
3.0 mm	0	0	1	Sunflower
3.5 mm	0	1	0	Rose
4.5 mm	1	0	0	Orchid

3.7 mm	1	0	0	Tulip

Estructura del árbol resultante tras codificación One-Hot:

- **Corte 1:** Si Ancho del Sépalo > 4.0 mm -> Predice 'Orquídea' (Orchid).
- **Corte 2:** Si Ancho <= 4.0 mm, evalúa C1 (Purple) == 1? Si es SÍ -> Predice 'Tulipán' (Tulip).
- **Corte 3:** Si C1 == 0, evalúa C2 (Red) == 1? Si es SÍ -> Predice 'Rosa' (Rose). Si es NO -> Predice 'Girasol' (Sunflower, pues implica C3=1).

5. Soporte e Implementación en Scikit-Learn

A diferencia de otros marcos de trabajo (como R's randomForest, H2O o XGBoost) que manejan internamente variables categóricas de forma nativa, la implementación actual de árboles de decisión en **Scikit-Learn utiliza una versión optimizada del algoritmo CART que no admite variables categóricas directamente**. Por lo tanto, es obligatorio transformar previamente todas las columnas categóricas a representaciones numéricas (mediante OneHotEncoder u OrdinalEncoder) antes del ajuste del modelo.

> **Resumen de Recomendaciones Académicas:**
>
> 1. Para atributos ordinales con orden claro (ej. Bajo < Medio < Alto): Utilizar OrdinalEncoding.
> 2. Para atributos nominales sin orden (ej. Colores, Ciudades): Utilizar OneHotEncoding para evitar imponer relaciones falsas de distancia u ordenamiento.
> 3. En Scikit-Learn: La transformación debe realizarse obligatoriamente en la etapa de preprocesamiento de datos.

Poda de Árboles de Decisión y Poda por Complejidad de Costo

Alternativas a las Condiciones de Parada Preespecificadas y Regularización Mediante Cost-Complexity Pruning

1. Limitaciones de las Condiciones de Parada Preespecificadas

El control del sobreajuste (*overfitting*) en los árboles de decisión se ha abordado históricamente mediante la imposición de condiciones de detención predefinidas, tales como la profundidad máxima (**max_depth**) o el número mínimo de observaciones en un nodo hoja (**min_samples_leaf**). Sin embargo, detener el crecimiento del árbol de forma previa presenta dilemas estructurales severos:

- **Riesgo de Parada Prematura (Subajuste):** Si el criterio de parada se establece de forma demasiado estricta (por ejemplo, una ganancia mínima de impureza elevada), el árbol puede detenerse antes de capturar interacciones no lineales complejas en los datos, resultando en un modelo con alto sesgo.
- **Riesgo de Parada Tardía (Sobreajuste):** Si las restricciones son demasiado permisivas, el árbol crecerá hasta memorizar el ruido de la muestra de entrenamiento, creando regiones compuestas por puntos individuales y alcanzando un 100% de precisión en entrenamiento a costa de una alta varianza fuera de muestra.
- **Alto Costo Computacional de Grid Search:** A evaluar múltiples combinaciones de hiperparámetros mediante validación cruzada antes del entrenamiento exige explorar un espacio discreto y discontinuo de modelos, lo que resulta sumamente costoso en conjuntos de datos de gran escala.

2. Concepto y Fundamento de la Poda (Pruning)

La poda (*Pruning*) constituye una estrategia conceptualmente opuesta a las condiciones de detención: en lugar de restringir el crecimiento inicial del árbol, se permite que el árbol crezca por completo hasta alcanzar la máxima pureza posible (generando un árbol completo denotado como $\mathbf{T_o}$) y, posteriormente, se eliminan de manera selectiva aquellas ramas o subárboles redundantes que no aportan capacidad de generalización.

Ejemplo Ilustrativo: Selección de Candidatos a Posgrado

Supóngase un modelo para decidir la admisión de estudiantes basado en tres variables: GRE, GPA y Años de Experiencia Laboral. Un árbol no restringido puede crear una rama muy profunda que evalúe la experiencia laboral para estudiantes cuyo GPA ya es extremadamente bajo. Dicha división resulta espuria y arbitraria. Al podar esa hoja sobrante, el modelo reduce su complejidad y mejora su rendimiento en futuros solicitantes.

3. Poda por Complejidad de Costo (Cost-Complexity Pruning)

El método de **Poda por Complejidad de Costo** (o *Minimal Cost-Complexity Pruning*) formaliza matemáticamente la penalización a la estructura del árbol. Se introduce una función de costo penalizada que equilibra la tasa de error con el número total de hojas:

$$C_\alpha(T) = Error(T) + \alpha \cdot |T|$$

- **Error(T):** Mide la tasa de error de clasificación (o impureza/MSE) del árbol T en los datos de entrenamiento.
- **$|T|$:** Representa el número total de nodos hoja en el árbol T, sirviendo como métrica directa de complejidad estructural.
- **α (Parámetro de Complejidad):** Hiperparámetro de regularización no negativo ($\alpha \geq 0$) que controla la penalización asignada por cada hoja adicional. Si $\alpha = 0$, el árbol óptimo es el árbol completo T_o. A medida que α aumenta, se favorecen árboles con menor cantidad de hojas.

3.1 Demostración Numérica del Impacto de la Poda (α = 0.2)

Considerando un parámetro de complejidad fijado en $\alpha = 0.2$, la siguiente tabla demuestra cómo la eliminación de una hoja reduce la complejidad total a pesar de incrementar marginalmente el error de entrenamiento:

Estado del Árbol	Error(T)	Número de Hojas ($\|T\|$)	Penalización ($\alpha \cdot \|T\|$)	Costo Total $C_{o.2}(T)$
Árbol Completo (T)	0.32	8	$0.2 \cdot 8 = 1.60$	1.92
Árbol Podado (T')	0.33	7	$0.2 \cdot 7 = 1.40$	**1.73 (Óptimo)**

El resultado confirma que $C_{o.2}(T') = 1.73$ es menor que $C_{o.2}(T) = 1.92$. Aunque la poda incrementó el error de entrenamiento en +0.01, redujo el término de regularización en -0.20, produciendo un modelo más simple con menor varianza esperada en producción.

4. Algoritmo de Poda Secuencial y Validación Cruzada

Dado un árbol completo T_o, existe un número finito de ubicaciones posibles de poda. El algoritmo de Poda por Complejidad de Costo opera generando una secuencia decreciente de subárboles anidados:

$$T_o \supset T_1 \supset T_2 \supset \ldots \supset T_L$$

donde T_L representa el árbol trivial que consta únicamente del nodo raíz.

4.1 Criterio de Selección del Nodo a Podar en Cada Paso

Para determinar qué nodo interno debe convertirse en hoja en cada iteración k, se evalúa el cociente entre el incremento de error y la reducción en el número de hojas:

Ratio = [Error(T_podado) - Error(T)] / [|T| - |T_podado|]

El algoritmo selecciona en cada etapa aquel subárbol que minimiza este cociente, garantizando la eliminación de aquellas ramas que producen el menor incremento de error por cada hoja simplificada.

4.2 Selección de α Óptimo Mediante Validación Cruzada

Una regla fundamental en el proceso de poda es que la selección del hiperparámetro α óptimo **NO se realiza utilizando la métrica C_α(T) en el conjunto de entrenamiento**, sino evaluando la tasa de error directo en un conjunto de validación o mediante validación cruzada (*Cross-Validation*):

- **Paso 1:** Ajustar el árbol completo T_o en los datos de entrenamiento y calcular la secuencia de valores críticos de α.
- **Paso 2:** Para cada candidato α, obtener el subárbol T* correspondiente usando el algoritmo de poda.
- **Paso 3:** Evaluar la precisión de cada subárbol sobre los pliegues de validación cruzada fuera de muestra y seleccionar el valor de α que minimice el error de validación medio.

Árboles de Decisión y Poda de Modelos

2.2.1 Ejemplo Práctico: Evaluación de Solicitudes de Máster

> ⚠ **FOR ILLUSTRATION ONLY (Aviso Pedagógico)**
> *Tenga en cuenta que la siguiente representación es puramente ficticia y sirve únicamente para fines pedagógicos—no refleja con precisión el proceso real del comité de admisión.*

1. Introducción y Contexto del Escenario

Antes de profundizar en la técnica de poda (pruning), se presenta un escenario de ejemplo para comprender mejor su aplicación práctica en los algoritmos de clasificación basados en árboles de decisión.

En este caso particular, se dispone de un árbol de decisión para evaluar solicitudes de admisión a programas de maestría. Cada nodo del árbol representa una pregunta o criterio de evaluación utilizado para evaluar a los solicitantes, como:

- **GRE (Graduate Record Examinations):** Puntuación cuantitativa en el examen de admisión.
- **GPA / CGPA (Cumulative Grade Point Average):** Promedio de calificaciones académicas acumuladas del solicitante.
- **TOEFL (Test of English as a Foreign Language):** Evaluación del nivel de idioma inglés.
- **Experiencia Laboral (Work Experience):** Años de experiencia laboral previa del candidato.

La estructura jerárquica del árbol guía el proceso de toma de decisiones, conduciendo finalmente a una decisión de admisión binaria: **Aceptar (Accept)** o **Rechazar (Reject)**.

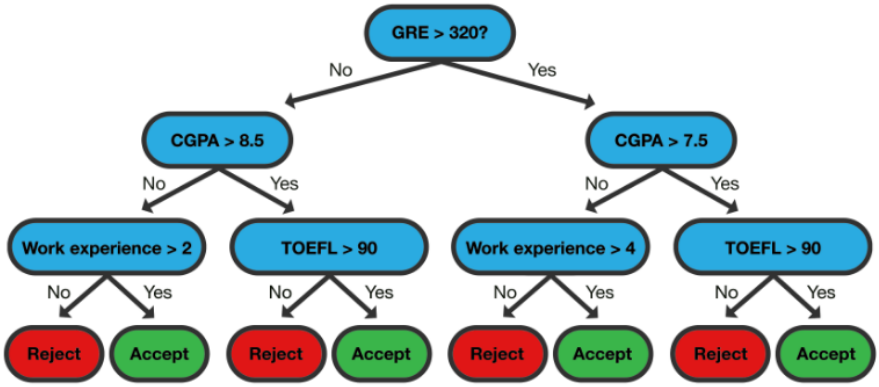
2. Estructura Jerárquica del Árbol de Decisión

A continuación se presenta el diagrama del árbol de decisión incluido en los materiales de estudio:

2.2.1 Ejemplo: Solicitudes de máster

Antes de profundizar en la técnica de poda, consideremos un escenario de ejemplo para comprender mejor su aplicación práctica.

> ⚠ **For Illustration Only**
> Tenga en cuenta que la siguiente representación es puramente ficticia y sirve únicamente para fines pedagógicos—no refleja con precisión el proceso real del comité de admisión.



En el ejemplo dado, tenemos un árbol de decisión para evaluar solicitudes de maestría. Cada nodo representa una pregunta o criterio utilizado para evaluar a los solicitantes, como el GPA, las cartas de recomendación y las puntuaciones del GRE. La estructura del árbol guía el proceso de toma de decisiones, conduciendo finalmente a una decisión de admisión.

Sin embargo, este árbol parece bastante complejo, lo que podría indicar sobreajuste. En lugar de detener prematuramente el crecimiento del árbol, podemos emplear técnicas de poda para obtener un árbol de decisión más simple, pero efectivo.

Figura 1: Árbol de decisión para la evaluación de solicitudes de maestría (Sección 2.2.1).

2.1 Tabla de Reglas y Caminos de Decisión

Navegando el árbol desde la raíz (**GRE > 320**) hasta cada hoja, se derivan las siguientes 8 reglas lógicas de clasificación:

Puntuación GRE	Calificación CGPA	Criterio Nivel 3	Condición Criterio	Resultado Final
<= 320 (No)	<= 8.5 (No)	Work experience	<= 2 (No)	**Rechazar (Reject)**
<= 320 (No)	<= 8.5 (No)	Work experience	> 2 (Yes)	**Aceptar (Accept)**
<= 320 (No)	> 8.5 (Yes)	TOEFL	<= 90 (No)	**Rechazar (Reject)**
<= 320 (No)	> 8.5 (Yes)	TOEFL	> 90 (Yes)	**Aceptar (Accept)**
> 320 (Yes)	<= 7.5 (No)	Work experience	<= 4 (No)	**Rechazar (Reject)**

> 320 (Yes)	<= 7.5 (No)	Work experience	> 4 (Yes)	**Aceptar (Accept)**
> 320 (Yes)	> 7.5 (Yes)	TOEFL	<= 90 (No)	**Rechazar (Reject)**
> 320 (Yes)	> 7.5 (Yes)	TOEFL	> 90 (Yes)	**Aceptar (Accept)**

3. Diagnóstico de Sobreajuste y Téchnicas de Poda

Sin embargo, al observar con detenimiento la estructura del árbol de decisión, se aprecia que es un modelo bastante complejo. En el contexto del aprendizaje automático, un árbol excesivamente ramificado suele indicar la presencia de **sobreajuste (overfitting)**.

> **Concepto Clave: Estrategia de Poda (Pruning)**
> En lugar de detener prematuramente el crecimiento del árbol durante su entrenamiento, la metodología recomendada consiste en permitir que el árbol se desarrolle completamente y, posteriormente, emplear técnicas de poda para simplificarlo y obtener un árbol de decisión más sencillo pero igualmente efectivo.

Resumen de Puntos Clave:

- **Evaluación Multicriterio:** Uso de condiciones secuenciales basadas en examen GRE, rendimiento académico (CGPA), nivel de inglés (TOEFL) y años de experiencia laboral.
- **Complejidad y Sobreajuste:** La presencia de múltiples ramificaciones paralelas idénticas o profundas suele indicar que el árbol está memorizando particularidades de los datos.

Técnica de Poda: Estrategia esencial post-entrenamiento para eliminar subárboles redundantes, reduciendo la varianza del modelo y mejorando su capacidad de generalización.

Técnicas de Poda en Árboles de Decisión

Algoritmos de Clasificación • Poda de Complejidad de Costo (Cost-Complexity Pruning)

---

1. Fundamentos de la Poda de Complejidad de Costo

La **poda (pruning)** es el proceso de reducir selectivamente el tamaño de un árbol de decisión mediante la eliminación de ramas o subárboles redundantes. Su objetivo principal es mitigar el sobreajuste (overfitting), mejorando la capacidad de generalización del modelo ante datos no vistos.

Entre los diferentes enfoques metodológicos, destaca el método de **poda de complejidad de costo (Cost-Complexity Pruning)**. Este enfoque asigna una puntuación de costo a cada árbol o subárbol basada en un compromiso equilibrado entre su precisión de clasificación y su grado de complejidad estructural.

Fórmula Fundamental del Coste de Complejidad C(T):

$$C(T) = Error(T) + \alpha \cdot |T|$$

Donde cada componente se define como:

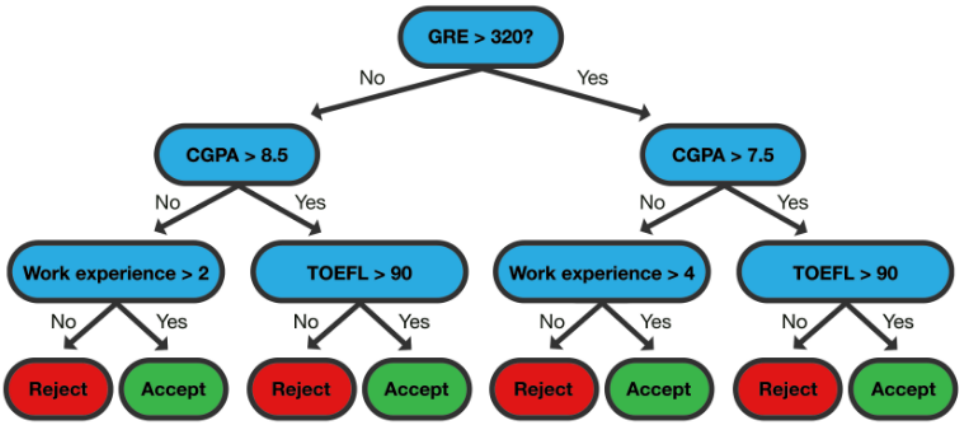
- **C(T):** Puntuación de complejidad de costo global del árbol de decisión T.
- **Error(T):** Tasa de error de clasificación del árbol T en el conjunto de entrenamiento.
- **α (Alfa):** Hiperparámetro de penalización por complejidad ($\alpha \geq 0$) que controla la compensación entre la precisión y la simplicidad estructural del árbol.
- **|T|:** Número total de nodos hoja (hojas finales) presentes en el árbol T.

Nota sobre el hiperparámetro α: Cuando $\alpha = 0$, el árbol no recibe penalización por su tamaño, lo que resulta en un árbol completo no podado T_o. A medida que α aumenta, la penalización por cada nodo hoja crece, favoreciendo árboles progresivamente más pequeños y simples.

2. Ejemplo Práctico de Evaluación y Poda (α = 0.2)

Para ilustrar el funcionamiento de este algoritmo, analicemos un escenario de clasificación sobre la admisión universitaria en un máster de Informática basado en métricas académicas (GRE, CGPA, Experiencia Laboral y TOEFL).

El proceso de poda se puede visualizar de la siguiente manera. Inicialmente, comenzamos con un árbol de decisión completamente desarrollado, representado como T_0. Se divide primero según el puntaje GRE, luego en CGPA en ambos lados. Por debajo de esa división, se examinan los años de experiencia laboral y los puntajes TOEFL. (Recuerda, este árbol es solo un ejemplo; no es así como el departamento de Informática realmente decide quién ingresa.)



Árbol	Error (T)	|T|	$Error(T) + \alpha \|T\|$
T_0	0.32	8	0.32 + 0.2 \cdot 8 = 19.2

Figura 1: Estructura del árbol inicial completamente desarrollado T_0 con 8 nodos hoja.

1. Evaluación del Árbol Inicial T_0:

El árbol inicial T_0 divide las solicitudes comenzando por el puntaje GRE (GRE > 320), seguido por CGPA en ambas ramas, y posteriormente evaluando los años de experiencia laboral y el puntaje TOEFL. Este árbol cuenta con $|T_0| = 8$ nodos hoja y presenta un error de entrenamiento de $Error(T_0) = 0.32$.

Aplicando $\alpha = 0.2$, su coste de complejidad es: **$C(T_0) = 0.32 + 0.2 \cdot 8 = 1.92$.**

Luego elegimos subárboles, los reemplazamos con nodos hoja y calculamos la complejidad del coste. A continuación, tenemos un ejemplo de ello.

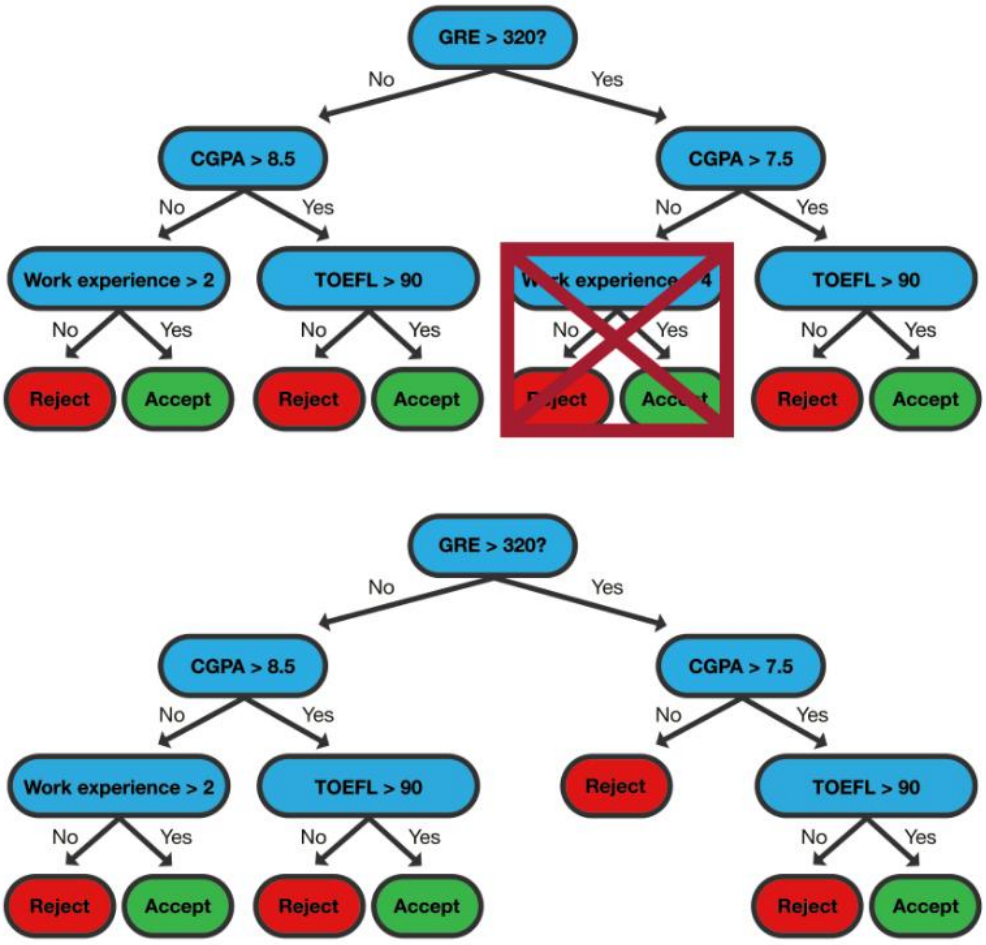


Figura 2: Proceso de sustitución del subárbol 'Work experience > 4' por una hoja de decisión ('Reject').

2. Sustitución de Subárbol por Nodo Hoja (Poda de Rama):
A continuación, seleccionamos un subárbol interno (en este caso, el nodo que evalúa 'Work experience > 4' bajo la rama de GRE > 320 y CGPA ≤ 7.5) y lo reemplazamos por un único nodo hoja con la clase predominante ('Reject').

Tabla 1: Comparativa de Métricas de Coste entre el Árbol Completo T_o y el Árbol Podado T_pequeño

Árbol de Decisión	Error(T)	|T| (Hojas)	Coste C(T) = Error(T) + α|T|
T_o (Árbol Completo)	0.32	8	**0.32 + 0.2 · 8 = 1.92**
T_pequeño (Podado)	0.33	7	**0.33 + 0.2 · 7 = 1.73**

Conclusión del Ejemplo: Al eliminar la rama, el error de entrenamiento aumenta de manera muy leve (de 0.32 a 0.33). Sin embargo, al reducir la cantidad de hojas de 8 a 7, la puntuación total de

complejidad de coste disminuye significativamente de **1.92 a 1.73**. Por lo tanto, bajo el hiperparámetro $\alpha = 0.2$, el árbol podado T_pequeño se considera superior a T_o por ser estructuralmente más simple sin perder precisión significativa.

3. Algoritmo General de Poda de Complejidad de Costo

El proceso sistemático para podar un árbol de decisión y seleccionar el modelo óptimo consta de cuatro pasos fundamentales:

Paso 1: Construcción del Árbol Completo (T_o)

Se hace crecer un árbol de decisión completamente desarrollado T_o sobre los datos de entrenamiento hasta que cada nodo hoja sea puro o alcance el límite de parada estándar.

Paso 2: Cálculo de la Tasa de Poda y Selección de Subárbol

Se evalúan todos los subárboles internos de T_o. Se selecciona para poda aquel subárbol que minimice el incremento del error por cada nodo hoja eliminado, expresado numéricamente mediante la relación:

- Tasa de Cambio = [Error(T_podado) - Error(T_actual)] / [α (|T_actual| - |T_podado|)]

Al colapsar dicho subárbol en un nodo hoja, se obtiene un nuevo árbol podado T_1.

Paso 3: Generación Ascendente de la Secuencia de Árboles (T_o, T_1, ..., T_L)

Se repite iterativamente el proceso de poda de forma ascendente (bottom-up), generando una secuencia finita de árboles podados T_o, T_1, T_2, ..., T_L, donde el último árbol T_L consiste únicamente en el nodo raíz inicial.

Paso 4: Selección del Árbol Óptimo T_i mediante Validación Cruzada

Dado que el proceso de poda optimiza implícitamente la puntuación C(T) en cada etapa, el árbol final óptimo T_i y el valor adecuado del hiperparámetro α se determinan evaluando el rendimiento de cada árbol de la secuencia en un conjunto de validación independiente (o mediante validación cruzada k-fold).

4. Resumen de Claves y Consideraciones Prácticas

Concepto Clave	Descripción / Regla Técnica
Objetivo Principal	Eliminar ramas ruidosas y evitar sobreajuste, mejorando la generalización sobre datos de prueba.
Hiperparámetro α	Determina el nivel de severidad de la poda. $\alpha = 0$ genera el árbol completo T_o; valores elevados de α producen árboles muy reducidos.
Evaluación Óptima	Utilizar siempre un conjunto de validación o validación cruzada (Cross-Validation) para seleccionar el α óptimo sin sesgo de entrenamiento.

En la práctica (por ejemplo, en librerías como **Scikit-Learn** a través del parámetro **ccp_alpha**), la ruta de poda de complejidad de costo extrae automáticamente los valores de α críticos y la secuencia correspondiente de subárboles para facilitar la selección óptima del modelo.

Evaluación y Optimización de Opciones de Poda en Árboles de Decisión

Apuntes Técnicos: Cost-Complexity Pruning, Derivación Matemática y Selección de Subárboles Óptimos

1. Estructura del Árbol Completo y Opciones de Poda

Un árbol de decisión crecido completamente sin restricciones se denomina **árbol completo T_0**. Este árbol maximiza el ajuste en el conjunto de entrenamiento, pero suele sufrir de sobreajuste (overfitting), reduciendo su capacidad de generalización ante nuevos datos.

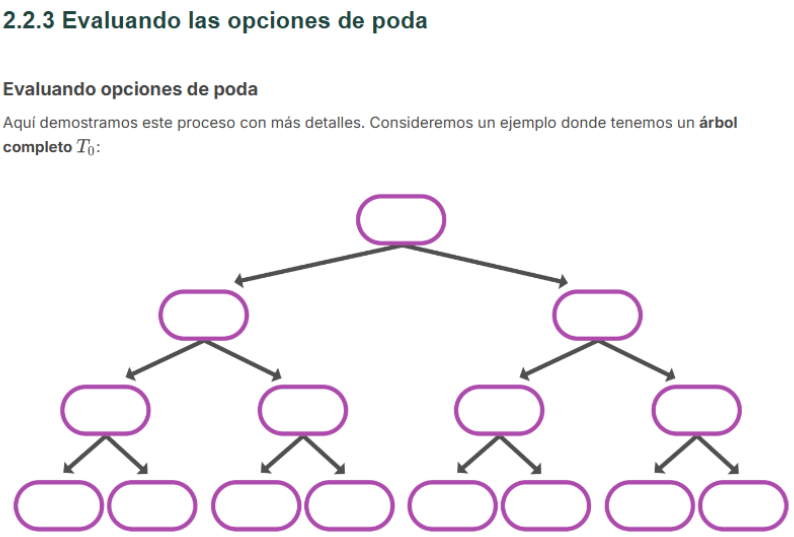


Figura 1: Representación de un árbol completo T_0 no podado.

Para reducir la complejidad del modelo, existen múltiples opciones de poda. La poda no se limita a eliminar la última capa de nodos hoja; es posible recortar ramas internas o subárboles enteros, dando lugar a una familia de posibles árboles podados denominados **T***.

Para podar este árbol, tenemos varias opciones posibles. Podemos podar ramas o subárboles individuales, resultando en diferentes árboles podados denominados como T^*. Tenga en cuenta que, en general, no tenemos que podar solo la última capa.

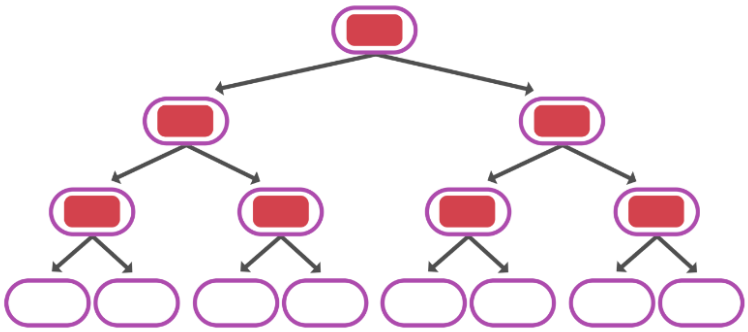
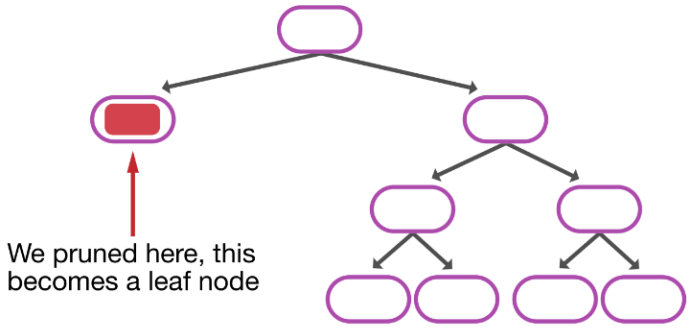


Figura 2: Identificación de nodos y subárboles candidatos para la poda (en rojo).

Cuando un subárbol interno es podado en un nodo específico, dicho nodo se convierte formalmente en un **nodo hoja (leaf node)**, simplificando la estructura global del árbol.

Para cada posible poda, obtenemos un conjunto de árboles podados.



Ahora, surge la pregunta: ¿Cómo determinamos el mejor árbol podado?

El proceso de selección implica maximizar la diferencia en las puntuaciones de complejidad de costo entre el árbol completo y cada árbol podado. Queremos encontrar el árbol podado que maximice $C(T) - C(T^*)$, utilizando la misma definición de C(T) que usamos en la última página:

$$C(T) = Error(T) + \alpha|T|$$

Figura 3: Resultado del corte en un nodo específico, transformándolo en nodo hoja.

2. Función de Complejidad de Costo C(T)

Para determinar cuál es el mejor árbol podado entre todas las opciones posibles, se evalúa una métrica que equilibra la precisión y la simplicidad estructural del árbol: la **puntuación de complejidad de costo C(T)**.

$$C(T) = Error(T) + \alpha\,|T|$$

Desglose de Componentes de la Función de Costo:

Variable / Término	Significado y Función en la Poda
C(T)	Puntuación de complejidad de costo total del árbol T.
Error(T) o E(T)	Medida del error de clasificación del árbol (por ejemplo, tasa de clasificación errónea o impureza de Gini/Entropía).
|T|	Tamaño del árbol, medido como el número total de nodos terminales (hojas).
α (Alfa)	Hiperparámetro de penalización por complejidad (α ≥ 0). Controla el compromiso entre el ajuste al entrenamiento y el tamaño del árbol.

3. Derivación Matemática del Criterio de Selección

El objetivo del proceso de selección es maximizar la diferencia en las puntuaciones de complejidad de costo entre el árbol no podado T (o árbol actual) y cada subárbol candidato podado T*:

Pasos de la Derivación Matemática:

Paso de Derivación	Expresión Matemática	Explicación Técnica
Paso 1: Planteamiento de la Diferencia	$\text{argmax}\ [\ C(T) - C(T^*)\ ] = \text{argmax}\ [\ (E(T) - E(T^*)) + \alpha\|T\| - \alpha\|T^*|\ ]$	Se expande la definición de C(T) y C(T*) sustituyendo la fórmula del costo.
Paso 2: Factorización y División	$= \text{argmax}\ [\ (E(T) - E(T^*)) / (\alpha\|T^*| - \alpha\|T|) - 1\ ]$	Se normaliza la expresión dividiendo por el cambio de tamaño ponderado por α.
Paso 3: Eliminación de Constante	$= \text{argmax}\ [\ (E(T) - E(T^*)) / (\alpha\|T^*| - \alpha\|T|)\ ]$	La constante -1 no altera la ubicación del máximo argumentos (argmax).
Paso 4: Ajuste de Signo en Denominador	$= \text{argmax}\ [\ (E(T) - E(T^*)) / (\alpha\|T| - \alpha\|T^*|)\ ]$	Se reordena el denominador como α(|T| - |T*|) para reflejar la reducción de hojas.
Paso 5: Conversión a Minimización	$= \text{argmin}\ [\ (E(T^*) - E(T)) / (\alpha(\|T| - \|T^*|))\ ]$	Al invertir el numerador (E(T*) - E(T)), la maximización se convierte equivalente a

una minimización.

▶ Explica las variables

En otras palabras, nuestro objetivo es maximizar $C(T) - C(T^\star)$ al comparar la medida de error $E(T)$ y $E(T^\star)$, así como los tamaños del árbol $|T|$ y $|T^\star|$. Nuestro objetivo es maximizar $C(T) - C(T^\star)$ tal que:

$$\begin{aligned} argmax\,[C(T) - C(T^\star)] &= argmax\,[E(T) - E(T^\star) + \alpha|T| - \alpha|T^\star|] \\ &= argmax\,[\frac{E(T) - E(T^\star)}{\alpha|T^\star| - \alpha|T|} - 1] \\ &= argmax\,[\frac{E(T) - E(T^\star)}{\alpha|T^\star| - \alpha|T|}] \\ &= argmax\,[\frac{E(T) - E(T^\star)}{\alpha|T| - \alpha|T^\star|}] \\ &= argmin\,[\frac{E(T^\star) - E(T)}{\alpha|T| - \alpha|T^\star|}] \end{aligned}$$

Este procedimiento iterativo de seleccionar el árbol podado óptimo de un árbol podado dado se repite hasta que obtenemos el árbol podado final $T^{(L)}$. Por lo tanto, podemos en su lugar, en cada paso, minimizar:

$$\frac{E(T^\star) - E(T)}{\alpha(|T| - |T^\star|)}$$

Este proceso iterativo de poda considera múltiples niveles de poda, dando lugar a una secuencia de árboles podados $T^{(1)}, T^{(2)}, \ldots, T^{(L)}$.

Finalmente, seleccionamos el árbol óptimo $T^{(i)}$ utilizando validación. Este árbol representa la versión podada que logra el mejor rendimiento en el conjunto de validación.

Figura 4: Derivación completa del criterio de minimización paso a paso.

4. Procedimiento Iterativo y Selección por Validación

En lugar de evaluar todos los subárboles posibles de forma exhaustiva (lo cual es computacionalmente inviable), el algoritmo aplica un procedimiento iterativo basado en el criterio minimizador:

Criterio a Minimizar en Cada Paso:
[E(T*) - E(T)] / [α (|T| - |T*|)]

Resumen del Algoritmo de Poda:

- **1. Secuencia de Árboles Podados:** Este procedimiento iterativo se repite sucesivamente, eliminando los nodos que ofrecen el menor incremento relativo de error por hoja eliminada, produciendo una secuencia finita de árboles podados $T^{(1)}$, $T^{(2)}$, ..., $T^{(L)}$.
- **2. Múltiples Niveles de Poda:** La secuencia abarca desde el árbol completo no podado $T^{(1)} = T_0$ hasta el árbol raíz de un solo nodo $T^{(L)}$.

- **3. Selección del Modelo Óptimo por Validación:** Finalmente, se evalúa cada subárbol candidato $T^{(i)}$ en un conjunto de validación independiente (o mediante Validación Cruzada). Se selecciona el árbol $T^{(i)}$ que minimiza el error de clasificación sobre los datos de validación.

2.2.4 Selección del Hiperparámetro Óptimo α en la Poda de Árboles

Optimización del Parámetro de Complejidad de Costo mediante Validación Cruzada

---

1. El Desafío de Seleccionar el Hiperparámetro α

En la poda de complejidad de costo (Cost-Complexity Pruning), el hiperparámetro **α (alpha)** regula el equilibrio exacto entre la tasa de error del árbol y su complejidad estructural (medida por el número de hojas terminales |T|). Determinar el valor preciso de α es crucial: un valor demasiado bajo mantendrá un árbol excesivamente ramificado que sobreajusta los datos de entrenamiento, mientras que un valor demasiado alto podará el árbol en exceso, reduciendo su capacidad predictiva (subajuste).

2. Procedimiento Sistemático en 3 Pasos

Para identificar el mejor valor del hiperparámetro α de forma objetiva y fundamentada, se sigue un algoritmo estructurado en tres etapas consecutivas:

Paso 1	**Fijar un valor específico para α** Se establece un valor concreto de penalización α dentro del espacio de búsqueda candidato.
Paso 2	**Evaluar el mejor subárbol T* en validación** Para el α seleccionado, se encuentra el subárbol óptimo T* evaluando la función de complejidad de costo C(T) = Error(T) + α|T| en el pliegue de validación correspondientes.
Paso 3	**Aplicar Validación Cruzada (Cross-Validation)** Se evalúa la medida de error promedio a través de todos los pliegues (folds) de validación para comparar distintos valores de α y seleccionar aquel que minimiza el error generalizado.

3. Impacto de la Optimización y Beneficios del Modelo Podado

Este proceso iterativo de optimización no solo determina matemáticamente el hiperparámetro α que produce el árbol podado más adecuado, sino que transforma la estructura del modelo aportando tres ventajas estratégicas fundamentales:

- **Simplificación Efectiva del Árbol:** Elimina ramas redundantes o con escaso aporte predictivo, reduciendo la profundidad total del gráfico de decisión.
- **Mitigación del Sobreajuste (Overfitting):** Al evitar que el modelo memorice el ruido del conjunto de entrenamiento, la poda garantiza un comportamiento estable ante datos no observados.
- **Mejora de la Interpretabilidad y Generalización:** Genera modelos finales más legibles para los especialistas del dominio y con una superior capacidad predictiva generalizada.

4. Resumen Comparativo de los Extremos de α

Valor de α	Efecto en la Estructura del Árbol	Riesgo en el Desempeño Predictivo
α = 0	Sin poda. Se conserva el árbol completo T_0 sin ninguna penalización.	Sobreajuste (Overfitting) alto; baja generalización.
α intermedio (Óptimo)	Subárbol T* podado con equilibrio óptimo entre complejidad y precisión.	Mínimo error en datos de validación/prueba.
α excesivamente alto	Poda extrema. El árbol se reduce a la raíz o a muy pocos nodos.	Subajuste (Underfitting) elevado; pérdida de patrones.

Apuntes Teóricos: La Geometría de los Datos y Algoritmos Voraces en Árboles de Decisión

Análisis de límites de decisión lineales y complejos, arquitectura voraz recursiva y criterios de pureza de nodos

Índice del Documento

(Nota: La tabla de contenidos se actualizará automáticamente al abrir el documento en Microsoft Word o Google Docs).

1. La Geometría de los Datos y Límites de Decisión

La capacidad de un modelo de aprendizaje automático para clasificar correctamente los datos depende de la compatibilidad entre la arquitectura del modelo y la geometría subyacente del espacio de características. En escenarios donde las clases son linealmente separables, modelos paramétricos básicos como la **Regresión Logística** pueden trazar una única hiperplano o línea recta que divide de forma aceptable las distintas categorías.

1.1 Fronteras Lineales Simples vs. Regresión Logística

En la figura a continuación se muestra la geometría de datos lineales simples, donde la distribución geográfica (Latitud vs. Longitud) entre tierra seca (*Dry Land*) y tierra agrícola (*Agricultural Land*) puede explicarse mediante un límite de decisión recto único:

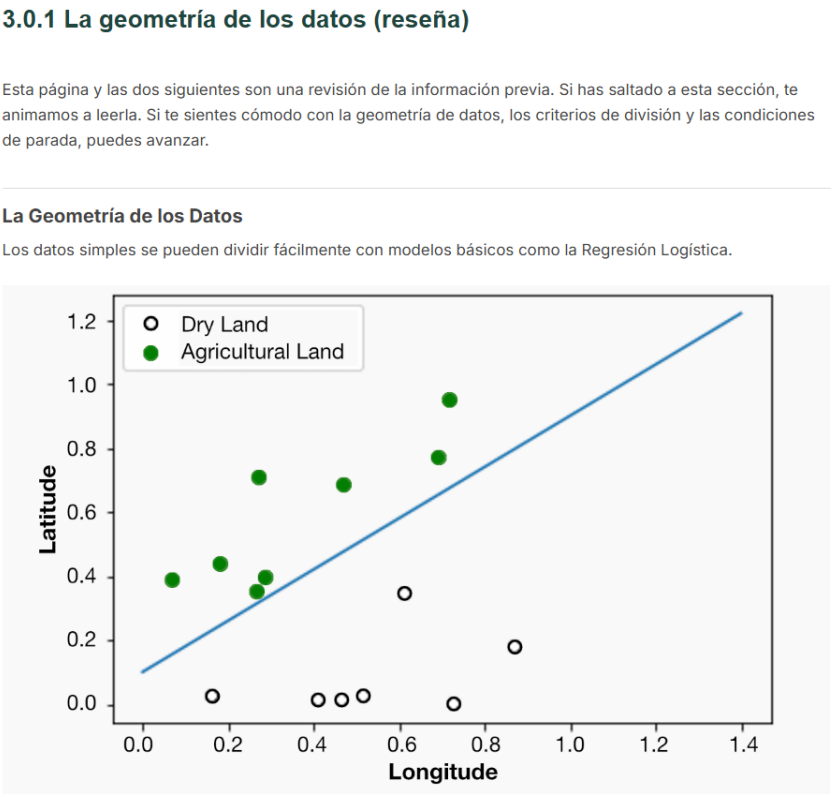


Figura 1: Límite de decisión lineal simple mediante Regresión Logística entre Dry Land y Agricultural Land.

1.2 Complejidad No Lineal: Disposición XOR y Patrones Concéntricos

Sin embargo, los límites de decisión en problemas reales suelen presentar complejidades geométricas no lineales que la regresión logística o las fronteras lineales son incapaces de resolver. Los dos patrones más representativos incluyen:

- **Patrón XOR (Bloques Cruzados):** Los puntos de datos secos y agrícolas están dispuestos de forma alternada en cuadrantes opuestos. Ninguna línea recta única puede dividir ambas clases para formar un límite de decisión aceptable.
- **Estructura Concéntrica (Agrupamiento Central):** Existe una alta concentración de tierra agrícola en el centro del gráfico de Latitud vs. Longitud, mientras que la tierra seca se extiende en la periferia a su alrededor. Se requieren fronteras cerradas o por tramos.

Sin embargo, los límites de decisión complicados no pueden explicarse con modelos tan simples. Un ejemplo de esto se puede ver en los siguientes 2 gráficos.

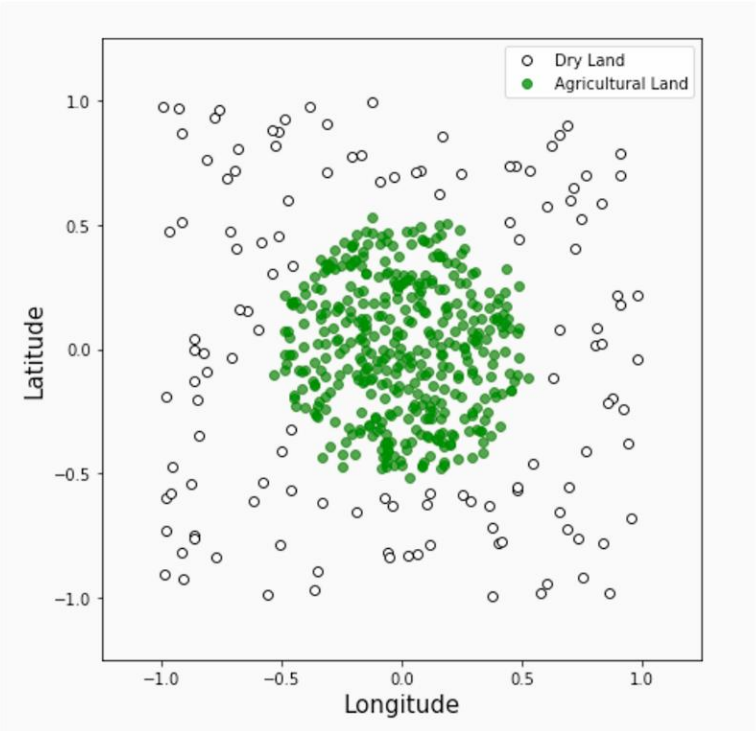


Figura 2: Frontera concéntrica no lineal con concentración central de tierra agrícola.

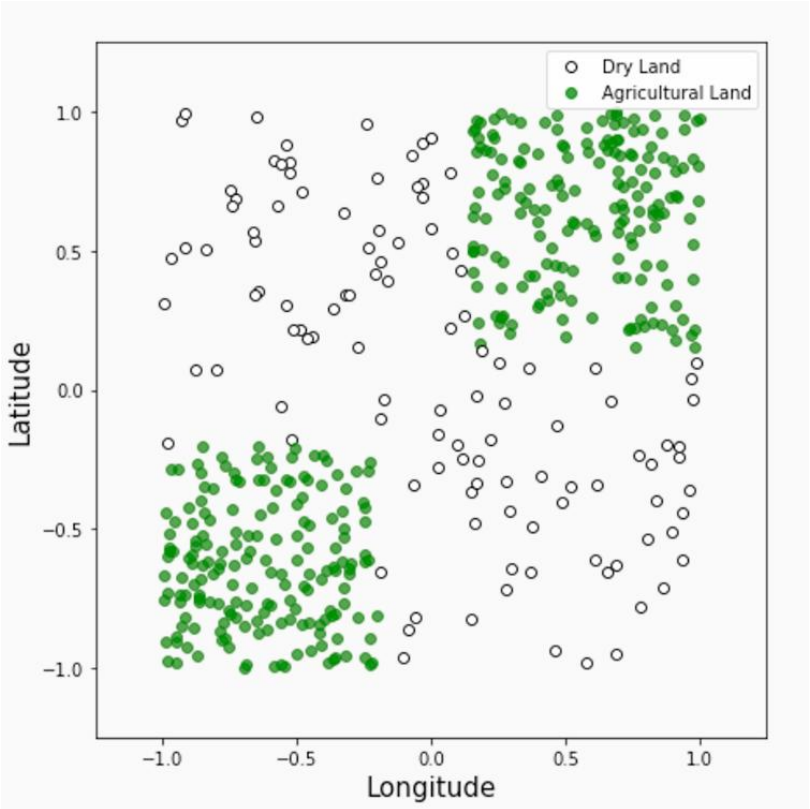


Figura 3: Disposición tipo XOR donde ninguna frontera lineal única puede separar las clases.

2. Algoritmo Voraz Recursivo para la Construcción de Árboles

Para superar las limitaciones de los clasificadores lineales en geometrías complejas, se emplean los **Árboles de Decisión**. Estos modelos son capaces de aprender funciones complejas fragmentando el espacio de entrada mediante divisiones ortogonales sucesivas (paralelas a los ejes del espacio de características).

2.1 Secuencia Paso a Paso del Enfoque Voraz (Greedy Approach)

Tanto para modelos de regresión como de clasificación, la construcción del árbol sigue un enfoque voraz (top-down, greedy) que busca la mejor división local en cada etapa:

1. **Paso 1: Nodo Raíz Global:** Comienza con un nodo único que contenga la totalidad del conjunto de datos original de entrenamiento.
2. **Paso 2: Evaluación de Condición de Detención:** Verifica si se cumple alguna condición de parada previa (por ejemplo, alcanzar la profundidad máxima, un número mínimo de muestras por nodo o una pureza total). Si se cumple, el nodo se convierte en una hoja final.
3. **Paso 3: Selección de Predictor y Umbral Óptimo:** Si la condición de detención no se cumple, evalúa todas las variables predictoras (ej. Latitud, Longitud) y selecciona el predictor óptimo junto con el umbral de corte (threshold) que maximice la separación o pureza de los subconjuntos.
4. **Paso 4: División y Recursión en Nodos Hijos:** Divide los datos del nodo en dos subconjuntos secundarios (izquierdo y derecho) y repite recursivamente el Paso 2 para cada nuevo nodo hijo formado.

Fase del Algoritmo	Acción Principal	Criterio de Control / Resultado
1. Inicialización	Creación del nodo raíz	Asignación del 100% de la muestra de datos al nodo superior.
2. Verificación de Parada	Comprobar condiciones de parada	Detener la división si se alcanza nodo puro, max_depth o min_samples_split.
3. Optimización Voraz	Búsqueda de predictor (X_j) y umbral (t)	Evaluación de todos los cortes posibles para maximizar la ganancia de pureza.
4. División Recursiva	Partición binaria en nodos hijos	Generación de ramas izquierda y derecha y llamada recursiva al algoritmo.

3. Criterios de División: Clasificación vs. Regresión

Aunque la estructura recursiva del algoritmo voraz se mantiene idéntica para problemas de clasificación y regresión, la diferencia fundamental radica en la **función de costo o condición de división** utilizada para seleccionar el punto de corte óptimo.

3.1 Árboles de Clasificación: Medidas de Pureza (Entropía e Índice Gini)

En modelos de clasificación, la pureza de las regiones resultantes es el indicador clave de rendimiento del modelo. Una región es pura cuando la totalidad de sus observaciones pertenecen a una sola clase. Las dos métricas estándar son:

- **Entropía (Entropy / Entropía Cruzada):** Mide la incertidumbre o el desorden del nodo. La división busca minimizar la entropía cruzada de los nodos hijos mediante un enfoque voraz, favoreciendo agrupaciones donde la probabilidad de una clase sea cercana a 0 o 1.
- **Índice de Gini (Gini Impurity):** Mide la probabilidad de clasificar incorrectamente un elemento elegido al azar si se etiquetara según la distribución de clases del nodo. Un valor de 0 indica pureza absoluta.

3.2 Árboles de Regresión: Reducción de Varianza

Cuando la variable objetivo es continua (regresión), la pureza basada en clases no es aplicable. En su lugar, el algoritmo busca la división que minimice la varianza intra-nodo o el Error Cuadrático Medio (MSE) respecto al valor promedio predicho en cada región final.

Tipo de Árbol	Métrica de División	Fórmula / Enfoque Objetivo	Propósito Principal
Clasificación	Entropía Cruzada	Minimizar - Σ p_i log2(p_i)	Medir incertidumbre y desorden en el nodo
Clasificación	Índice de Gini	Minimizar 1 - Σ (p_i)^2	Evaluar impureza y probabilidad de error
Regresión	Reducción de Varianza (MSE)	Minimizar Σ (y_i - ŷ_node)^2	Obtener subconjuntos con menor dispersión continua

En conclusión, los árboles de decisión ofrecen una respuesta flexible a la geometría no lineal de los datos al descomponer el espacio multidimensional en regiones ortogonales simples. Comprender la relación entre la distribución de las características y el criterio de pureza permite configurar modelos robustos capaces de evitar tanto el subajuste en límites complejos como el sobreajuste mediante condiciones de parada o poda adecuada.

Criterios de División en Árboles de Decisión

Revisión Técnica: Clasificación vs. Regresión, Métricas de Impureza y Error Cuadrático Medio

1. Fundamentos de Asignación y Etiquetado en Regiones

En la construcción de árboles de decisión, la estrategia empleada para dividir el espacio de características depende fundamentalmente de la naturaleza del problema de aprendizaje supervisado: **Clasificación** o **Regresión**. Cada nodo hoja del árbol define una región del espacio y requiere un criterio preciso para asignar la predicción final a las observaciones que caen dentro de ella.

Modelos de Clasificación: Criterio de Pluralidad

Para problemas de clasificación, cada región en el modelo se etiqueta según la clase de puntos más numerosa. En términos técnicos, se utiliza el principio de **pluralidad** de puntos dentro de la región para elegir la etiqueta definitiva del nodo.

Modelos de Regresión: Criterio de la Media

Para problemas de regresión, se predice el valor de salida utilizando la **media (promedio)** de los valores continuos de las observaciones de entrenamiento contenidas en dicha región.

2. Criterios de División para Clasificación: Entropía e Índice de Gini

En clasificación, las divisiones buscan maximizar la pureza de los nodos resultantes. Para cuantificar el nivel de impureza o desorden en una región se utilizan principalmente dos métricas de evaluación:

2.1 Entropía H(S)

La entropía mide el grado de incertidumbre o desorden en un nodo determinado. La formulación matemática de la entropía **H(S)** se expresa de la siguiente manera:

$$H(S) = (1 / n) * \Sigma_{i=1}^{C} -p_i * \log(p_i)$$

Donde ***p_i*** es la proporción de observaciones pertenecientes a la clase *i* en el nodo en cuestión, y ***C*** representa el número total de clases únicas presentes.

2.2 Índice de Gini

De forma similar, el índice de Gini evalúa la probabilidad de clasificar erróneamente un elemento elegido al azar si se etiquetara aleatoriamente según la distribución de clases en el nodo. Se expresa formalmente como:

$$Gini(S) = (1 / n) * \sum_{i=1}^{C} p_i * (1 - p_i)$$

3. Criterio de División para Regresión: Error Cuadrático Medio (MSE)

En los modelos de árboles de regresión, la meta es minimizar la dispersión de las respuestas continuas dentro de cada región. El **Error Cuadrático Medio (MSE)** es la opción preferida e estándar para guiar la selección de divisiones en la variable predictora.

$$MSE = (1 / n) * \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

- **n:** Número total de observaciones en el nodo o región evaluada.
- **y_i:** Valor real continuo correspondiente a la observación i-ésima.
- **$\hat{y}_i$:** Valor predicho para la región (que corresponde a la media de todas las observaciones y_i de esa misma región).

4. Cuadro Comparativo Resumen

Tipo de Modelo	Predicción Nodo	Criterio de División	Fórmula Matemática
Clasificación	Pluralidad (Moda)	Entropía	$H(S) = (1/n) \sum -p_i \log(p_i)$
Clasificación	Pluralidad (Moda)	Índice de Gini	$Gini(S) = (1/n) \sum p_i (1 - p_i)$
Regresión	Media (Promedio)	Error Cuadrático Medio (MSE)	$MSE = (1/n) \sum (y_i - \hat{y}_i)^2$

5. Material Visual de Origen

A continuación se adjunta la captura de pantalla de referencia correspondiente a la sección 3.0.2 sobre Criterios de División:

3.0.2 Criterios de división (revisión)

Para **clasificación**, cada región en el modelo se etiqueta según la clase de puntos más numerosa. En términos técnicos, utilizamos el **pluralidad** de puntos dentro de la región para elegir la etiqueta de la región. Se utilizan la entropía y el índice de Gini para los modelos de clasificación como criterios de división.

La entropía se define como:

$$H(S) = \frac{1}{n}\sum_{i=1}^{C} - p_i log(p_i)$$

donde p_i es la proporción de la clase i observaciones en el nodo en cuestión y C representa el número total de clases.

De forma similar, el índice de Gini se expresa como:

$$H(S) = \frac{1}{n}\sum_{i=1}^{C} p_i(1 - p_i)$$

Para **regresión**, predecimos con la **media** de los valores de salida para los puntos de entrenamiento contenidos en la región. El MSE es generalmente la opción preferida como criterio de división para los modelos de árboles de regresión.

MSE significa error cuadrático medio:

$$MSE = \frac{1}{n}\sum_{i}^{n} (y_i - \hat{y}_i)^2$$

La variable n en el cálculo del MSE es el número de observaciones, y_i es el valor real de los datos, y $\hat{y}_i$ es el valor predicho.

Condiciones de Parada y Estrategias de Crecimiento en Árboles de Decisión

Control de Complejidad, Búsqueda en Profundidad vs. Best-First e Hiperparámetros de Scikit-Learn

Índice de Contenidos

(El índice de contenidos se actualizará automáticamente al abrir el documento en Microsoft Word o Google Docs).

1. Control de Complejidad Mediante Profundidad Máxima (`max_depth`)

La condición de detención más común y extendida, aplicable tanto a árboles de clasificación como de regresión, consiste en limitar la **profundidad máxima (max_depth)** del árbol. Un árbol no restringido continuará dividiendo regiones hasta lograr una separación perfecta del conjunto de entrenamiento, lo que conduce de manera inevitable al sobreajuste (overfitting).

> **Concepto Clave: Árbol de Profundidad 1 (Decision Stump)**
>
> Un árbol de decisión con profundidad máxima igual a 1 realiza una única partición binaria en el nodo raíz. Aunque presenta un alto sesgo, sirve como bloque fundamental en modelos de ensamble como AdaBoost y Gradient Boosting.

En la estructura de nivel 1, el nodo raíz evalúa una única variable para dividir el conjunto inicial de observaciones en dos sub-regiones:

3.0.3 Condición de parada (revisión)

La condición de detención más común utilizada tanto para árboles de clasificación como de regresión es limitar el **profundidad máxima** del árbol.

Un ejemplo de un árbol de decisión de profundidad 1 es el siguiente.

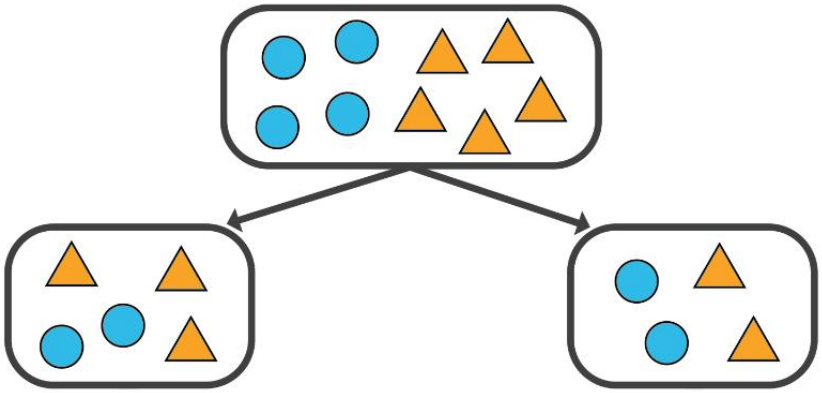


Figura: Árbol de decisión con profundidad máxima igual a 1 (Decision Stump)

Al incrementar la profundidad máxima a **Depth = 3**, la arquitectura permite hasta 3 niveles consecutivos de ramificación ($2^3 = 8$ nodos hoja potenciales). Esto incrementa la capacidad del modelo para capturar fronteras de decisión complejas:

Al aumentar la profundidad del árbol de decisión a 3, obtenemos el siguiente árbol.

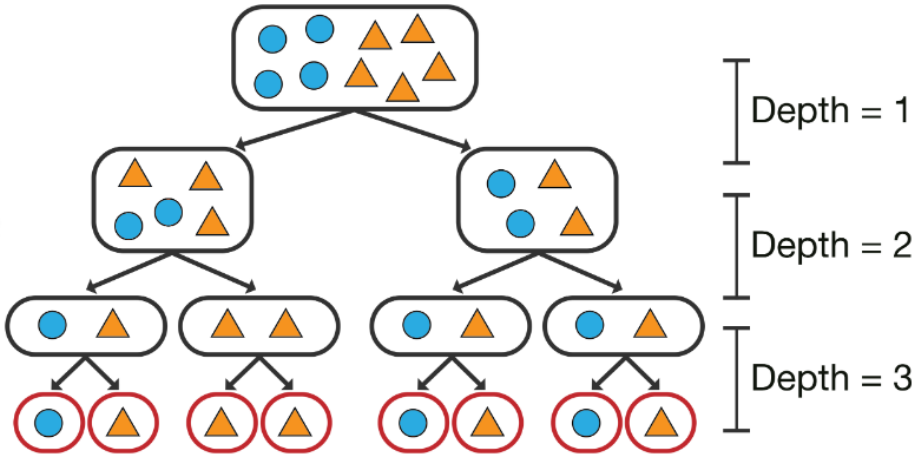


Figura: Estructura jerárquica de un árbol de decisión al aumentar la profundidad a Depth = 3

2. Criterios Simples de Detención Predefinida

Además de la profundidad global, existen condiciones de detención locales que evalúan las propiedades de las sub-regiones candidatas a división:

- **Nodos Hoja Puros (Pure Leaf Nodes):** No se divide una región si todas las instancias contenidas pertenecen exactamente a la misma clase target. En este punto, la impureza del nodo (medida mediante Gini o Entropía) es cero, por lo que una división adicional no aportaría ganancia de información.

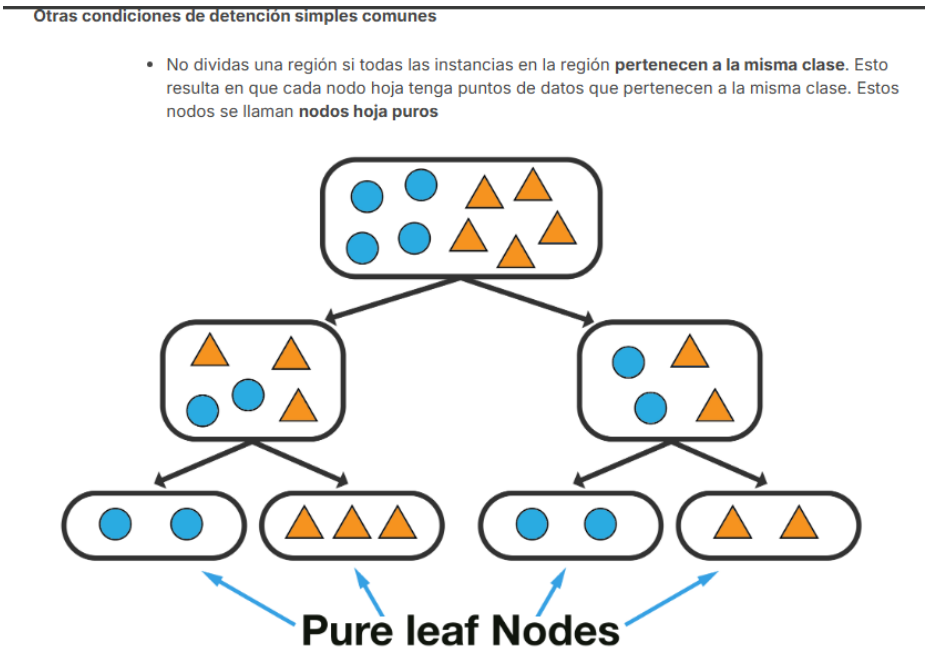


Figura: Detención natural del algoritmo al alcanzar nodos hoja puros

- **Mínimo de Muestras en Hoja (`min_samples_leaf`):** Se prohíbe la división de una región si el número de instancias en cualquiera de las sub-regiones resultantes caería por debajo de un umbral prefijado. Por ejemplo, al establecer **min_samples_leaf = 4**, cualquier partición que genere una hoja con menos de 4 observaciones es descartada automáticamente.

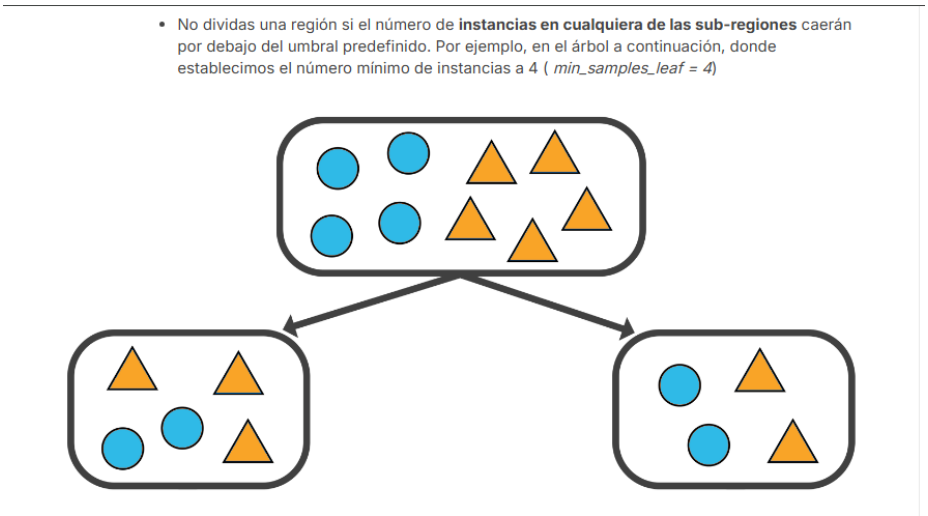


Figura: Restricción de división basada en el umbral min_samples_leaf = 4

- **Número Máximo de Nodos Hoja (`max_leaf_nodes`):** Limita la cantidad total de nodos hoja permitidos en todo el árbol. Al alcanzar dicho número (por ejemplo, **max_leaf_nodes = 3**), la construcción del árbol finaliza de inmediato, independientemente de que existan nodos aún no puros.

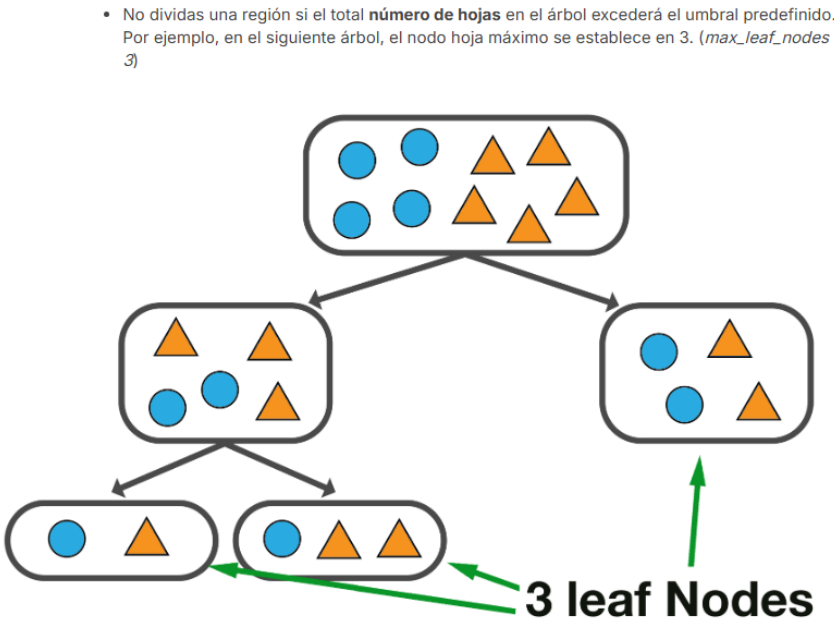


Figura: Arbol restringido a un máximo de 3 nodos hoja (max_leaf_nodes = 3)

3. Estrategias de Crecimiento: Búsqueda en Profundidad vs. Best-First

El algoritmo de construcción de árboles en librerías como Scikit-Learn adopta distintas estrategias de expansión en función de las restricciones configuradas:

> **Diferencia Fundamental de Crecimiento en Scikit-Learn**
>
> • Búsqueda en Profundidad (Depth-First): Comportamiento por defecto cuando max_leaf_nodes es None. El árbol expande completamente una rama descendente antes de retornar a ramas superiores.
> • Crecimiento Best-First: Se activa automáticamente al especificar max_leaf_nodes. El árbol hace crecer primero los nodos que ofrecen la mayor disminución global de impureza.

3.1 Crecimiento por Búsqueda en Profundidad (Depth-First)

Consideremos un modelo de diagnóstico médico que predice la presencia de enfermedad cardíaca basándose en variables clínicas (edad, sexo, presión arterial BP y colesterol):

Búsqueda en profundidad

Considera el siguiente árbol de decisión que predice si una persona tiene enfermedad cardíaca basándose en *edad, sexo, presión arterial (BP),* y colesterol

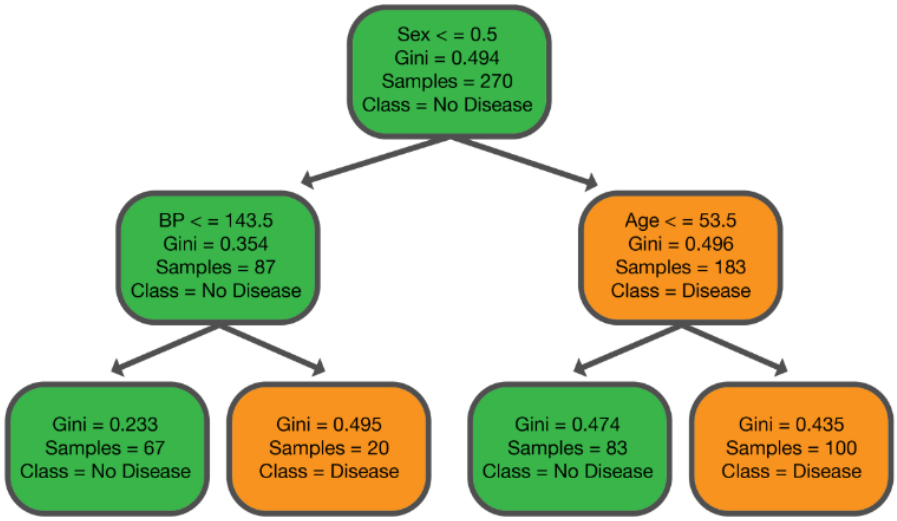


Figura: Estructura de árbol desarrollada mediante búsqueda en profundidad (Depth-First)

En el árbol anterior, el nodo raíz evalúa **Sex <= 0.5** ($N = 270$, Gini $= 0.494$). La rama izquierda evalúa **BP <= 143.5** ($N = 87$, Gini $= 0.354$) dividiéndose en dos hojas ($N=67$ con Gini 0.233 y $N=20$ con Gini 0.495). La rama derecha evalúa **Age <= 53.5** ($N = 183$, Gini $= 0.496$) dividiéndose en dos hojas ($N=83$ con Gini 0.474 y $N=100$ con Gini 0.435).

3.2 Crecimiento Best-First Basado en Disminución de Impureza

En el crecimiento **Best-First**, la selección del nodo a dividir se rige estrictamente por la **disminución de impureza (Impurity Decrease)**. La fórmula matemática para calcular el beneficio de dividir un nodo específico es:

```
Impurity Decrease = (N_nodo / N_total) * [ I(R) - (N_1 / N_nodo) * I(R_1)
                    - (N_2 / N_nodo) * I(R_2) ]
```

Para el nodo de presión arterial **BP <= 143.5** ($N=87$ sobre $N_{\text{total}}=270$), el cálculo es:

Crecimiento best-first

Sklearn determina la mejor división basada en **disminución de impureza**. El árbol resultante será el mismo cuando esté completamente desarrollado, solo que el orden en el que se construye es diferente. Así, para el mismo ejemplo que en búsqueda en profundidad obtenemos el siguiente árbol.

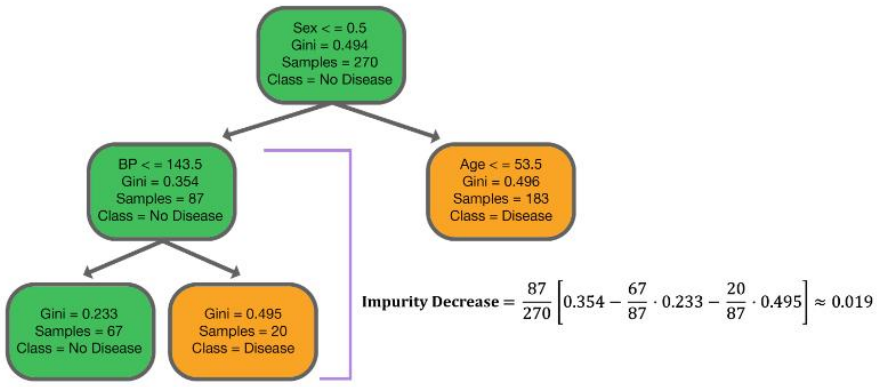


Figura: Cálculo cuantitativo de la disminución de impureza en la rama izquierda (BP <= 143.5)

Sustituyendo los valores del ejemplo:

```
Impurity Decrease (BP) = (87 / 270) * [ 0.354 - (67 / 87)*0.233 - (20 /
                         87)*0.495 ] ≈ 0.019
```

Al realizar la misma evaluación en la rama derecha (nodo de edad **Age <= 53.5**), se obtiene una disminución de impureza de **≈ 0.35**, un valor casi 18 veces superior al de la rama izquierda (0.019).

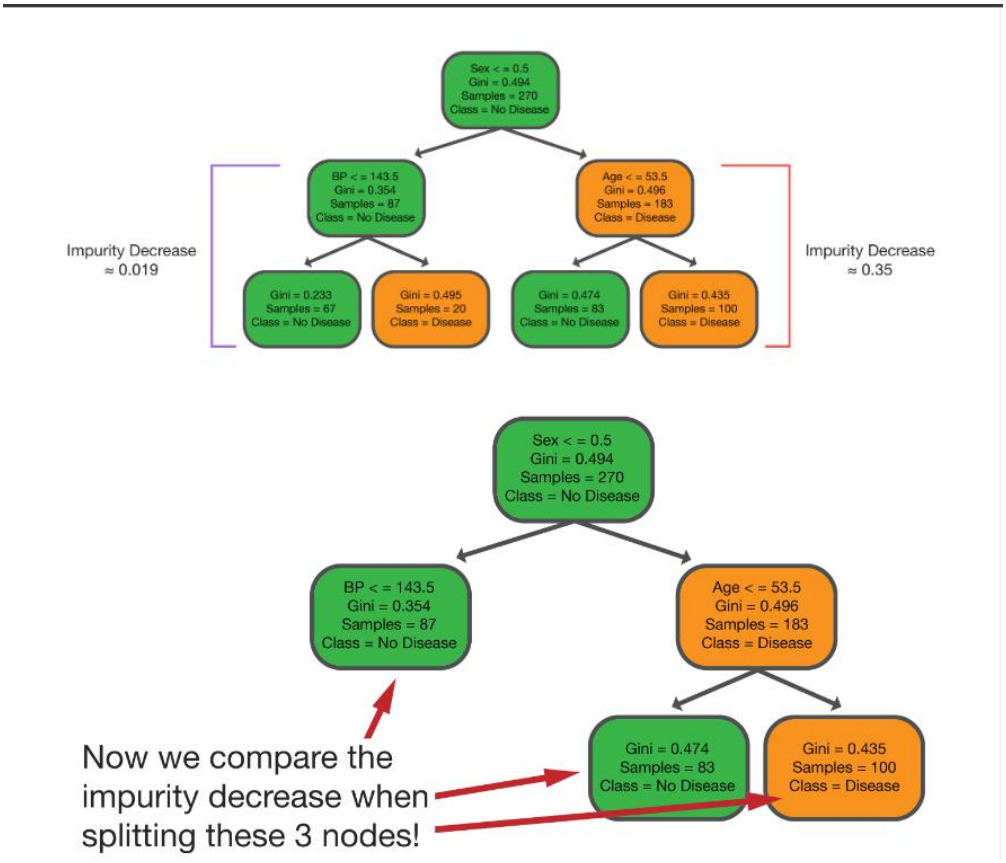


Figura: Comparativa de disminución de impureza entre candidatos en crecimiento Best-First

Por consiguiente, el algoritmo Best-First otorgará prioridad absoluta a la expansión de la rama con mayor reducción de impureza (0.35), garantizando que cada hoja añadida reduzca al máximo la varianza o error residual del modelo global.

4. Detención por Ganancia Mínima de Impureza (`min_impurity_decrease`)

Una condición de detención formal y cuantitativa consiste en requerir un umbral mínimo de ganancia en pureza, información o reducción de entropía antes de autorizar la partición de una región R en R_1 y R_2:

Una condición de detención más restrictiva es calcular la ganancia en pureza, de información o la reducción en la entropía al dividir una región R en R_1 y R_2;

$$Gain(R) = \delta(R) = m(R) - \frac{N_1}{N} m(R_1) - \frac{N_2}{N} m(R_2)$$

Aquí, $m(R)$ representa el error de clasificación, el índice Gini, o la entropía.

It is important to note that no split is made if the gain is less than some pre-defined threshold (`min_impurity_decrease`)

Figura: Formulación matemática de la ganancia de impureza (min_impurity_decrease)

La expresión matemática de la ganancia es:

$$Gain(R) = \delta(R) = m(R) - (N_1 / N) * m(R_1) - (N_2 / N) * m(R_2)$$

Donde **m(R)** representa la medida de impureza seleccionada (Error de clasificación, Índice Gini o Entropía), y N_1, N_2, N son las frecuencias absolutas en las sub-regiones y región madre respectivamente. Si **Gain(R) < min_impurity_decrease**, la división es abortada y el nodo se consolida como hoja.

5. Cuadro Comparativo de Hiperparámetros de Control en Scikit-Learn

La siguiente tabla resume los hiperparámetros clave para gestionar las condiciones de parada y la estrategia de crecimiento en modelos `DecisionTreeClassifier` y `DecisionTreeRegressor`:

Hiperparámetro	Estrategia	Mecanismo de Control	Efecto Principal
max_depth	Depth-First	Limita el número máximo de niveles desde la raíz hasta las hojas.	Evita sobreajuste global y restringe la altura del árbol.
max_leaf_nodes	Best-First	Activa crecimiento Best-First y limita el total de nodos hoja.	Prioriza las divisiones con mayor reducción de impureza.
min_samples_leaf	Ambas	Exige un número mínimo de observaciones en cada sub-hoja.	Suaviza el modelo y previene la creación de hojas ruidosas.
min_impurity_decrease	Ambas	Exige un umbral mínimo de ganancia δ(R) para autorizar splits.	Filtra divisiones marginales que no mejoran la pureza.

Bagging (Bootstrap Aggregating) y Métodos Ensamble

Superación de las limitaciones de la alta varianza en árboles de decisión mediante muestreo bootstrap y agregación de estimadores

Índice de Contenidos

1. Motivación: El Dilema de Sesgo-Varianza en Árboles Individuales

Los modelos basados en un único árbol de decisión presentan grandes ventajas operativas: son intuitivos, altamente interpretables y se entrenan con rapidez gracias a sus algoritmos voraces de particionamiento. Sin embargo, en escenarios con fronteras de decisión complejas o relaciones no lineales continuas, se enfrentan a un compromiso estructural severo entre el subajuste (underfitting) y el sobreajuste (overfitting).

Vídeo: Motivación para Bagging

Motivación para Bagging

> **LAS LIMITACIONES DE LOS MODELOS DE ÁRBOLES DE DECISIÓN**
>
> Los modelos de árboles de decisión son altamente interpretables y rápidos de entrenar, utilizando nuestros algoritmos de aprendizaje voraces. Sin embargo, para capturar una frontera de decisión compleja (o aproximar una función compleja) necesitamos usar un árbol grande (ya que cada vez solo podemos hacer particiones alineadas a los ejes).
>
> Los árboles grandes tienen una alta varianza y son propensos al sobreajuste. Por estas razones, en la práctica, los modelos de árboles de decisión a menudo tienen un rendimiento inferior cuando se comparan con otros métodos de clasificación o regresión.

Como viste en la última página, los modelos de árboles de decisión a menudo tienen un rendimiento inferior en situaciones de fronteras complejas y varianza. Por lo tanto, necesitamos un modelo mejor que supere las limitaciones de los árboles de decisión. Por ello, introducimos modelos Bagging.

Bagging se basa en 2 conceptos importantes:

- Aprendizaje en conjunto
- Bootstrapping

Examinemos eso con mayor detalle.

Figura 1: Captura del módulo 'Motivación para Bagging' donde se exponen las limitaciones por particiones alineadas con los ejes.

1.1 El Impacto de la Profundidad en el Ajuste

Impacto estructural: El comportamiento predictivo de un árbol está condicionado por su profundidad máxima:

- **Árboles Superficiales (Alta Varianza/Baja Capacidad):** No logran dividir el espacio de características en suficientes regiones. Como consecuencia, el modelo incurre en subajuste (underfitting), presentando un alto sesgo tanto en el conjunto de entrenamiento como en el de prueba.
- **Árboles Profundos (Alta Varianza/Alta Sensibilidad):** Fragmentan el espacio de entrada en un número excesivo de regiones diminutas. Esto hace que el árbol memorice el ruido específico del conjunto de datos en lugar de aprender el patrón subyacente, derivando en sobreajuste (overfitting).

1.2 Particiones Alineadas con los Ejes (Axis-Aligned Splits)

La restricción fundamental de los árboles de decisión tradicionales es que cada división espacial es ortogonal (perpendicular) a una sola variable continua. Para aproximarse a una frontera diagonal, circular o no alineada, el algoritmo debe generar una 'escalera' de múltiples cortes perpendiculares. Esto requiere árboles de gran profundidad que disparan la varianza del modelo.

> **Problema Clave de los Árboles Individuales**
> Para capturar fronteras complejas mediante cortes alineados con los ejes, se requieren árboles muy profundos. Sin embargo, la gran profundidad implica una alta varianza predictiva. Esta limitación provoca que un único árbol de decisión tenga un rendimiento inferior en la práctica frente a métodos más robustos.

2. Fundamentos del Aprendizaje en Conjunto (Ensemble Learning)

El aprendizaje por conjuntos es un meta-algoritmo que combina las predicciones de múltiples estimadores base (o modelos individuales) independientes para generar una única predicción consensuada con mayor capacidad de generalización.

2.1 Analogía Médica: El Diagnóstico por Junta de Especialistas

Ejemplo conceptual: Consideremos la tarea médica de diagnosticar la presencia de un tumor cerebral mediante resonancia magnética:

- **Enfoque Individual:** Confiar en el criterio de un único médico especialista formado en un solo hospital.

- **Enfoque Ensamble:** Consultar independientemente a una junta de múltiples especialistas entrenados en diversos centros médicos con diferentes historiales de pacientes.
- **Criterio de Agregación:** Si la mayoría de los médicos diagnostican la presencia del tumor, se confirma la predicción positiva. La votación mayoritaria reduce drásticamente el riesgo de un diagnóstico erróneo por sesgo personal o falta de experiencia en un caso raro.

2.2 Reglas de Agregación en Clasificación y Regresión

La combinación de resultados de los M estimadores independientes depende de la naturaleza de la variable objetivo:

Tipo de Problema	Mecanismo de Agregación	Formulación Matemática
Clasificación (Binaria / Multiclase)	Voto Mayoritario (Moda)	y_final = mode(h_1(x), h_2(x), ..., h_M(x))
Regresión (Variable Continua)	Promedio Aritmético (Media)	y_final = (1 / M) * sum(h_m(x))

3. Bootstrapping: Generación de Diversidad a partir de un Único Dataset

Para que un ensemble funcione eficazmente, los modelos base deben estar entrenados en conjuntos de datos diversos. En la práctica real no disponemos de múltiples datasets independientes. Para solucionar la escasez de datos se utiliza la técnica de Bootstrapping.

> **Definición de Bootstrapping**
> El Bootstrapping es el procedimiento estadístico de muestreo aleatorio CON REEMPLAZO (sampling with replacement) sobre un conjunto de datos original de N observaciones. Permite crear M versiones o muestras bootstrap sintéticas para realizar inferencias agregadas.

3.2 Propiedades Fundamentales del Muestreo Bootstrap

- **Tamaño Constante (N):** Cada dataset bootstrap contiene exactamente el mismo número de muestras (N) que el dataset original.
- **Reemplazo y Duplicados:** Al seleccionar cada elemento con reemplazo, algunas observaciones originales aparecerán repetidas múltiples veces en una muestra bootstrap, mientras que aproximadamente el 36.8% de los datos quedarán fuera (datos Out-Of-Bag, OOB).

4. Bagging (Bootstrap Aggregating)

Introducido por Leo Breiman en 1996, el algoritmo Bagging integra el muestreo Bootstrap y la Agregación en un marco unificado de reducción de varianza.

4.1 Flujo Algorítmico de Bagging

Algoritmo de 3 Pasos: El proceso general de entrenamiento e inferencia se resume en tres etapas:

Paso	Fase	Descripción Detallada
Paso 1	**Bootstrap**	Generar M subconjuntos de datos de entrenamiento aleatorios mediante muestreo con reemplazo sobre el dataset original D.
Paso 2	**Entrenamiento**	Entrenar un árbol de decisión profundo h_m(x) de manera completamente independiente sobre cada uno de los M conjuntos bootstrap.
Paso 3	**Agregación**	Para una nueva entrada x, combinar las M predicciones individuales calculando la media (regresión) o el voto por mayoría (clasificación).

4.2 Suavizado de la Superficie de Predicción en Regresión

En problemas de regresión, un único árbol de decisión produce una función escalonada con saltos abruptos. Al promediar la respuesta de M árboles entrenados sobre muestras bootstrap, la función combinada resultante se convierte en una curva suave, eliminando el patrón en 'zigzag' característico y reduciendo significativamente la varianza de la predicción final.

5. Evaluación Crítica: Ventajas y Desventajas de Bagging

A pesar de sus sólidas propiedades para el control de varianza, Bagging no es un modelo perfecto y presenta ciertos inconvenientes estructurales que deben gestionarse en la práctica.

☑ Ventajas de Bagging	✗ Desventajas y Limitaciones
Alta Expresividad: Al utilizar árboles base profundos, cada estimador captura fronteras y patrones no lineales complejos.	Pérdida de Interpretabilidad: El modelo final es un promedio de decenas o cientos de árboles. Se pierde la capacidad de trazar reglas simples de decisión.
Reducción de Varianza: El promediado de estimadores independientes cancela	Riesgo de Subajuste Persistente: Si los árboles base son demasiado superficiales (p. ej. profundidad 2), el ensamble no capturará el patrón y

las fluctuaciones aleatorias y el ruido de los datos.	seguirá subajustando.
Paralelización Eficiente: Dado que cada árbol bootstrap se entrena de forma independiente, el proceso es altamente paralelizable en múltiples núcleos.	Riesgo de Sobreajuste Residual / Correlación: Si los árboles son extremadamente profundos o las muestras están correlacionadas, la reducción de varianza disminuye.

6. Consideraciones Prácticas y Control de Complejidad

Para garantizar que Bagging alcance su rendimiento óptimo, es crucial regular la profundidad de los árboles individuales mediante condiciones de parada o poda. Aunque la validación cruzada permite ajustar el número de estimadores y su profundidad, resulta computacionalmente costosa. Por esta razón, se emplean métricas como el error Out-Of-Bag (OOB) para evaluar el rendimiento del ensamble sin necesidad de conjuntos de validación externos.

Sobreajuste y Subajuste en Árboles de Decisión

Análisis de Complejidad, Fronteras de Decisión y Estrategias de Control de Profundidad (Sección 3.1.1)

1. Conceptos Fundamentales: Subajuste vs. Sobreajuste

En el aprendizaje automático, el equilibrio entre la complejidad del modelo y su capacidad de generalización es un pilar crítico. Se definen formalmente dos estados indeseados en el ajuste de un modelo de clasificación:

- **Subajuste (Underfitting):** Ocurre cuando el modelo carece del detalle o la capacidad expresiva suficiente para capturar las relaciones subyacentes en el conjunto de datos. La frontera de decisión resulta demasiado rígida o simplista.
- **Sobreajuste (Overfitting):** Ocurre cuando el modelo es demasiado detallado y complejo, adaptándose con extrema precisión a los datos de entrenamiento específicos (incluyendo el ruido aleatorio y los valores atípicos), lo que genera un rendimiento deficiente e impredecible al clasificar datos futuros inobservados.

En la estructura específica de los **árboles de decisión**, el fenómeno de subajuste o sobreajuste está directamente gobernado por la **profundidad máxima del árbol (Depth):**

> **DIFERENCIACIÓN POR PROFUNDIDAD EN ÁRBOLES DE DECISIÓN**
>
> • Árboles demasiado superficiales (Depth bajo): No logran dividir el espacio de características en suficientes regiones. La frontera de decisión no envuelve adecuadamente los patrones de datos (Subajuste).
>
> • Árboles demasiado profundos (Depth alto): Cortan el espacio de entrada en un número excesivo de hiperplanos ortogonales, memorizando el ruido. Generan fronteras complejas e inestables (Sobreajuste).

2. Evolución Visual de la Frontera de Decisión según la Profundidad

Al analizar un problema de clasificación en un espacio bidimensional de características (por ejemplo, coordenadas de **Latitude vs. Longitude**), se observa cómo la forma de las fronteras de decisión se transforma drásticamente conforme se incrementa la profundidad permitida:

Nivel de Profundidad	Comportamiento del Modelo	Efecto sobre la Frontera y Generalización
Profundidad Baja (Depth = 1 - 2)	**Subajuste Severo**	Divide el plano con 1 o 2 cortes ortogonales rígidos. Es fácil de interpretar pero no logra separar las clases adecuadamente.
Profundidad Óptima	**Equilibrio Apropiado**	Capta la forma general del clúster central sin

(Depth = 5)		encerrar puntos individuales ruidosos. Equilibra sesgo y varianza.
Profundidad Excesiva (Depth = 36 - 100)	**Sobreajuste Extremo**	Fragmenta el plano en múltiples micro-regiones cuadriláteras para aislar cada punto con ruido. Presenta una varianza muy elevada.

Aquí hay otra vista. La siguiente figura muestra tres gráficos de complejidad creciente de la frontera de decisión, de izquierda a derecha.

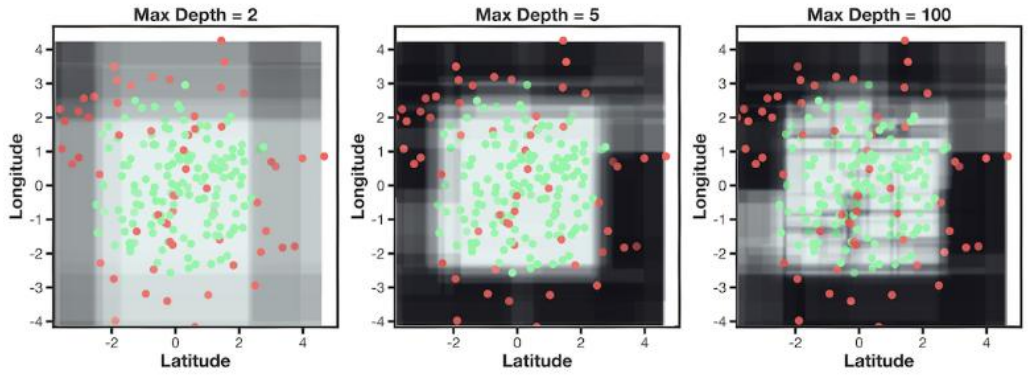
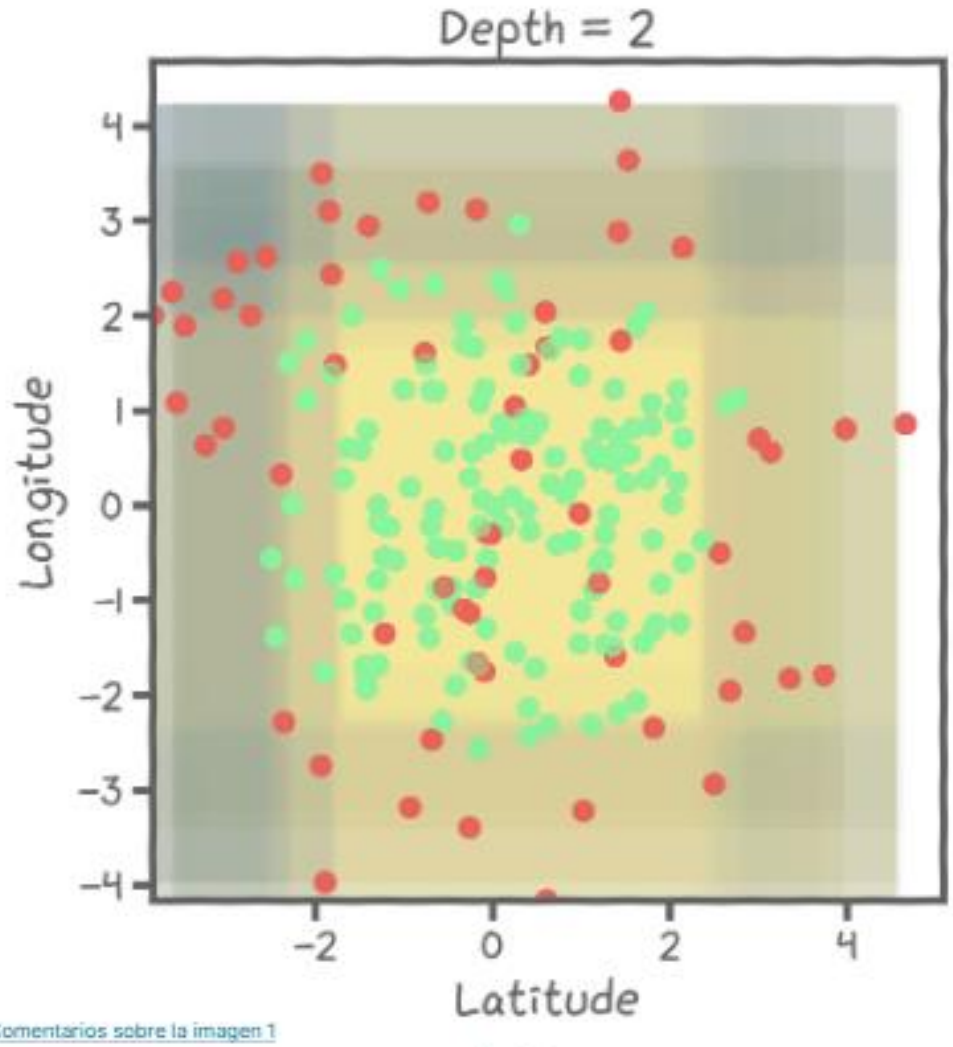


Figura 3.1: Comparativa de fronteras de decisión según la complejidad creciente del árbol (Max Depth = 2, 5 y 100). Fuente: Captura de pantalla de la plataforma.

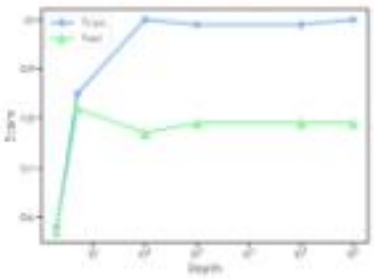
3. Curva de Complejidad y Brecha de Generalización (Train vs. Test Score)

Para cuantificar empíricamente el sobreajuste, se analiza la evolución del rendimiento (puntuación de precisión o **Accuracy Score**) evaluado de forma simultánea en el conjunto de entrenamiento (**Train**) y en el conjunto de prueba (**Test**):



Comentarios sobre la imagen 1

El gráfico de la derecha muestra la profundidad frente a la puntuación de precisión para este tipo de clasificación. Demuestra que a medida que aumenta la profundidad del árbol, la diferencia entre las puntuaciones de entrenamiento y prueba aumenta.



Hay dos técnicas que puede utilizar para evitar el sobreajuste:

- Poda del árbol
- Limitar la profundidad máxima del árbol

Hablaremos sobre ambos en las próximas páginas.

Figura 3.2: Representación del modelo con Depth = 2 y curva de precisión (Train vs. Test) según el incremento de profundidad. Fuente: Material original.

El comportamiento observatorio en la gráfica de precisión revela dos fenómenos clave:

- **Comportamiento en Entrenamiento (Train Score):** A medida que la profundidad del árbol aumenta, la precisión sobre el conjunto de entrenamiento crece de manera monótona hasta alcanzar valores cercanos al 100% (1.0). El árbol aprende la posición de cada muestra individual.
- **Comportamiento en Prueba (Test Score):** La precisión en el conjunto de prueba aumenta inicialmente hasta alcanzar su punto óptimo a profundidades moderadas (por ejemplo, profundidad entre 2 y 5). A partir de ese punto, la precisión de prueba se estanca o decae.
- **Ampliación de la Brecha de Generalización:** La diferencia creciente entre la puntuación de entrenamiento y la puntuación de prueba es la prueba directa del sobreajuste. El modelo ha memorizado el ruido de los datos de entrenamiento en lugar de aprender patrones generalizables.

4. Técnicas para Evitar el Sobreajuste

Para controlar la complejidad excesiva y garantizar que el árbol de decisión mantenga una alta precisión predictiva sobre datos no vistos, existen dos técnicas fundamentales:

Técnica de Control	Mecanismo de Acción	Ventaja Principal
Limitar la Profundidad Máxima (Max Depth / Pre-pruning)	Establece un umbral estricto durante la fase de crecimiento del árbol para detener la división de nodos cuando se alcanza una profundidad dada.	Evita de forma preventiva la creación de ramas innecesarias, reduciendo el tiempo de cómputo y la varianza.
Poda del Árbol (Tree Pruning / Post-pruning)	Permite que el árbol crezca por completo hasta memorizar los datos y posteriormente elimina de forma sistemática las ramas secundarias con menor impacto predictivo.	Permite evaluar la importancia global de cada subdivisión, conservando solo las estructuras que mejoran la generalización.

RECOMENDACIÓN PRÁCTICA

• El ruido inherente a los datos de entrenamiento hace que intentar construir fronteras hiper-complejas sea inútil y contraproducente.

• Un modelo con una estructura más simple y menor profundidad no solo es más fácil de interpretar y ejecutar, sino que ofrece una precisión equivalente o superior en entornos de producción reales.

Aprendizaje en Conjunto (Ensemble Learning)

Combinación de modelos base para optimización predictiva, control de varianza y superación de limitaciones en árboles individuales

---

Índice de Contenidos

1. Definición y Principios del Aprendizaje en Conjunto

El **aprendizaje en conjunto (Ensemble Learning)** es una técnica avanzada de aprendizaje automático basada en la combinación de múltiples modelos predictivos individuales (conocidos como modelos base o *weak learners*) para producir un modelo global con una capacidad predictiva óptima y superior a la de cualquiera de sus componentes por separado.

El principio subyacente radica en que, aunque un modelo individual pueda cometer errores o sufrir de alta varianza, la agregación sistemática de un grupo diverso de modelos diluye los errores no correlacionados, generando predicciones más robustas y estables.

> **Mecanismos de Agregación según el Tipo de Problema:**
>
> - **Clasificación:** El resultado final se determina calculando la **pluralidad** o voto mayoritario entre las predicciones emitidas por todos los modelos individuales del conjunto.
> - **Regresión:** El resultado final se obtiene calculando el **promedio continuo** de los valores numéricos devueltos por cada modelo individual.

2. Analogía del Mundo Real: Diagnóstico por Junta Médica

Para comprender intuitivamente el comportamiento del aprendizaje en conjunto, considérese el escenario de evaluación de resonancias magnéticas (MRI) para la detección de tumores cerebrales:

- **Diversidad de Entrenamiento:** Diversos médicos especialistas se han formado en distintas escuelas de medicina (Hospital A, Hospital B, Hospital C), habiendo estado expuestos a diferentes conjuntos de casos prácticos y muestras de entrenamiento durante su carrera.
- **Evaluación Independiente:** Ante un nuevo paciente con una resonancia magnética inédita, cada médico analiza la imagen de forma totalmente independiente y emite un diagnóstico individual (SÍ/NO a la presencia del tumor).

- **Decisión Consensuada:** El diagnóstico final entregado al paciente no depende de la opinión aislada de un solo facultativo, sino de la agregación por mayoría de todos los votos. Si la mayoría determina la presencia de tumor, el diagnóstico global es positivo; de lo contrario, es negativo.

3.1.2 Aprendizaje en Conjunto

Aprendizaje en Conjunto

El aprendizaje en conjunto es una técnica de aprendizaje automático que combina varios modelos base para producir un modelo predictivo óptimo.

- Para la clasificación, devolvemos un *pluralidad* de los modelos.
- Para la regresión, devolvemos el *promedio* de la salida del modelo.

Un ejemplo del mundo real de aprendizaje en conjunto se muestra en la figura a continuación. Los médicos son entrenados con diferentes muestras de resonancia magnética en sus respectivas escuelas de medicina. En el hospital en el que actualmente trabajan, se les muestra una resonancia magnética que ninguno ha visto antes. Cada uno de ellos toma una decisión sobre la presencia de un tumor cerebral. La decisión final es una agregación de todos sus votos.

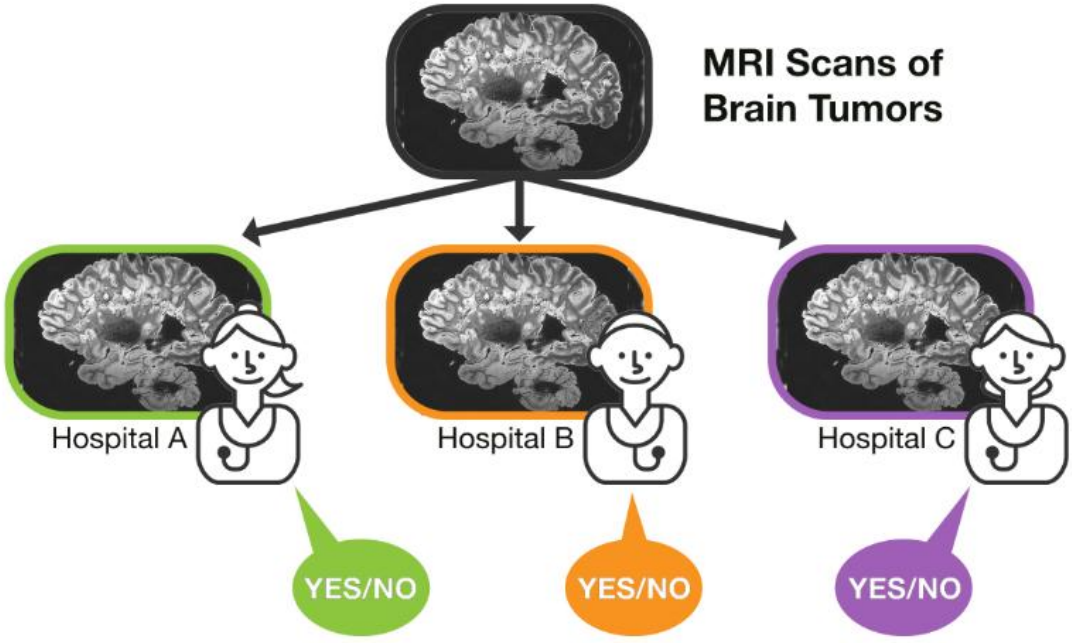


Figura 1: Representación del aprendizaje en conjunto mediante un diagnóstico por comité médico de tres hospitales (A, B y C) evaluando un escaneo de resonancia magnética.

3. Ventajas Estructurales frente a Modelos Individuales

El empleo de ensambles resuelve directamente las principales deficiencias de los modelos individuales aislados (como los árboles de decisión profundos o muy superficiales). Las tres ventajas cardinales son:

Dimensión de Mejora	Efecto Técnico en el Aprendizaje Automático
Mayor Precisión Predictiva	La combinación de múltiples estimadores diversificados alcanza una precisión significativamente superior a la de cualquier modelo individual aislado del grupo.
Resistencia al Sobreajuste	Mientras que un modelo único profundo tiende a memorizar el ruido específico de los datos de entrenamiento, la votación o promediado del conjunto cancela las desviaciones espurias.
Reducción de Varianza y Sesgo	Equilibra el sesgo y minimiza drásticamente la varianza de la predicción final, ofreciendo una alta estabilidad general.

4. Taxonomía de los Métodos de Conjunto

Dentro del aprendizaje en conjunto existen cuatro familias metodológicas principales según la forma en que construyen y combinan sus estimadores base:

- **Bagging (Bootstrap Aggregating):** Entrena estimadores independientes en paralelo utilizando subconjuntos de datos generados mediante muestreo con reemplazo (bootstrap), combinando sus salidas por promedio o voto.
- **Boosting:** Entrena estimadores de forma secuencial, donde cada nuevo modelo se enfoca en corregir los errores cometidos por los modelos anteriores.
- **Stacking:** Combina modelos heterogéneos y utiliza un meta-modelo de nivel superior para aprender a ponderar y combinar las predicciones de los modelos base.
- **Blending:** Variante simplificada de Stacking que utiliza un conjunto de validación retenido (hold-out set) para entrenar el meta-modelo de combinación.

Enfoque del Curso: *Las metodologías de* **Bagging** *y* **Boosting** constituyen los dos pilares centrales de estudio para potenciar el rendimiento de los árboles de decisión en problemas complejos de clasificación y regresión.

Ventajas

- Un grupo de modelos ofrece una mejor precisión en comparación con los modelos individuales
- El uso de un solo modelo causa sobreajuste, mientras que el grupo es menos propenso a sobreajustarse.
- Esto ayuda a reducir el sesgo y la varianza

Tipos de métodos de conjunto

Los tipos de métodos de conjunto incluyen...

- Bagging
- Boosting
- Stacking
- Blending

El bagging y el boosting serán nuestro enfoque principal a medida que avancemos en este curso.

Figura 2: Resumen sintético de ventajas y principales familias metodológicas de aprendizaje en conjunto (Bagging, Boosting, Stacking y Blending).

3.1.3 Arranque: Bootstrapping y Bagging

Apuntes de Clase y Síntesis Teórico-Práctica

> **MOTIVACIÓN: Generación de Múltiples Modelos con un Único Conjunto de Datos**
> *En la práctica, no disponemos de conjuntos de datos ilimitados o diferentes para entrenar varios modelos. Sin embargo, resulta deseable contar con múltiples modelos entrenados sobre datos distintos. La técnica de Bootstrapping es el procedimiento matemático que nos permite lograr este objetivo a partir de una única muestra de datos.*

1. Concepto y Mecánica del Muestreo Bootstrap (Bootstrapping)

El **Bootstrapping** es un proceso estadístico de **muestreo con reemplazo** (sampling with replacement) aplicado sobre un conjunto de datos original. Al realizar múltiples extracciones aleatorias con reemplazo, se construyen varios subconjuntos de entrenamiento (muestras bootstrap) sobre los cuales se ejecutan cálculos independientes para obtener diferentes predicciones.

Para comprender la mecánica, considérese un conjunto de datos original constituido por las escaneografías por resonancia magnética (MRI) de 5 pacientes: **A, B, C, D y E**. A partir de esta población inicial de 5 observaciones, se pueden generar distintas versiones de conjuntos bootstrapped mediante selección aleatoria con reemplazo:

- **Muestra 1:** Escaneos A, C, A (repetido), E, B
- **Muestra 2:** Escaneos E, B, A, C, C (repetido)
- **Muestra 3:** Escaneos D, B, A, E, A (repetido)

Propiedades Clave del Muestreo Bootstrap:

1. Cada subconjunto bootstrapped contiene exactamente la **misma cantidad de datos (N)** que el conjunto original.

2. La selección aleatoria con reemplazo permite que algunas observaciones se dupliquen o tripliquen en una muestra, mientras que otras quedan excluidas (datos Out-Of-Bag).

2. Bagging (Bootstrap Aggregating): Definición y Algoritmo

El término **Bagging** es un acrónimo de **B**ootstrap **Agg**regating. Representa un enfoque de aprendizaje en conjunto (ensemble learning) fundamentado en dos fases consecutivas:

- **Fase 1: Bootstrapping.** Se generan múltiples subconjuntos de datos de entrenamiento aleatorios mediante muestreo con reemplazo. Posteriormente, se entrena un árbol de decisión profundo e independiente sobre cada una de las muestras bootstrap.

- **Fase 2: Agregación (Aggregation).** Frente a una nueva entrada de prueba (test input), se obtienen las predicciones de todos los árboles entrenados y se combinan para emitir una única predicción colectiva final.

Representación Esquemática del Proceso de Bagging

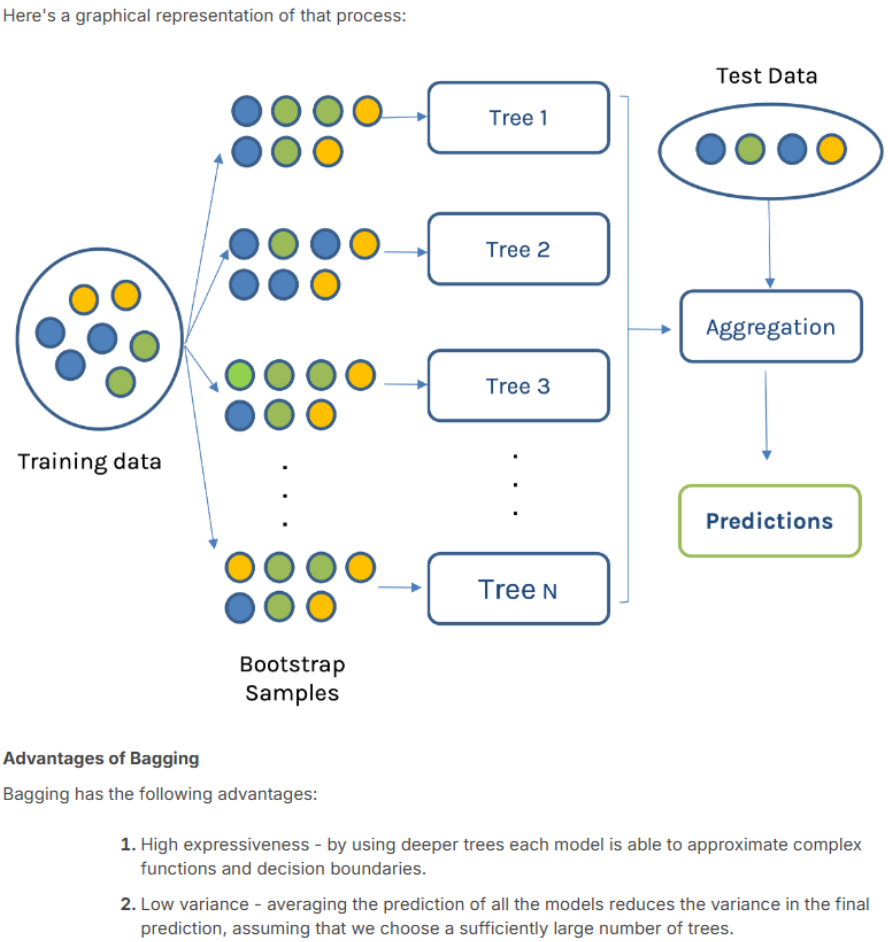


Figura 1: Diagrama de flujo de Bagging desde el conjunto original hasta la predicción agregada.

3. Ventajas y Tipos de Bagging: Clasificación vs. Regresión

La técnica de Bagging destaca por proporcionar dos propiedades deseables en el modelado predictivo:

- **1. Alta Expresividad (High Expressiveness):** Al utilizar árboles de decisión individuales profundos, cada estimador es capaz de aproximar funciones complejas y fronteras de decisión altamente no lineales.
- **2. Baja Varianza (Low Variance):** Promediar o agregar las predicciones de múltiples estimadores independientes reduce de forma sustancial la varianza en la predicción final, siempre que se seleccione un número suficientemente elevado de árboles (N).

Estrategias de Agregación Según la Tarea de Aprendizaje

Tipo de Tarea	Mecanismo de Agregación	Resultado y Comportamiento
Clasificación	**Voto Mayoritario (Majority Vote)**	Se asigna la clase más frecuente entre las predicciones individuales de los N árboles.
Regresión	**Promedio Matemático (Mathematical Mean)**	Se calcula la media aritmética de las salidas continuas emitidas por cada árbol estimador.

Ejemplo Práctico de Bagging en Regresión (Temperatura vs. Ozono)

Bagging types

For each bootstrap, we build a decision tree. The result is a combination (majority) of the predictions from all trees for classification tasks. While for regression, we take the average of all individual predictions of the trees or estimators.

An example of regression bagging is shown in the following plot. Each of the thin green lines is a prediction from an individual tree from a different bootstrapped sample. The thick green line is the average of all the individual predictions. It gives us a stronger predictor overall than any one of the individual trees.

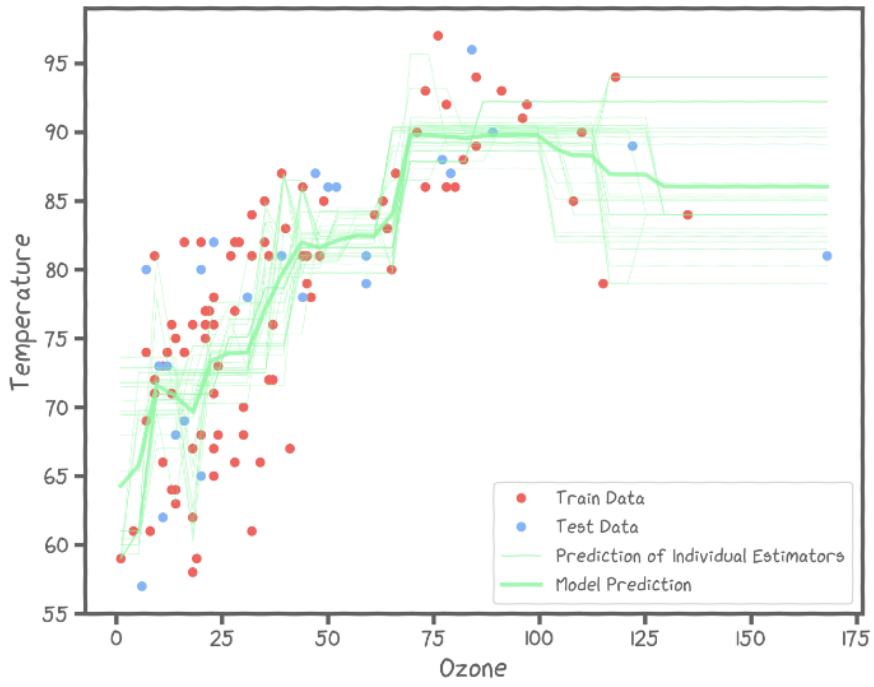


Figura 2: Curvas de predicción de estimadores individuales (líneas verdes finas) frente a la predicción promediada del ensamble (línea verde gruesa).

Como se aprecia en la gráfica de regresión, cada una de las líneas verdes finas corresponde a la predicción de un árbol individual entrenado sobre una muestra bootstrap diferente. La línea verde gruesa representa el promedio de todas las predicciones individuales. Se observa claramente que el promedio produce una curva de ajuste mucho más suave y robusta, eliminando la volatilidad y los escalones abruptos propios de un solo árbol.

4. Limitaciones y Desventajas de Bagging

A pesar de sus amplias ventajas predictivas, la técnica de Bagging presenta tres limitaciones fundamentales que se deben considerar en la práctica:

- **Subajuste si los árboles son muy superficiales:** Si los árboles individuales que componen el ensamble tienen poca profundidad, el modelo de conjunto continuará subajustando (underfitting), ya que la combinación de múltiples modelos débiles e incapaces no logra capturar la estructura subyacente de los datos.
- **Riesgo residual de sobreajuste si los árboles son excesivamente grandes:** Si los árboles individuales crecen desmedidamente sin ningún tipo de restricción o si existe alta correlación entre las muestras, persiste el riesgo de sobreajuste (overfitting).

Pérdida de Interpretabilidad Directa: A diferencia de un único árbol de decisión cuya lógica se puede rastrear fácilmente mediante una secuencia clara de reglas 'si-entonces', un modelo promediado formado por decenas o cientos de árboles pierde su interpretabilidad directa. Resulta imposible rastrear de forma intuitiva la trayectoria de una decisión individual a través del conjunto agregador.

3.1.4 Árbol de Decisión vs Bagging

Análisis Comparativo de Rendimiento y Sobreajuste según la Profundidad del Modelo

Apuntes Técnicos de Aprendizaje Automático — Algoritmos de Clasificación y Ensambles

Índice de Contenidos

(El índice de contenidos se actualizará automáticamente al abrir este documento en Microsoft Word o Google Docs).

1. Impacto de la Profundidad del Árbol (Tree Depth) en la Capacidad de Generalización

En el modelado mediante árboles de decisión individuales, la profundidad del árbol (**Tree Depth**) actúa como el hiperparámetro fundamental que controla la complejidad del modelo. A medida que se añade más profundidad a un árbol de decisión único, la estructura divide el espacio de características en regiones cada vez más pequeñas, lo que provoca que el modelo se sobreajuste rápidamente al ruido presente en los datos de entrenamiento.

Por el contrario, el algoritmo de ensamblado **Bagging (Bootstrap Aggregating)** demuestra una respuesta sensiblemente más estable ante el incremento de la profundidad. Al promediar las predicciones de múltiples árboles entrenados de forma independiente sobre muestras bootstrap, Bagging logra mantener un nivel superior de rendimiento generalizado incluso cuando la profundidad de los estimadores individuales aumenta. No obstante, dicho incremento en la profundidad no produce mejoras indefinidas: eventualmente, el rendimiento en validación se estabiliza y deja de mejorar a partir de cierto punto crítico.

> **Principio Clave de Comparación:**
> Mientras que un árbol de decisión individual sufre una degradación severa en el conjunto de validación al aumentar su profundidad debido a la alta varianza, Bagging mitiga dicha varianza mediante promediado masivo, manteniendo el error de validación acotado y estable.

2. Análisis de las Curvas de Error: Training Error vs. Validation Error

Las diferencias entre ambos enfoques quedan reflejadas en el comportamiento dinámico de las curvas de error de entrenamiento (**Training Error**) y de validación (**Validation Error**) en función de la profundidad del árbol:

- **Error de Entrenamiento (Training Error):** Tanto en los árboles individuales como en Bagging, el error de entrenamiento disminuye al aumentar la profundidad. Sin embargo, el árbol individual alcanza rápidamente un error prácticamente nulo (memorización total), mientras que Bagging se estabiliza en valores reducidos y ligeramente superiores, reflejando una menor memorización de ruido individual.
- **Error de Validación (Validation Error):** En un árbol de decisión individual, el error de validación desciende inicialmente hasta alcanzar un mínimo local a profundidad baja, pero luego se dispara exponencialmente hacia niveles muy elevados. En Bagging, el error de validación disminuye de forma sustancialmente más profunda y permanece plano y controlado sin sufrir el repunte del árbol único.

Figura 3.1.4: Curvas Comparativas de Error de Entrenamiento y Validación

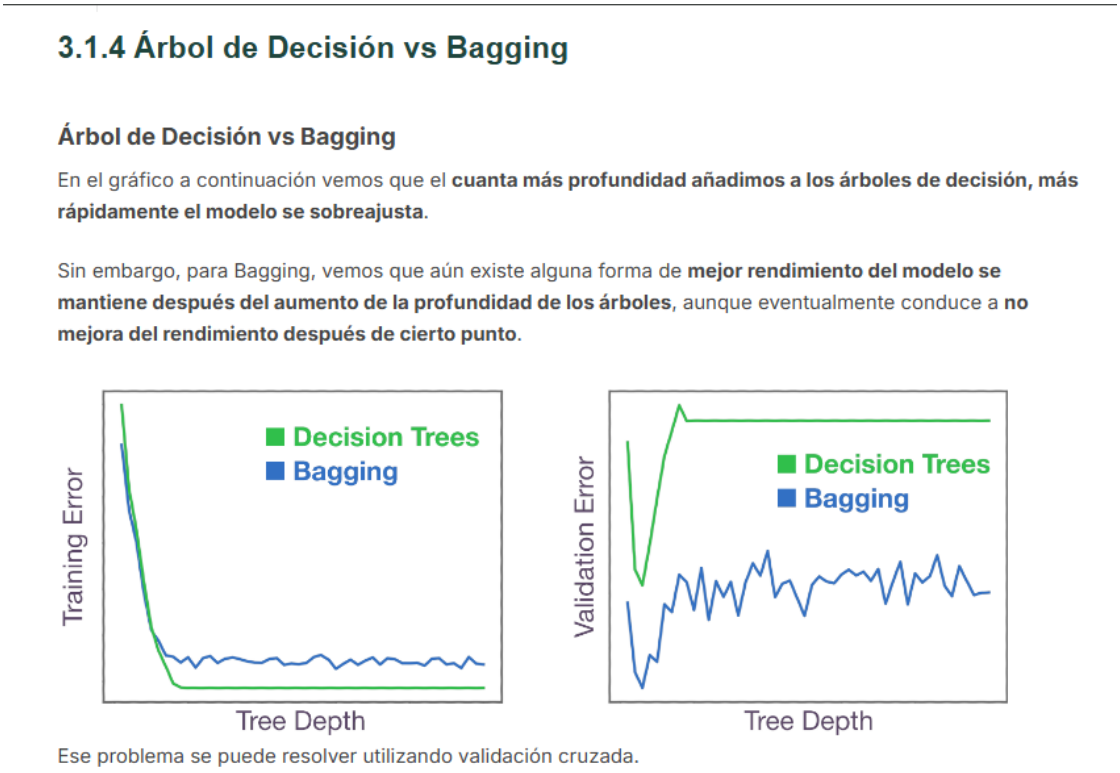


Gráfico de la Sección 3.1.4: Comparación de Training Error (izquierda) y Validation Error (derecha) entre Decision Trees (verde) y Bagging (azul) frente a Tree Depth.

Tabla Comparativa: Árbol de Decisión Individual vs. Ensamble Bagging

Métrica / Dimensión	Árbol de Decisión Individual	Ensamble Bagging
Error de Entrenamiento	Cae a casi cero muy rápido a profundidades elevadas.	Disminuye constantemente hasta estabilizarse en un valor bajo y sostenido.

Error de Validación	Inicia bajo a profundidad óptima pero se dispara por sobreajuste severo.	Alcanza un mínimo más bajo y se mantiene plano y estable sin explotar.
Sensibilidad a Profundidad	Extremadamente sensible: alta profundidad produce alta varianza.	Altamente robusto: la agregación cancela la varianza individual.
Límite de Rendimiento	Se degrada severamente a gran profundidad.	Se estabiliza; a mayor profundidad no mejora más, pero tampoco empeora.

3. Optimización de la Profundidad y Selección del Modelo mediante Validación Cruzada

Como se observa en el gráfico de error de validación, determinar la profundidad idónea de un árbol de decisión es un problema técnico crítico. Si la profundidad seleccionada es insuficiente, el modelo incurre en subajuste (**underfitting**); si es excesiva, sufre sobreajuste (**overfitting**).

Para resolver este dilema de ajuste de hiperparámetros de manera rigurosa, se emplea la **validación cruzada (Cross-Validation)**. Esta técnica permite evaluar sistemáticamente distintas profundidades sobre múltiples particiones de datos (folds), identificando con precisión la profundidad óptima donde el error de validación es mínimo antes de que se desencadene el sobreajuste.

> **Conclusión Práctica:**
> Aunque Bagging ofrece una resistencia sustancialmente mayor al sobreajuste frente al aumento de profundidad, la combinación de validación cruzada con la búsqueda de hiperparámetros sigue siendo la herramienta fundamental para seleccionar tanto la profundidad adecuada de los estimadores base como el número de estimadores del ensamble.

Error Fuera de la Bolsa (Out-Of-Bag Error - OOB)

Evaluación Eficiente de Modelos de Ensamble y Estimación de Generalizabilidad sin Fuga de Datos

---

Índice de Contenidos

1. Concepto y Fundamentos de la Estimación Out-Of-Bag (OOB)

La estimación **Out-Of-Bag (OOB)** o error fuera de la bolsa es una métrica de evaluación interna diseñada específicamente para algoritmos de ensamble basados en *bagging* (Bootstrap Aggregating). Permite evaluar el rendimiento predictivo y la capacidad de generalización del modelo de forma simultánea a su entrenamiento, eliminando la necesidad obligatoria de reservar un conjunto de validación independiente.

En la construcción de cada árbol del ensamble, se genera una muestra de entrenamiento mediante **muestreo con reemplazo (bootstrapping)** a partir del conjunto de datos original de tamaño *N*. Debido a la naturaleza probabilística del reemplazo, la probabilidad de que una observación particular no sea seleccionada en una muestra bootstrap de tamaño *N* viene dada analíticamente por:

> **Proporción Teórica de Observaciones OOB:**
> P(observación i ∉ Muestra Bootstrap) = (1 - 1/N)^N ≈ e^(-1) ≈ 36.8%

Esto implica que aproximadamente el **36.8% de los datos originales** quedan excluidos de la bolsa de entrenamiento de cada árbol individual. Estas observaciones omitidas constituyen el grupo *Out-Of-Bag* para ese árbol específico y funcionan como un conjunto de prueba natural e inalterado.

2. Mapeo Muestral y Diagrama de Observaciones OOB

Para ilustrar el mecanismo de aislamiento de muestras OOB, considérese un conjunto de datos inicial compuesto por 5 observaciones etiquetadas: {Gato, Caballo, Perro, Vaca, Rata}. Al extraer múltiples muestras bootstrap independientes, la presencia de duplicados en la muestra de entrenamiento genera automáticamente un subconjunto complementario de validación fuera de la bolsa:

Muestra Bootstrap	Datos de Entrenamiento (Bolsa - In-	Observaciones Excluidas

	Bag)	(Fuera de la Bolsa - OOB)
Muestra 1	{Gato, Perro, Caballo, Caballo, Perro}	{Rata, Vaca}
Muestra 2	{Rata, Rata, Vaca, Perro, Vaca}	{Caballo}
Muestra 3	{Perro, Rata, Caballo, Vaca, Perro}	{Gato}

Vídeo: Error fuera de la bolsa/Estimación fuera de la bolsa

OOB

Error fuera de la bolsa (o **Estimación fuera de la bolsa**) es un método para determinar el error de predicción que permite que los árboles se ajusten y se validen mientras se entrenan. Se llama fuera de la bolsa porque se usan los puntos que fueron *no* incluidos en un conjunto de entrenamiento bootstrap agregado (la "bolsa"). Normalmente, fuera de la bolsa se abrevia como OOB.

El diagrama a continuación muestra elementos que están "fuera de la bolsa" para una bolsa. Por ejemplo, "Rata" y "Vaca" no aparecen en la primera muestra, por lo que están en el grupo "fuera de la bolsa".

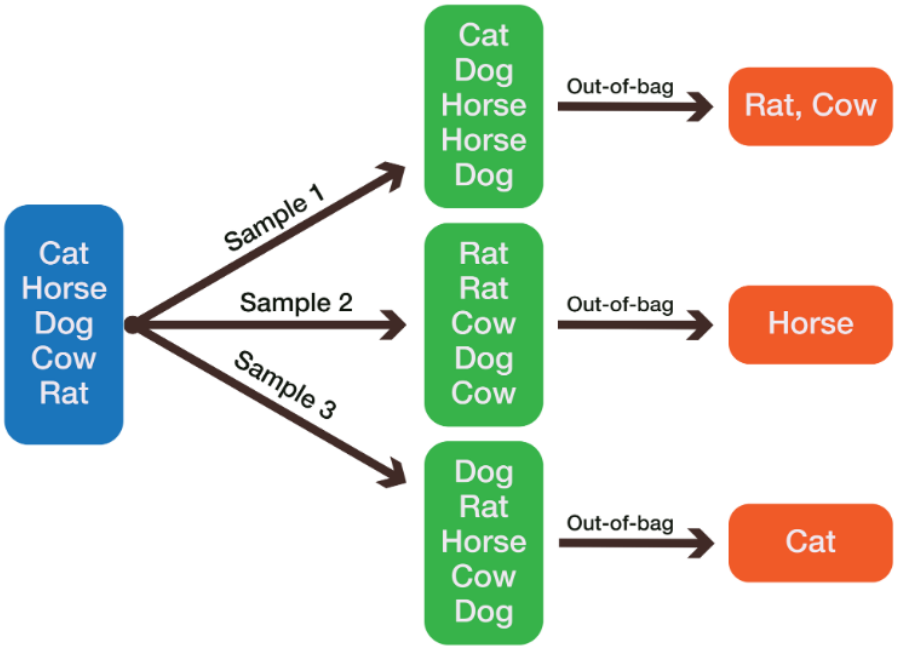


Figura 1: Muestreo Bootstrap y aislamiento de observaciones Out-Of-Bag (OOB) por árbol.

3. Algoritmo Paso a Paso para el Cálculo del Error OOB

El cálculo formal del error out-of-bag se ejecuta mediante la agregación restringida de predicciones exclusivamente sobre los modelos que no vieron cada punto individual durante la fase de ajuste.

Paso 1: Identificación de Modelos OOB por Punto: Para cada observación (x_i, y_i) del conjunto de datos de entrenamiento original, se identifica el conjunto de árboles B_i = {b_1, b_2, ..., b_k} cuya muestra bootstrap no incluyó a la observación x_i.

Paso 2: Generación de Predicción Puntual Agregada: Se evalúa la entrada x_i únicamente en los árboles pertenecientes a B_i. La predicción agregada ŷ_i se calcula según el tipo de problema:

Paso 3: Cálculo del Error Puntual OOB: Se compara la predicción puntual agregada ŷ_i con el valor real y_i para obtener el error específico de la observación i.

Paso 4: Promediado Global sobre la Muestra: Se calcula el promedio de los errores puntuales sobre la totalidad de los N datos del conjunto de entrenamiento.

Formulación Matemática Formal del Error OOB:

• Clasificación (Voto Mayoritario Puntual):

$\hat{y}_i = \text{Moda}_{b \in B_i} \{ f^b(x_i) \}$

$\text{Error Puntual}_i = I(\hat{y}_i \neq y_i)$ [1 si hay error de clasificación, 0 si coincide]

$\text{Error OOB Global} = (1 / N) * \sum_{i=1}^{N} I(\hat{y}_i \neq y_i)$

• Regresión (Promedio Continuo Puntual):

$\hat{y}_i = (1 / |B_i|) * \sum_{b \in B_i} f^b(x_i)$

$\text{Error Puntual}_i = (y_i - \hat{y}_i)^2$ [Error Cuadrático]

$\text{Error OOB Global (MSE OOB)} = (1 / N) * \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$

4. Ventajas Comparativas: OOB vs. Validación Cruzada (CV)

Aunque la validación cruzada (K-Fold CV) es una técnica estándar de evaluación, presenta inconvenientes teóricos y computacionales importantes cuando se aplica a métodos de ensamble basándose en bagging. La estimación OOB resuelve directamente estas limitaciones:

Criterio de Evaluación	Validación Cruzada (K-Fold CV)	Estimación Out-Of-Bag (OOB)
Fuga de Datos (Data Leakage)	Existe riesgo de fuga si las particiones de validación comparten muestras con modelos de iteraciones adyacentes.	Aislamiento estricto: cada observación se evalúa exclusivamente en árboles que jamás la vieron en entrenamiento.
Costo Computacional	Elevado: requiere reentrenar el ensamble completo K veces (K ejecuciones independientes).	Cero costo adicional: el error se calcula 'al vuelo' durante el entrenamiento único del bagging.
Varianza del Estimador	Mayor varianza debido a la	Menor varianza y ajuste más suave

subdivisión del conjunto de entrenamiento en pliegues.

sin necesidad de reducir el tamaño efectivo de los datos.

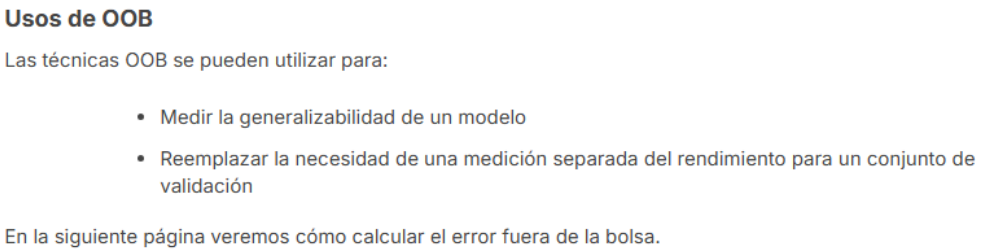


Figura 2: Aplicaciones principales del error Out-Of-Bag en la medición de generalizabilidad.

5. Aplicaciones, Limitaciones y Correlación de Árboles

A pesar de que el error OOB permite controlar la profundidad de los árboles y medir la generalizabilidad con precisión, persistieron limitaciones estructurales en el método bagging convencional:

- **Pérdida de Interpretabilidad:** Al agregar decenas o cientos de árboles de decisión independientes, se pierde la trazabilidad lógica directa de un único conjunto de reglas de decisión simples.
- **Efecto de Subajuste o Sobreajuste Residual:** Si los árboles individuales son excesivamente superficiales (e.g., profundidad 2), el ensamble seguirá sufriendo subajuste. Si son extremadamente profundos y correlacionados, el ensamble puede seguir sobreajustando el ruido de los datos.
- **El Problema de la Correlación entre Árboles:** En el bagging tradicional, si existe un predictor o característica extremadamente fuerte (x_j), el algoritmo voraz (greedy) seleccionará x_j en el nodo raíz de casi todos los árboles. Como resultado, los árboles generados se vuelven altamente correlacionados entre sí, limitando la reducción teórica de la varianza.

Esta limitación de la correlación entre árboles es el detonante teórico que motiva la transición hacia los **Bosques Aleatorios (Random Forests)**, donde además del muestreo bootstrap de datos, se introduce el subespaciado aleatorio de características (feature subspacing) en cada división para descorrelacionar los árboles del ensamble.

Procedimiento Detallado de Cálculo del Error Out-Of-Bag (OOB)

Guía estructurada en 5 pasos para la evaluación técnica de ensambles sin conjunto de validación externo

Índice de Contenidos

1. Fundamento General del Procedimiento OOB

El cálculo del error Out-Of-Bag (OOB) permite estimar la capacidad de generalización de un conjunto de árboles de decisión (Bagging) de manera nativa durante la fase de entrenamiento. Al emplear muestreo bootstrap con reemplazo, aproximadamente un **36.8% de las observaciones originales queda fuera de cada muestra de entrenamiento individual**. Estas instancias no vistas actúan como conjuntos de validación independientes para cada árbol específico.

> 📌 **Principio Out-Of-Bag: Aprovechamiento Eficiente de Datos:** Dado que cada árbol del ensamble se entrena únicamente sobre un subconjunto bootstrap B_b, los datos excluidos (Out-Of-Bag) de dicho árbol proporcionan una evaluación totalmente insesgada sin necesidad de dividir los datos originales ni hacer validación cruzada K-Fold.

Gráficamente, el enfoque se ve así, partiendo de los datos originales a la izquierda y generando un árbol de decisión a partir de la muestra de bootstrap.

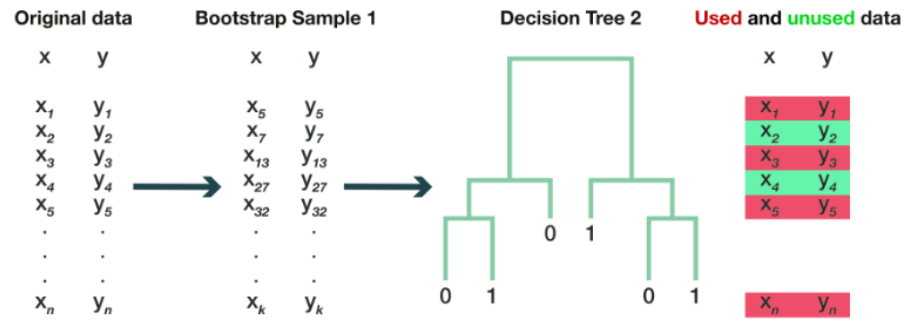


Figura 3.2.1a: Flujo de remuestreo bootstrap con reemplazo y separación de muestras OOB.

2. Desglose Algorítmico en 5 Pasos

Paso 1: Muestreo Bootstrap con Reemplazo

Dado un dataset original de tamaño N, se generan B muestras bootstrap seleccionando aleatoriamente N observaciones con reemplazo. Cada subconjunto bootstrap B_b mantiene la misma dimensión que el dataset inicial, introduciendo variabilidad mediante la duplicación de algunas instancias y la omisión deliberada de otras.

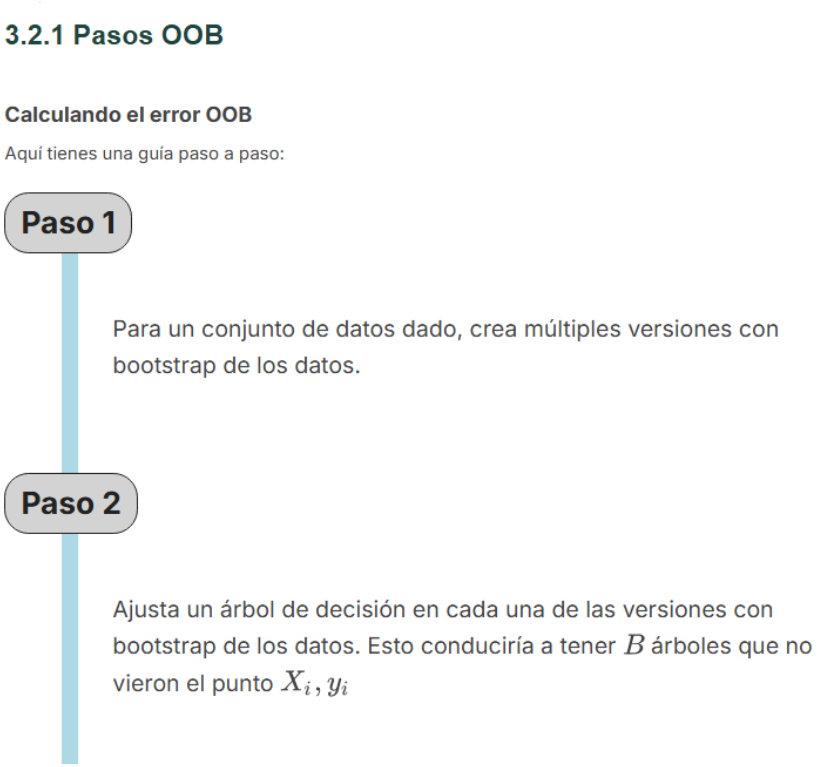


Figura 3.2.1b: Paso 1 y 2 — Generación de muestras bootstrap y entrenamiento de modelos individuales.

Paso 2: Entrenamiento Independiente de Árboles

Sobre cada muestra bootstrap B_b se entrena de forma completamente independiente un árbol de decisión profundo T_b. Debido a las diferencias en la composición de datos de cada subconjunto, los modelos resultantes construyen reglas de división únicas y rutas predictivas diversificadas.

Un conjunto de datos puede proporcionarnos una gran cantidad de árboles diferentes para trabajar, incluso cuando trabajamos únicamente con modelos que no han visto un dato en particular.

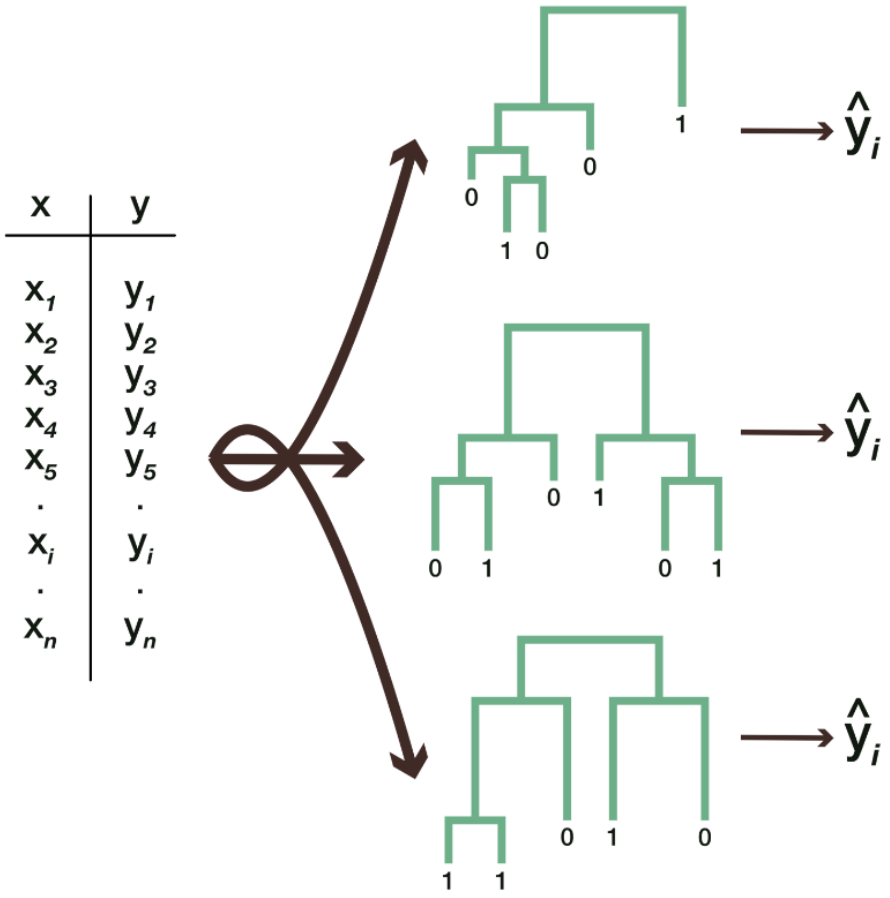


Figura 3.2.1c: Generación de múltiples caminos de decisión y diversidad en el ensamble.

Paso 3: Predicción Puntual Out-Of-Bag (Pointwise Prediction)

Para cada observación original (x_i, y_i) del dataset de entrenamiento, se identifican únicamente aquellos árboles b en los que dicha observación NO estuvo presente en su muestra de entrenamiento. La predicción puntual OOB, denotada como ŷ_{i,pw}, se obtiene agregando las salidas de este subgrupo específico de árboles:

- **Clasificación: ŷ_{i,pw} = Moda { T_b(x_i) | (x_i, y_i) ∉ B_b } (Voto mayoritario)**
- **Regresión: ŷ_{i,pw} = Promedio { T_b(x_i) | (x_i, y_i) ∉ B_b } (Media aritmética)**

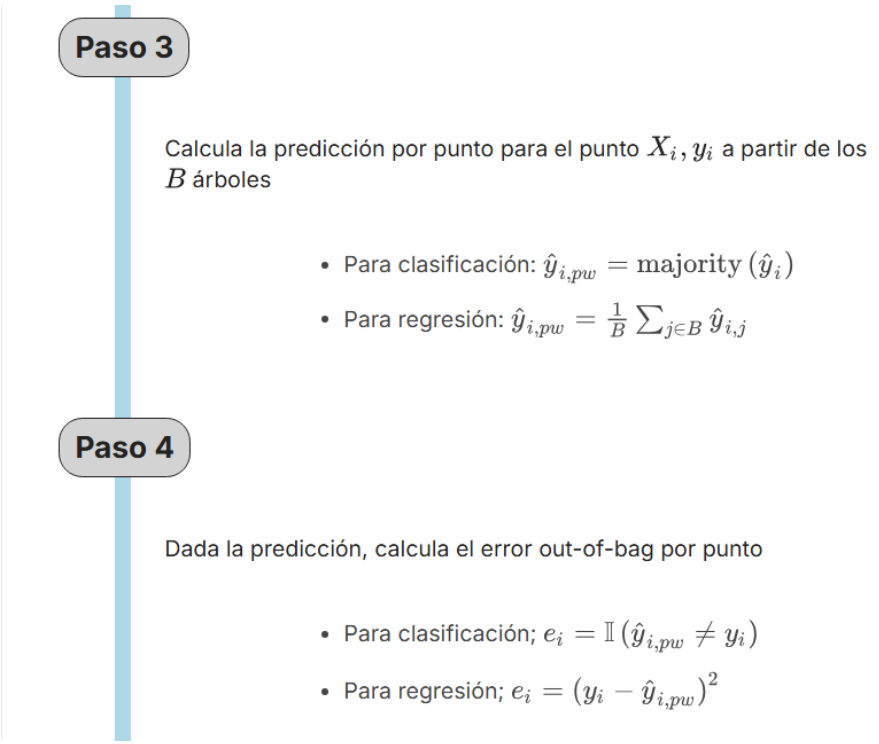


Figura 3.2.1d: Paso 3 y 4 — Predicción puntual agregada y evaluación de la discrepancia de predicción.

Paso 4: Cálculo del Error Puntual OOB

Se evalúa la diferencia directa entre la predicción puntual OOB ŷ_{i,pw} y la etiqueta/valor real y_i de la instancia:

- **Clasificación: e_i = I(ŷ_{i,pw} ≠ y_i) [Devuelve 1 si hay error de clase, 0 si acierta]**
- **Regresión: e_i = (ŷ_{i,pw} - y_i)² [Diferencia cuadrática puntual]**

Paso 5: Agregación del Error Global OOB

Finalmente, el error OOB global se calcula promediando los errores puntuales e_i sobre el total de N observaciones del conjunto de datos:

Error_OOB = (1 / N) * ∑_{i=1}^N e_i

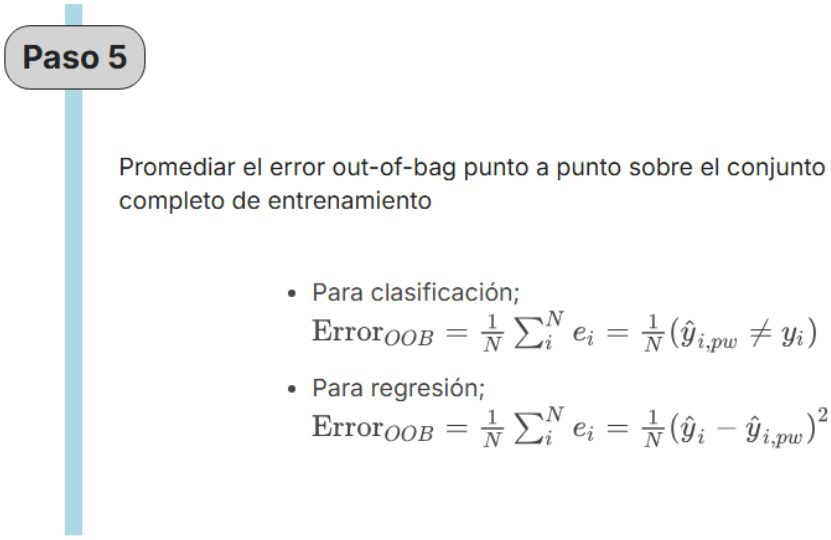


Figura 3.2.1e: Paso 5 — Promediado global del error Out-Of-Bag sobre el total de observaciones.

3. Resumen de Fórmulas y Mecanismos por Tipo de Tarea

Componente / Fase	Clasificación Multiclase/Binaria	Regresión Continua
Agregación Puntual (ŷ_{i,pw})	Voto Mayoritario (Moda) entre árboles que excluyeron a x_i	Promedio Aritmético (Media) entre árboles que excluyeron a x_i
Error Puntual (e_i)	Indicador de Clasificación Incorrecta: I(ŷ_{i,pw} ≠ y_i)	Error Cuadrático Puntual: (ŷ_{i,pw} - y_i)²
Métrica Global (Error_OOB)	Tasa de Error de Clasificación OOB (Misclassification Rate)	Error Cuadrático Medio OOB (Mean Squared Error OOB)

CAPÍTULO 3.2.2

Beneficios y Desventajas del Error OOB y del Bagging

1. Evaluación Eficiente y Ventajas del Error Out-Of-Bag (OOB)

La estimación mediante el error **Out-Of-Bag (OOB)** surge como un mecanismo alternativo directo a la validación cruzada (Cross-Validation / CV) tradicional para evaluar modelos ensamblados por Bagging. A continuación se detallan sus dos justificaciones fundamentales:

1. Prevención de la Fuga de Datos (Data Leakage) y Reducción de Varianza

En la validación cruzada tradicional aplicada a modelos ensamblados, existe el riesgo de que los datos del conjunto de validación hayan sido vistos o utilizados previamente durante el entrenamiento de algunos árboles de decisión individuales. Esto genera una fuga de datos que infla artificialmente la precisión esperada y produce mayor varianza. El error OOB elimina esta fuga al evaluar cada observación únicamente con aquellos árboles que jamás la incluyeron en su muestra bootstrap, garantizando una estimación de varianza sensiblemente menor.

2. Eficiencia Computacional Superior

El costo computacional de calcular el error OOB es drásticamente menor en comparación con la validación cruzada k-fold. En k-fold CV se requiere re-entrenar k veces todo el conjunto de árboles de bagging. En cambio, el error OOB se obtiene 'gratis' durante el propio proceso de entrenamiento del ensemble aprovechando los datos excluidos por el muestreo bootstrap.

1.1 Análisis del Comportamiento del Error vs. Profundidad del Árbol

El gráfico extraído del material de estudio demuestra cuantitativamente cómo evoluciona el error de validación conforme se incrementa la profundidad de los árboles de decisión (**Depth of Trees**) en diferentes arquitecturas:

3.2.2 Beneficios y Desventajas del OOB

¿Por qué usar el error OOB?

- Al utilizar la técnica de validación cruzada, cada conjunto de validación ya ha sido visto o utilizado en el entrenamiento por algunos árboles de decisión. Esto resulta en una fuga de datos, y por lo tanto, mayor varianza. El error OOB previene la fuga y ofrece un mejor modelo con baja varianza.
- El costo computacional del error OOB es menor en comparación con la validación cruzada para el bagging. Puedes ver en el gráfico a continuación que el error OOB para bagging es menor, y disminuye rápidamente.

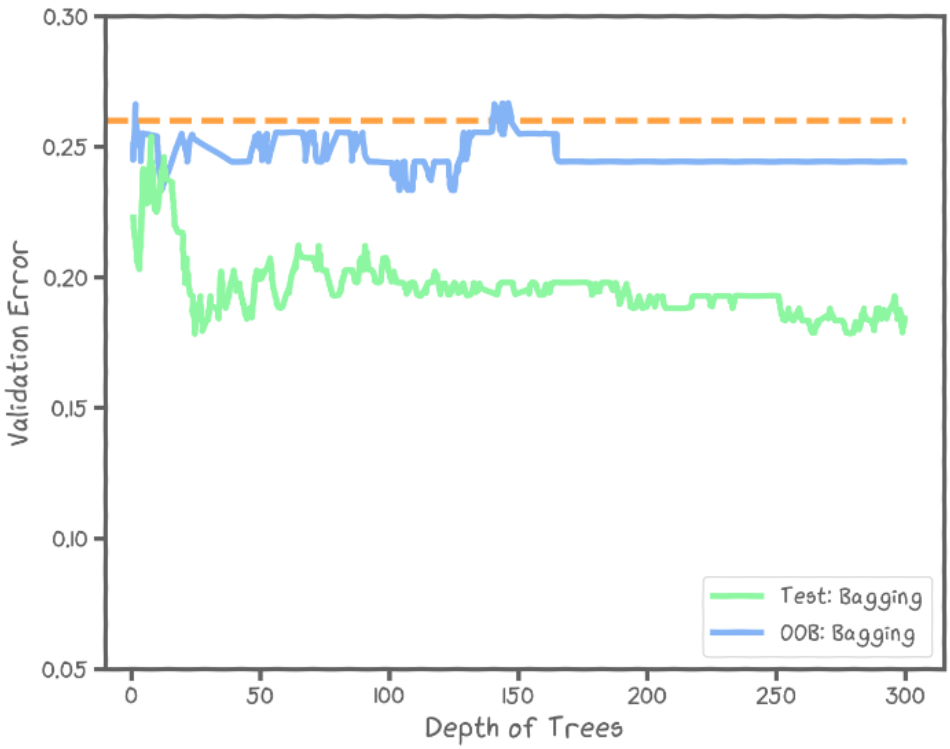


Figura 3.2.2: Comparación del error de validación frente a la profundidad del árbol para la línea base (naranja), OOB Bagging (azul) y Test Bagging (verde).

- **Línea Base (Naranja Discontinua):** Mantiene un nivel de error elevado (~0.26) de forma constante, lo que refleja las limitaciones de un modelo simple o de una evaluación convencional sin ensamblado.
- **OOB Bagging (Curva Azul):** Proporciona una estimación interna del error que disminuye rápidamente respecto al modelo base y se estabiliza alrededor de 0.24, actuando como una cota conservadora pero precisa del rendimiento.
- **Test Bagging (Curva Verde):** Muestra un descenso pronunciado desde ~0.24 hasta alcanzar un mínimo óptimo de error (~0.18 - 0.19) a profundidades moderadas (entre 25 y 50 niveles), manteniendo una meseta estable sin sufrir sobreajuste severo a profundidades altas.

2. Desventajas Estructurales y Limitaciones del Bagging

A pesar de sus sólidas propiedades predictivas y reducción de varianza, el Bagging presenta dos limitantes críticas que restringen su aplicabilidad directa en ciertos contextos:

Limitación	Mecanismo del Problema e Impacto
Pérdida de Interpretabilidad ('Caja Negra')	Un árbol de decisión individual es altamente interpretable gracias a su estructura de reglas si-entonces alineadas con los ejes. En contraste, el promedio de decenas o cientos de árboles convierte al ensamble en un modelo complejo ('caja negra'), haciendo muy difícil rastrear la lógica exacta de una predicción específica.
Árboles Altamente Correlacionados	Si el dataset contiene un predictor dominante (extremadamente fuerte, xj), el algoritmo voraz (greedy) seleccionará invariablemente ese predictor xj para la división inicial en la mayoría de las muestras bootstrap. Esto hace que gran parte de los árboles tengan estructuras iniciales idénticas, rompiendo el supuesto de independencia.

3. De Bagging a Bosques Aleatorios (Random Forests)

La teoría matemática de Bagging asume originalmente que los árboles individuales del ensamble están distribuidos de manera **independiente e idénticamente distribuida (i.i.d.)**, de modo que la varianza del promedio se reduce a un factor de 1/B respecto a la varianza de un solo árbol. Sin embargo, debido a la presencia de predictores dominantes:

Fallo del Supuesto de Independencia en Bagging

Dado que todos los árboles seleccionan la misma variable dominante xj en el nodo raíz, las predicciones de los árboles están positivamente correlacionadas. La varianza de la media de B variables aleatorias con correlación ρ viene dada por:

$$\text{Var}(\text{Media}) = \rho \cdot \sigma^2 + ((1 - \rho) / B) \cdot \sigma^2$$

Incluso si B tiende a infinito, el término $\rho \cdot \sigma^2$ permanece invariable. Por lo tanto, si la correlación ρ es alta, el beneficio de promediar muchos árboles se satura rápidamente.

Para resolver esta limitante y descorrelacionar los árboles, la técnica de **Bosques Aleatorios (Random Forests)** introduce un nivel adicional de aleatoriedad: en cada división de cada árbol, solo se permite considerar un subconjunto aleatorio reducido de predictores ($m < p$). Esto fuerza a los árboles a explorar variables secundarias, reduciendo drásticamente la correlación ρ y optimizando el rendimiento global del ensamble.

Bosques Aleatorios (Random Forests)

Descorrelación de Árboles, Selección Aleatoria de Características e Hiperparámetros Clave

Índice de Contenidos

(El índice de contenidos se actualizará automáticamente al abrir el archivo en Word o Google Docs).

1. La Limitación del Bagging: Presencia de Características Dominantes

El método de **Bagging (Bootstrap Aggregating)** es un ensamble sumamente efectivo para reducir la alta varianza y prevenir el sobreajuste al promediar múltiples árboles de decisión entrenados sobre muestras bootstrap. Sin embargo, sufre una limitación estructural crítica cuando el conjunto de datos contiene una **característica dominante** (un predictor extremadamente fuerte frente a los demás).

Debido a que el algoritmo de construcción de árboles de decisión utiliza un enfoque voraz (greedy approach), en cada iteración evalúa todas las características disponibles para seleccionar la división óptima. Si una característica presenta una ganancia de información o reducción de impureza sustancialmente superior al resto, la gran mayoría de los árboles del ensamble seleccionarán esa misma característica como su nodo raíz.

> **Ejemplo Ilustrativo: Clasificación de Formas Geométricas**
>
> En un conjunto de datos de formas geométricas con atributos de color, forma y tamaño, si el atributo 'color' es el predictor dominante, todos los árboles de decisión generados mediante Bagging comenzarán dividiéndose por 'color' en la raíz. Esto genera árboles altamente correlacionados entre sí, reduciendo drásticamente la diversidad del ensamble y limitando la capacidad de reducción de varianza.

2. Solución mediante Bosques Aleatorios: Descorrelación de Árboles

Un **Bosque Aleatorio (Random Forest)** es una modificación directa de Bagging diseñada explícitamente para descorrelacionar los árboles individuales. Para lograrlo, introduce un nivel adicional de aleatoriedad durante el proceso de división de cada nodo.

Mecanismo de Selección de Características:

- **Muestreo Bootstrap:** Se genera una muestra con reemplazo del conjunto de datos original (igual que en Bagging).
- **Subconjunto Aleatorio en Cada Nodo:** Al construir cada nodo de un árbol, en lugar de considerar el conjunto completo de J predictores, se selecciona aleatoriamente un subconjunto m de predictores (donde m < J).

- **División Óptima Restringida:** El algoritmo encuentra el mejor predictor y su umbral óptimo de división únicamente de entre los m predictores seleccionados aleatoriamente para ese nodo.

> **Caso Práctico: Diagnóstico de Tumor / Diagnóstico Cardíaco**
>
> Considerando un conjunto de datos de salud cardíaca con 5 predictores (edad, sexo biológico, frecuencia cardíaca máxima, colesterol y dolor en el pecho):
> • Nodo Raíz: Se elige aleatoriamente {edad, sexo, colesterol} -> se selecciona la mejor división entre esas 3.
> • Nodo Hijo Izquierdo: Se elige aleatoriamente {colesterol, freq. cardíaca máx., dolor en el pecho} -> se selecciona la mejor.
> • Nodo Hijo Derecho: Se elige aleatoriamente {sexo, freq. cardíaca máx., dolor en el pecho} -> se selecciona la mejor.
> Este procedimiento obliga a la arquitectura a explorar predictores secundarios, construyendo árboles verdaderamente independientes.

3. Algoritmo General de Bosques Aleatorios

El algoritmo completo de Random Forest sigue los siguientes pasos secuenciales:

1. Muestreo Bootstrap: Crear B muestras bootstrap independientes del conjunto de entrenamiento original de tamaño N con todos los J predictores.

2. Crecimiento de Árboles Independientes: Para cada muestra b = 1, ..., B, entrenar un árbol de decisión de gran profundidad sin podar, aplicando la siguiente regla en cada división de nodo:

a. Seleccionar aleatoriamente m predictores del conjunto total de J predictores.
b. Elegir el predictor óptimo y el umbral de división dentro del subconjunto m.
c. Dividir el nodo en dos nodos hijos.

3. Agregación e Inferencia: Para clasificar una nueva observación, combinar las predicciones de los B árboles mediante votación por mayoría (clasificación) o promediando las salidas continuas (regresión).

4. Hiperparámetros Clave y Reglas Prácticas de Ajuste

Para evitar una búsqueda exhaustiva en el espacio de hiperparámetros, los profesionales con amplia experiencia recomiendan reglas prácticas (rules of thumb) bien definidas para iniciar la optimización.

Hiperparámetro	Valor Recomendado (Rule of Thumb)	Propósito y Comportamiento
Cantidad de predictores por división (m)	• Clasificación: $m = \sqrt{J}$ • Regresión: $m = J / 3$	Controla la aleatoriedad y la descorrelación entre árboles. Un valor menor descorrelaciona más pero puede perder señal.

Cantidad total de árboles (B / n_estimators)	Monitoreo vía Error Out-Of-Bag (OOB)	Aumentar B no provoca sobreajuste; el error de validación/OOB disminuye y se estabiliza eventualmente.
Criterios de detención locales	Profundidad máxima (max_depth), min_samples_leaf	Limita la complejidad individual de los árboles si se requiere reducir el costo computacional.

5. Estabilización del Error e Importancia de Variables

Estabilización de la Curva de Error OOB:

A diferencia de un árbol de decisión individual donde aumentar la profundidad genera un sobreajuste severo, en los Bosques Aleatorios el incremento en el número de árboles B no sobreajusta el modelo. A medida que B aumenta, el error de validación Out-Of-Bag disminuye progresivamente hasta alcanzar una asíntota y estabilizarse. Por lo tanto, el error OOB se utiliza como criterio directo para determinar cuándo detener la adición de nuevos árboles.

Importancia de Variables (Variable Importance):

Aunque los Bosques Aleatorios pierden la interpretabilidad directa de un único árbol visual, proporcionan un mecanismo métrica robusto denominado Importancia de Variables. Esta métrica cuantifica la contribución de cada predictor registrando la disminución media en la impureza (Índice Gini o Entropía en clasificación, o MSE en regresión) acumulada a lo largo de todos los nodos y árboles donde se utilizó dicho predictor.

Importancia de Variables en Bosques Aleatorios

Evaluación de Características Mediante Disminución Media de Impureza (MDI) e Importancia por Permutación

Tabla de Contenidos

(La tabla de contenidos se actualizará automáticamente al abrir este documento en Microsoft Word o Google Docs).

1. Evaluación de Características: Modelos Paramétricos vs. Ensambles

En la regresión lineal y en la regresión logística, la relevancia de cada predictor se evalúa directamente a través de sus coeficientes estimados. Un coeficiente de magnitud elevada refleja una fuerte influencia en la predicción final. Sin embargo, en los modelos de Bosque Aleatorio (Random Forest) y ensambles no lineales basados en árboles, la estructura fragmentada y no paramétrica imposicilita la inspección de coeficientes únicos.

Para solucionar este desafío, se recurre a métricas agregadas globales que miden el aporte de cada variable a lo largo de todos los árboles del ensamble. Existen cuatro métodos principales para determinar la importancia de características en Bosques Aleatorios:

- **Disminución Media de la Impureza (MDI - Mean Decrease Impurity):** Mide la reducción promedio en la impureza (como el índice de Gini o la entropía) aportada por las divisiones basadas en una variable específica a lo largo de todo el bosque.
- **Importancia por Permutación (Permutation Importance):** Evalúa el incremento en el error del modelo al desordenar aleatoriamente los valores de una característica en el conjunto de validación o prueba.
- **Valores LIME (Local Interpretable Model-agnostic Explanations):** Método agnóstico al modelo enfocado en explicaciones locales interpretables.
- **Valores SHAP (SHapley Additive exPlanations):** Aproximación basada en la teoría de juegos cooperativos para la atribución justa del impacto de cada característica.

> **DIFERENCIA FUNDAMENTAL**
>
> A diferencia de los coeficientes lineales que indican dirección (+/-) y magnitud, el MDI mide la contribución total de la variable a la pureza estructural del árbol independientemente de la linealidad de la relación.

2. Disminución Media de la Impureza (MDI / Gini Importance)

Durante el crecimiento de un árbol de decisión, cada partición se selecciona para maximizar la pureza (o maximizar la disminución de la impureza). La Disminución Media de la Impureza (MDI) cuantifica la reducción del índice de Gini asignable a cada variable en cada nodo.

$$\Delta I(n) = (N_n / N_total) \times [\ Gini(n) - (\ (N_left / N_n) \times Gini(left) + (N_right / N_n) \times Gini(right)\ )\ ]$$

En esta formulación, n representa el nodo padre y m representa el nodo hijo. El término de la derecha corresponde al promedio ponderado de la impureza de Gini de los nodos hijos. La diferencia entre la impureza del nodo padre y el promedio de los hijos se multiplica por la proporción (N_n / N_total) que el nodo padre representa respecto al conjunto total de datos de entrenamiento.

3. Algoritmo de Cálculo de MDI Paso a Paso

Tomando como referencia un conjunto de datos clínico de diagnóstico cardíaco con 270 muestras y 5 características principales (Edad, Sexo biológico, Frecuencia cardíaca máxima, Colesterol y Dolor en el pecho), el cálculo global del MDI se estructura en cuatro fases:

Paso	Fase del Algoritmo	Descripción y Operación Matemática
Paso 1	Disminución de Impureza por Nodo	Se calcula ΔI(n) para cada nodo individual del árbol donde se realiza una división sobre la variable objetivo.
Paso 2	Suma por Característica en el Árbol	Se acumula la disminución de impureza de todos los nodos del árbol b en los que aparece la variable j: I_j(b) = Σ ΔI(n).
Paso 3	Normalización por Árbol	Se divide la suma de la característica por la suma total de las importancias de todas las variables en ese árbol para obtener un valor normalizado entre 0 y 1.
Paso 4	Promediado a Nivel de Bosque	Se promedian los valores de importancia normalizados de la variable j a través de los B árboles del ensamble: MDI_j = (1/B) Σ I_j,norm(b).

4. Hallazgos Clínicos e Importancia por Permutación

Tras la aplicación del algoritmo MDI al conjunto de datos de salud cardíaca, se constata visualmente que la frecuencia cardíaca máxima y el tipo de dolor en el pecho destacan como predictores sumamente potentes de la presencia de enfermedad cardíaca. Por el contrario, los valores de presión arterial en reposo actúan como predictores débiles dentro del ensamble.

Como alternativa complementaria, el enfoque de **Importancia por Permutación (Permutation Importance)** evalúa la degradación en el rendimiento del modelo (ej. pérdida de precisión o incremento

en el error cuadrático) al permutarse aleatoriamente las observaciones de una característica en un conjunto de datos fuera de la bolsa (OOB) o de validación. Si permutar una variable incrementa drásticamente el error, la variable se clasifica como crítica.

> **ADVERTENCIA METODOLÓGICA**
>
> MDI tiende a estar sesgado hacia variables continuas o categóricas de alta cardinalidad porque ofrecen más oportunidades de división.
>
> La Importancia por Permutación corrige este sesgo al evaluarse sobre datos no vistos y medir directamente el impacto en el rendimiento predictivo final.

Importancia de Permutación, Evaluación de Variables y Bosques Aleatorios

Guía Teórica y Práctica sobre Permutation Importance, Comparativa MDI vs. Permutación y Comportamiento Asintótico en Random Forest

Índice de Contenidos

(El índice de contenidos se actualizará automáticamente al abrir el archivo en Word o Google Docs).

1. Importancia de Permutación (Permutation Importance)

La **importancia de permutación** es una métrica de evaluación que mide la relevancia de una característica observando el impacto que tiene su alteración en el rendimiento del modelo. La intuición fundamental reside en que, si un modelo de aprendizaje automático depende fuertemente de un predictor específico para realizar sus predicciones, mezclar o permutar aleatoriamente los valores de ese predictor provocará un incremento notable en el error de predicción. Por el contrario, si la característica no es significativa para el modelo, la permutación de sus valores dejará el error de validación prácticamente inalterado.

> **INTUICIÓN DE LA IMPORTANCIA DE PERMUTACIÓN**
> • Predictor Relevante: Al permutar sus valores en el conjunto de validación/OOB, se rompe la relación entre la característica y la variable respuesta, reduciendo significativamente la precisión del modelo.
> • Predictor No Relevante: La alteración aleatoria de sus valores genera un cambio mínimo o nulo en la precisión del modelo.

1.1 Algoritmo de Cálculo Paso a Paso

Para evaluar la importancia de cada característica *j* en el conjunto de datos mediante un modelo de bosque aleatorio, se ejecuta la siguiente secuencia matemática estructurada:

Paso 1: Registro de la línea base: Se mide y registra la precisión de validación u Out-of-Bag (OOB) del modelo de Bosque Aleatorio sobre el conjunto de datos original sin modificar, denotada como s (por ejemplo, s = 0.88 en el estudio de aptitud física).

Paso 2: Permutación de la característica j: Para evitar alterar otras relaciones, únicamente se permutan aleatoriamente los valores del predictor objetivo j dentro del conjunto de validación/OOB. Se mide la nueva precisión sobre los datos modificados.

Paso 3: Repetición K veces para control de variabilidad: Para evitar sesgos derivados de una permutación particular, el proceso de mezcla se repite K veces. Se registra la precisión en cada repetición k y se calcula la precisión permutada promedio (por ejemplo, con K = 3 repeticiones se obtiene un promedio permutado de 0.85).

Paso 4: Cálculo de la diferencia de precisión: Se determina la importancia de la característica j restando la precisión permutada promedio de la precisión baseline original: Importancia(j) = s - s_permutada_promedio (en el ejemplo, 0.88 - 0.85 = 0.03).

Al representar gráficamente los resultados de importancia de permutación, las **barras de error** asociadas a cada variable reflejan el nivel de variabilidad resultante de la cantidad de repeticiones K realizadas, ofreciendo una medida de la estabilidad de la métrica.

2. Comparativa: Disminución Media de Impureza (MDI) vs. Importancia de Permutación

Tanto la Disminución Media de la Impureza (MDI / Gini Importance) como la Importancia de Permutación permiten jerarquizar las variables en Bosques Aleatorios, pero presentan diferencias fundamentales en velocidad de cálculo y sesgo hacia la cardinalidad de los predictores.

Criterio de Evaluación	Disminución Media de Impureza (MDI)	Importancia de Permutación
Eficiencia Computacional	Muy rápida: se calcula directamente durante el proceso de entrenamiento del árbol sin costo adicional.	Más costosa: requiere reevaluar el modelo en la validación K veces por cada predictor.
Sesgo de Cardinalidad	Sesgado: favorece a predictores con alta cardinalidad (muchos valores o categorías únicas), aumentando artificialmente su importancia.	No sesgado: selección más justa de predictores, independiente de su número de categorías únicas.
Mecanismo de Selección	Mide la ganancia acumulada de impureza (Gini/Entropía) en los nodos donde aparece la variable.	Mide la caída directa en la precisión de predicción al romper la estructura de la variable en validación.

Nota sobre la alta cardinalidad: MDI favorece variables con categorías numerosas porque los árboles de decisión encuentran divisiones que logran relaciones casi 1 a 1 con la variable respuesta sobre el conjunto de entrenamiento. La importancia de permutación soluciona esta limitación al evaluar la generalización sobre datos de validación, garantizando una evaluación imparcial.

3. Distribución de Importancia: Bagging vs. Random Forest

El análisis comparativo de la importancia de variables entre Bagging y Random Forest pone de manifiesto el impacto de la descorrelación de árboles:

- **Bagging y el Predicador Dominante:** En Bagging, un predictor extremadamente fuerte (como 'dolor en el pecho' en el dataset cardíaco) suele ser seleccionado sistemáticamente en el nodo raíz de casi todos los árboles debido al algoritmo voraz. Esto provoca que dicho

predictor concentre de manera desproporcionada la importancia de las características, reflejando una alta correlación entre los estimadores del conjunto.

- **Random Forest y la Distribución Uniforme:** Al restringir la división de cada nodo a un subconjunto aleatorio de predictores (m < J), Random Forest impide que la variable dominante aparezca siempre en la raíz. Esto fuerza al modelo a explorar la capacidad predictiva de otras variables (como la frecuencia cardíaca máxima o la edad), distribuyendo la importancia de forma más uniforme y balanceada.

4. Consideraciones Finales y Comportamiento Asintótico en Random Forest

4.1 Presencia Masiva de Características No Importantes (Ruido)

Cuando el conjunto de datos contiene una elevada proporción de características irrelevantes o ruida (por ejemplo, solo 5 predictores importantes de un total de 100), la probabilidad de seleccionar un predictor verdaderamente relevante en el subconjunto aleatorio *m* disminuye sensiblemente. Esto lleva a Random Forest a construir árboles basados primordialmente en características ruidosas, degradando su rendimiento predictivo.

> **SOLUCIÓN PRÁCTICA EN CIENCIA DE DATOS (DOS FASES)**
>
> 1. Aplicar Bagging inicialmente para identificar los predictores con mayor relevancia general.
> 2. Filtrar el conjunto de datos conservando únicamente las características importantes identificadas.
> 3. Entrenar el modelo final de Random Forest exclusivamente sobre el subconjunto de predictores seleccionados.

4.2 Control del Sobreajuste e Incremento del Número de Árboles

Aumentar de manera indefinida la cantidad de árboles en Random Forest **NO incrementa el riesgo de sobreajuste**. A diferencia de los hiperparámetros de profundidad o tamaño de nodo, añadir más estimadores al conjunto reduce la varianza de la predicción agregada hasta alcanzar un punto de estabilización asintótica del error de validación.

4.3 Interpretación de la Fracción de Votos como Probabilidad de Clase

Desde una perspectiva matemática rigurosa, la fracción de votos pura atribuida a una clase no es idéntica a una probabilidad calibrada de pertenencia. Sin embargo, en la práctica de aprendizaje automático, la probabilidad de clase predicha para una muestra de entrada se estima formalmente mediante la **probabilidad media de clase predicha por el conjunto de todos los árboles** del bosque aleatorio.

Introducción a los Bosques Aleatorios

Superación de la Correlación entre Árboles y Atributos Dominantes en Bagging

Resumen del Módulo

Analiza la transición conceptual desde la Agregación Bootstrap (Bagging) hacia los Bosques Aleatorios (Random Forests), identificando la limitación crítica que suponen los atributos dominantes y la alta correlación entre estimadores individuales.

Índice de Contenidos

(El índice se actualizará automáticamente al abrir el archivo en Word o Google Docs).

1. Fundamentos de Bagging (Bootstrap Aggregating)

La técnica de **Bagging (Bootstrap Aggregating)** es un método de aprendizaje en conjunto (ensemble learning) diseñado para reducir la alta varianza y prevenir el sobreajuste (*overfitting*) inherente a los árboles de decisión profundos. El procedimiento opera construyendo múltiples modelos independientes sobre diferentes subconjuntos de datos generados mediante muestreo con reemplazo.

- **Muestreo Bootstrap:** A partir de un conjunto de datos original con N observaciones, se crean B muestras aleatorias del mismo tamaño N realizando extracción con reemplazo. Esto provoca que algunas observaciones se dupliquen y un ~36.8% queden excluidas (datos Out-Of-Bag).
- **Entrenamiento Independiente:** Sobre cada muestra bootstrap se entrena un árbol de decisión profundo de forma autónoma sin podar, maximizando la expresividad del modelo.
- **Agregación de Predicciones:** Durante la inferencia, las predicciones individuales se combinan mediante voto mayoritario (en clasificación) o promedio aritmético (en regresión).

Como primer paso del Bagging, se crean múltiples subconjuntos del conjunto de datos como se muestra a continuación. Utilizamos reemplazo durante el proceso de muestreo, por lo que algunos elementos podrían aparecer más de una vez en el mismo conjunto.



Luego, se ajustan múltiples árboles de decisión a partir de cada uno de los conjuntos de datos bootstrapped.

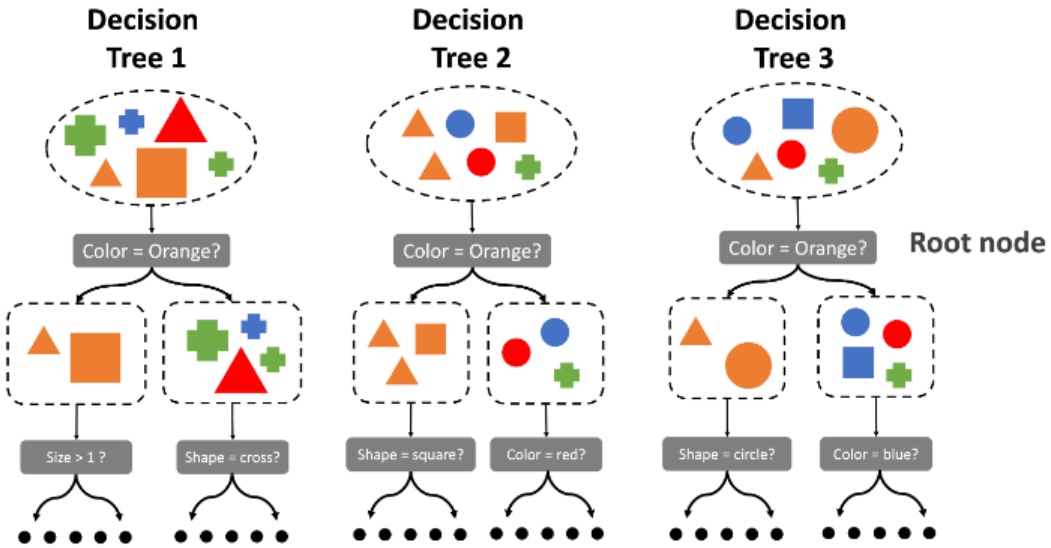


Figura 1: Arquitectura general del método Bagging (Muestreo Bootstrap → Entrenamientos independientes → Agregación final).

2. Vulnerabilidad de Bagging ante Atributos Dominantes

A pesar de la eficacia teórica de Bagging, el algoritmo presenta una limitación crítica cuando el conjunto de datos contiene un **atributo fuertemente dominante** (un predictor cuyo poder de división supera con creces al resto de las variables). En estas circunstancias, la diversidad estructural del ensamble se colapsa.

4.1.1 Introducción a los Bosques Aleatorios

¿Por qué Bosques Aleatorios?

Motivación

En nuestra última sección, exploramos varios obstáculos típicos encontrados en tareas de clasificación, incluyendo alta varianza y sobreajuste. Además, también aprendimos el concepto de bagging, una robusta técnica de aprendizaje en conjunto destinada a mitigar estos desafíos. Sin embargo, una pregunta que surgió al final de la última sesión es si el bagging puede verse afectado cuando una sola característica domina. Es una preocupación válida y que debemos considerar. Si una sola característica pesa más que las demás, los modelos creados pueden terminar haciendo predicciones similares, lo que resulta en una falta de diversidad en las predicciones finales. En resumen, aunque el bagging es un enfoque efectivo para mitigar la alta varianza y el sobreajuste, debemos ser cautelosos sobre el impacto potencial de una característica dominante. Es importante considerar el uso de técnicas para mitigar este problema, lo cual será el enfoque de las próximas sesiones.

Demos un ejemplo del tipo de problema que esperamos resolver. Considera un conjunto de datos de formas geométricas con características de *Color*, *Forma* y *Tamaño*. Queremos utilizar el método de aprendizaje en conjunto, Bagging (Agregación de Bootstrap).

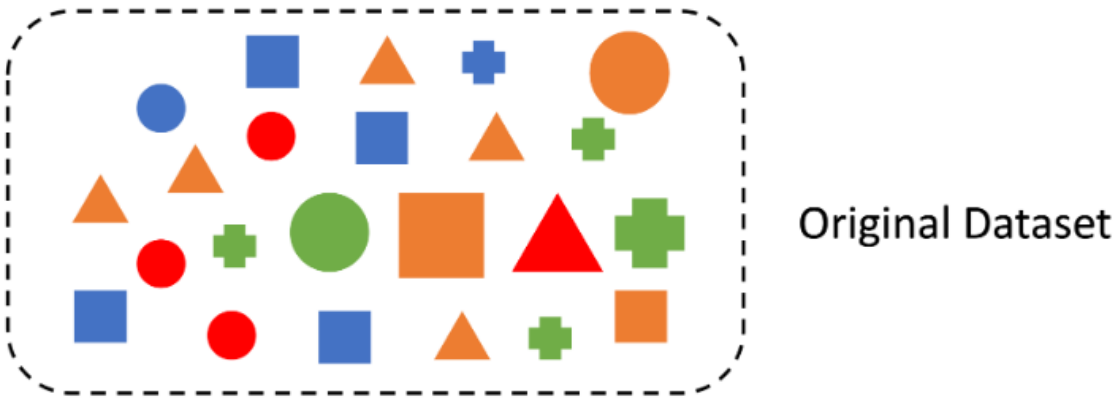


Figura 2: Impacto de las características dominantes en el aprendizaje en conjunto.

Efecto de la Característica Dominante

Cuando un predictor es extremadamente fuerte, el algoritmo voraz (greedy algorithm) elegirá siempre dicha variable para realizar la división inicial en la raíz de casi todos los árboles del ensamble. Esto provoca que los árboles generados compartan una estructura casi idéntica.

3. El Problema de la Correlación entre Árboles (ρ > 0)

Considérese un conjunto de datos geométricos con características de *color, forma y tamaño*. Al aplicar Bagging, se generan múltiples subconjuntos bootstrap y se entrena un árbol sobre cada uno. Si la característica 'Color' es dominante (p. ej., distinguir naranja vs. otros colores), el resultado se ilustra a continuación:

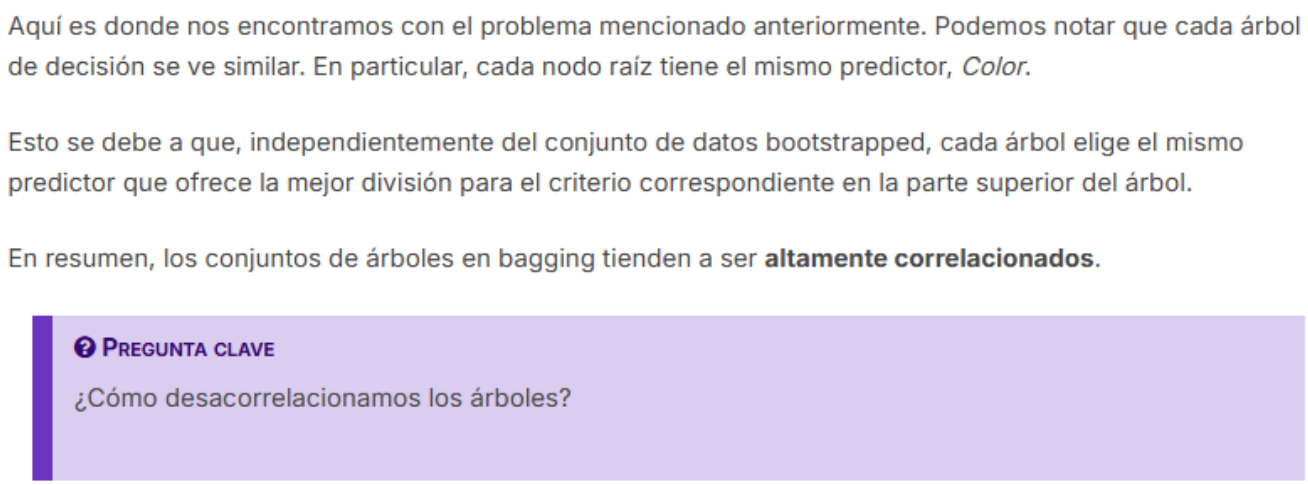


Figura 3: Repetición sistemática del nodo raíz (Color = Orange?) en todos los árboles de Bagging debido a un predictor dominante.

Desde el punto de vista estadístico, la varianza del promedio de B variables aleatorias idénticamente distribuidas, cada una con varianza σ^2 y correlación par a par ρ, viene dada por la fórmula:

> **Formulación de la Varianza del Ensamble**
>
> $\text{Var}(\bar{X}) = \rho \cdot \sigma^2 + ((1 - \rho) / B) \cdot \sigma^2$
>
> • Si $\rho = 0$ (árboles completamente independientes): a medida que $B \to \infty$, el segundo término tiende a cero y $\text{Var}(\bar{X}) \to 0$.
> • Si $\rho > 0$ (árboles altamente correlacionados): el primer término ($\rho \cdot \sigma^2$) actúa como un límite inferior infranqueable. Por muchos árboles B que agreguemos, la varianza del ensamble no se podrá reducir por debajo de $\rho \cdot \sigma^2$.

4. Solución: Descorrelación mediante Bosques Aleatorios

Para solucionar la dependencia estructural y forzar la diversidad entre los árboles, **Random Forests** introduce una modificación fundamental sobre el algoritmo de Bagging: el submuestreo aleatorio de características (*Random Subspace Method*).

Aspecto de Comparación	Bagging Estándar	Bosque Aleatorio (Random Forest)
Selección de Predictores en Nodos	Considera TODOS los J predictores disponibles en cada división.	Evalúa solo un subconjunto aleatorio m de predictores (m < J).
Estructura del Nodo Raíz	Idéntica en la mayoría de árboles si existe un predictor dominante.	Variada y diversa, al obligar al árbol a evaluar otras variables.
Correlación entre Árboles (ρ)	Alta (ρ > 0), limitando la reducción total de la varianza.	Baja (ρ ≈ 0), logrando una reducción óptima de la varianza del ensamble.

Regla Práctica para la Selección de m

En la práctica, el tamaño del subconjunto de características m se configura convencionalmente como:
• Clasificación: $m = \sqrt{J}$ (raíz cuadrada del total de predictores).
• Regresión: $m = J / 3$ (un tercio del total de predictores).

Esta restricción obliga a los árboles a explorar predictores secundarios, construyendo una representación verdaderamente diversa y descorrelacionada.

Fundamentos y Arquitectura de Bosques Aleatorios

Sección 4.1.2 — Subespacios Aleatorios de Predictores y Agregación por Votación

---

Índice de Contenidos

(Nota: En Microsoft Word o Google Docs, actualice el campo para visualizar el número de páginas exacto).

1. Definición y Principio Fundamental de Random Forest

El método de **Bosque Aleatorio (Random Forest)** es una técnica modificada de aprendizaje por conjuntos (ensemble learning) basada en *bagging* que genera conjuntos de árboles de decisión altamente independientes.

En el *bagging convencional*, cada árbol evalúa la totalidad de los **J** predictores disponibles para seleccionar el corte óptimo en cada nodo. Esto provoca que si existe una variable excesivamente dominante, todos los árboles la elijan de forma recurrente en su nodo raíz, generando modelos fuertemente correlacionados. **Random Forest resuelve esta limitación mediante el submuestreo aleatorio de predictores:** en cada división de cada nodo, el algoritmo restringe la búsqueda a un subconjunto aleatorio de **j** variables (donde **j < J**), forzando a que los árboles exploren combinaciones diversas de características.

> **Regla Clave de Descorrelación:** *Al restringir las variables disponibles a un subconjunto aleatorio j en cada nodo, la variable dominante no siempre estará disponible. Esto permite que otros predictores moderados sean elegidos como divisores principales, garantizando que la estructura de los árboles sea diversa e independiente.*

2. Ejemplo Práctico: Selección Aleatoria en el Diagnóstico Cardiaco

Considérese un conjunto de datos clínico para el diagnóstico de cardiopatías con **5 características clave (J = 5)**: *Edad, Sexo, Frecuencia Cardíaca Máxima (FC Máxima), Colesterol y Dolor en el Pecho.*

1. Muestra Bootstrap inicial (B_1): Se extrae una muestra con reemplazo de los datos de entrenamiento conteniendo todos los predictores originales.
2. Selección de subconjunto aleatorio en la raíz: En lugar de evaluar los 5 predictores, el algoritmo selecciona aleatoriamente un subconjunto de 3 variables (j = 3): {Edad, Sexo, Colesterol}.

3. División óptima restringida: De esos 3 predictores, la variable con mayor criterio de pureza resulta ser Sexo, estableciendo la primera división del árbol.
4. Aleatorización en nodos descendientes: Para el nodo hijo izquierdo, se selecciona aleatoriamente un nuevo subconjunto {Colesterol, FC Máxima, Dolor en el pecho}; para el derecho, {Sexo, FC Máxima, Dolor en el pecho}. Este proceso se repite en cada división.

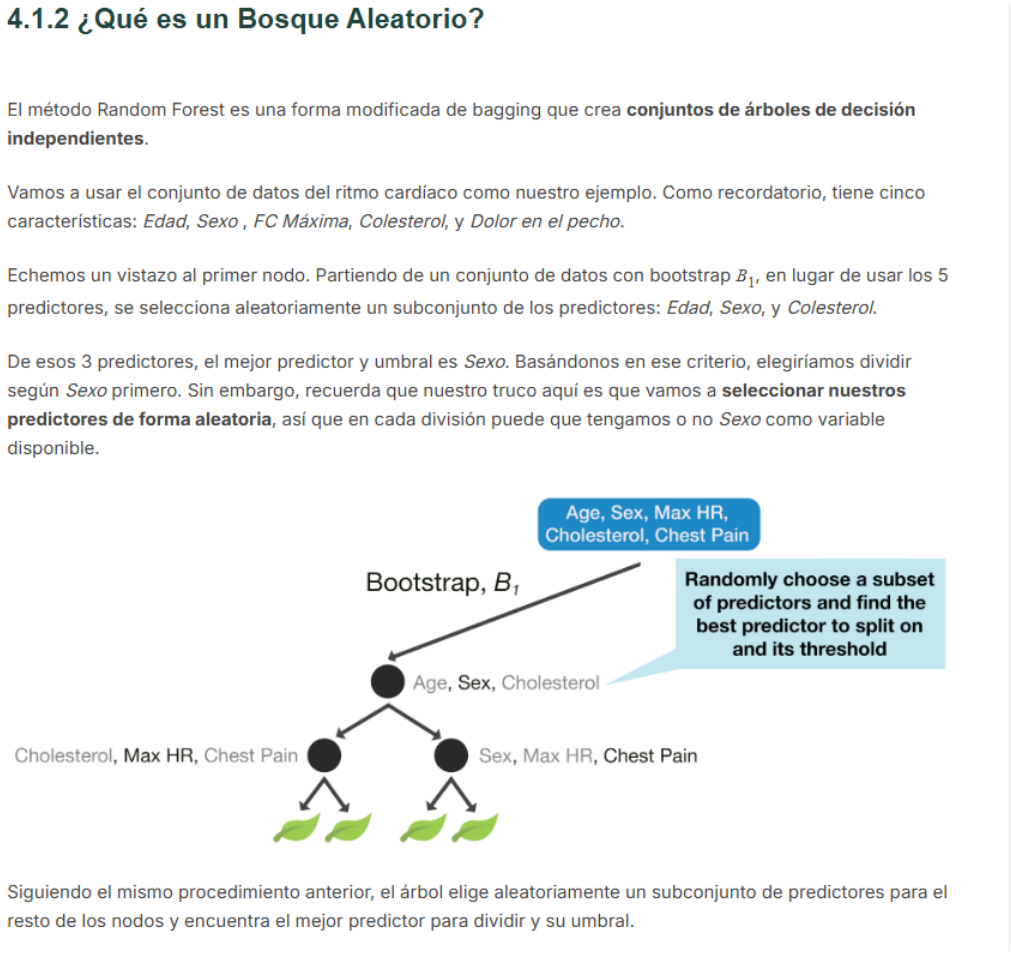


Figura 1: Proceso de selección aleatoria del subconjunto de predictores en cada nodo para la muestra Bootstrap B_1.

3. Agregación de Predicciones por Votación de Mayoría

Una vez entrenados los **T** árboles de decisión independientes a partir de sus respectivas muestras bootstrap (B_1, B_2, ..., B_T), la inferencia para un nuevo paciente se realiza agregando sus respuestas individuales.

En problemas de clasificación, el resultado del bosque aleatorio se obtiene aplicando el principio de **votación por mayoría (majority voting)**:

- **Árbol 1 (B_1):** Predice Heart Disease (Presencia de enfermedad cardíaca).
- **Árbol 2 (B_2):** Predice No Heart Disease (Ausencia de enfermedad cardíaca).
- **Árbol 3 (B_3):** Predice No Heart Disease (Ausencia de enfermedad cardíaca).

Decisión del Ensamble: Dado que 2 de los 3 árboles votaron por la ausencia de enfermedad, la predicción agregada del modelo final es **No Heart Disease**.

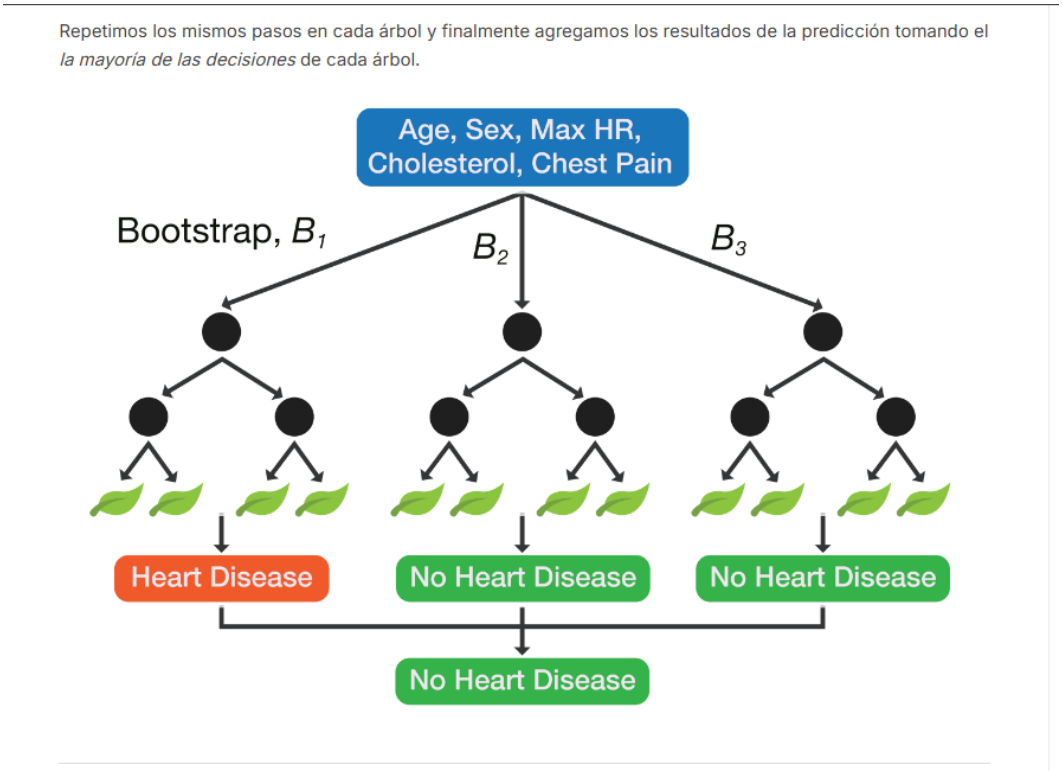


Figura 2: Esquema de agregación mediante votación de mayoría para clasificar la presencia de cardiopatía.

4. Algoritmo General de Random Forest Paso a Paso

El procedimiento formal para la construcción e inferencia de un modelo Random Forest se estructura en los siguientes cuatro pasos secuenciales:

Paso	Descripción del Algoritmo
Paso 1	Muestreo Bootstrap: Crear conjuntos de datos bootstrap (B_1, B_2, ..., B_T) muestreando con reemplazo a partir de los datos originales, manteniendo los J predictores iniciales.
Paso 2	Inicialización: Configurar e inicializar un bosque aleatorio compuesto por T árboles de decisión independientes.
Paso 3	Subespacio Aleatorio de Características: Para cada árbol y en cada corte de nodo individual, seleccionar aleatoriamente un subconjunto j de los J predictores totales (j < J).
Paso 4 & Repetición	División Óptima y Recursión: Entre los j predictores seleccionados, determinar la variable óptima y el umbral de corte ideal. Repetir recursivamente en cada nodo hasta cumplir los criterios de detención.

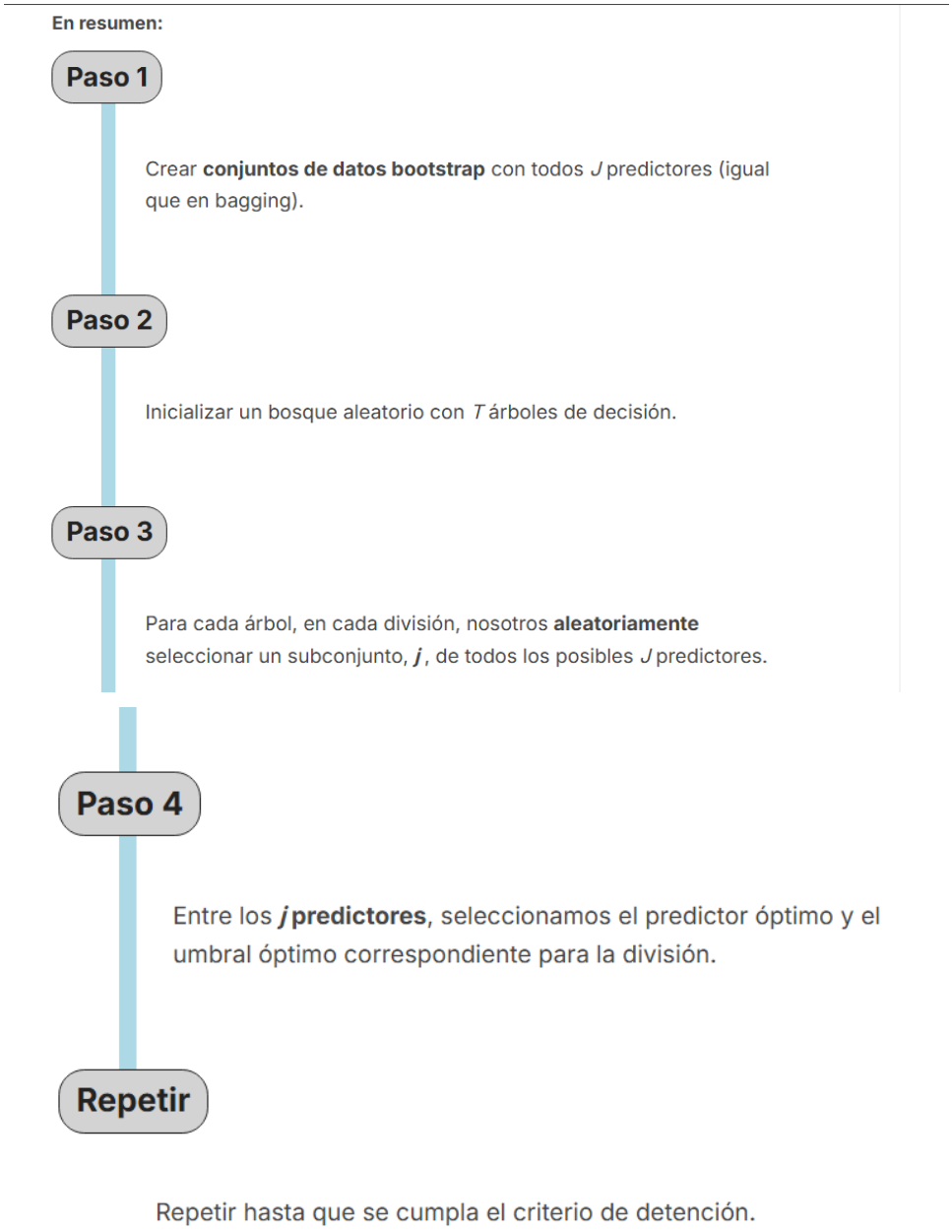


Figura 3: Resumen visual de la secuencia algorítmica de 4 pasos para la construcción de Random Forests.

5. Cuadro Comparativo: Bagging Estándar frente a Random Forest

La siguiente tabla sintetiza las diferencias operativas clave entre la agregación bootstrap tradicional y el algoritmo Random Forest:

Dimensión Técica	Bagging Estándar	Random Forest
Muestreo de Datos	Bootstrap con reemplazo (B muestras de tamaño N)	Bootstrap con reemplazo (B muestras de tamaño N)

Predictores por Nodo	Todos los J predictores disponibles en cada corte	Subconjunto aleatorio j < J en cada corte
Correlación de Árboles	Alta si existe una variable dominante en el dataset	Baja (árboles descorrelacionados)
Reducción de Varianza	Limitada por la covarianza positiva entre estimadores	Máxima reducción de varianza sin incrementar sesgo

4.1.3 Ajuste de Bosques Aleatorios

Optimización de Hiperparámetros, Reglas Empíricas y Estimación Out-Of-Bag (OOB)

Índice de Contenidos

(El índice de contenidos se actualizará automáticamente al abrir el archivo en Microsoft Word o Google Docs).

1. Hiperparámetros Clave en Bosques Aleatorios

A diferencia de los árboles de decisión individuales que se construyen mediante un algoritmo voraz determinista, los modelos de **Bosque Aleatorio (Random Forest)** dependen de múltiples hiperparámetros de estructura y ensamble que deben configurarse adecuadamente para equilibrar la capacidad expresiva y la varianza del modelo. Los tres hiperparámetros fundamentales son:

- **Número de predictores por división (m o N_pred):** Representa la cantidad de características seleccionadas de manera aleatoria en cada nodo para evaluar la mejor partición. Es el parámetro principal para descorrelacionar los árboles del ensamble.
- **Número total de árboles en el conjunto (B o N_trees):** Determina cuántos estimadores individuales componen el bosque. Un número suficientemente alto garantiza la estabilización del promedio y reduce la varianza de la predicción final sin causar sobreajuste.
- **Criterios de parada locales y estructurales:** Incluyen la profundidad máxima del árbol (max_depth), el número mínimo de muestras para permitir una división (min_samples_split) y el tamaño mínimo de los nodos hoja (min_samples_leaf).

> **Recomendación Práctica de Configuración:**
> Existen valores predeterminados estándar respaldados por la literatura técnica y la práctica profesional. Sin embargo, dado que el comportamiento óptimo depende estrictamente de la distribución y dimensionalidad del conjunto de datos específico, se recomienda optimizar estos parámetros mediante el método **Fuera de la Bolsa (Out-Of-Bag / OOB)**

2. Reglas Empíricas para la Selección de Predictores

Para determinar la cantidad de características m que deben evaluarse al azar en cada división de nodo, si N (o J) es el número total de predictores disponibles en el dataset, se utilizan las siguientes fórmulas estándar como punto de partida:

Tipo de Problema	Regla Estándar (m)	Justificación Teórica
Clasificación	**$m = \sqrt{N}$**	Mantiene baja correlación entre árboles al restringir severamente los candidatos en datasets con muchas variables.
Regresión	**$m = N / 3$**	Permite considerar un subconjunto mayor de variables continuas para capturar variaciones cuantitativas suaves.

Por ejemplo, en un conjunto de datos con **N = 100 predictores**, un clasificador de Bosque Aleatorio evaluará solo **m = 10 predictores** elegidos al azar en cada nodo, mientras que un modelo de regresión evaluará aproximadamente **m = 33 predictores**.

3. Validación y Detención Mediante Error Out-Of-Bag (OOB)

El error **Out-Of-Bag (OOB)** es una métrica interna de evaluación que aprovecha el proceso de muestreo bootstrap. Dado que cada árbol se entrena utilizando aproximadamente el 63.2% de los datos originales, el 36.8% restante de las observaciones queda fuera de la muestra de entrenamiento (datos OOB) para dicho árbol específico.

Ajuste del Número de Árboles (B) con OOB

- **Entrenamiento y Validación Integrados:** Con el método OOB, el entrenamiento y la validación cruzada se realizan en una sola secuencia de ejecución, eliminando la necesidad de reservar un conjunto de validación separado.
- **Criterio de Estabilización del Error:** A medida que se añaden árboles al ensamble, el error OOB disminuye progresivamente hasta alcanzar un valor asintótico. La regla de parada consiste en interrumpir el entrenamiento una vez que la curva del error fuera de la bolsa se estabiliza.
- **Inexistencia de Sobreajuste por Número de Árboles:** Aumentar el número de árboles B no genera sobreajuste en el ensamble; únicamente incrementa el costo computacional sin alterar negativamente el error de generalización.

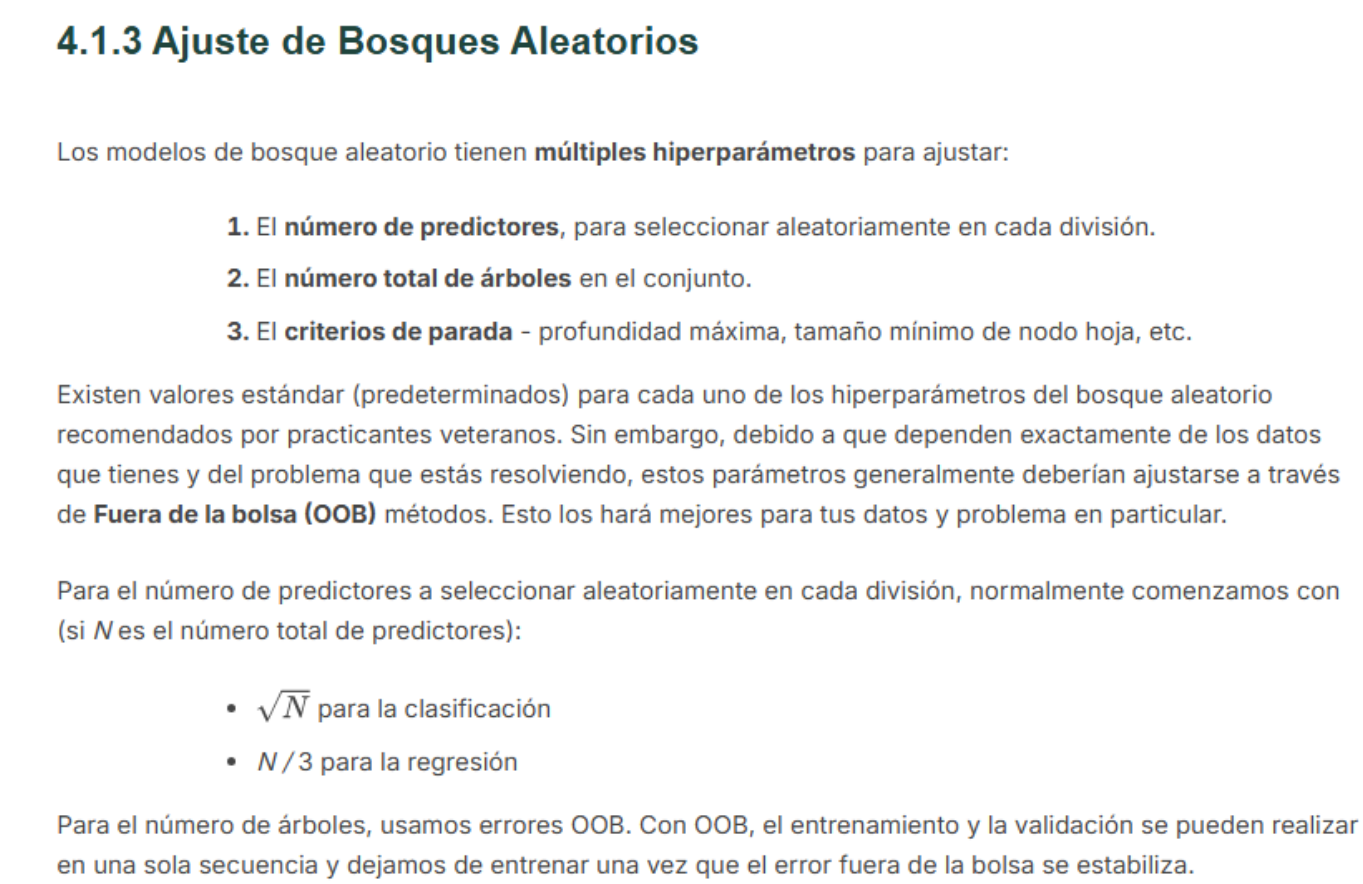
4. Resumen de Parámetros y Estrategias de Optimización

Hiperparámetro	Valor Inicial Predeterminado	Estrategia de Optimización OOB
Número de Predictores (m)	$\sqrt{N}$ (Clasificación) / N/3 (Regresión)	Ajustar evaluando una rejilla de valores pequeños alrededor de $\sqrt{N}$ o

		N/3 utilizando el error OOB.
Número de Árboles (B)	100 - 500 árboles	Incrementar B secuencialmente hasta que el error OOB se estabilice en una meseta.
Criterios de Parada Locales	Arboles profundos (max_depth = None)	Controlar min_samples_leaf o max_depth si se requiere reducir el tamaño en memoria o el tiempo de inferencia.

5. Material Gráfico de Origen

A continuación se presenta la lámina original correspondiente a la sección **4.1.3 Ajuste de Bosques Aleatorios**, que resume de manera esquemática las fórmulas y principios expuestos:



4.1.3 Ajuste de Bosques Aleatorios

Los modelos de bosque aleatorio tienen **múltiples hiperparámetros** para ajustar:

1. El **número de predictores**, para seleccionar aleatoriamente en cada división.
2. El **número total de árboles** en el conjunto.
3. El **criterios de parada** - profundidad máxima, tamaño mínimo de nodo hoja, etc.

Existen valores estándar (predeterminados) para cada uno de los hiperparámetros del bosque aleatorio recomendados por practicantes veteranos. Sin embargo, debido a que dependen exactamente de los datos que tienes y del problema que estás resolviendo, estos parámetros generalmente deberían ajustarse a través de **Fuera de la bolsa (OOB)** métodos. Esto los hará mejores para tus datos y problema en particular.

Para el número de predictores a seleccionar aleatoriamente en cada división, normalmente comenzamos con (si N es el número total de predictores):

- $\sqrt{N}$ para la clasificación
- $N/3$ para la regresión

Para el número de árboles, usamos errores OOB. Con OOB, el entrenamiento y la validación se pueden realizar en una sola secuencia y dejamos de entrenar una vez que el error fuera de la bolsa se estabiliza.

Figura 4.1: Lámina explicativa sobre el Ajuste de Hiperparámetros y Estimación OOB en Bosques Aleatorios.

SECCIÓN 4.1.4

Importancia de Variables en Random Forest

Resumen Clave: A diferencia de los modelos lineales que evalúan importancia mediante coeficientes, Random Forest la determina evaluando cuánto contribuye cada variable a la pureza del árbol (reducción de impureza Gini o ganancia de información). Existan cuatro métodos principales para su cuantificación.

1. Evaluación de Importancia: Modelos Lineales vs. Random Forest

En algoritmos paramétricos tradicionales como la **regresión lineal y regresión logística**, la relevancia de cada característica se evalúa directamente a través de la magnitud de sus coeficientes asociados. Un coeficiente con un valor absoluto elevado indica un fuerte impacto sobre la variable de respuesta, asumiendo que las variables han sido previamente estandarizadas.

En **Random Forest**, al ser un modelo no paramétrico basado en ensambles de árboles de decisión, la importancia no puede derivarse de coeficientes lineales. En su lugar, se mide la contribución de cada variable a la pureza u homogeneidad global del modelo, analizando cómo las particiones generadas por esa variable reducen la incertidumbre en los nodos de los árboles componentes.

2. Criterios de Pureza e Impureza en la Selección de Características

El cálculo de la importancia se fundamenta en la capacidad de las características para separar eficazmente las clases o reducir la varianza de la variable objetivo. Las métricas primarias empleadas son:

- **Impureza Gini (Gini Impurity):** Mide la probabilidad de clasificar incorrectamente un elemento elegido al azar si se etiquetara aleatoriamente según la distribución de clases del nodo. Una mayor reducción acumulada en el índice de Gini asigna mayor jerarquía a la característica.
- **Ganancia de Información (Information Gain / Entropía):** Cuantifica la disminución en la entropía (grado de desorden o incertidumbre) tras realizar la división por un atributo específico a lo largo de todos los árboles del bosque.

3. Métodos Principales para Cuantificar la Importancia

Se identifican cuatro enfoques clave para medir la relevancia de las variables en arquitecturas de Random Forest, estructurados según su nivel de granularidad e interpretabilidad:

Método de Importancia	Enfoque / Métrica	Ámbito de Aplicación
1. Disminución Media de la	Suma acumulada de reducciones de	Métrica nativa global basada en la

Impureza (MDI)	impureza (Gini/Entropía) en todos los nodos divididos por la variable.	estructura del árbol durante el entrenamiento.
2. Importancia por Permutación (PFI)	Evaluación de la caída en el rendimiento (precisión/OOB) tras permutar aleatoriamente los valores del atributo.	Métrica global post-entrenamiento e insensible al sesgo de cardinalidad.
3. LIME (Local Interpretable Model-agnostic Explanations)	Aproximación lineal local alrededor de una predicción individual específica.	Explicabilidad local modelo-agnóstica para instancias individuales.
4. SHAP (Shapley Additive exPlanations)	Cálculo de valores Shapley basados en teoría de juegos cooperativos.	Explicabilidad global y local teóricamente fundamentada y consistente.

4. Material Gráfico Adjunto de Referencia

A continuación se integra la captura del texto original de la lección (4.1.4 Importancia de Variables para Random Forest):

4.1.4 Importancia de Variables para Random Forest

En la regresión lineal y logística, evaluamos la importancia de las características analizando sus respectivos coeficientes, donde un valor grande del coeficiente indica un alto nivel de importancia para la característica correspondiente. Sin embargo, en Random Forest, la importancia de las características se determina típicamente en función de métricas como la impureza Gini o la ganancia de información, que miden hasta qué punto cada característica contribuye a la pureza u homogeneidad general de los árboles de decisión en el bosque. Se consideran más importantes aquellas características que conducen a mayores reducciones en la impureza cuando se utilizan en los árboles de decisión.

Existen cuatro métodos principales para determinar la importancia de las variables en Random Forest. Aunque no profundizaremos en los métodos 3 y 4 aquí, a continuación encontrarás enlaces que puedes seguir para obtener más información.

1. Disminución Media de la Impureza (a continuación).
2. Importancia por Permutación (más adelante en esta sección).
3. LIME (Explicaciones Locales Interpretables Agnósticas al Modelo).
4. SHAP (Explicaciones Aditivas de SHapley).

4.1.5 Disminución Media de la Impureza (MDI)

Cálculo Paso a Paso, Formulación Matemática, Ejemplo Práctico y Ranking de Importancia de Variables en Random Forests

---

Índice de Contenidos

(El índice numérico de páginas se actualiza automáticamente al abrir el archivo en Microsoft Word o Google Docs).

1. Concepto Fundamental de la Disminución Media de la Impureza (MDI)

Los árboles de decisión construyen sus divisiones (*splits*) seleccionando iterativamente las características y umbrales que maximizan la disminución de impureza en cada nodo. Al calcular el promediado de esta disminución a lo largo de todos los árboles del modelo, se obtiene la métrica de **Importancia de Variables por Disminución Media de la Impureza (MDI)** (conocida en inglés como *Mean Decrease in Impurity* o *Gini Importance*). Este método es aplicable idénticamente tanto a modelos de **Bagging** como a **Random Forests**.

> **Idea Clave del Método MDI:**
>
> 1. Cada división en un árbol reduce la impureza del nodo padre respecto a los nodos hijos.
> 2. La ganancia generada por una variable se acumula en todos los nodos donde dicha variable es seleccionada.
> 3. El valor final a nivel de Bosque Aleatorio es el promedio de las importancias normalizadas de cada árbol individual.

Para ilustrar el procedimiento numérico completo, se toma como referencia un árbol de decisión entrenado sobre el conjunto de datos de **frecuencia cardíaca** (270 muestras), compuesto por 5 características clave: *edad (Age)*, *sexo biológico (Sex)*, *presión arterial en reposo / frecuencia cardíaca (BP / MaxHR)*, *colesterol (Cholesterol)* y *dolor en el pecho (ChestPain)*.

4.1.5 Disminución Media de la Impureza (MDI)

Decision trees make splits that maximize the decrease in impurity. By calculating the mean decrease in impurity for each feature across all trees, we arrive at the variable importance for a random forest model. Note that this approach works the same for bagging models.

Consider the following decision tree, once again trained on the heart rate dataset, which comprises 5 features: age, sex, maximum heart rate, cholesterol, and chest pain.

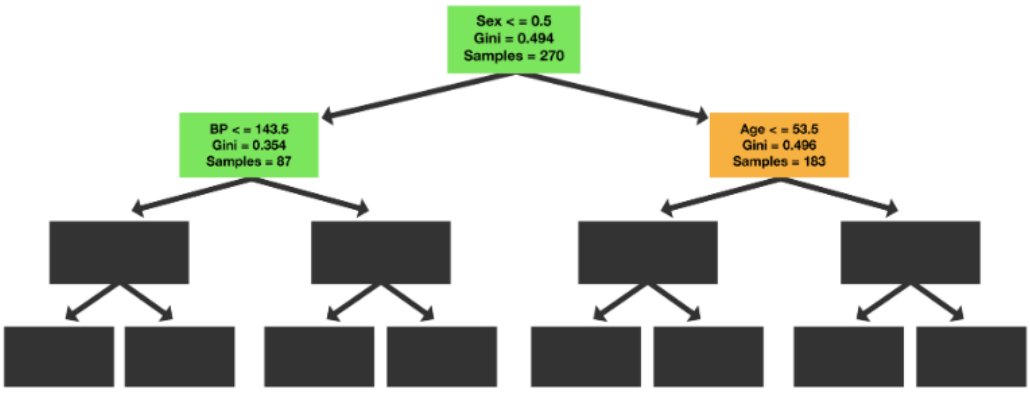


Figura 1: Estructura inicial del árbol de decisión sobre el conjunto de datos de salud cardíaca (270 muestras).

2. Procedimiento Algorítmico Paso a Paso para el Cálculo de MDI

El cálculo formal de la importancia MDI sigue una secuencia de 4 pasos estructurados, desde la evaluación de la impureza en un nodo individual hasta la agregación global en todo el bosque aleatorio.

Paso 1: Cálculo de la Disminución de Impureza por Nodo (ΔI_n)

Para cada nodo interno *n* en el árbol de decisión, la disminución ponderada de la impureza **ΔI_n** se calcula comparando el índice de Gini del nodo padre con la suma ponderada del índice de Gini de sus nodos hijos:

> **Formulación Matemática de la Disminución de Impureza por Nodo:**
>
> **ΔI_n = (n / N) * [Gini_n - Σ_{m ∈ Child(n)} (m / n) * Gini_m]**
>
> *Donde:*
>
> • ΔI_n: Disminución ponderada de la impureza en el nodo n.
>
> • (n / N): Proporción de muestras en el nodo n respecto al total de datos del conjunto de entrenamiento N.
>
> • Gini_n: Índice de impureza de Gini en el nodo padre n.
>
> • (m / n): Fracción de muestras que migran al nodo hijo m respecto al nodo padre n.
>
> • Gini_m: Índice de impureza de Gini en el nodo hijo m.

Ejemplo numérico en el primer nodo (Nodo Raíz - Sex <= 0.5):
En la raíz, *n = 270* y *N = 270* (fracción = 1). El valor inicial de Gini es *0.494*. La división divide las muestras en una rama izquierda con *m_izq = 87* (Gini = 0.354) y una rama derecha con *m_der = 183* (Gini = 0.496):

> **Cálculo del Nodo Raíz (Sex):**
>
> **ΔI_raiz = (270 / 270) * [0.494 - (87 / 270) * 0.354 - (183 / 270) * 0.496]**
>
> ***ΔI_raiz = 1.0 * [0.494 - 0.1141 - 0.3362] = 0.044***

A medida que el árbol continúa dividiéndose (por ejemplo, en el nodo de *Edad* o *Colesterol*), la fracción de muestras respecto al dataset completo (*n/N*) se va reduciendo proporcionalmente, ajustando el peso relativo de cada ganancia local de impureza.

Step 1

Calculate the mean decrease in impurity for each node n in the decision tree as:

$$\Delta I_n = \left(\frac{n}{N}\right)\left[Gini_n - \sum_{m \in \text{Child}(n)} \left(\frac{m}{n}\right) Gini_m\right]$$

where:

ΔI_n represents the mean decrease in impurity for each node n

$\left(\frac{n}{N}\right)$ is the fraction of n samples from the node out of the whole dataset

$\sum_{m \in \text{Child}(n)}$ is the sum over all children of none n

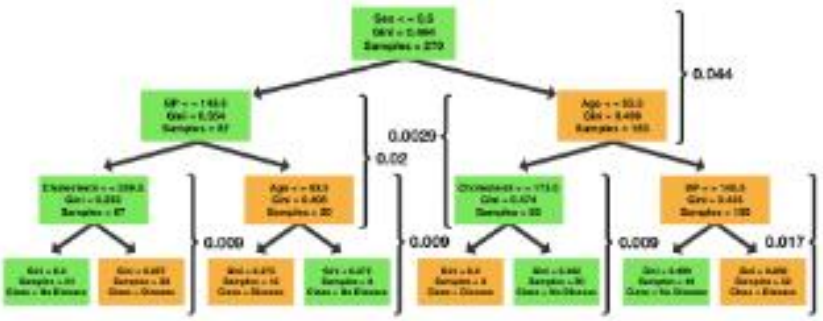
For example, the decrease in impurity for the first node:



$$\frac{270}{270}\left[0.494 - \frac{87}{270}0.354 - \frac{183}{270}0.496\right] = 0.0044$$

We then proceed to the next step. In this example, we split based on the "age" feature. Everything remains the same as before, but it's important to note that the fraction of the parent node out of the entire dataset changes as we continue the split.

Again, we follow the same procedure as before, with the target predictor to split being "cholesterol" in this case. We continue this process for all features, and finally, we obtain the mean decrease in impurity for each node.



Now, let us try to compute the importance of the feature

Figura 2: Paso 1 del algoritmo: cálculo de la disminución de impureza ΔI_n por nodo y ejemplo en la raíz.

Paso 2: Acumulación de la Importancia por Característica en el Árbol (FI_j)

La importancia no normalizada de una característica determinada *j* en un único árbol de decisión (**FI_j**) se obtiene sumando la disminución de impureza *ΔI_n* de todos los nodos *n* donde se utiliza dicha característica para realizar la partición:

> **Acumulación para una Característica (Ejemplo: Colesterol):**
>
> **FI_{Cholesterol} = Σ_{n ∈ nodos divididos por Cholesterol} ΔI_n**
>
> En el árbol del ejemplo, el Colesterol aparece en dos nodos de división con ganancias de 0.009 y 0.008:
>
> **FI_{Cholesterol} = 0.009 + 0.008 = 0.017**

Paso 3: Normalización de Importancias a Nivel de Árbol (Norm FI_j)

Para garantizar la comparabilidad entre características dentro de un árbol, se divide el valor acumulado *FI_j* por la suma total de las importancias de todas las características del árbol. Esto asegura que la suma de todas las importancias en el árbol sume exactamente 1 (o 100%):

> **Fórmula de Normalización en un Árbol Individual:**
>
> **Norm FI_{Cholesterol} = FI_{Cholesterol} / [Σ_{j ∈ todas las características} FI_j]**

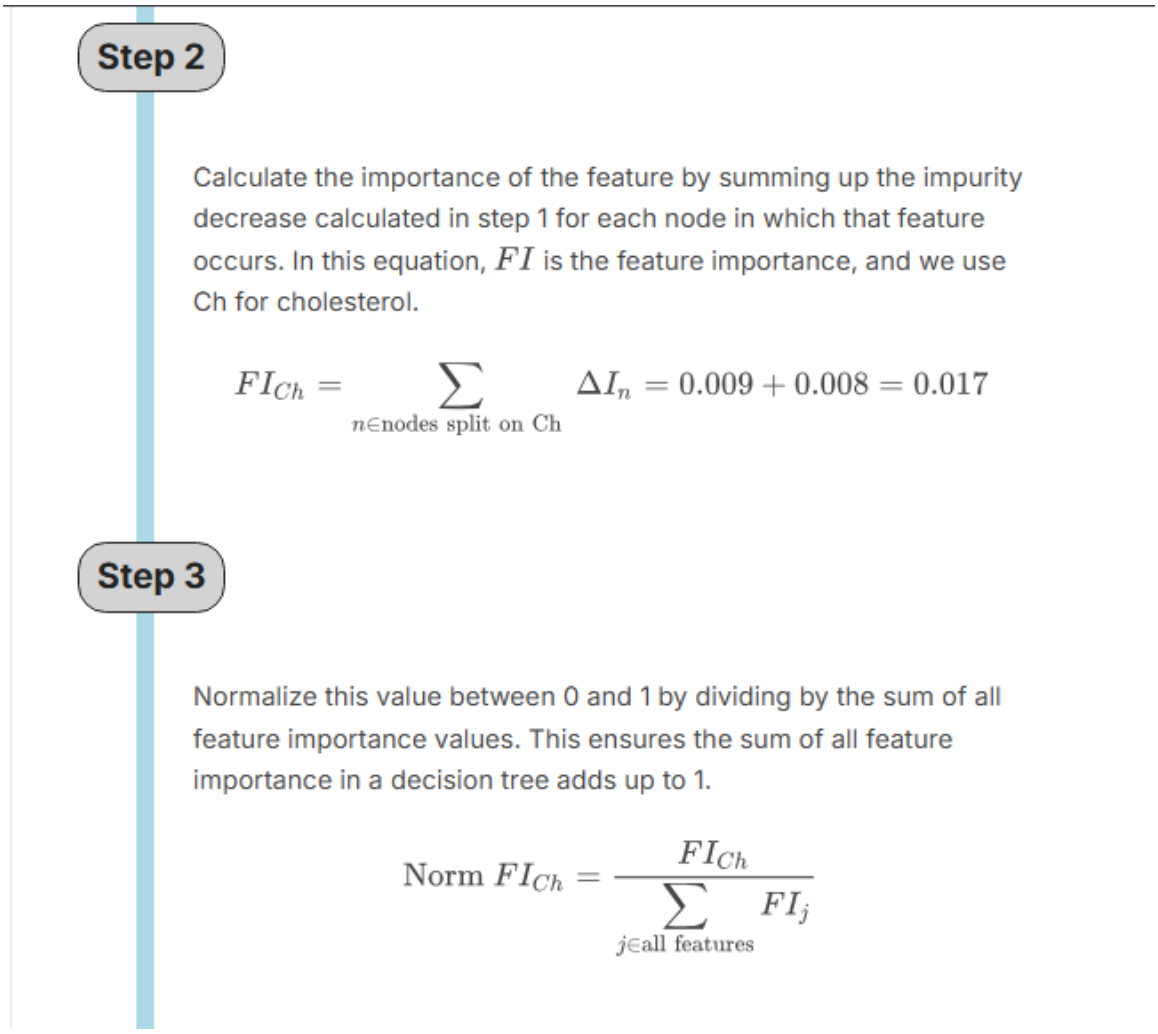


Figura 3: Pasos 2 y 3 del algoritmo: suma de ganancias por variable y normalización unitaria.

Paso 4: Promediado Global a Nivel de Ensamble (Random Forest / Bagging)

Hasta el Paso 3, la métrica solo evalúa un árbol individual. Para extender el cálculo a nivel de **Random Forest** o **Bagging**, se calcula la media aritmética de la importancia normalizada del predictor a lo largo de los *B* árboles que componen el ensamble:

> **Fórmula de MDI a Nivel de Bosque Aleatorio:**
>
> **Norm RF / Bagging FI_j = (Σ_{t ∈ todos los árboles} Norm FI_{j, t}) / [Número de árboles]**
>
> Donde t representa cada árbol entrenado sobre una muestra bootstrap y Norm FI_{j,t} es la importancia normalizada de la característica j en el árbol t.

Step 4

Up to step 3, we only consider the feature importance at a single tree level. To calculate the feature importance at the **Random Forest or Bagging level**, we average the normalized feature importance of the given predictor over all the trees.

$$\begin{matrix}\text{Norm RF} \\ \text{or} \\ \text{Bagging FI}_{Ch}\end{matrix} = \frac{\sum_{t \in \text{all features}} \text{Norm}FI_t}{\text{Number of trees}}$$

Finally

Finally, the overall feature importance for each feature would look like this:

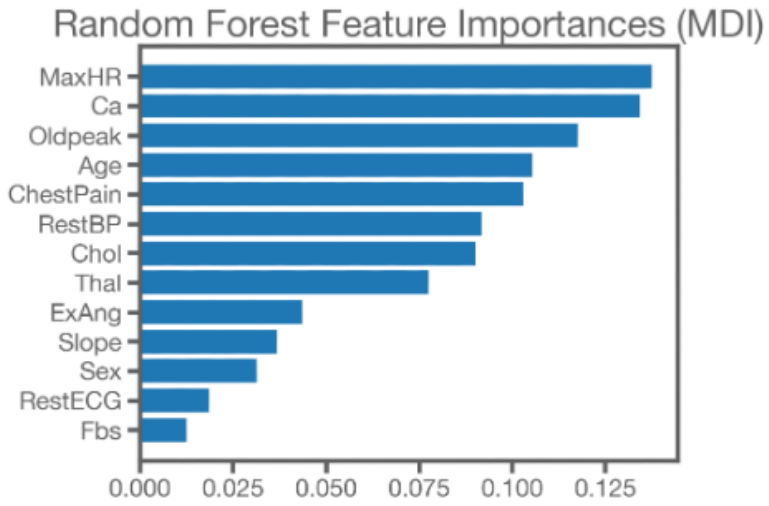


Figura 4: Paso 4 del algoritmo y gráfico final de importancia de características MDI en Random Forest.

3. Interpretación de Resultados y Ranking de Predictoras

El gráfico de barras MDI resultante proporciona una interpretación clara de cuáles son las variables más determinantes para la predicción del diagnóstico cardíaco:

- **Predictores de Máximo Impacto:** Frecuencia Cardíaca Máxima (MaxHR ~ 0.13), Ca (número de vasos principales), Oldpeak, Edad (Age) y Dolor en el pecho (ChestPain) encabezan la importancia con valores MDI superiores a 0.09.
- **Predictores de Impacto Moderado:** Presión Arterial en Reposo (RestBP), Colesterol (Chol), Thal y Angina Ejercitada (ExAng) aportan entre 0.04 y 0.08 de importancia relativa.
- **Predictores Débiles:** Slope, Sexo (Sex), RestECG y Azúcar en Sangre en Ayunas (Fbs < 0.02) muestran una contribución muy reducida al cómputo total de la impureza.

Tabla 1: Resumen del Proceso MDI para Importancia de Variables en Random Forest

Paso	Fase / Nivel	Operación Matemática Principales
Paso 1	**Nodo Individual (n)**	$\Delta I_n = (n/N) * [Gini_n - \Sigma (m/n) Gini_m]$ Calcula la reducción ponderada de impureza por división.
Paso 2	**Árbol Individual (FI_j)**	$FI_j = \Sigma_{n \in$ nodos split j$} \Delta I_n$ Suma la ganancia de impureza en todos los nodos que dividen por j.
Paso 3	**Normalización Árbol**	$Norm\ FI_j = FI_j / \Sigma_k FI_k$ Ajusta los valores para que la suma en el árbol sea igual a 1.
Paso 4	**Ensamble Global (RF)**	$MDI_{RF} = (\Sigma_{t=1}^{B} Norm\ FI_{j,t}) / B$ Promedia la importancia normalizada a lo largo de los B árboles.

4. Evaluación Crítica: Ventajas y Limitaciones de MDI

El método MDI es el estándar por defecto en librerías como Scikit-Learn debido a sus propiedades teóricas y computacionales, aunque presenta advertencias importantes que el analista debe conocer:

- **Ventaja Computacional (Velocidad):** MDI se calcula de forma in-situ durante el proceso de entrenamiento del árbol, sin requerir evaluaciones adicionales sobre conjuntos de validación ni permutaciones intensivas.
- **Sesgo de Cardinalidad (Advertencia Principal):** MDI tiende a inflar artificialmente la importancia de variables con alta cardinalidad (características continuas o categóricas con muchas categorías únicas). En estos casos, se recomienda complementar la evaluación utilizando **Importancia por Permutación (Permutation Importance)** sobre datos fuera de la bolsa (OOB) o de validación.

Conclusión Práctica:

MDI ofrece una primera estimación instantánea y efectiva de la relevancia de variables en Random Forests. Si el dataset contiene características heterogéneas con gran diferencia en cardinalidad, MDI debe validarse mediante la importancia de permutación para evitar diagnósticos sesgados.

Importancia por Permutación de Características

Sección 4.1.6 — Apuntes Técnicos y Metodología de Evaluación en Bosques Aleatorios

---

1. Intuición Fundamental y Concepto Teórico

La evaluación de la relevancia de una variable predictora mediante el método de Importancia por Permutación (Permutation Feature Importance) se basa en medir la degradación del rendimiento del modelo cuando se destruye intencionadamente la relación entre dicha variable y la variable respuesta objetivo.

Supongamos un modelo de Bosque Aleatorio entrenado que depende fuertemente de una característica específica **j**. Al reorganizar o barajar aleatoriamente (permutar) los valores de la columna **j** dentro del conjunto de datos de validación u Out-Of-Bag (OOB), la asociación predictiva entre esa característica y el objetivo se rompe por completo. Si la variable era crucial para la capacidad predictiva del modelo, el error de predicción aumentará significativamente (disminuirá la precisión). Por el contrario, si la variable es ruidosa o irrelevante, barajar sus valores no afectará al rendimiento general del modelo.

> **PRINCIPIO DE EVALUACIÓN DE IMPORTANCIA**
>
> • Variable Imprescindible: La permutación destruye el patrón aprendido → Gran caída en la precisión.
> • Variable Irrelevante: La permutación no aporta ruido adicional → Caída nula o despreciable en la precisión.
> • Evaluación Imparcial: A diferencia de MDI (Mean Decrease Impurity), no presenta sesgo hacia variables numéricas continuas o de alta cardinalidad.

2. Ejemplo Práctico e Ilustración del Algoritmo Paso a Paso

Para comprender la métrica en detalle, considere un conjunto de datos diseñado para predecir el **Nivel de Condición Física (1-5)** a partir de predictores físicos como **Altura (cm)** y **Peso (kg)**:

Altura (cm)	Peso (kg)	Nivel de Condición Física (1-5)
150	65	2
140	50	3
...	...	...
170	70	4
160	80	1

▪ Paso 1: Registro de la Precisión Baseline

Se evalúa el modelo de Bosque Aleatorio entrenado sobre el conjunto de datos de validación u OOB original. Se registra la precisión de referencia (baseline):

Precisión Original (s) = 0.88

▪ Paso 2: Permutación de la Columna Objetivo

Se seleccionan aleatoriamente los valores de la columna objetivo j (en este ejemplo, Peso (kg)) y se permutan (intercambian) únicamente en el conjunto de validación. Al evaluar el modelo sobre el conjunto modificado, se obtiene una nueva precisión:

Precisión Permutada (s_1) = 0.87

▪ Paso 3: Repetición K Veces para Estabilización

Para evitar sesgos asociados a un único reordenamiento aleatorio, el proceso de permutación se repite K veces (ejemplo K = 3). Se calcula el promedio de las precisiones obtenidas:

- Repetición 1: Precisión = 0.87
- Repetición 2: Precisión = 0.86
- Repetición 3: Precisión = 0.82

Precisión Permutada Promedio (s_j) = (0.87 + 0.86 + 0.82) / 3 = 0.85

▪ Paso 4: Cálculo del Valor Final de Importancia

Se resta la precisión promedio permutada de la precisión baseline original. La magnitud de esta diferencia cuantitativa representa la importancia relativa de la característica dentro del Bosque Aleatorio:

Diferencia de Importancia = s - s_j = 0.88 - 0.85 = 0.03

3. Algoritmo General y Formulación Matemática

Para cada característica **j** en el conjunto de datos completo (donde *j = 1, 2, ..., J*), el procedimiento de cálculo de la Importancia por Permutación formaliza el siguiente proceso iterativo:

Paso 1	Registrar la precisión de validación u OOB del modelo de Bosque Aleatorio no modificado: s
Paso 2	Para cada repetición k ∈ {1, ..., K}, permutar aleatoriamente los datos de la columna j y registrar la precisión resultante: s_{k, j}
Paso 3	Calcular la precisión promedio del conjunto de datos permutado: s_j = (1 / K) * ∑_{k=1}^{K} s_{k, j}
Paso 4	Calcular la diferencia entre las precisiones no permutada y permutada: RF FeatureImportance_j = s - s_j

4. Capturas de Fuente y Material Gráfico de Referencia

A continuación se incorporan de forma secuencial las diapositivas originales del curso que documentan cada etapa del algoritmo:

4.1.6 Importancia por permutación

Supongamos que tenemos un modelo entrenado que depende en gran medida de cierto predictor. Cuando los valores de este predictor se barajan aleatoriamente dentro del conjunto de datos y el conjunto de datos modificado se proporciona al modelo para la predicción, el error de la predicción aumentará. Por otro lado, si el impacto de la característica en la predicción del modelo no es significativo, el error de la predicción se mantendrá sin cambios.

Esta es una intuición básica para **Importancia por Permutación**.

Considere el siguiente conjunto de datos:

Altura (cm)	Peso (kg)	...	Nivel de condición física (1-5)
150	65	...	2
140	50	...	3
...	...	...	...
170	70	...	4
160	80	...	1

Paso 1

Registre la precisión de validación/OOB del modelo RF: Precisión = 0.88

Figura 1: Introducción a la Importancia por Permutación y Paso 1 (Baseline s = 0.88)

Paso 2

Permuta aleatoriamente los datos de la columna j (en este caso Peso). La permutación, en caso de que no estés familiarizado con ella, consiste en intercambiar los valores dentro de la misma columna, como se muestra en la imagen a continuación.

Registre la precisión de validación/OOB del modelo RF en el conjunto de datos modificado: en este caso, Precisión Permutada = 0.87

Height (cm)	Weight (kg)	...	Fitness Level (1 – 5)
150	65	...	2
140	50	...	3
...	...	...	...
170	70	...	4
160	80	...	1

Figura 2: Paso 2 — Permutación de los valores de la columna Peso (kg)

Paso 3

Repita el paso anterior K veces y calcule el promedio de todas las precisiones.

Suponiendo que permutamos la característica 3 veces, obtenemos:

$$\text{Precisión Permutada}_1 = 0.87$$
$$\text{Precisión Permutada}_2 = 0.86 \longrightarrow \text{Precisión Permutada Promedio} = \frac{0.87+0.86+0.82}{3} = 0.85$$
$$\text{Precisión Permutada}_3 = 0.82$$

Paso 4

Calcule la diferencia entre la precisión promedio permutada y la no permutada para obtener la importancia de la característica en el bosque aleatorio: en este caso, Diferencia = 0.88 - 0.85 = **0.03**

Figura 3: Pasos 3 y 4 — Promediado de K repeticiones y cálculo final de la diferencia (0.03)

Resumen: Importancia de Permutación

Para cada característica *j* en el conjunto de datos:

Paso 1

Registre la precisión de validación/OOB del modelo RF.

Paso 2

Para cada repetición k en 1...*K*: Permuta aleatoriamente los datos de la columna *j*. Registre la precisión de validación/OOB $s_{k,j}$ del modelo RF en el conjunto de datos modificado.

Paso 3

Calcule el promedio de las precisiones de todos los conjuntos de datos permutados.

$$s_j = \frac{1}{K}\sum_{k=1}^{K} s_{k,j}$$

Figura 4: Resumen del Algoritmo de Importancia por Permutación (Pasos 1 a 3)

Paso 4

Calcule la diferencia entre la precisión promedio permutada y no permutada para obtener la importancia de la característica en el bosque aleatorio.

$$RF\ FeatureImportance_j = s - s_j$$

Figura 5: Resumen del Algoritmo (Paso 4 — Formulación matemática de la métrica)

Sección 4.1.7: MDI vs. Importancia por Permutación

Análisis Comparativo, Evaluación de Sesgos y Cardinalidad en Random Forest

Índice de Contenidos

(El índice se actualizará automáticamente al abrir el documento en Microsoft Word o Google Docs).

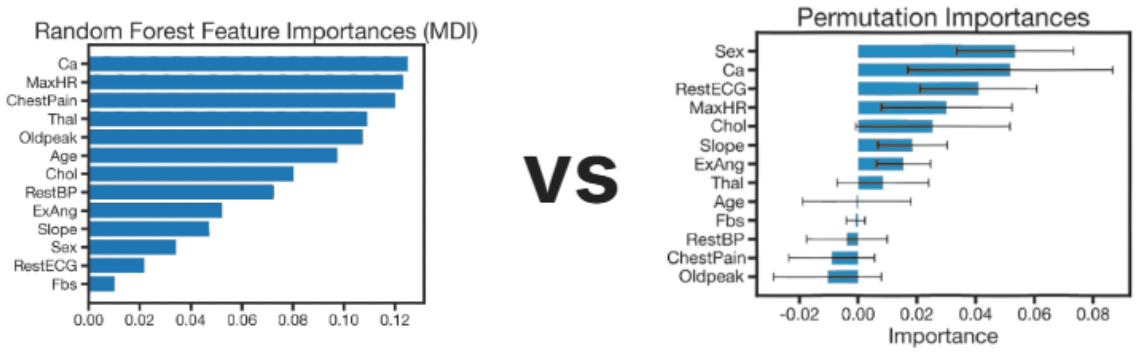
1. Diferencias Prácticas en la Evaluación de Importancia de Características

En la práctica de Machine Learning, las métricas de **Disminución Media de la Impureza (MDI / Random Forest Feature Importances)** y de **Importancia por Permutación (Permutation Importances)** producen resultados notablemente dispares sobre un mismo conjunto de datos. Aunque ambas metodologías persiguen identificar cuáles predictores son fundamentales para las decisiones del modelo, difieren profundamente en sus mecanismos de cálculo y en sus vulnerabilidades sistemáticas.

4.1.7 MDI vs Importancia por Permutación

Ventajas y Desventajas

Las importancias de las características del Random Forest (MDI) y las importancias de permutación se ven muy diferentes en la práctica. Aquí hay un ejemplo:



La mayor ventaja del MDI es **una velocidad de computación**. Todos los valores necesarios se calculan durante el entrenamiento del Random Forest.

El inconveniente del MDI es su tendencia a preferir características numéricas y características categóricas con **alta cardinalidad**. En otras palabras, selecciona características con un alto número de valores únicos como importantes. Esto se debe a que los random forests tienden a seleccionar predictores que pueden dar lugar a una relación uno a uno con la variable objetivo.

En el ejemplo mostrado, Max_HR se selecciona como una característica importante en el MDI debido a la alta cardinalidad. Por lo tanto, podría preferirse la importancia por permutación si el conjunto de datos contiene características con alta cardinalidad con el fin de evitar sesgos.

Figura 4.1.7: Comparativa de importancias calculadas por MDI frente a Importancia por Permutación.

2. Análisis Comparativo del Ranking de Predictores

Al observar el experimento presentado en la Figura 4.1.7, se evidencia un reordenamiento drástico en la jerarquía de las variables predictoras entre ambos métodos. Variables que MDI posiciona en los lugares más altos pasan a un rango inferior o insignificante en Permutación, y viceversa.

Característica	MDI (Random Forest)	Importancia por Permutación
Ca	Muy Alta (~0.125) - Rango 1	Alta (~0.052) - Rango 2
MaxHR	Muy Alta (~0.123) - Rango 2	Moderada (~0.038) - Rango 4
ChestPain	Alta (~0.120) - Rango 3	Negativa / Nula (~ -0.012) - Rango 12
Thal	Alta (~0.110) - Rango 4	Baja (~0.008) - Rango 8
Oldpeak	Alta (~0.108) - Rango 5	Negativa (~ -0.015) - Rango 13
Age	Moderada-Alta (~0.098) - Rango 6	Nula (~0.002) - Rango 9
Chol	Moderada (~0.075) - Rango 7	Moderada (~0.022) - Rango 5
RestBP	Baja-Moderada (~0.052) - Rango 8	Negativa (~ -0.008) - Rango 11
ExAng	Baja (~0.048) - Rango 9	Baja (~0.010) - Rango 7
Slope	Baja (~0.035) - Rango 10	Baja-Moderada (~0.018) - Rango 6
Sex	Muy Baja (~0.022) - Rango 11	Máxima (~0.055) - Rango 1
RestECG	Muy Baja (~0.015) - Rango 12	Alta (~0.042) - Rango 3
Fbs	Insignificante (~0.005) - Rango 13	Negativa (~ -0.005) - Rango 10

Destaca de manera notable la variable **Sex (Sexo biológico)**: mientras que bajo MDI ocupa la penúltima posición (Rango 11 con ~0.022), bajo Importancia por Permutación se posiciona como la característica más relevante de todas (Rango 1 con ~0.055). Del mismo modo, variables como **ChestPain, Oldpeak y Age** obtienen valores inflados en MDI debido a su alta variabilidad numérica/categorías múltiples, pero al permutar sus datos muestran un impacto mínimo o ligeramente negativo en la precisión fuera de muestra.

3. Ventajas y Desventajas Estructurales

Cálculo de MDI (Mean Decrease in Impurity)

• **Ventaja Principal:** Alta velocidad de computación. Todos los valores requeridos se calculan de manera nativa e inercial durante el propio entrenamiento del Random Forest, sin requerir pasadas adicionales sobre los datos.
• **Inconveniente Principal:** Sesgo sistemático a favor de características continuas (numéricas) y categóricas con alta cardinalidad (múltiples niveles únicos).

Cálculo de Importancia por Permutación (Permutation Importance)

• **Ventaja Principal:** Evaluación imparcial e insesgada sobre el conjunto de validación/OOB, reflejando el impacto real en la capacidad de generalización independientemente del tipo o escala de la variable.
• **Inconveniente Principal:** Mayor costo computacional, al requerir reevaluar repetidamente la precisión sobre el dataset tras permutar individualmente cada una de las columnas K veces.

4. El Sesgo de Alta Cardinalidad en MDI y Soluciones Prácticas

El mecanismo de construcción de los árboles de decisión explica la tendencia de MDI a sobreestimar la relevancia de características con alta cardinalidad. Al evaluar divisiones en un nodo, un predictor continuo o con un elevado número de categorías únicas presenta múltiples puntos de corte posibles. Esto incrementa exponencialmente la probabilidad de encontrar de forma aleatoria un umbral que divida 'perfectamente' las muestras del nodo de entrenamiento, logrando una drástica reducción aparente de impureza (Gini/Entropía).

En esencia, los Random Forests tienden a seleccionar preferentemente predictores que puedan dar lugar a una **relación uno a uno con la variable objetivo** sobre los datos de entrenamiento. En el ejemplo práctico de la lección:

- **Caso Max_HR (Frecuencia cardíaca máxima):** Al ser una variable numérica continua de alta cardinalidad, MDI la posiciona como la segunda más importante (~0.123). Sin embargo, al permutar sus valores en validación, la caída en precisión es moderada (~0.038), revelando que MDI estaba inflando su rol debido al sobreajuste local en las divisiones.
- **Caso Sex (Sexo biológico):** Al ser una variable binaria (baja cardinalidad, solo 2 valores únicos), un solo corte genera una ganancia de impureza limitada por paso en MDI (~0.022). No obstante, en la prueba de permutación se demuestra que es la variable con mayor poder explicativo global (~0.055), ya que su alteración destruye drásticamente la capacidad predictiva del ensamble.

Recomendación Metodológica: Para evitar sesgos y seleccionar características de manera robusta en proyectos reales de ciencia de datos, **debe preferirse la Importancia por Permutación** siempre que el conjunto de datos contenga variables con cardinalidades mixtas (continuas vs. binarias/categóricas). MDI puede emplearse como una aproximación inicial rápida durante la exploración, pero no como la métrica definitiva de selección de variables.

4.1.8 Importancia de las Variables: Random Forest vs. Bagging

Comparativa estructural de la distribución de importancia de características y efecto de la descorrelación de árboles

Tabla de Contenidos

1. Comparación General entre Métodos de Importancia

Al comparar la importancia de variables entre distintos algoritmos de ensamble, como Bagging y Random Forest, es habitual observar variaciones significativas en la jerarquía y ordenación de los predictores principales. Esta discrepancia no constituye una inconsistencia del análisis, sino que refleja las diferencias fundamentales en cómo cada método explora el espacio de características durante el entrenamiento.

Si todos los algoritmos de ensamble evaluaran y ponderaran las variables de manera idéntica, no existirá una razón teórica ni práctica para disponer de múltiples arquitecturas. La diversidad en las métricas de importancia permite diagnosticar sesgos estructurales en los datos, como la presencia de predictores dominantes o la correlación entre árboles.

2. Comportamiento en Bagging: Dominancia de Predictores y Sesgo Raíz

En el método de Bagging tradicional, el algoritmo de aprendizaje evalúa la totalidad de los predictores disponibles (J) en cada nodo de decisión para seleccionar la partición óptima según la reducción de impureza. Como consecuencia de este enfoque voraz (greedy), si existe una característica que posee una potencia predictiva o ganancia de información superior a las demás (por ejemplo, el predictor 'dolor en el pecho' o 'Ca'), esta será seleccionada sistemáticamente como el nodo raíz en la gran mayoría o totalidad de los árboles del ensamble.

Debido a que el nodo raíz concentra la mayor proporción de muestras del conjunto de datos, las particiones realizadas en este nivel generan las reducciones de impureza más drásticas. Esto provoca que la variable dominante acumule una proporción desproporcionada de la importancia total, solapando el aporte de otros predictores secundarios que también poseen capacidad discriminativa.

3. Distribución MDI en Random Forest: Descorrelación de Árboles

A diferencia de Bagging, el algoritmo de Random Forest introduce una restricción intencionada: en cada división de nodo, se selecciona aleatoriamente un subconjunto reducido de m predictores (donde $m < J$, típicamente $m = \sqrt{J}$ en clasificación). Esta restricción obliga al modelo a explorar cortes en

variables secundarias cuando el predictor dominante no forma parte del subconjunto aleatorio disponible.

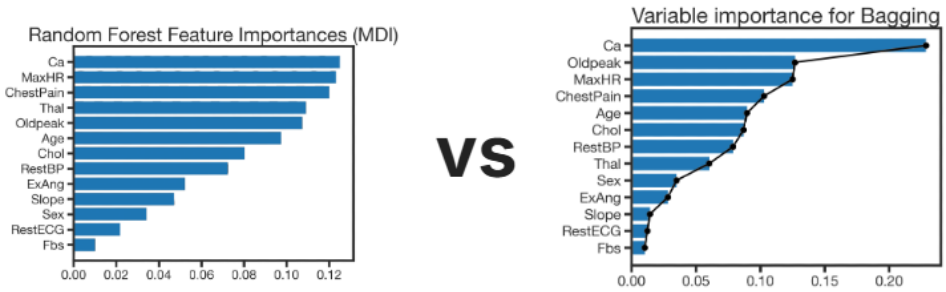
Como resultado de esta aleatorización en la selección de características, el predictor dominante (como 'dolor en el pecho') no siempre ocupa el nodo raíz. Esto descorrelaciona la estructura de los árboles individuales y permite que la ganancia de impureza se distribuya de manera mucho más uniforme y homogénea entre todas las variables informativas del modelo, reflejando una visión más completa del fenómeno.

4. Evidencia Visual y Matriz Comparativa de Jerarquías

La siguiente figura, tomada del material original de clase, ilustra claramente la diferencia en los perfiles de importancia de variables entre Random Forest (MDI) y Bagging:

4.1.8 Importancia de las variables para Random Forest vs Bagging

Ahora, comparemos la importancia de las variables para bagging y random forest. Veremos si los resultados coinciden con lo que esperamos. Tenga en cuenta que los métodos que estamos utilizando no siempre ofrecen las mismas clasificaciones en lo que se refiere a cuál característica es la más importante. De hecho, si lo hicieran, ¡no habría razón para tener más de un método! Aquí hay un ejemplo de cómo podrían valorar las cosas de manera diferente:



En bagging se puede observar fácilmente que cierto predictor parece ser dominante en todos los predictores. Sin embargo, el enfoque de random forest demuestra una importancia de variables más distribuida de manera uniforme entre todos los predictores. Esto se debe a la correlación entre los árboles en bagging, como discutimos anteriormente. Bagging selecciona el mismo predictor y proporciona la mejor partición de entre todos los predictores. En este caso, "dolor en el pecho" siempre aparecerá en el nodo raíz en bagging, lo que resulta en que el predictor ofrezca consistentemente la mayor ganancia en impureza. Sin embargo, en random forest, dado que selecciona el mejor predictor de un subconjunto de todos los predictores, "dolor en el pecho" puede que no siempre se encuentre en el nodo raíz. Por ello, los random forest muestran una importancia de variables más distribuida de manera uniforme entre todos los predictores.

Figura 4.1.8: Comparación de la Importancia de Variables (MDI) entre Random Forest y Bagging.

A continuación se presenta un resumen comparativo de las características clave observadas en ambos enfoques:

Criterio de Evaluación	Bagging Estándar	Random Forest (MDI)
Selección de Predictores por Nodo	Todos los predictores disponibles (J)	Subconjunto aleatorio de predictores ($m < J$)
Presencia en Nodo Raíz	El predictor más fuerte siempre aparece en la raíz	La raíz varía según el subconjunto m seleccionado
Perfil de Distribución de Importancia	Altamente concentrado en 1 o 2 variables dominantes	Distribuido de forma más uniforme entre las variables
Correlación entre Árboles ($\|\rho\|$)	Alta correlación (estructuras de árbol similares)	Baja correlación (árboles descorrelacionados e independientes)

5. Conclusiones y Consideraciones Prácticas

El análisis comparativo de importancia de características confirma que Random Forest ofrece una evaluación de predictores más equilibrada e insensible a la dominancia de una sola variable. Mientras que Bagging tiende a enmascarar variables secundarias útiles debido a la presencia constante del mejor predictor global en la raíz, Random Forest garantiza la participación de todo el espectro de variables explicativas, mejorando tanto la capacidad de generalización del ensamble como la riqueza del diagnóstico de importancia.

Sección 4.1.9: Reflexiones Finales sobre Bosques Aleatorios

Apuntes Técnicos sobre Sensibilidad al Ruido, Comportamiento Asintótico del Ensamble y Estimación de Probabilidades

Índice de Contenidos

1. Sensibilidad al Ruido: Las Características Irrelevantes Reducen la Precisión

El rendimiento de un modelo de Bosques Aleatorios (Random Forests) se degrada cuando el número total de predictores es extremadamente grande pero la cantidad de predictores verdaderamente informativos o relevantes es pequeña. Este fenómeno ocurre debido al mecanismo interno de selección aleatoria de predictores en cada división de nodo. Al restringir cada división a evaluar un subconjunto aleatorio de **$m = \sqrt{J}$** predictores (en clasificación) o **$m = J/3$** (en regresión), la presencia masiva de características sin señal ("noise features") reduce drásticamente la probabilidad de que un nodo incluya al menos un predictor relevante. Como consecuencia, la gran mayoría de los árboles individuales construidos terminan siendo modelos débiles con poca capacidad discriminatoria.

Pregunta de Análisis 1: ¿Qué pasa si el número de predictores es grande, pero el número de predictores relevantes es pequeño? ¿Pueden los bosques aleatorios funcionar mejor o peor?

Respuesta Técnica: En cada división, la probabilidad de seleccionar un predictor relevante disminuye a medida que aumenta el volumen de características de ruido. Al ser bajas las posibilidades de incluir variables informativas en los subconjuntos seleccionados aleatoriamente, la mayoría de los árboles del conjunto se convierten en clasificadores débiles, provocando una caída continua en la precisión general del modelo.

4.1.9 Reflexiones Finales sobre Bosques Aleatorios

Pregunta 1:

¿Qué pasa si el número de predictores es grande, pero el número de predictores relevantes es pequeño? ¿Pueden los bosques aleatorios funcionar mejor o peor?

▼ Mostrar respuesta

En cada división, las posibilidades de seleccionar un predictor relevante serán bajas y, por lo tanto, la mayoría de los árboles en el conjunto serán modelos débiles. En el gráfico a continuación se puede observar que un aumento en "noise features" -aquellas que no son predictores relevantes- resulta en una disminución general en la precisión.

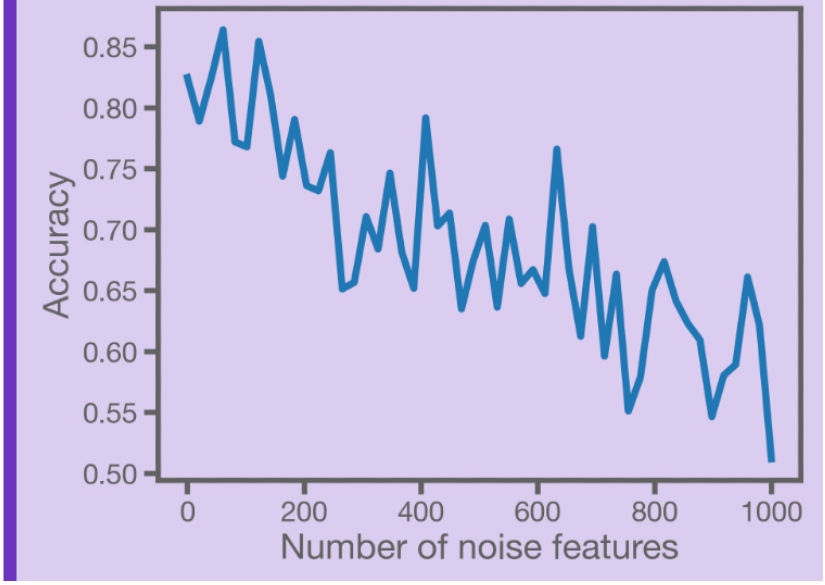


Figura 4.1.9a: Impacto del número de características de ruido (Number of noise features) sobre la precisión (Accuracy) en Random Forests.

La siguiente tabla sintetiza la relación cuantitativa observada en el estudio empírico entre la proporción de características de ruido y el rendimiento predictivo:

Nivel de Ruido (Noise Features)	Precisión Aproximada (Accuracy)	Efecto Estructural en el Ensamble
Bajo (0 - 100 features)	80% - 85%	Alta probabilidad de incluir predictores relevantes en las divisiones de cada nodo.
Moderado (200 - 500 features)	65% - 75%	Degradación progresiva; mayor proporción de árboles divididos sobre ruido.
Severo (600 - 1000 features)	50% - 65%	La mayoría de los árboles son clasificadores aleatorios débiles; pérdida de precisión.

2. Comportamiento Asintótico: Agregar Árboles No Provoca Sobreajuste

A diferencia de los árboles de decisión individuales no podados o de algoritmos iterativos como el Boosting, **aumentar el número de árboles en un Bosque Aleatorio no incrementa el riesgo de sobreajuste (overfitting).** Al descomponer el error de generalización en sus componentes clave de sesgo y varianza, el promedio de B árboles independientes reduce la varianza del modelo sin alterar el sesgo individual de los árboles componentes.

Pregunta de Análisis 2: Si aumentamos el número de árboles en el conjunto, ¿también aumenta el riesgo de sobreajuste?

Respuesta Técnica: No. Aumentar el número de árboles produce un modelo que es al menos tan robusto como un solo árbol. Los valores de error de entrenamiento y de validación disminuyen rápidamente al agregar árboles iniciales y posteriormente se nivelan (asíntota horizontal), permaneciendo estables sin volver a aumentar a medida que se continúan añadiendo cientos o miles de árboles.

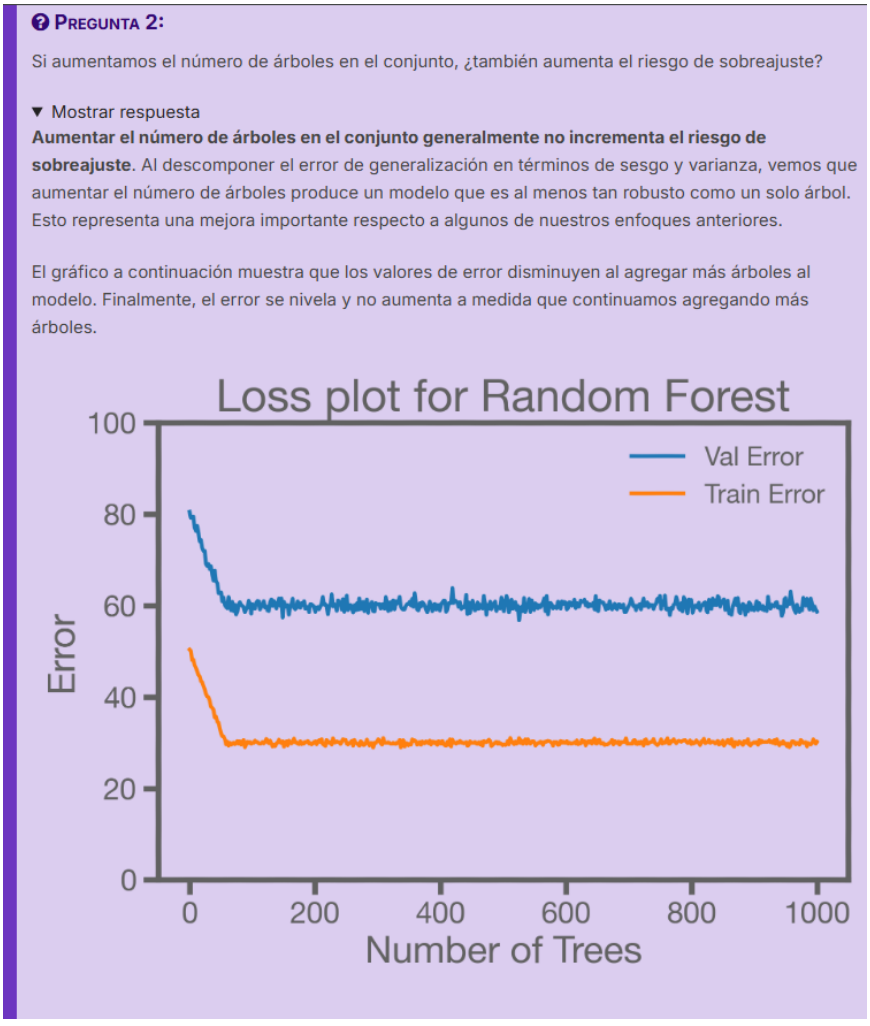


Figura 4.1.9b: Curvas de error de entrenamiento (Train Error) y validación (Val Error) frente al número de árboles (Number of Trees) en Random Forest.

2.1. Descomposición del Error de Generalización: Sesgo vs. Varianza

La varianza de la media de B variables aleatorias con varianza σ^2 y correlación por pares ρ está dada formalmente por la relación:

$$\mathbf{Var(\bar{X}) = \rho \cdot \sigma^2 + ((1 - \rho) / B) \cdot \sigma^2}$$

A medida que el número de árboles **B → ∞**, el segundo término $((1 - \rho) / B) \cdot \sigma^2$ tiende a cero. Esto demuestra matemáticamente por qué el error de validación no puede crecer sin control al agregar más árboles: la varianza se reduce hasta un límite inferior determinado por la correlación residual ρ entre los árboles.

3. Estimación de Probabilidades de Clase mediante el Promedio del Ensamble

Existe una distinción rigurosa entre la simple proporción de votos de un ensamble y la definición matemática formal de probabilidad posterior. En la práctica de aprendizaje automático, el modelo de Bosque Aleatorio puede utilizarse para proporcionar estimaciones probabilísticas continuas de pertenencia a cada clase.

Pregunta de Análisis 3: ¿Puede el clasificador de Bosque Aleatorio (y bagging) devolver probabilidades?

Respuesta Técnica: Técnicamente, la fracción de votos para una clase determinada de entre todas las clases no es matemáticamente equivalente a una probabilidad bien calibrada. Sin embargo, las probabilidades de clase predichas para una muestra de entrada se pueden calcular como las probabilidades medias de clase predichas por cada uno de los árboles individuales del bosque.

QUESTION 3:

¿Puede el clasificador de Bosque Aleatorio (y ensacado) devolver probabilidades?

▼ Mostrar respuesta

Técnicamente, la fracción de votos para una determinada clase de entre todas las clases es **no equivalente** a la probabilidad en una definición matemática. Sin embargo, las probabilidades de clase predichas de una muestra de entrada se pueden calcular como **las probabilidades medias de clase predichas de los árboles** en el bosque.

Figura 4.1.9c: Justificación conceptual sobre la estimación de probabilidades de clase en clasificadores de ensamble.

A continuación se comparan los dos enfoques de inferencia en clasificadores de Bosque Aleatorio:

Métrica de Salida	Mecanismo de Cálculo	Interpretación Teórica y Uso
Fracción de Votos (Hard Voting)	Proporción de árboles que asignan la etiqueta discreta Y = k entre los B árboles.	Frecuencia empírica de decisión. Útil para clasificación discreta por mayoría simple.
Probabilidad Media de Clase (Soft Voting)	Promedio de las distribuciones de probabilidad predichas p_b(Y = k | x) por cada árbol b.	Aproximación suave de probabilidad continua. Permite ajustar umbrales de decisión (ROC/AUC).

4. Cuadro Síntesis de Decisiones Metodológicas

La siguiente matriz resume los tres pilares conceptuales abordados en la sección de reflexiones finales sobre Bosques Aleatorios:

Aspecto Evaluado	Comportamiento del Modelo	Recomendación Práctica
Presencia de Ruido Masivo	Caída de precisión al reducirse la probabilidad de seleccionar predictores útiles en cada división.	Filtrar características irrelevantes previamente (p. ej., mediante Bagging o selección de variables).
Aumento de Árboles (B)	Estabilización asintótica del error sin riesgo de sobreajuste por reducción de varianza.	Fijar B lo suficientemente alto hasta observar la convergencia de la curva de error OOB/Validación.
Inferencia Probabilística	Las probabilidades se estiman promediando las probabilidades predichas de los árboles individuales.	Utilizar el promedio de probabilidades de los árboles para calibrar umbrales y evaluar curvas ROC.

Manejo de Datos Faltantes en Árboles de Decisión: Divisiones Sustitutas

Apuntes de Estudio — Algoritmos de Clasificación y Estrategias Nativas de Imputación

Índice de Contenidos

(Nota: El índice de contenidos se generará automáticamente al abrir este documento en Microsoft Word o Google Docs).

1. El Desafío de los Datos Faltantes en Clasificación

Los conjuntos de datos del mundo real frecuentemente contienen observaciones con **valores faltantes (missing data)** debido a fallos en la recolección, omisiones del usuario o pruebas médicas no realizadas. En un caso típico de diagnóstico médico —por ejemplo, para predecir si un paciente padece una enfermedad cardíaca basada en características como la edad, el peso, el dolor de pecho o el estado de las arterias— la ausencia de una variable clave durante la inferencia puede bloquear el recorrido del dato por el árbol de decisión.

Existen dos enfoques principales para abordar este problema:

- **Imputación Convencional (Preprocesamiento):** Rellenar los huecos antes del entrenamiento mediante la media, la mediana, la moda o algoritmos más complejos como K-Nearest Neighbors (KNN).
- **Divisiones Sustitutas (Surrogate Splits):** Estrategia nativa en algoritmos de árbol (como CART) donde el modelo aprende divisiones de respaldo durante la fase de entrenamiento para utilizarlas automáticamente cuando falta el divisor primario durante la predicción.

2. Mecanismo de Funcionamiento de las Divisiones Sustitutas

El objetivo de una división sustituta es imitar lo más fielmente posible la partición generada por el **divisor primario (primary splitter)**. Durante el entrenamiento de un árbol de decisión, en cada nodo se determina la característica y el umbral que maximizan la pureza (por ejemplo, mediante el índice de Gini o la ganancia de información). Una vez seleccionado dicho divisor primario, el algoritmo evalúa todos los demás predictores para calcular su grado de similitud con la división óptima.

Fases del Procedimiento con Sustitutos:

- **Fase de Entrenamiento:** Para cada nodo, se genera una lista ordenada (ranking) de predictores sustitutos según su capacidad de reproducir la división del divisor primario.
- **Fase de Inferencia/Predicción:** Si a un nuevo registro le falta el valor de la característica primaria, el

árbol consulta la lista ordenada y utiliza la primera división sustituta disponible que tenga un valor válido.

3. Ejemplo Práctico: Selección entre Predictores Alternativos

Considérese un conjunto de datos médico para predecir si un paciente presenta una enfermedad cardíaca (variable de respuesta binaria: **Sí / No**). Supóngase que la división óptima seleccionada como divisor primario es la variable **Arterias Bloqueadas (Blocked Arteries)**. Para encontrar el mejor sustituto si este dato llegase a faltar, se evalúan dos variables alternativas: *Pecho Congestionado (Congested Chest)* y *Buena Circulación Sanguínea (Good Blood Circulation)*.

Distribución de las divisiones según el estado de cada predictor:

Predictor	Estado = Verdadero (Sí / No)	Estado = Falso (Sí / No)	Diferencia de Distribución vs. Primario	Resultado como Sustituto
Arterias Bloqueadas (Primario)	2 Sí / 0 No	1 Sí / 3 No	— (Referencia)	Divisor Primario Óptimo
Pecho Congestionado	1 Sí / 0 No	2 Sí / 3 No	Diferencia = 2	**1ª Opción (Mejor Sustituto)**
Buena Circulación	0 Sí / 1 No	3 Sí / 2 No	Diferencia = 6	2ª Opción (Sustituto Secundario)

Análisis del Cálculo: Al comparar la partición de *Arterias Bloqueadas* frente a *Pecho Congestionado*, la disparidad en las ramas Falso/Verdadero arroja una diferencia de distribución de **2**, mientras que con *Buena Circulación* la diferencia asciende a **6**. Como una menor diferencia implica mayor similitud en la agrupación de clases, **Pecho Congestionado es seleccionado como la primera división sustituta**.

4. Propiedades Clave y Consideraciones Metodológicas

Las divisiones sustitutas presentan características fundamentales que deben ser consideradas al diseñar modelos basados en árboles:

- **Interpretación del Divisor Primario:** Analizar las variables sustitutas ayuda a comprender mejor qué características del conjunto de datos comparten información y desempeñan roles equivalentes en la toma de decisiones.
- **Rendimiento ante Multicolinealidad:** Los sustitutos funcionan con mayor eficacia cuando existen predictores altamente correlacionados, ya que una variable correlacionada reproducirá con alta precisión el comportamiento del divisor primario.

- **Sin Garantía de Sustitutos Útiles:** Si el divisor primario se basa en una característica única y sin correlación con las demás, es posible que ninguna variable alternativa logre una similitud aceptable, limitando la efectividad del procedimiento ante datos faltantes.

Resumen Comparativo de Estrategias:

Estrategia	Mecanismo Principal	Ventaja Clave
Imputación Previa (Media/KNN)	Sustituye valores ausentes antes de entrenar usando estadísticos globales.	Permite reutilizar modelos estándar que no admiten valores nulos.
Divisiones Sustitutas (Nativa)	Aprende particiones alternativas ordenadas por similitud durante el entrenamiento.	Mantiene la coherencia estructural del árbol sin distorsionar la distribución original.

4.2.1 Datos Faltantes, Motivaciones

Tratamiento de Valores Ausentes en Árboles de Decisión mediante Divisiones Sustitutas

---

Índice de Contenidos

(El índice de contenidos se actualizará automáticamente al abrir este documento en Microsoft Word o Google Docs).

1. El Desafío de los Datos Faltantes en el Mundo Real

Los conjuntos de datos del mundo real a menudo contienen **valores faltantes (ausentes)**, causados por la falta de registros, errores en la recolección o sensores defectuosos. Para gestionar esta limitación, la práctica convencional recurre a la **imputación de datos** (rellenando huecos mediante la media, mediana o algoritmos de vecinos cercanos).

> **MOTIVACIÓN Y ESTRATEGIA ALTERNATIVA**
>
> En lugar de imputar previamente los datos, existe una estrategia nativa diseñada específicamente para modelos basados en árboles de decisión: las ***Divisiones Sustitutas (Surrogate Splits)***. La idea básica consiste en que, durante la fase de entrenamiento, el algoritmo identifica divisiones alternativas en otras variables correlacionadas. Estas divisiones sustitutas se emplean automáticamente durante la fase de predicción si falta el valor de la característica primaria.

2. Comparativa: Imputación Previa vs. Divisiones Sustitutas

La siguiente tabla resume la diferencia metodológica entre abordar datos faltantes mediante preprocesamiento o mediante mecanismos nativos del árbol:

Característica	Imputación Convencional	Divisiones Sustitutas
Momento de Ejecución	Fase previa de preprocesamiento de datos (antes de entrenar el modelo).	Fase de entrenamiento e inferencia interna del árbol de decisión.
Mecanismo de Operación	Sustituye el valor ausente por una estimación estadística global (media, mediana, KNN).	Aprovecha la correlación estructural entre predictores para hallar variables sustitutas.

3. Estructura del Conjunto de Datos de Ejemplo

Para ilustrar la aplicación de las divisiones sustitutas, consideremos el escenario de diagnóstico médico donde se pretende entrenar un modelo clasificador. El objetivo es predecir si un paciente padece una **Enfermedad Cardíaca (Sí / No)** a partir de múltiples variables de entrada clínicas, abarcando desde medidas continuas como *Peso* y *Altura* hasta síntomas dicotómicos como *¿Arterias bloqueadas?*.

Peso (kg)	Altura (cm)	¿Pecho congestionado?	¿Buena circulación sanguínea?	¿Arterias bloqueadas?	Enfermedad Cardíaca
58.3	125.3	No	No	No	**No**
65.7	193.2	Sí	No	Sí	**Sí**
75.2	112.0	No	Sí	No	**No**
112.0	165.7	No	No	Sí	**Sí**
45.0	135.0	No	No	No	**No**
40.0	120.0	No	No	No	**Sí**

Tabla 4.2.1: Registro clínico de entrenamiento con características antropométricas, síntomas cardiovasculares y diagnóstico final

.

4. Material Visual de Referencia

A continuación se adjunta la captura de pantalla de la lección original utilizada como fuente primaria:

4.2.1 Datos faltantes, Motivaciones

Datos faltantes

Motivación

Los conjuntos de datos del mundo real a menudo tienen valores faltantes. Para lidiar con los datos faltantes, podemos imputarlos (por ejemplo, como hicimos casi al final del primer curso de esta serie). Sin embargo, existe otra estrategia para tratar los datos faltantes en árboles. La idea básica es que durante el entrenamiento, encontramos divisiones alternativas, que llamamos *Divisiones sustitutas*, que pueden usarse durante la predicción.

Imagine que tenemos un conjunto de datos como se muestra a continuación. Nos gustaría entrenar un modelo para predecir si un sujeto tiene enfermedad cardíaca o no, dado múltiples características de entrada desde *Peso* a *Arterias bloqueadas*. El objetivo es predecir los valores en la columna Enfermedad Cardíaca, en verde a la derecha abajo.

Peso (kg)	Altura (cm)	¿Pecho congestionado?	¿Buena circulación sanguínea?	¿Arterias bloqueadas?	Enfermedad Cardíaca
58.3	125.3	No	No	No	No
65.7	193.2	Sí	No	Sí	Sí
75.2	112	No	Sí	No	No
112	165.7	No	No	Sí	Sí
45	135	No	No	No	No
40	120	No	No	No	Sí

Figura 4.2.1a: Captura de pantalla de la sección 4.2.1 'Datos faltantes, Motivaciones'.

Sección 4.2.2 — Ejemplo 1: Evaluación del Predictor 'Pecho Congestionado'

Análisis detallado de división de nodos y distribución de frecuencias para el cálculo de divisiones sustitutas

---

Índice de Contenidos

(El índice de contenidos se actualizará automáticamente al abrir el archivo en Word o Google Docs).

1. Contexto y Procedimiento Habitual de División

En la construcción de árboles de decisión y en la evaluación de divisiones sustitutas (**surrogate splits**) para el manejo de datos faltantes, el primer paso fundamental consiste en realizar el análisis estándar de división sobre cada predictor candidato. En este ejemplo, se analiza el comportamiento del predictor categórico binario **Pecho Congestionado** sobre el conjunto de entrenamiento de 6 observaciones de diagnóstico cardíaco.

2. Matriz de Datos de Entrenamiento (6 Observaciones)

A continuación se muestra el conjunto de datos completo utilizado en la lección, compuesto por 6 pacientes con atributos físicos y síntomas clínicos, evaluando la variable de respuesta binaria **Enfermedad Cardíaca**

Peso (kg)	Altura (cm)	¿Pecho congestionado?	¿Buena circulación sanguínea?	¿Arterias bloqueadas?	Enfermedad cardíaca
58.3	125.3	No	No	No	No
65.7	193.2	Sí	No	Sí	Sí
75.2	112.0	No	Sí	No	No
112.0	165.7	No	No	Sí	Sí
45.0	135.0	No	No	No	No
40.0	120.0	No	No	No	Sí

Tabla 4.2.2: Distribución de las 6 observaciones y sus valores en el predictor candidato 'Pecho Congestionado'.

3. Diagrama de División y Distribución de Clases

Al realizar el split binario en el nodo **Chest Congested (Pecho Congestionado)**, las 6 observaciones del conjunto de datos quedan distribuidas en dos ramas diferenciadas:

- **Rama 'NO' (Pecho Congestionado = No):** Comprende 5 observaciones (Filas 1, 3, 4, 5 y 6). De estas 5 observaciones, **3 corresponden a 'No Enfermedad'** (Filas 1, 3 y 5) y **2 corresponden a 'Sí Enfermedad'** (Filas 4 y 6).
- **Rama 'YES' (Pecho Congestionado = Sí):** Comprende 1 observación (Fila 2). Esta observación corresponde a **0 'No Enfermedad'** y **1 'Sí Enfermedad'**.

4. Captura de Referencia de la Lección

4.2.2 Ejemplo 1: Pecho congestionado

Como primer paso, lo que necesitamos hacer es realizar nuestro análisis de árbol habitual. Comencemos con el predictor *Pecho congestionado* y crear una división.

Peso (kg)	Altura (cm)	¿Pecho congestionado?	¿Buena circulación sanguínea?	¿Arterias bloqueadas?	Enfermedad cardíaca
58.3	125.3	No	No	No	No
65.7	193.2	Sí	No	Sí	Sí
75.2	112	No	Sí	No	No
112	165.7	No	No	Sí	Sí
45	135	No	No	No	No
40	120	No	No	No	Sí

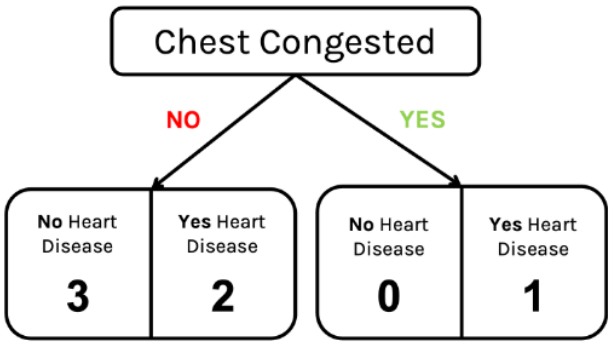


Figura 4.2.2: Ejemplo 1 — Proceso de división del nodo 'Chest Congested' y conteo final de ramas.

5. Tabla Síntesis del Split 'Pecho Congestionado'

La siguiente tabla resume los recuentos absolutos y las proporciones resultantes de la división, los cuales sirven como base para medir la reducción de impureza (Gini/Entropía) o la disparidad frente al nodo primario al evaluar divisiones sustitutas:

Rama del Nodo	Filtro Aplicado	Total Muestras	No Enfermedad (Y=0)	Sí Enfermedad (Y=1)	Proporción (Y=1)
Izquierda (NO)	Pecho Congestionado = No	5	3	2	40.0% (2/5)
Derecha (YES)	Pecho Congestionado = Sí	1	0	1	100.0% (1/1)
Total Nodo	**Conjunto Completo**	**6**	**3**	**3**	**50.0% (3/6)**

Importancia en el Algoritmo: Este procedimiento de conteo y mapeo de frecuencias se repite para cada predictor secundario disponible (como **Buena circulación sanguínea** o **Arterias bloqueadas**). Posteriormente, la distribución de respuestas en cada rama se compara contra la distribución del divisor primario para seleccionar la mejor división sustituta.

Apuntes de Machine Learning: Divisiones Sustitutas

Sección 4.2.3 — Ejemplo 2: Evaluación del Predictor 'Buena Circulación Sanguínea'

Índice de Contenidos

1. Marco Conceptual del Predictor Candidato

En el contexto del entrenamiento de árboles de decisión con datos faltantes, las **divisiones sustitutas (surrogate splits)** permiten reemplazar el divisor primario cuando una observación presenta un valor ausente. Para determinar qué predictor actúa como la mejor alternativa, se evalúa de manera individual la distribución de las respuestas obtenidas al dividir el conjunto de datos según cada variable secundaria.

En este segundo ejemplo, se analiza el comportamiento de la variable binaria **¿Buena circulación sanguínea?** sobre el conjunto de datos clínico de 6 individuos, observando cómo se reparten las clases de la variable objetivo (**Enfermedad cardíaca**) entre sus dos ramas (SÍ y NO).

2. Registro de Datos Clínicos y Distribución de Muestras

El conjunto de datos consta de 6 observaciones con atributos antropométricos y síntomas clínicos. A continuación se detalla la matriz de datos con el predictor bajo análisis destacado:

Peso (kg)	Altura (cm)	¿Pecho congestionado?	¿Buena circulación sanguínea?	¿Arterias bloqueadas?	Enfermedad cardíaca
58.3	125.3	No	No	No	No
65.7	193.2	Sí	No	Sí	Sí
75.2	112.0	No	Sí	No	No
112.0	165.7	No	No	Sí	Sí
45.0	135.0	No	No	No	No
40.0	120.0	No	No	No	Sí

3. Desglose del Split y Diagrama del Árbol de Decisión

Al realizar la división binaria según el valor de **Buena circulación sanguínea**, las 6 observaciones se distribuyen en dos subconjuntos:

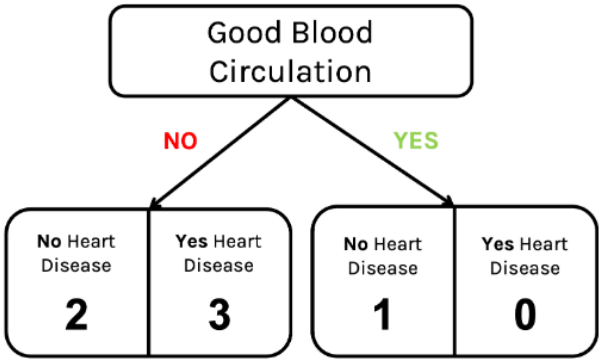
- **Rama VERDADERO (SÍ):** Contiene únicamente **1 observación** (Registro #3: Peso 75.2 kg, Altura 112 cm). En este grupo, la respuesta observada es **1 caso de 'NO Enfermedad'** y **0 casos de 'SÍ Enfermedad'**.
- **Rama FALSO (NO):** Contiene un total de **5 observaciones** (Registros #1, #2, #4, #5 y #6). En este grupo, la distribución resulta en **2 casos de 'NO Enfermedad'** (Registros #1 y #5) y **3 casos de 'SÍ Enfermedad'** (Registros #2, #4 y #6).

Figura 4.2.3 — Esquema de División para Buena Circulación Sanguínea

4.2.3 Ejemplo 2: Buena circulación sanguínea

Para el predictor *Buena circulación sanguínea*, seguimos el mismo procedimiento. Si *Circulación sanguínea* es VERDADERO (SÍ), tenemos sólo una instancia de "NO". Sin embargo, si es FALSO (NO), tenemos un total de 5 ejemplos, con 3 instancias de "SÍ" y 2 instancias de "NO".

Peso (kg)	Altura (cm)	¿Pecho congestionado?	¿Buena circulación sanguínea?	¿Arterias bloqueadas?	Enfermedad cardíaca
58.3	125.3	No	No	No	No
65.7	193.2	Sí	No	Sí	Sí
75.2	112	No	Sí	No	No
112	165.7	No	No	Sí	Sí
45	135	No	No	No	No
40	120	No	No	No	Sí



4. Resumen Cuantitativo del Nodo y Conclusiones

La siguiente tabla resume las frecuencias absolutas obtenidas tras la división por el predictor **Buena circulación sanguínea**:

Valor del Predictor	Total Muestras	Sin Enfermedad (NO)	Con Enfermedad (SÍ)	Proporción Predominante
SÍ (Verdadero)	1	1	0	100% Sin Enfermedad
NO (Falso)	5	2	3	60% Con Enfermedad

Conclusión metodológica: El predictor **Buena circulación sanguínea** genera un subconjunto altamente desbalanceado (5 muestras en NO frente a solo 1 en SÍ). Al comparar esta división con la del predictor primario objetivo (por ejemplo, **Arterias bloqueadas**), se calculará la diferencia cuantitativa en la separación de clases para determinar el ranking final de sustitutos en caso de valores ausentes.

Selección Final de Divisiones Sustitutas y Variables Continuas

Análisis del predictor primario 'Arterias bloqueadas', comparación de disparidad de distribuciones y extensión a predictores continuos.

---

1. Evaluación del Predictor Primario 'Arterias Bloqueadas'

En la construcción del árbol de decisión para el diagnóstico de enfermedad cardíaca, el predictor primario seleccionado para la división óptima es **¿Arterias bloqueadas?**. Para evaluar su capacidad de partición sobre el conjunto de entrenamiento de 6 pacientes, identificamos la distribución de la variable respuesta (Enfermedad cardíaca) en cada una de sus ramas:

- **Rama VERDADERO (YES / Sí):** Agrupa 2 instancias, ambas correspondientes a pacientes con enfermedad cardíaca (0 'NO', 2 'SÍ').
- **Rama FALSO (NO / No):** Agrupa 4 instancias, compuestas por 3 pacientes sin enfermedad y 1 paciente con enfermedad (3 'NO', 1 'SÍ').

Tabla 1: Conjunto de Datos de Entrenamiento (6 Pacientes) con el Predictor Objetivo Relevado

Peso (kg)	Altura (cm)	¿Pecho congestionado?	¿Buena circulación?	¿Arterias bloqueadas?	Enfermedad cardíaca
58.3	125.3	No	No	**No**	No
65.7	193.2	Sí	No	**Sí**	Sí
75.2	112.0	No	Sí	**No**	No
112.0	165.7	No	No	**Sí**	Sí
45.0	135.0	No	No	**No**	No
40.0	120.0	No	No	**No**	Sí

4.2.4 Ejemplo 3: Arterias bloqueadas

Para el predictor *Arterias bloqueadas*, seguimos el mismo procedimiento. Si *Arterias bloqueadas* es TRUE (YES), tenemos dos instancias de "YES" y ninguna instancia de "NO". Sin embargo, si es FALSE (NO), tenemos una instancia de "YES" y tres instancias de "NO".

Peso (kg)	Altura (cm)	¿Pecho congestionado?	¿Buena circulación sanguínea?	¿Arterias bloqueadas?	Enfermedad cardíaca
58.3	125.3	No	No	No	No
65.7	193.2	Sí	No	Sí	Sí
75.2	112	No	Sí	No	No
112	165.7	No	No	Sí	Sí
45	135	No	No	No	No
40	120	No	No	No	Sí

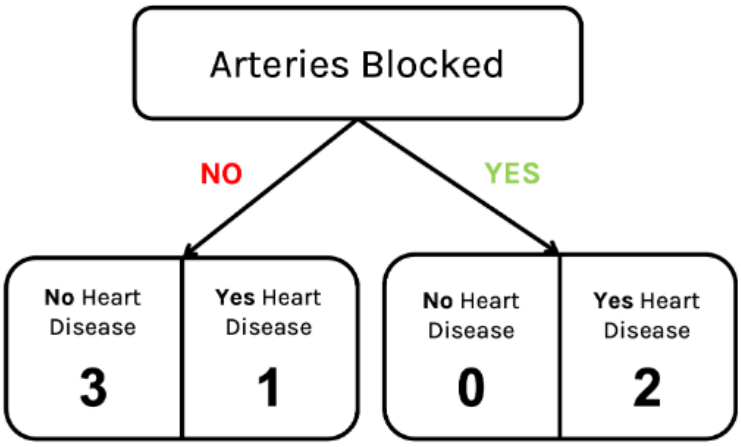


Figura 1: División del predictor primario 'Arterias Bloqueadas' y sus distribuciones resultantes en los nodos hoja.

2. Comparación de Distribuciones y Medida de Disparidad

Para determinar qué predictor funcionará mejor como **división sustituta (surrogate split)** cuando el valor de 'Arterias bloqueadas' esté ausente, comparamos cuantitativamente cómo dividen las observaciones los predictores candidatos frente al divisor primario.

Resumen de distribuciones observadas por predictor:

- **Arterias Bloqueadas (Divisor Primario / Referencia):**
  - **FALSO:** 3 'NO', 1 'SÍ'
  - **VERDADERO:** 0 'NO', 2 'SÍ'
- **Pecho Congestionado (Candidato 1):**

- **FALSO:** 3 'NO', 2 'SÍ'
- **VERDADERO:** 0 'NO', 1 'SÍ'

- **Buena Circulación Sanguínea (Candidato 2):**
  - **FALSO:** 2 'NO', 3 'SÍ'
  - **VERDADERO:** 1 'NO', 0 'SÍ'

Al calcular la **diferencia absoluta en la distribución de instancias** (discrepancia en la asignación de observaciones entre ramas FALSO y VERDADERO) respecto al patrón de 'Arterias Bloqueadas', se obtienen los siguientes valores:

Predictor Candidato	Distribución FALSO	Distribución VERDADERO	Diferencia de Disparidad
Pecho Congestionado	3 NOs, 2 SÍ	0 NOs, 1 SÍ	**2 (Mínima Discrepancia - ÓPTIMO)**
Buena Circulación Sanguínea	2 NOs, 3 SÍ	1 NO, 0 SÍ	6 (Alta Discrepancia)

Comparemos las distribuciones divididas de *Arterias Bloqueadas* con los otros dos predictores, *Pecho Congestionado* y *Buena Circulación Sanguínea*.

- *Arterias Bloqueadas*
  - FALSO: [3 NOs, 1 SÍ]
  - VERDADERO: [0 NOs, 2 SÍ]
- *Pecho Congestionado*
  - FALSO: [3 NOs, 2 SÍ]
  - VERDADERO: [0 NOs, 1 SÍ]

La diferencia en la distribución de instancias entre FALSO y VERDADERO para *Pecho Congestionado* es **2**.

- *Buena Circulación Sanguínea*
  - FALSO: [2 NOs, 3 SÍ]
  - VERDADERO: [1 NO, 0 SÍ]

La diferencia en la distribución de instancias entre FALSO y VERDADERO para *Buena Circulación Sanguínea* es **6**.

Figura 2: Comparación explícita de las distribuciones divididas y cálculo de la diferencia numérica de disparidad.

3. Selección del Sustituto Óptimo y Extensión a Variables Continuas

Dado que **Pecho Congestionado** exhibe la distribución de partición más similar a 'Arterias Bloqueadas' (diferencia de solo 2 frente a 6), el algoritmo lo designa oficialmente como el **sustituto primario (Sustituto de 1ª elección)**. Si durante la fase de predicción o inferencia se presenta una observación donde falta el valor de 'Arterias Bloqueadas', el árbol evaluará automáticamente la regla basada en 'Pecho Congestionado'.

Dado que *Pecho Congestionado* exhibe **la distribución de partición más similar** a *Arterias Bloqueadas*, tomaremos *Pecho Congestionado* valor en lugar de *Arterias bloqueadas* valor faltante, utilizando así *pecho congestionado* como un "Sustituto"!

Peso (kg)	Altura (cm)	¿Pecho Congestionado?	¿Buena Circulación Sanguínea?	¿Arterias Bloqueadas?	Enfermedad Cardíaca
58.3	125.3	No	No	No	No
65.7	193.2	Sí	No	Sí	Sí
75.2	112	No	Sí	No	No
112	165.7	No	No	Sí	Sí
45	135	No	No	No	No
40	120	No	No	No	Sí

Nota: **La medida predictiva de asociación** es una métrica que se puede usar para encontrar divisiones sustitutas. Indica la similitud entre las reglas de decisión que dividen las observaciones utilizando probabilidades. Lee más sobre ello aquí.

Figura 3: Selección de 'Pecho Congestionado' como sustituto de 'Arterias Bloqueadas' y nota sobre la medida predictiva de asociación.

Nota Técnica: Medida Predictiva de Asociación
La 'medida predictiva de asociación' (predictive measure of association) es la métrica probabilística cuantitativa utilizada para encontrar divisiones sustitutas. Mide el grado de similitud entre las reglas de decisión de dos predictores al clasificar las observaciones, evaluando la probabilidad de que ambas reglas envíen un dato a la misma rama.

Sustitutos para Variables Continuas

El principio de sustitución se extiende de manera idéntica a las variables cuantitativas continuas. Por ejemplo, si el divisor primario es la variable continua **Peso (kg)** y en una observación de prueba dicho dato está ausente, el sustituto natural y lógico identificado por alta correlación/asociación física y estadística es la variable **Altura (cm)**.

Ahora, ¿qué hay de **variable continua**? ¿Qué piensas que es el sustituto de *Peso*?

La suposición lógica sería *Altura*!

Peso (kg)	Altura (cm)	¿Pecho congestionado?	¿Buena circulación sanguínea?	¿Arterias bloqueadas?	Enfermedad cardíaca
58.3	125.3	No	No	No	No
65.7	193.2	Sí	No	Sí	Sí
75.2	112	No	Sí	No	No
112	165.7	No	No	Sí	Sí
45	135	No	No	No	No
40	120	No	No	No	Sí

Durante el entrenamiento, para cada división óptima, creamos una lista de clasificación de divisiones sustitutas. Esta clasificación se basa en la similitud entre las distribuciones de división del predictor óptimo y cada otro predictor.

Algunos puntos importantes sobre las divisiones sustitutas

- Los sustitutos pueden ayudarnos a comprender el divisor principal.
- Los sustitutos funcionan mejor cuando existe multicolinealidad.
- ¡No hay garantía de que se puedan encontrar sustitutos útiles!

Figura 4: Identificación de sustitutos en variables continuas (Peso vs. Altura) y resumen de propiedades clave de las divisiones sustitutas.

4. Procedimiento General y Propiedades de los Sustitutos

Durante el proceso de entrenamiento de un árbol de decisión, para cada división óptima (divisor primario) se construye una **lista de clasificación ordenada de divisiones sustitutas**. Esta jerarquía se fundamenta estrictamente en la similitud de las distribuciones de partición de todos los predictores restantes respecto al primario.

Puntos Clave y Consideraciones Metodológicas:

- **Comprensión del Divisor Principal:** Analizar los sustitutos de un nodo permite a los científicos de datos entender qué variables alternativas comparten estructura de decisión con el divisor primario.
- **Comportamiento ante Multicolinealidad:** Las divisiones sustitutas alcanzan su máximo rendimiento y utilidad cuando existe alta colinealidad entre características, ya que las variables correlacionadas replican casi a la perfección el comportamiento del divisor primario.

Ausencia de Garantías: Si las características del conjunto de datos son completamente independientes (sin multicolinealidad ni asociación), no hay garantía de encontrar sustitutos útiles. En tales casos, si falta el predictor primario, el árbol asigna la observación a la rama mayoritaria.

Tratamiento de Conjuntos de Datos Desequilibrados en Clasificación

Métricas de Evaluación, Submuestreo (NearMiss), Sobremuestreo (SMOTE) y Ponderación de Clases

1. El Desafío del Desequilibrio de Clases y la Insuficiencia de la Accuracy

En problemas de clasificación binaria donde la distribución de las clases es fuertemente asimétrica (por ejemplo, una relación de 90 muestras de la Clase 1 frente a solo 10 de la Clase 2), la exactitud convencional (**Accuracy**) deja de ser una métrica representativa del rendimiento real del modelo. Un clasificador trivial que prediga de forma indiscriminada la clase mayoritaria alcanzará un 90% de exactitud; sin embargo, este resultado es engañoso, ya que la red no posee capacidad predictiva sobre la clase de interés minoritaria.

La exactitud resulta adecuada únicamente cuando el conjunto de datos está equilibrado y los costos asociados a los errores de falsos positivos y falsos negativos son equivalentes. Para evaluar de manera rigurosa modelos expuestos a datos desequilibrados, se emplean métricas especializadas:

- **Puntuación F1 (F1-Score):** Representa la media armónica entre la precisión y la exhaustividad (recall). Proporciona una medida equilibrada al penalizar de forma estricta los extremos donde una de las dos métricas sea deficiente.
- **Curva ROC y Área Bajo la Curva (AUC):** Cuando los costos o las causas de los falsos negativos y falsos positivos difieren, la curva ROC evalúa el rendimiento del modelo a lo largo de diversos umbrales de decisión, permitiendo identificar la relación óptima entre la tasa de verdaderos positivos (TPR) y la tasa de falsos positivos (FPR). El valor AUC sintetiza este comportamiento en una sola métrica integral.

Debido a que métricas como F1 y AUC no son funciones diferenciables, no es posible utilizarlas directamente como funciones de pérdida (**loss functions**) durante la optimización del modelo. Por este motivo, se aplican técnicas de preparación de datos o modificaciones algorítmicas enfocadas en mejorar estas métricas durante la etapa de entrenamiento.

2. Estrategias Principales para el Tratamiento del Desequilibrio

Existen tres enfoques metodológicos fundamentales para mitigar el sesgo provocado por clases desproporcionadas: submuestreo de la clase mayoritaria, sobremuestreo de la clase minoritaria y ajuste de la ponderación de costo en la función de pérdida.

Estrategia	Mecanismo de Acción	Ventaja Principal	Riesgo / Desventaja
Submuestreo Aleatorio	Elimina instancias al azar de la clase mayoritaria.	Reduce el tamaño de datos y el tiempo de entrenamiento.	Pérdida de datos informativos críticos.
NearMiss (Submuestreo)	Selecciona muestras mayoritarias cercanas al límite de decisión.	Conserva los puntos más informativos cerca de la frontera.	Mayor sensibilidad al ruido e instancias atípicas.
Sobremuestreo Aleatorio	Duplica instancias de la clase minoritaria con reemplazo.	No descarta información del conjunto original.	Riesgo de sobreajuste por duplicación exacta.
SMOTE (Sobremuestreo)	Genera muestras sintéticas mediante interpolación entre vecinos.	Crea distribuciones continuas e informativas.	Mayor costo computacional en alta dimensión.
Ponderación de Clase	Aumenta la penalización por error en la clase minoritaria.	Directo y sin alterar la cantidad de observaciones.	Depende del soporte algorítmico del modelo.

3. Submuestreo de la Clase Mayoritaria: Muestreo Aleatorio vs. NearMiss

El submuestreo iguala el volumen de observaciones reduciendo la cantidad de puntos de la clase mayoritaria hasta equipararlos con la clase minoritaria. La principal limitación del submuestreo aleatorio radica en el descarte indiscriminado de información. Debido a la aleatoriedad, se pueden seleccionar observaciones distantes del límite de decisión que aportan escasa capacidad discriminativa, perdiendo la estructura general de los datos.

Para optimizar este proceso, se utiliza el método **NearMiss**, una técnica de submuestreo inteligente. NearMiss selecciona explícitamente aquellas muestras de la clase mayoritaria que se encuentran más próximas a las observaciones de la clase minoritaria en el espacio de características. Al priorizar los datos situados en el entorno de la frontera de decisión, NearMiss retiene la información más valiosa para definir el hiperplano clasificador sin conservar redundancias en zonas profundas de la clase mayoritaria.

4. Sobremuestreo de la Clase Minoritaria: Muestreo Bootstrap vs. SMOTE

A diferencia del submuestreo, el sobremuestreo incrementa el número de observaciones de la clase minoritaria. El sobremuestreo aleatorio utiliza un esquema tipo **bootstrap** con reemplazo. Sin embargo, este método presenta la desventaja de replicar exactamente puntos existentes, lo que genera alta densidad en ubicaciones concretas pero deja vacíos continuos en el espacio de características, propiciando el sobreajuste.

La técnica **SMOTE (Synthetic Minority Over-sampling Technique)** resuelve este inconveniente mediante la generación de puntos sintéticos realistas. El procedimiento de SMOTE se sintetiza en las siguientes etapas:

- **Identificación de Vecinos:** Para cada observación de la clase minoritaria, se ubican sus k vecinos más cercanos pertenencientes a la misma clase.
- **Interpolación en el Espacio de Características:** Se trazado un segmento de línea recta que conecta la observación original con uno de sus vecinos seleccionados al azar.
- **Generación Sintética:** Se genera una nueva muestra en una posición aleatoria a lo largo de dicha línea de interpolación. Este enfoque rellena los vacíos entre datos existentes, construyendo una representación continua y realista de la clase minoritaria.

5. Ponderación de Clase (Class Weighting) en la Función de Pérdida

La ponderación de clase es un método directo que no altera el número físico de observaciones ni elimina datos. Consiste en incorporar un factor de peso diferencial en la función de pérdida del algoritmo de aprendizaje, aumentando la penalización impuesta al cometer errores sobre la clase minoritaria.

El peso asignado a cada clase es inversamente proporcional a su frecuencia relativa en el conjunto de entrenamiento:

$$w_c = Total_Muestras / (N_Clases * Muestras_Clase_c)$$

En la librería **Scikit-Learn**, este mecanismo se activa de forma automática configurando el parámetro **class_weight='balanced'** en estimadores como bosques aleatorios, árboles de decisión o regresión logística, permitiendo compensar el desequilibrio de forma computacionalmente eficiente.

4.3.1 Clases Desequilibradas: Evaluación y Limitaciones de la Exactitud

Análisis crítico del impacto del desequilibrio de datos en modelos de clasificación y la falacia de la métrica de precisión (Accuracy) en casos raros.

Índice de Contenidos

1. Motivación y Desafío de las Clases Desequilibradas

> **MOTIVACIÓN TEÓRICA:**
> Entrenar un Random Forest (o cualquier modelo de aprendizaje automático) en un conjunto de datos desequilibrado puede introducir desafíos únicos al problema de aprendizaje. Por ejemplo, si usamos un conjunto de datos sesgado para entrenar un modelo, el modelo resultante también será sesgado.
>
> Desafortunadamente, el desequilibrio de datos es bastante común en el mundo real (médica, fraude, detección de anomalías). En tales casos, ¿cómo podemos evaluar efectivamente el desempeño del modelo?

Prevalencia del desequilibrio de datos en el mundo real: En problemas prácticos de clasificación binaria, las observaciones pertenecientes a una clase de interés primario (como la presencia de una enfermedad rara, transacciones fraudulentas o fallas mecánicas) suelen representar solo una pequeña fracción del total de datos disponible. Cuando se entrena un algoritmo sin ajustes específicos, el proceso de optimización tiende a favorecer a la clase mayoritaria para maximizar la tasa global de aciertos.

2. La Métrica Tradicional de Exactitud o Precisión Global (Accuracy)

Al evaluar el desempeño de un clasificador binario, la métrica estándar por excelencia en la literatura introductoria es la **Precisión o Exactitud (Accuracy)**. En resumen, la exactitud mide la proporción de predicciones correctas realizadas por el modelo respecto al volumen total de muestras evaluadas.

Precisión (Accuracy) = Número de predicciones correctas / Número total de predicciones

3. Limitaciones de la Exactitud: Ejemplo de Prueba COVID-19

Para evidenciar el fallo estructural de la exactitud como métrica de evaluación, supongamos un problema de clasificación binaria clínica en el que se requiere predecir si una persona padece COVID-19. El conjunto de datos consta de **1.000 muestras** con la siguiente distribución extrema:

- **Clase Positiva (Personas con COVID-19):** 10 muestras (1.0% del total de la población).
- **Clase Negativa (Personas sin COVID-19):** 990 muestras (99.0% del total de la población).

Supongamos que construimos un modelo de clasificación binaria altamente sesgado hacia la clase mayoritaria. Al ser evaluado sobre la población de prueba, el modelo clasifica correctamente 990 de las 990 muestras negativas, pero solo logra identificar correctamente 1 de las 10 muestras positivas.

Métrica de Matriz de Confusión	Valor Absoluto	Interpretación Clínica
Verdaderos Positivos (TP)	1	Pacientes con COVID correctamente identificados
Falsos Positivos (FP)	0	Sanos clasificados erróneamente como enfermos
Verdaderos Negativos (TN)	990	Sanos correctamente identificados
Falsos Negativos (FN)	9	Pacientes enfermos no detectados (peligro clínico)
Exactitud Global (Accuracy)	**991 / 1000 = 0.991**	**Puntuación de 99.1% (Engañosa)**

El peligro de la alta puntuación (99.1%): En este escenario, la puntuación de precisión resultante es de 991/1000 = **0.991 (99.1%)**, lo que podría parecer un rendimiento casi perfecto a primera vista. Sin embargo, esta alta cifra es una **ilusión métrica extremadamente peligrosa**, ya que el modelo **falló en identificar correctamente al 90% de los casos positivos (9 de 10 enfermos de COVID quedaron sin diagnosticar)**. En un entorno de salud pública, un clasificador tan sesgado resulta inútil y perjudicial.

4. Material Visual Adjunto

A continuación se integra la lámina original de la lección sobre la sección 4.3.1 Clases Desequilibradas, donde se detalla el marco conceptual y el ejemplo práctico analizado:

4.3.1 Clases desequilibradas

> **Motivación**
>
> Entrenar un Random Forest (o cualquier modelo de aprendizaje automático) en un conjunto de datos desequilibrado puede introducir desafíos únicos al problema de aprendizaje. Por ejemplo, si usamos un conjunto de datos sesgado para entrenar un modelo, el modelo resultante también será sesgado. Desafortunadamente, el desequilibrio de datos es bastante común en el mundo real. En tales casos, ¿cómo podemos evaluar efectivamente el desempeño del modelo?

Al evaluar la precisión de clasificación del modelo, la métrica más comúnmente utilizada es la Precisión. En resumen, la Precisión resume el desempeño del modelo al calcular el número de predicciones correctas dividido por el número total de predicciones.

$$\text{Precisión} = \frac{\text{Número de predicciones correctas}}{\text{Número total de predicciones}}$$

Limitación de la puntuación de Precisión: ejemplo de prueba de COVID-19

Supongamos que tenemos un problema de clasificación binaria en el que necesitamos predecir si una persona tiene COVID o no. En nuestro conjunto de datos, tenemos 1,000 muestras, de las cuales solo 10 pertenecen a la clase positiva (personas con COVID) y las 990 restantes pertenecen a la clase negativa (personas sin COVID).



Ahora, supongamos que construimos un modelo de clasificación binaria. Clasifica correctamente 990 de 990 muestras negativas, pero solo identifica correctamente 1 de 10 muestras positivas. En resumen,

- Verdaderos Positivos (TP): 1
- Falsos Positivos (FP): 0
- Verdaderos Negativos (TN): 990
- Falsos Negativos (FN): 9

En este caso, la puntuación de Precisión sería 991/1000 = **0.991**, lo que podría parecer impresionante a primera vista. Sin embargo, esta alta puntuación de Precisión es engañosa porque ***el modelo no logró identificar correctamente la mayoría de los casos positivos***.

Entonces, ¿cómo abordamos este problema?

Figura 4.3.1: Captura original de la lección sobre la métrica de Precisión y la paradoja del test de COVID-19.

5. Conclusión y Alternativas Evaluativas

Para superar la trampa de la exactitud en escenarios de clases desequilibradas, es imprescindible abandonar la evaluación monolítica de la exactitud global y adoptar métricas sensibles al equilibrio de errores, tales como:

- **Sensibilidad o Exhaustividad (Recall / TPR):** Mide la proporción de verdaderos positivos identificados sobre el total real de positivos. En el caso de COVID-19, el Recall sería solo del 10% (1/10).
- **Puntuación F1 (F1-Score):** Promedio armónico entre la Precisión (Precision) y el Recall, castigando severamente a los modelos que ignoran la clase minoritaria.
- **Área Bajo la Curva ROC (ROC-AUC):** Evalúa la capacidad de discriminación del modelo a través de diversos umbrales de decisión independientemente de la prevalencia de clase.

4.3.2 Mejores Métricas en el Desequilibrio de Datos

Evaluación robusta de clasificadores binarios mediante Sensibilidad, Precisión, F1-Score, Curva ROC y Área Bajo la Curva (AUC)

———————————————————————————————————————

Índice de Contenidos

1. Sensibilidad y Precisión: Evaluación Unilateral de Clases

Cuando un conjunto de datos presenta un desequilibrio severo de clases (por ejemplo, 10 casos positivos de COVID-19 frente a 990 casos negativos en un total de 1.000 pacientes), la exactitud global (*Accuracy*) deja de ser una métrica representativa. Un clasificador ingenuo que prediga siempre la clase mayoritaria obtendría un 99.0% de exactitud pero fallaría estrepitosamente en la detección médica. Para exponer las verdaderas deficiencias del modelo, es imprescindible desagregar la evaluación mediante la **Sensibilidad** y la **Precisión**.

1.1 Sensibilidad (Recall / Tasa de Verdaderos Positivos)

La **Sensibilidad** (también conocida como *Recall* o **Tasa de Verdaderos Positivos - TPR**) mide la capacidad del modelo para identificar correctamente todas las instancias pertenecientes a la clase positiva. Una puntuación baja de sensibilidad resalta de inmediato la incapacidad del algoritmo para diagnosticar pacientes enfermos.

> **Fórmula de Sensibilidad (Recall):**
> Sensibilidad = Verdaderos Positivos / (Verdaderos Positivos + Falsos Negativos)
>
> Ejemplo práctico (Escenario COVID-19 con 1 VP y 9 FN):
> Sensibilidad = 1 / (1 + 9) = 0.10 (10.0%)

En el diagnóstico médico, la sensibilidad es la métrica más crítica: un valor de 0.10 revela que el modelo deja sin diagnosticar al 90% de las personas infectadas, un riesgo inaceptable en salud pública.

1.2 Precisión (Precision)

La **Precisión** cuantifica la capacidad del modelo para evitar falsos positivos. Refleja la proporción de casos verdaderamente positivos de entre todas las observaciones que el modelo catalogó como positivas.

> **Fórmula de Precisión:**
> Precisión = Verdaderos Positivos / (Verdaderos Positivos + Falsos Positivos)
>
> Ejemplo práctico (Escenario COVID-19 con 1 VP y 0 FP):
> Precisión = 1 / (1 + 0) = 1.00 (100.0%)

Aunque la precisión de 1.00 indica confiabilidad absoluta cuando el modelo emite una alerta positiva, por sí sola oculta el hecho de que el modelo casi nunca se atreve a predecir la clase positiva.

4.3.2 Mejores métricas en el desequilibrio de datos

Mejores métricas en el desequilibrio de datos

Sensibilidad

En este escenario, la sensibilidad (también conocida como sensibilidad o tasa de verdaderos positivos) es un buen indicador. La sensibilidad mide la capacidad del modelo para identificar correctamente todos los casos positivos. Dada la naturaleza desequilibrada del conjunto de datos con solo 10 casos positivos, una baja puntuación de sensibilidad destacaría la deficiencia del modelo para detectar con precisión las instancias positivas (casos de COVID).

$$\text{Sensibilidad} = \frac{\text{Verdaderos Positivos}}{\text{Verdaderos Positivos} + \text{Falsos Negativos}}$$

En nuestro ejemplo anterior, la sensibilidad sería 1/(1+9) = **0.1**. Esta métrica proporciona una mejor visión ya que nos interesa más cuán bien nuestro modelo clasifica correctamente los casos positivos.

Precisión

La precisión mide la capacidad del modelo para evitar falsos positivos. Idealmente, queremos que nuestro modelo maximice la proporción de casos verdaderamente positivos de entre todas las instancias predichas como positivas.

$$\text{Precisión} = \frac{\text{Verdaderos Positivos}}{\text{Verdaderos Positivos} + \text{Falsos Positivos}}$$

De igual manera, en nuestro ejemplo, la precisión sería 1/(1+0) = **1**. Dado que no existe ningún caso de falso positivo, la puntuación de precisión es 1.

Figura 4.3.2a: Formulación explícita y cálculo de Sensibilidad (10%) y Precisión (100%) en presencia de desequilibrio de datos.

2. F1-Score: Integración mediante Media Armónica

Evaluar la sensibilidad y la precisión por separado obliga al analista a ponderar un compromiso de compensación (*trade-off*). El **F1-Score** resuelve este desafío combinando ambas métricas en una única puntuación mediante su **media armónica**.

> **Fórmula del F1-Score:**
> F1-Score = 2 * (Sensibilidad * Precisión) / (Sensibilidad + Precisión)
>
> Aplicación al ejemplo (Sensibilidad = 0.10, Precisión = 1.00):
> F1-Score = 2 * (0.10 * 1.00) / (0.10 + 1.00) = 0.20 / 1.10 = 0.1818 (18.18%)

A diferencia de la media aritmética tradicional (que daría 0.55), la media armónica penaliza fuertemente los valores extremos. Si cualquiera de las dos métricas es cercana a cero, el F1-Score colapsa hacia abajo, proporcionando una evaluación rigurosa y equilibrada ante desequilibrios de datos.

F1-score: Media armónica de Precisión y Sensibilidad

Tanto la sensibilidad como la precisión son métricas reveladoras, especialmente ante la presencia de desequilibrio en los datos. Pero, ¿cómo consideramos ambas simultáneamente? El F1-score combina la precisión y la sensibilidad tomando su media armónica. La media armónica es una métrica útil cuando se desea una tasa promedio.

$$F1 = \frac{2\text{Sensibilidad} * \text{Precisión}}{\text{Sensibilidad} + \text{Precisión}}$$

Da igual peso a la precisión y la sensibilidad, proporcionando una evaluación equilibrada del rendimiento del modelo al considerar eficazmente tanto falsos positivos como falsos negativos.

Curva ROC

Una curva ROC (curva de caracterización operativa del receptor) se refiere a un gráfico que muestra el rendimiento de un modelo de clasificación en todos los umbrales de clasificación. Entonces, ¿qué significa umbral de clasificación?

Supongamos que tenemos un modelo de clasificación binaria que determina si un sujeto es positivo para COVID o no (negativo). Este modelo de clasificación proporcionará la probabilidad predicha de obtener un resultado de prueba positivo cada vez que se aplique a un sujeto.

Aquí hay un ejemplo hipotético de datos:

Sujeto (entrada)	Probabilidad Predicha	Valor real
Sujeto 1	0.95	Positivo
Sujeto 2	0.86	Negativo
Sujeto 3	0.75	Negativo
Sujeto 4	0.88	Positivo
Sujeto 5	0.89	Negativo
Sujeto 6	0.80	Negativo

El modelo de clasificación asignará una etiqueta estimada basada en un umbral dado. Si la probabilidad predicha supera el umbral, el modelo clasificará al sujeto como COVID positivo.

Figura 4.3.2b: Definición matemática del F1-Score y planteamiento inicial de la Curva ROC con un conjunto de datos hipotético.

3. La Curva ROC y el Impacto del Umbral de Clasificación

Un modelo de clasificación probabilístico asigna a cada observación una probabilidad continua de pertenecer a la clase positiva (por ejemplo, $P(\text{COVID} = \text{Positivo})$). Para convertir esta

probabilidad en una decisión binaria, se debe definir un **umbral de clasificación** (t). Si $P \ge t$, se asigna la clase positiva; en caso contrario, la clase negativa.

Para ilustrar cómo varía el comportamiento del modelo al modificar el umbral t, analicemos la siguiente muestra hipotética de 6 sujetos:

Sujeto (Entrada)	Probabilidad Predicha P(COVID+)	Valor Real
Sujeto 1	0.95	**Positivo**
Sujeto 2	0.86	**Negativo**
Sujeto 3	0.75	**Negativo**
Sujeto 4	0.88	**Positivo**
Sujeto 5	0.85	**Negativo**
Sujeto 6	0.80	**Negativo**

3.1 Métricas Fundamentales de la Curva ROC

La **Curva ROC** (Receiver Operating Characteristic) grafica la relación entre dos tasas a lo largo de todos los posibles umbrales:

- **Tasa de Verdaderos Positivos (TVP / TPR):** TVP = Verdaderos Positivos / (Verdaderos Positivos + Falsos Negativos)
- **Tasa de Falsos Positivos (TFP / FPR):** TFP = Falsos Positivos / (Falsos Positivos + Verdaderos Negativos)

3.2 Comparativa Metodológica de Umbrales de Clasificación

Umbral (t)	Resultados Predichos Positivos	Resultados Predichos Negativos	TVP (TPR)	TFP (FPR)
t = 0.85	Sujeto 1 (VP), Sujeto 4 (VP), Sujeto 2 (FP), Sujeto 5 (FP)	Sujeto 3 (VN), Sujeto 6 (VN)	2 / (2+0) = 1.00 (100%)	2 / (2+2) = 0.50 (50%)
t = 0.70	Sujeto 1 (VP), Sujeto 4 (VP), Sujeto 2 (FP), Sujeto 3 (FP), Sujeto 5 (FP), Sujeto 6 (FP)	Ninguno	2 / (2+0) = 1.00 (100%)	4 / (4+0) = 1.00 (100%)

Dibujar una curva ROC requiere dos métricas:

$$\text{Tasa de Verdaderos Positivos (TVP)} = \frac{\text{Verdaderos Positivos}}{\text{Verdaderos Positivos} + \text{Falsos Negativos}}$$

$$\text{Tasa de Falsos Positivos (TFP)} = \frac{\text{Falsos Positivos}}{\text{Falsos Positivos} + \text{Verdaderos Negativos}}$$

Consideremos dos umbrales diferentes a continuación.

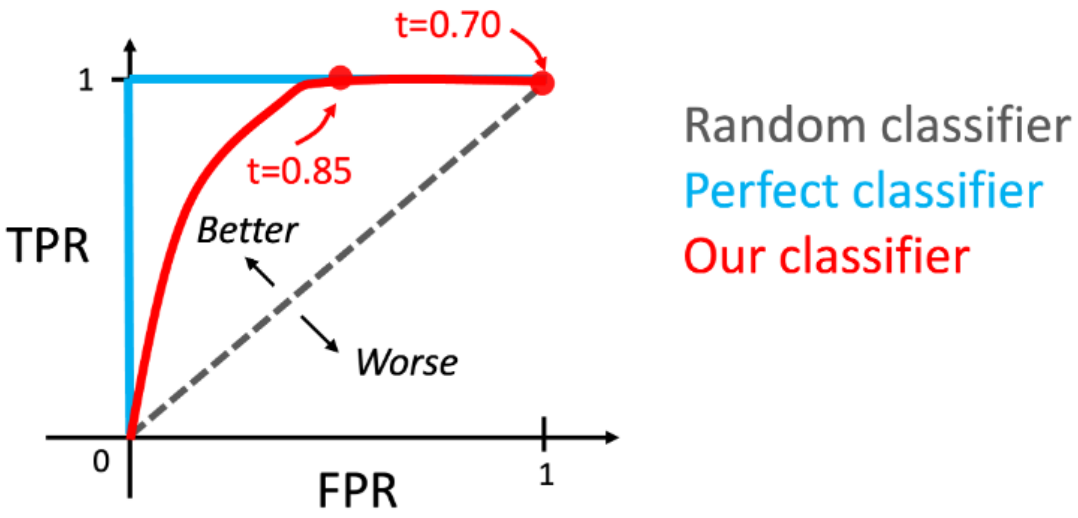
- Umbral: **0.85**
    - Resultados predichos
        - Positivo: Sujeto 1 (VP), Sujeto 2 (FP), Sujeto 4 (VP), Sujeto 5 (FP)
        - Negativo : Sujeto 3 (VN), Sujeto 6 (VN)
    - Tasa de Verdaderos Positivos: 2/(2+0) = **1** (2 verdaderos positivos de 2 positivos reales)
    - Tasa de Falsos Positivos: 2/(2+2) = **0.5** (2 falsos positivos de 4 negativos reales)
- Umbral: **0.70**
    - Resultados predichos
        - Positivo: Sujeto 1 (TP), Sujeto 2 (FP), Sujeto 3 (FP), Sujeto 4 (TP), Sujeto 5 (FP), Sujeto 6 (FP)
        - Negativo : Ninguno
    - Tasa de verdaderos positivos: 2/(2+0) = **1** (2 verdaderos positivos de 2 positivos reales)
    - Tasa de falsos positivos: 4/(4+0) = **1** (4 falsos positivos de 4 negativos reales)

Figura 4.3.2c: Cálculo detallado de la Tasa de Verdaderos Positivos (TVP) y Tasa de Falsos Positivos (TFP) para umbrales t = 0.85 y t = 0.70.

4. Interpretación de la Curva ROC y Área Bajo la Curva (AUC)

Al graficar la TVP en el eje vertical (Y) frente a la TFP en el eje horizontal (X) para cada punto de umbral t, se obtiene la trayectoria continua de la **Curva ROC**.

El siguiente gráfico representa la curva ROC con varios ajustes de umbral. Intuitivamente, tiene sentido que tanto la TPR como la FPR aumenten a medida que el umbral disminuye. En otras palabras, si nuestro modelo clasifica todo como verdadero, el número de casos de falsos positivos también aumentará.



La línea diagonal en la curva ROC representa el rendimiento de un clasificador aleatorio. Independientemente del umbral utilizado, la tasa de verdaderos positivos (TPR) y la tasa de falsos positivos (FPR) permanecerán iguales. En otras palabras, el clasificador aleatorio no proporciona una discriminación útil entre las clases.

El objetivo de un buen clasificador es lograr métricas de rendimiento que sean mejores que adivinar al azar. Por lo tanto, nos esforzamos por posicionar nuestro clasificador lo más lejos posible de la línea diagonal. Al hacerlo, podemos maximizar la TPR (capturando más verdaderos positivos) mientras minimizamos la FPR (limitando los falsos positivos).

Figura 4.3.2d: Representación gráfica de la Curva ROC comparando nuestro clasificador frente al clasificador perfecto y el clasificador aleatorio.

4.1 Referencias Geométricas en el Espacio ROC

1. Clasificador Aleatorio (Línea Diagonal Nula): La línea diagonal trazada desde (0,0) hasta (1,1) representa un modelo que asigna clases al azar. Para cualquier umbral, la TVP coincide exactamente con la TFP. No aporta capacidad discriminativa alguna.

2. Clasificador Perfecto: Alcanza una TVP de 1.0 manteniendo una TFP de 0.0 en el punto $(0,1)$, formando una trayectoria rectangular perfecta.

3. Clasificador Óptimo: El objetivo de entrenamiento es empujar la curva hacia la esquina superior izquierda, maximizando la captura de verdaderos positivos mientras se frena el crecimiento de falsos positivos.

4.2 Área Bajo la Curva ROC (AUC)

El **Área Bajo la Curva (AUC - Area Under the Curve)** resume globalmente la capacidad del modelo para ordenar probabilidades e independiza la evaluación de un umbral específico.

> **Criterios de Evaluación por AUC:**
> • Clasificador Perfecto: AUC = 1.00 (Separación de clases absoluta)
> • Clasificador de Nuestro Ejemplo: AUC = 0.88 (Excelente capacidad discriminativa)
> • Clasificador Aleatorio: AUC = 0.50 (Sin capacidad predictiva)

Área bajo la curva ROC (AUC)

La curva ROC nos permite encontrar el umbral de clasificación que proporciona el mejor equilibrio entre la tasa de falsos positivos (FPR) y la tasa de verdaderos positivos (TPR).

Resumimos la ROC calculando el Área bajo la curva ROC (AUC). Para el clasificador perfecto el AUC es 1, el clasificador aleatorio tiene un AUC de 0.5.



Si deseas encontrar más información sobre las curvas ROC, hay una página sobre ellas en nuestro primer curso, Introducción a la Ciencia de Datos con Python.

Figura 4.3.2e: Área Bajo la Curva ROC (AUC) y síntesis cuantitativa de rendimiento global del modelo.

Sección 4.3.3: Tratamiento de Clases Desequilibradas

Estrategias Metodológicas de Submuestreo, Sobremuestreo (SMOTE) y Ponderación de Clases

Nota de lectura: El índice anterior se actualizará automáticamente al abrir este documento en Microsoft Word o Google Docs.

1. Visión General de las Estrategias frente al Desequilibrio

En tareas de clasificación donde la distribución de las clases objetivo está severamente sesgada (por ejemplo, detección de fraudes, diagnóstico de enfermedades raras o fallos en líneas de producción), los algoritmos de aprendizaje automático tienden de forma natural a optimizar la métrica global de exactitud (accuracy) favoreciendo sistemáticamente a la clase mayoritaria. Para mitigar esta deficiencia estructural y permitir que el modelo capture los patrones clave de la clase minoritaria, existen tres metodologías fundamentales de reequilibrio:

- **1. Submuestreo (Undersampling):** Reducción intencionada del número de observaciones de la clase mayoritaria para igualar el volumen de datos de la clase minoritaria.
- **2. Sobremuestreo (Oversampling):** Generación o replicación de observaciones para la clase minoritaria hasta alcanzar la paridad con la clase mayoritaria.
- **3. Ponderación de Clases (Class Weighting):** Modificación de la función de costo durante el entrenamiento asignando una penalización superior a los errores cometidos sobre la clase minoritaria, sin alterar la distribución original de datos.

> **Principio Fundamental de Reequilibrio:** *El desequilibrio de datos no debe abordarse modificando arbitrariamente las métricas de evaluación sin antes adecuar el conjunto de entrenamiento o la función de pérdida. Las técnicas de muestreo adaptan la estructura del espacio de datos, mientras que la ponderación de clases ajusta directamente la dinámica de optimización del algoritmo.*

Tabla Comparativa de Enfoques de Reequilibrio

Estrategia	Mecanismo de Acción	Ventajas Principales	Consideraciones y Riesgos
Submuestreo (Undersampling)	Elimina muestras de la clase mayoritaria hasta equiparar a la minoritaria.	Disminuye drásticamente el tiempo de cómputo y el consumo de memoria.	Riesgo alto de descartar información crítica y muestras altamente informativas.
Sobremuestreo (Oversampling)	Duplica o sintetiza nuevas muestras para la clase minoritaria.	Conserva toda la información original de la clase mayoritaria.	Riesgo de sobreajuste (overfitting) o generación de puntos fuera de distribución.
Ponderación (Class Weighting)	Aplica pesos inversos a la frecuencia en la función de costo.	No altera el dataset ni pierde/crea información sintética.	Requiere algoritmos que soporten pesos explícitos en su función de pérdida.

2. Submuestreo (Undersampling) y Selección de Muestras Mayoritarias

El submuestreo reduce intencionalmente el número de observaciones de la clase dominante (Clase A) para emparejar la cantidad de elementos disponibles en la clase minoritaria (Clase B). Este procedimiento busca reequilibrar la proporción $1:1$ entre clases, garantizando que el modelo no reciba un estímulo desproporcionado durante la fase de optimización.

Variantes Principales del Submuestreo

- **1. Muestreo Aleatorio (Random Undersampling):** Consiste en extraer aleatoriamente observaciones de la clase mayoritaria (con o sin reemplazo) hasta alcanzar la cardinalidad de la clase minoritaria. Aunque es rápido y sencillo, presenta el inconveniente de eliminar puntos de datos potencialmente valiosos situados cerca del límite de decisión.
- **2. Algoritmo NearMiss (Fallo Cercano):** Constituye un enfoque heurístico más sofisticado basado en distancias. En lugar de seleccionar al azar, analiza la proximidad espacial entre observaciones y selecciona aquellas muestras de la clase mayoritaria cuya distancia promedio a los puntos vecinos de la clase minoritaria sea mínima (o cumpla un criterio específico de densidad). Esto preserva la frontera de decisión con mayor fidelidad.

Representación Esquemática del Submuestreo

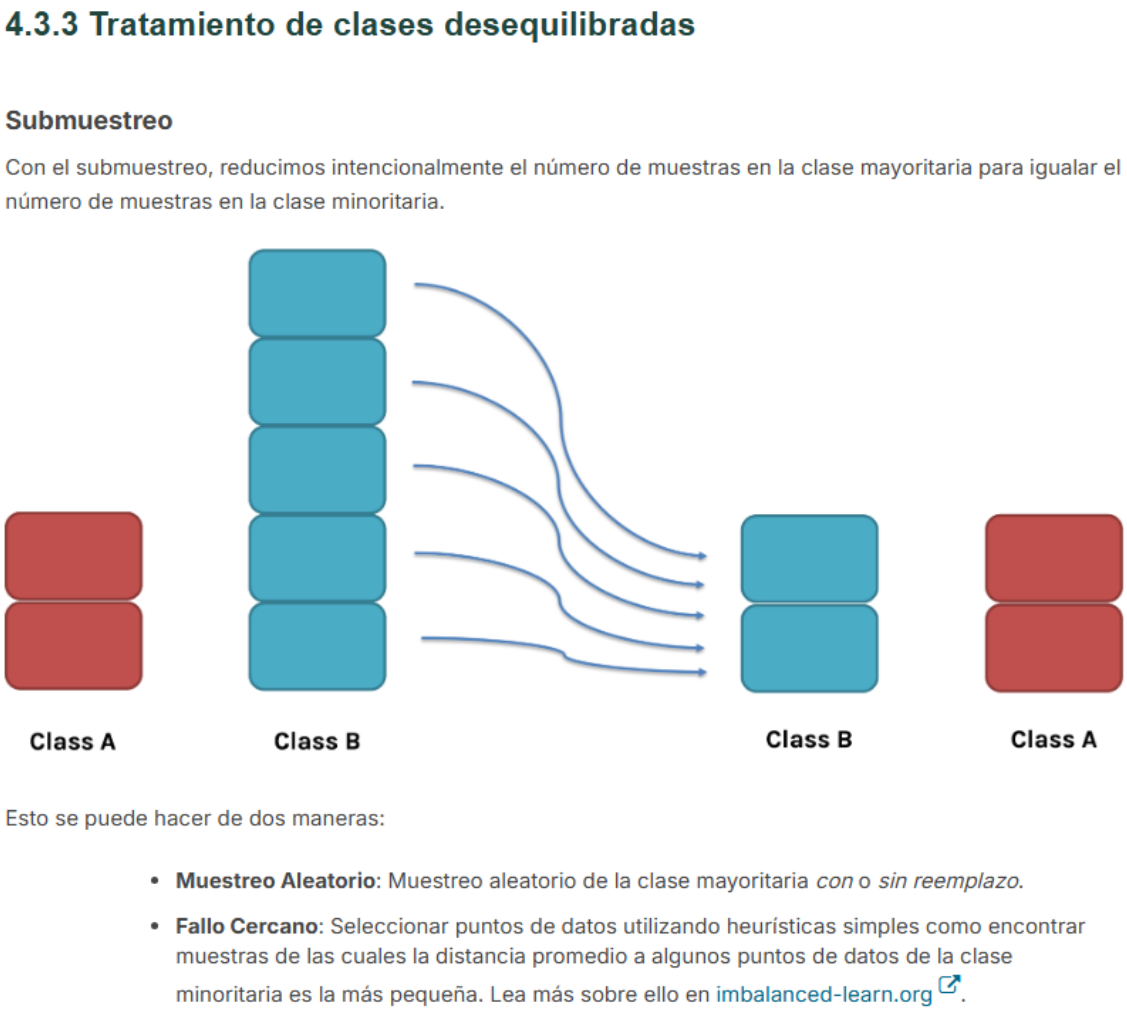


Figura 4.3.3a: Reducción intencionada de la Clase A para emparejar la dimensión de la Clase B mediante muestreo aleatorio o NearMiss.

3. Sobremuestreo (Oversampling) e Interpolación Sintética con SMOTE

El sobremuestreo combate el desequilibrio de datos incrementando la representación de la clase minoritaria. En lugar de descartar información de la clase dominante, genera o multiplica observaciones en la clase con menor densidad de datos.

Técnicas de Sobremuestreo: Replicación vs. Interpolación

- **1. Muestreo Aleatorio con Reemplazo:** Toma muestras aleatorias con reemplazo a partir de los datos existentes de la clase minoritaria. Sin embargo, al duplicar exactamente las mismas

observaciones, incrementa notablemente el riesgo de que el árbol de decisión memorice patrones específicos (sobreajuste).

- **2. Técnica SMOTE (Synthetic Minority Over-sampling Technique):** Representa una alternativa avanzada que supera las limitaciones de la duplicación simple. En lugar de replicar registros idénticos, SMOTE genera observaciones totalmente sintéticas pero plausibles en el espacio de características.

Mecanismo Geométrico de Generación Sintética en SMOTE

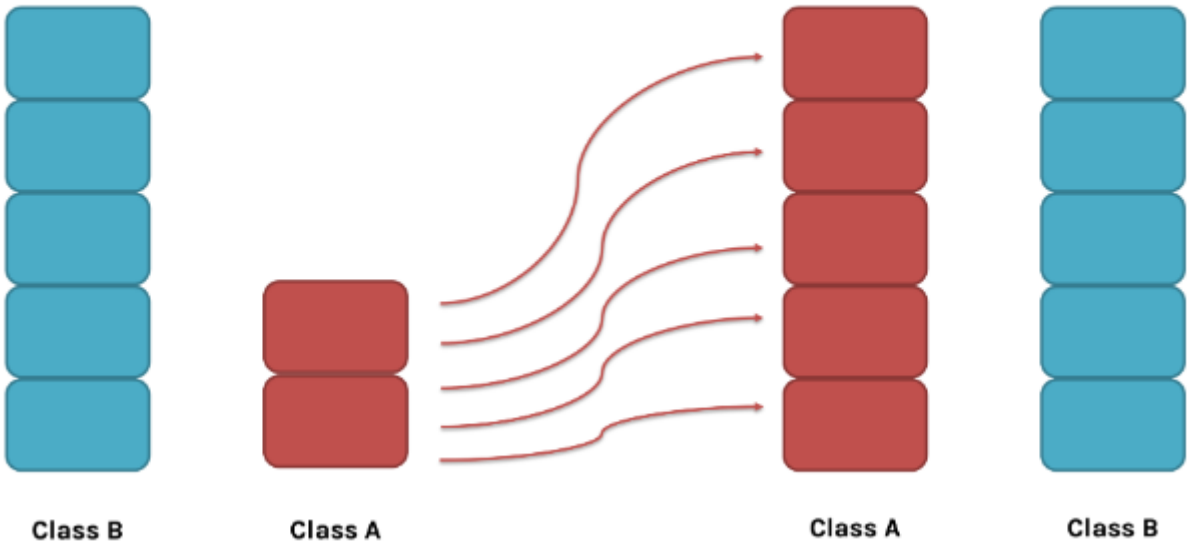
El algoritmo SMOTE opera bajo los siguientes pasos geométricos:

- **Paso 1 (Búsqueda de Vecinos):** Para cada instancia x_i perteneciente a la clase minoritaria, calcula sus k-vecinos más cercanos dentro de su misma clase utilizando la distancia euclídea.
- **Paso 2 (Selección Vectorial):** Selecciona aleatoriamente uno de estos k-vecinos, denominado x_{zi}.
- **Paso 3 (Segmento de Interconexión):** Traza un segmento de recta en el espacio de características que conecta x_i con x_{zi}.
- **Paso 4 (Interpolación):** Genera un nuevo punto sintético x_{nuevo} a lo largo de dicha línea mediante la fórmula: $x_{nuevo} = x_i + \lambda (x_{zi} - x_i)$, donde $\lambda \in [0, 1]$ es un número aleatorio uniforme.

Comparativa Espacial: Antes y Después de Aplicar SMOTE

Sobremuestreo

En el sobremuestreo, combatimos el desequilibrio de datos generando nuevas muestras para la clase minoritaria.



Esto se puede hacer de dos maneras:

- **Muestreo Aleatorio**: Muestreo aleatorio de la clase minoritaria *con reemplazo*.
- **SMOTE**: SMOTE es una alternativa mejorada para el sobremuestreo. SMOTE funciona encontrando puntos que están más cercanos en el espacio de características, trazando una línea entre esos puntos y generando nuevos puntos de datos a lo largo de esta línea. Puedes leer más sobre SMOTE si estás interesado.

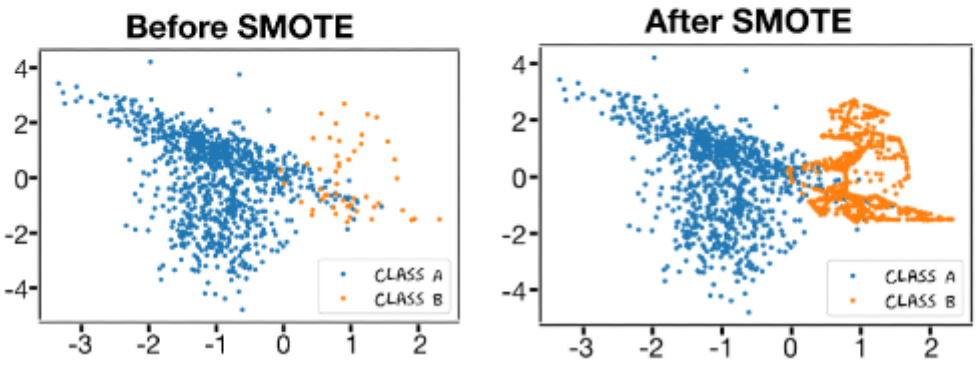


Figura 4.3.3b: Relleno continuo del espacio de características de la clase minoritaria mediante líneas de interpolación entre puntos vecinos con SMOTE.

4. Ponderación de Clases (Class Weighting) y Modificación de la Función de Pérdida

A diferencia del submuestreo y sobremuestreo, la ponderación de clases no modifica la composición cuantitativa del conjunto de datos original. En su lugar, ajusta la función de pérdida asignando un peso superior a los errores cometidos sobre las observaciones de la clase minoritaria. Esto obliga al algoritmo de aprendizaje a priorizar la correcta clasificación de las instancias poco frecuentes.

Fórmula de Ajuste Inverso en Scikit-Learn (`class_weight = 'balanced'`)

En librerías especializadas como Scikit-Learn (`sklearn`), el parámetro `class_weight='balanced'` calcula de forma automática los pesos asignados a cada clase k de manera inversamente proporcional a su frecuencia de aparición en los datos de entrada:

> **Ecuación de Ponderación Inversamente Proporcional:** *W_k = N / (K * N_k)*
>
> *Donde:*
> - *N = Número total de muestras en el conjunto de datos de entrenamiento.*
> - *K = Número total de clases distintas (K = 2 en clasificación binaria).*
> - *N_k = Número total de muestras pertenecientes a la clase k.*

Ejemplo Numérico Ilustrativo

Considérese un problema de clasificación médica con $N = 1.000$ pacientes totales, distribuidos en $K = 2$ clases:

- • Clase 1 (Sanos / Mayoritaria): $N_1 = 900$ muestras.
- • Clase 2 (Enfermos / Minoritaria): $N_2 = 100$ muestras.

Aplicando la fórmula de ponderación automática:

- • Peso para la Clase Mayoritaria (1): W_1 = 1.000 / (2 * 900) = 0,556
- • Peso para la Clase Minoritaria (2): W_2 = 1.000 / (2 * 100) = 5,000

Como resultado, cometer una equivocación al clasificar a un paciente enfermo (Clase 2) le cuesta al algoritmo 9 veces más penalización ($5,000 / 0,556 = 9$) que cometer un error sobre un paciente sano (Clase 1), obligando al árbol de decisión a ajustar sus divisiones para proteger a la clase minoritaria.

Configuración Práctica en Scikit-Learn

Ponderación de clases

Una forma sencilla de abordar el desequilibrio de clases es proporcionar un **peso para cada clase** que otorga más énfasis a las clases minoritarias.

En sklearn podemos proporcionar el peso de clase como un diccionario o usar *class_weight = balanceado*

Luego ajusta automáticamente los pesos de forma inversamente proporcional a las frecuencias de clase en los datos de entrada como: * N es el número total de muestras * N_k es el número de muestras en la clase K * K es el número total de clases.

$$W_k = \frac{N}{K * N_K}$$

Figura 4.3.3c: Especificación del parámetro class_weight = 'balanced' en modelos de clasificación de sklearn.

5. Guía Metodológica para la Selección de la Técnica de Reequilibrio

La elección de la técnica adecuada depende fundamentalmente del volumen total de observaciones disponibles y de la calidad de los datos:

- **Conjuntos de Datos Pequeños o Medianos (< 10.000 muestras):** Se recomienda utilizar 'class_weight = balanced' o SMOTE. El submuestreo debe evitarse porque descartaría muestras valiosas de una clase mayoritaria ya de por sí reducida.
- **Conjuntos de Datos Masivos (> 100.000 muestras):** El submuestreo (especialmente mediante técnicas inteligentes como NearMiss) resulta sumamente eficiente, ya que reduce sustancialmente el coste computacional sin perder la capacidad de representación general del problema.
- **Entornos con Alto Nivel de Ruido o Solapamiento entre Clases:** La ponderación de clases ('class_weight') o el uso combinado de SMOTE con submuestreo de fronteras (por ejemplo, SMOTE + Tomek Links) ofrece el mejor equilibrio entre estabilidad y generalización.

Introducción a Boosting: Intuición, Aprendices Débiles y Comparativa con Random Forest

Apuntes Académicos sobre Algoritmos de Ensambles y Aprendizaje Secuencial

Índice de Contenidos

1. Evolución de los Algoritmos para Datos Tabulares

Los datos tabulares —aquellos estructurados formalmente en filas y columnas como en hojas de cálculo— constituyen el núcleo de las aplicaciones prácticas de la ciencia de datos. A diferencia de dominios con datos no estructurados (imágenes, video, audio o procesamiento de lenguaje natural), la modelización de tablas ha estado históricamente liderada por los algoritmos basados en árboles de decisión.

Durante años, el algoritmo **Random Forest** se posicionó como el modelo dominante indiscutible en las competiciones de Data Science (como Kaggle) y en entornos de producción. Sin embargo, la aparición y perfeccionamiento de las técnicas de **Boosting** transformaron el panorama, convirtiéndose en el método estado del arte para la predicción en estructuras tabulares.

> **Dominio Estadístico en Datos Tabulares**
> Tanto Random Forest como las familias de Boosting representan los dos pilares fundamentales del aprendizaje supervisado para datos tabulares. Comprender la transición analítica de uno a otro es esencial para cualquier profesional en la disciplina.

2. Compromiso Sesgo-Varianza y Limitaciones de Random Forest

Para comprender la motivación de Boosting, es necesario analizar el comportamiento de los árboles de decisión según su profundidad y cómo Bagging / Random Forest gestionan el error de generalización:

- **Árboles Superficiales (Shallow Trees):** Exhiben un alto sesgo (subajuste o underfitting) y una baja varianza. Al realizar pocas divisiones, no logran capturar patrones complejos en los datos.
- **Árboles Profundos (Deep Trees):** Presentan un bajo sesgo pero una alta varianza (sobreajuste o overfitting). A medida que el árbol crece en profundidad, la complejidad del modelo aumenta de forma sustancial.

2.1. Problemas Estructurales de Random Forest

El enfoque de Bagging y Random Forest consiste en construir múltiples árboles muy complejos (profundos) e independientes, para posteriormente promediar sus predicciones. Aunque esta agregación reduce significativamente la varianza, presenta dos inconvenientes principales:

- **Error de Generalización Residencial:** A pesar de reducir la varianza promediando modelos complejos, el error de generalización total puede mantenerse elevado si los árboles individuales están fuertemente correlacionados o si el sesgo residual persiste.
- **Costo Computacional y Velocidad:** Requiere entrenar una cantidad grande n de árboles profundos e independientes. Generar cientos o miles de árboles de gran profundidad implica un consumo computacional y de memoria elevado.

3. Filosofía del Boosting: Aprendizaje Secuencial a partir de los Errores

En lugar de crear modelos profundos y complejos para intentar reducir la varianza a posteriori, la filosofía de Boosting invierte el planteamiento: **comienza con aprendices débiles (weak learners) — árboles muy simples o de baja profundidad— y construye el ensamble de forma secuencial, aprendiendo directamente de los errores cometidos en las iteraciones anteriores.**

3.1. Estrategia de Reducción del Error

Existen dos vías conceptuales para minimizar el error de generalización de un modelo predictivo:

- **Opción 1 (Bagging / Random Forest):** Partir de árboles individuales complejos (sobreajustados) y promediar sus predicciones para mitigar la varianza global del ensamble.
- **Opción 2 (Boosting):** Partir de árboles muy simples (subajustados) con alto sesgo, e incrementar gradualmente la complejidad del modelo combinándolos de forma inteligente iteración tras iteración, reduciendo paulatinamente el sesgo de izquierda a derecha.

Criterio de Diseño	Bagging / Random Forest	Boosting
Base de Aprendizaje	Árboles profundos y complejos (alto sobreajuste individual)	Árboles superficiales / aprendices débiles (alto subajuste)
Objetivo Principal	Reducir la varianza del modelo mediante promediado	Reducir el sesgo mediante combinación inteligente
Secuencia de Construcción	Paralela e independiente entre árboles	Secuencial e iterativa (aprende de errores previos)

4. Intuición Práctica: La Analogía del Examen y Reglas Débiles

Para ilustrar cómo la combinación de reglas simples logra un modelo robusto, consideremos la formulación teórica introducida por **Robert Schapire y Yoav Freund (1996)** mediante un ejemplo intuitivo de preparación para un examen final:

- **Primera Regla Débil:** Se consulta a un primer experto que proporciona una regla sumamente simple: 'Nunca elijas la opción D en el examen'. Esta regla logra un 60% de precisión. Aunque es útil, no alcanza la calificación requerida (A).
- **Segunda Regla Débil (Enfocada en Errores):** Se acude a un segundo experto para analizar únicamente el 40% de las preguntas que se respondieron de forma incorrecta con la primera regla. Este sugiere: 'Si ves la palabra sobreajuste, esa es la respuesta correcta'. La precisión acumulada aumenta.
- **Tercera Regla Débil:** Se consulta a un tercer experto sobre las preguntas que aún se fallaron: 'La validación cruzada siempre es una opción correcta'. La precisión del conjunto alcanza un 70%.

4.1. Combinación Ponderada de Reglas Débiles

El paso crucial en Boosting es determinar cuánta confianza se le otorga a cada regla individual. Matemáticamente, el modelo final asigna un peso específico (α, β, γ) a cada aprendiz débil según su precisión sobre los errores:

> **Formulación Ponderada de Aprendices Débiles**
> Modelo Ensamble Final = α · (Regla 1) + β · (Regla 2) + γ · (Regla 3)
>
> Donde α, β y γ representan los coeficientes de ponderación asignados a cada regla en función de su fiabilidad predictiva. Combinando múltiples aprendices débiles mediante esta estructura ponderada, se obtiene un aprendiz fuerte (strong learner) capaz de aprobar el examen con un rendimiento óptimo.

5. Resumen de Criterios Metodológicos

En conclusión, Boosting no busca construir modelos individuales perfectos, sino articular un proceso iterativo en el cual cada nuevo submodelo corrige las falencias de los anteriores. En las siguientes sesiones se profundizará en la formalización matemática detallada del algoritmo (AdaBoost, Gradient Boosting y XGBoost).

Capítulo 5.1.1: Introducción al Boosting

Fundamentos del Aprendizaje Secuencial, Construcción de Aprendices Débiles y Transición desde Random Forest

Módulo / Tema:	Árboles de Decisión y Algoritmos de Clasificación / Boosting
Tipo de Documento:	Apuntes Académicos de Referencia Técnica

Índice de Contenidos

(Nota: Haga clic derecho sobre el área del índice y seleccione 'Actualizar campos' al abrir en Microsoft Word o Google Docs).

1. Motivación y Contexto Histórico de los Algoritmos de Ensamble

Los datos tabulares —estructurados en filas y columnas como las hojas de cálculo, a diferencia de imágenes, series temporales, audio o lenguaje natural— representan el formato dominante en problemas reales de aprendizaje automático. **Históricamente, los algoritmos basados en Random Forest dominaban las competiciones de predicción sobre datos estructurados. Sin embargo, a partir de 2015 se produjo un cambio de paradigma hacia los métodos de Boosting.**

> *«Solía ser Random Forest el gran ganador, pero en los últimos seis meses un nuevo algoritmo llamado XGBoost ha surgido y está ganando prácticamente todas las competiciones en la categoría de datos estructurados.»*
> **— Anthony Goldbloom, Fundador y CEO de Kaggle (2015)**

En la actualidad, tanto Random Forest como las diferentes variantes de Boosting (XGBoost, LightGBM, CatBoost) constituyen los pilares fundamentales que cualquier científico de datos utiliza en su práctica profesional para el modelado de datos tabulares.

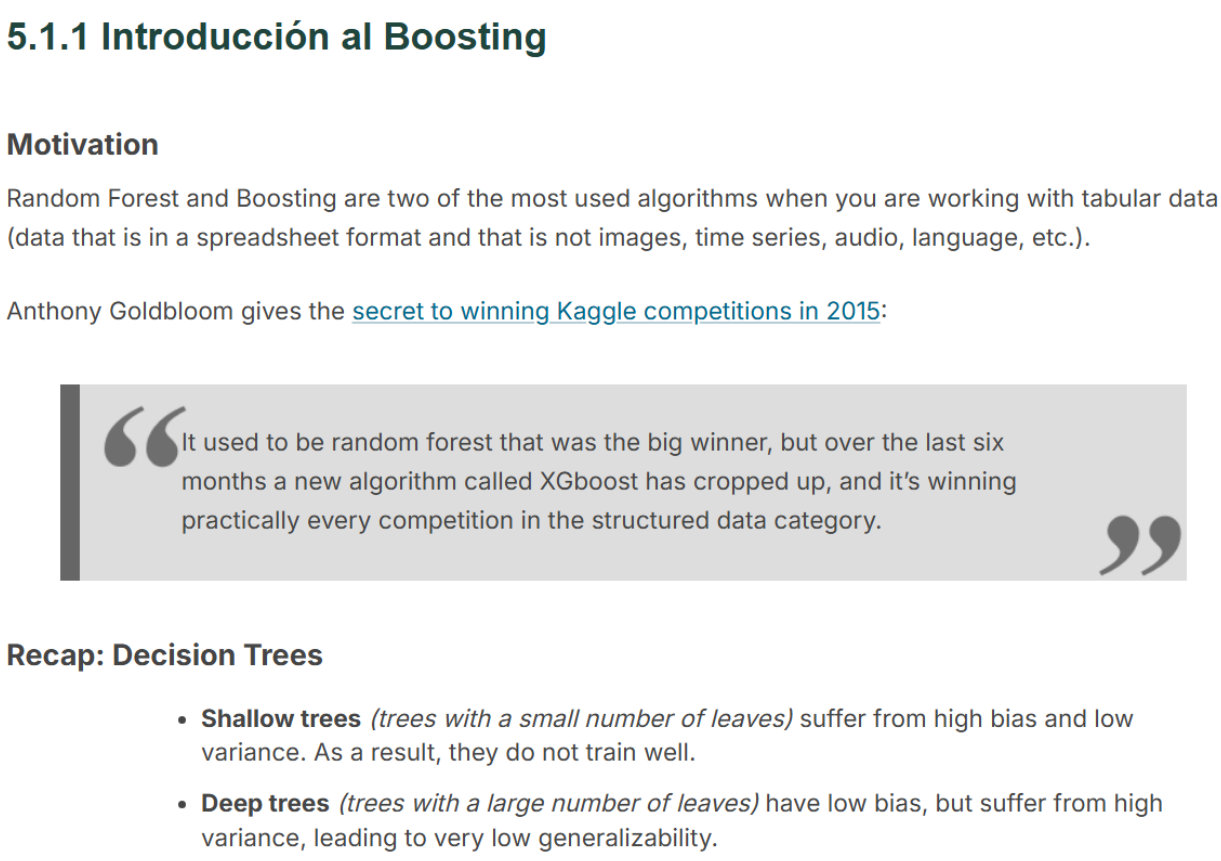


Figura 5.1.1a: Introducción al Boosting, cita de Anthony Goldbloom y repaso de árboles de decisión.

2. Compromiso Sesgo-Varianza en Árboles de Decisión

La capacidad predictiva de un árbol individual depende críticamente de su profundidad estructural y del número de hojas que genera. El comportamiento de los árboles de decisión se analiza bajo el dilema sesgo-varianza (*bias-variance tradeoff*):

Arquitectura del Árbol	Sesgo (Bias)	Varianza	Comportamiento / Ajuste
Árboles Poco Profundos (Shallow Trees / Stumps)	Alto	Baja	Subajuste (Underfitting). No aprenden patrones complejos.
Árboles Profundos (Deep Trees)	Bajo	Alta	Sobreajuste (Overfitting). Baja capacidad de generalización.

A medida que la complejidad del modelo aumenta (añadiendo divisiones y hojas), el sesgo decrece de forma monotónica mientras que la varianza se incrementa. El error total alcanza un punto óptimo en un nivel intermedio de complejidad antes de elevarse debido al sobreajuste.

This might sound familiar. You've seen this before:

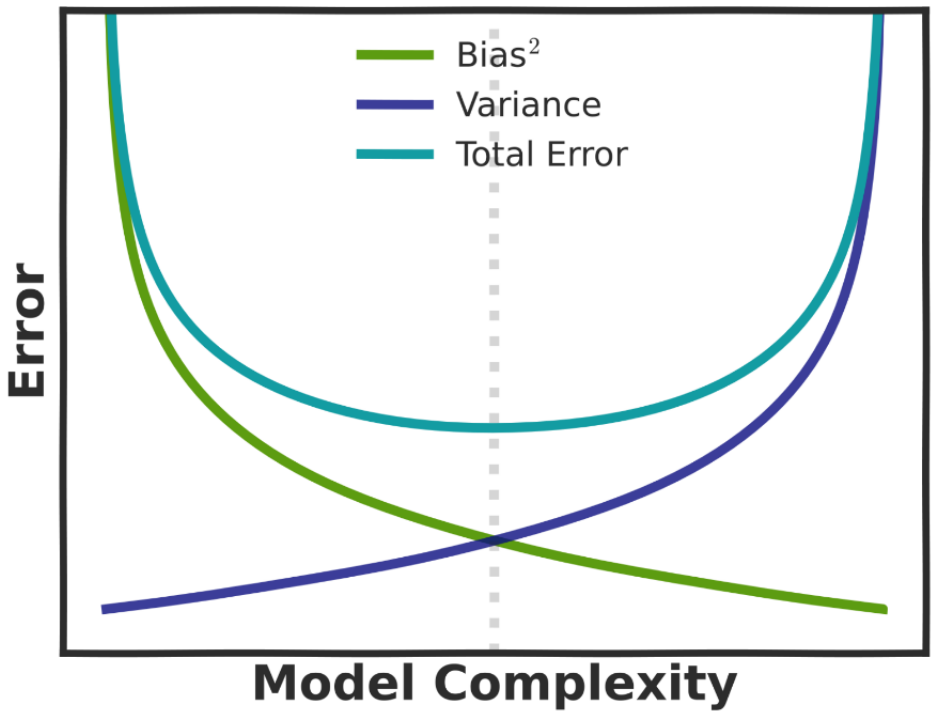


Figura 5.1.1b: Relación entre Complejidad del Modelo, Sesgo, Varianza y Error Total.

3. Deficiencias Estructurales de Random Forest y la Pregunta Clave

A pesar de que los métodos de Bagging y Random Forest reducen eficazmente la varianza mediante el promediado de múltiples árboles profundos e independientes, presentan dos limitaciones operativas críticas:

- **Error de Generalización Residual:** Aunque Random Forest ofrece mayor reducción de varianza que el Bagging estándar, el error de generalización en datos no vistos puede mantenerse elevado si los árboles individuales están fuertemente sobreajustados o correlacionados.
- **Velocidad e Ineficiencia Computacional:** Construir un gran número de árboles profundos (donde n es elevado) requiere un cómputo intensivo en memoria y tiempo, limitando su aplicación en sistemas de predicción e inferencia en tiempo real.

> **CUESTIONAMIENTO METODOLÓGICO CENTRAL**
> **Pregunta de Investigación:** ¿Es posible abordar las deficiencias de los modelos de árbol de decisión mediante una estrategia alternativa? En lugar de realizar una reducción de varianza sobre árboles complejos, ¿podemos reducir el sesgo de árboles simples e imperfectos para hacerlos más expresivos?

Recap: Random Forest Issues

- **Variance:** Although variance reduction is better in RF than bagging, the generalization error could still be high.
- **Speed:** Large number of trees can make the algorithm very slow and ineffective for real-time predictors.

> **QUESTION**
>
> Is it possible to address the shortcomings of single decision tree models in some other way? For example, rather than performing variance reduction on complex trees, can we decrease the bias of simple trees - make them more expressive?
>
> ▶ Show Answer

How does boosting work?

Let's take an exam scenario. Let's imagine that the only passing grade in the exam is A. How would you go about passing the exam? There are a couple of options that you have.

Figura 5.1.1c: Problemas de Random Forest (Varianza y Velocidad) e hipótesis de trabajo de Boosting.

4. Intuición del Boosting: La Analogía del Examen y Reglas Empíricas

Para comprender de manera intuitiva el funcionamiento del Boosting, considérese un escenario académico en el cual la única calificación de aprobación en un examen final es 'A'. Se analizan dos enfoques para alcanzar dicho objetivo:

Opción 1: Viaje en el Tiempo (Enfoque Ideal e Inviable)

Viajar a 1996 para conocer a Rob Schapire y Yoav Freund (creadores de AdaBoost), seguir su trabajo de investigación durante una década hasta dominar los fundamentos del boosting y regresar al presente para aprobar el examen. Ante la imposibilidad del viaje temporal, se recurre a la Opción 2.

Opción 2: Aprendizaje Secuencial a partir de Reglas Empíricas Débiles

Consiste en recopilar preguntas de años anteriores y consultar a distintas fuentes para obtener reglas heurísticas simples (reglas empíricas o aprendices débiles) y combinarlas ponderadamente:

- **Paso 1 (Regla 1 - Estudiante Ayudante):** Se adopta la regla simple «nunca elegir la opción D». Al evaluarla sobre el examen, se obtiene una precisión del 60%. La nota de aprobación requiere un rendimiento mayor.
- **Paso 2 (Regla 2 - Asistente de Docencia):** Se solicita un segundo consejo aplicado únicamente a las preguntas que resultaron incorrectas en el paso anterior. La regla es «elegir opciones que contengan la palabra overfitting». Aplicada sobre los errores residuales, corrige el 50% de ellos, elevando la precisión global al 64%.

- **Paso 3 (Regla 3 - Profesor):** Se solicita una tercera regla aplicada nuevamente a los errores remanentes: «la respuesta correcta suele ser cross-validation». Al integrar esta regla, la precisión alcanza el 70%.

Option 1: Time Travel

1. Steal the time-stone from Dr. Strange.
2. Go back to 1996 and meet Rob Schapire and Yoav Freund.
3. Befriend them by giving stock trading tips from the future.
4. Follow their work for at least a decade to understand everything about boosting.
5. Return to the present and nail the test.

Option 2: Learn from Past Exams

In the disappointing event that we cannot achieve time travel, we'll have to resort to less accurate methods. Here's an alternative, with some sample numbers included.

1. Go to the library and get previous year question papers.
2. Assemble a set of simple rules from different sources:
    1. Find a helpful student and ask them to give you a "rule of thumb" to get at least some answers right. The advice that the student gives you is to never choose option "D" on the exam. Testing out the given rule you find out that it works 60% of the time.
    2. Find a teaching assistant and ask them for a different "rule of thumb". This time the received rule is to always choose an option that has the word "overfitting" in it. You apply this rule only to the questions you got wrong in the previous step, and you get half of those questions correct now.
    3. Call your professor and ask them for a third "rule of thumb". They tell you that the right answer is almost always "cross-validation". Apply this rule on the ones that you got wrong before.
3. Combine the rules together.

Figura 5.1.1d: Enfoque de preparación del examen mediante recopilación secuencial de reglas empíricas.

As a result of the first rule you get 60% correct answers, after the second one you get 64% correct, and after the last third rule 70% answers are correct. However, you have to decide which person you trust more. If your trust one person more than others, you put more weight to their rule. In your final strategy each rule will have some weight assigned to it:

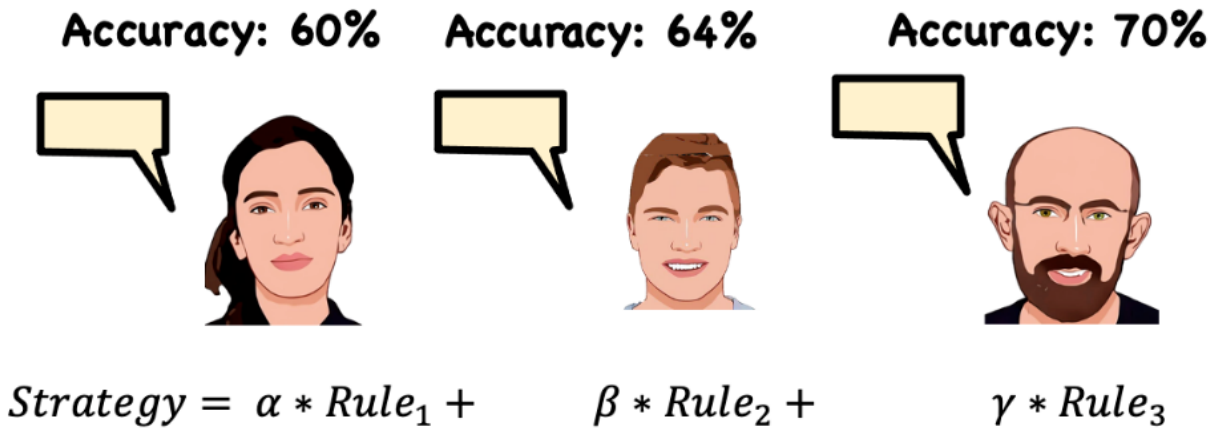


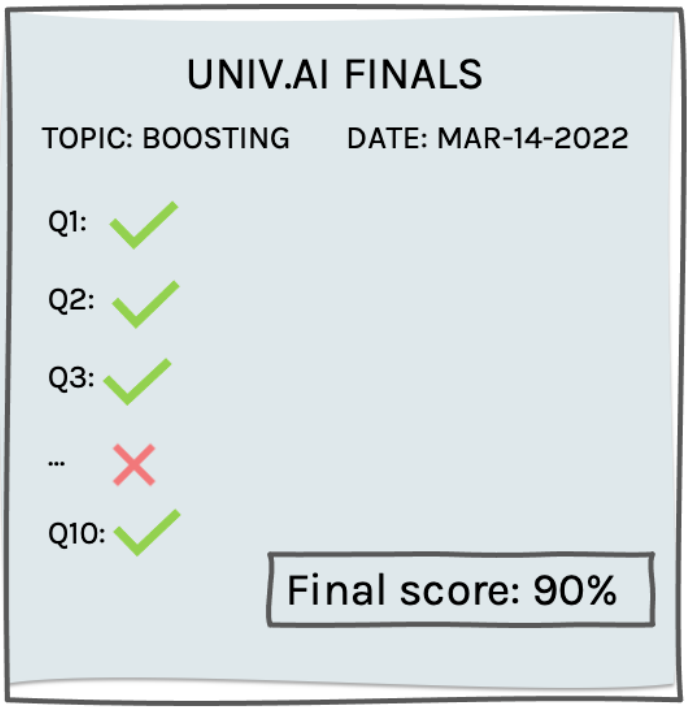
Figura 5.1.1e: Acumulación de precisión secuencial (60% -> 64% -> 70%) y estrategia ponderada.

5. Formulación Matemática de la Combinación Ponderada

El elemento diferenciador de Boosting radica en asignar una importancia o peso diferencial a cada regla según el grado de confianza en la fuente o su rendimiento sobre las observaciones. La estrategia de predicción final se expresa formalmente como una combinación lineal ponderada:

> **FÓRMULA DEL MODELO COMBINADO EN BOOSTING**
>
> **Ecuación General de Estrategia Ponderada:** Estrategia Final = alpha * Regla_1 + beta * Regla_2 + gamma * Regla_3
>
> **Donde:** alpha, beta, gamma representan los coeficientes de confianza (pesos) asignados a cada aprendiz débil según su capacidad predictiva y tasa de error.

Your final step is, of course, taking the test! Use a model consisting of the three simple rules combined appropriately.



Obtaining simple rules and combining them together with respective weights into one strong model is the basic idea of boosting. Boosting methods are general algorithms which combine several "weak learners" to produce a strong rule. In our example above, weak learners were "rules of thumb", in practice usually some sort of simple model is used, like a shallow tree.

Figura 5.1.1f: Modelo final de Boosting integrando aprendices débiles en un modelo fuerte unificado.

6. Definición Formal y Cuadro Comparativo: Bagging vs Boosting

En la terminología formal de aprendizaje automático, las reglas empíricas simples corresponden a **aprendices débiles (weak learners)**, que en la práctica suelen ser árboles de decisión muy poco profundos (como los tocones de decisión o *decision stumps*). Los algoritmos de Boosting son métodos generales que combinan secuencialmente múltiples aprendices débiles para construir un predictor altamente preciso o **aprendiz fuerte (strong learner)**.

Dimensión / Criterio	Bagging & Random Forest	Boosting (AdaBoost / XGBoost)
Arquitectura de Base	Árboles profundos (alto sesgo, alta varianza)	Árboles poco profundos / Stumps (alto sesgo, baja varianza)
Construcción del Ensamble	Paralela e independiente mediante	Secuencial e iterativa basada en

	muestras Bootstrap	errores previos
Objetivo Principal	Reducir la Varianza sin alterar sustancialmente el sesgo	Reducir el Sesgo aumentando progresivamente la complejidad
Ponderación de Modelos	Voto por mayoría o promediado simple no ponderado	Combinación lineal ponderada según la precisión de cada árbol
Riesgo Principal	Variaza residual y costo computacional en inferencia	Sensibilidad al ruido y sobreajuste si no se regulariza

Gradient Boosting: Aprendizaje Iterativo y Ajuste sobre Residuos

Sintesis Técnica sobre la Construcción Secuencial de Modelos Fuertes a partir de Aprendices Débiles

---

1. Principios Fundamentales del Gradient Boosting

El algoritmo de **Gradient Boosting** se basa en la combinación lineal de múltiples modelos simples o aprendices débiles (denotados como *T_h*) para construir un predictor global fuertemente expresivo (denotado como *T*). A diferencia de algoritmos como Random Forest, donde cada árbol se entrena de manera independiente y paralela sobre muestras bootstrap, Gradient Boosting opera de forma estrictamente iterativa y secuencial.

En cada iteración del proceso, el nuevo modelo agregado no intenta predecir la variable objetivo original de forma aislada, sino que se diseña para compensar las deficiencias y errores cometidos por el ensamble acumulado hasta ese momento. La premisa central del boosting radica en **aprender de los propios errores**, reenfocando la capacidad de aprendizaje del nuevo árbol hacia las observaciones donde el modelo actual presenta un rendimiento insatisfactorio.

> **Concepto Clave: Iteratividad y Aprendizaje Secuencial**
> Mientras que en Bagging y Random Forest los árboles no se comunican entre sí durante el entrenamiento, en Gradient Boosting la iteración m depende directamente del estado del modelo en la iteración $m-1$. Cada nuevo árbol simple actúa como un corrector especializado en los residuos del modelo precedente.

2. El Mecanismo de Ajuste Paso a Paso mediante Residuos

Para ilustrar el funcionamiento practico de este algoritmo en un problema de regresión, se utiliza un conjunto de aprendices extremadamente simples conocidos como **árboles tocón (stumps)**, los cuales consisten en árboles de decisión con una única división elegida para minimizar el error cuadrático medio (MSE).

Paso 1: Ajuste del Árbol Inicial sobre la Variable Objetivo

Se entrena el primer árbol tocón sobre los datos originales. La división óptima que minimiza el MSE ocurre en la condición **x > 6.8**:

- Para la región donde **x > 6.8**, el árbol calcula la media de los valores observados de y, prediciendo un valor constante de **7.0**.

- Para la región donde **x <= 6.8**, el árbol asigna la media correspondiente a ese grupo, prediciendo un valor constante de **0.01**.

Paso 2: Cálculo de los Residuos Iniciales

Los errores del primer modelo se expresan cuantitativamente mediante sus **residuos**, definidos como la diferencia matemática entre el valor real observado (y_i) y la predicción emitida por el primer árbol ($\hat{y}_i^{(1)}$):

Residuo_i = y_i - y_hat_i^(1)

Paso 3: Entrenamiento del Segundo Árbol sobre los Residuos

En lugar de reentrenar sobre y, el segundo árbol de tocón toma los residuos calculados como su nueva variable objetivo. Para este segundo árbol, la división óptima que minimiza el MSE sobre los residuos se localiza en **x = 3.5**. Este árbol captura el patrón del error que el primer árbol no logró explicar.

3. Acumulación Ponderada mediante Tasa de Aprendizaje (lambda)

Para evitar el sobreajuste y permitir un aprendizaje gradual, la predicción del segundo árbol no se suma directamente de forma íntegra. En su lugar, se escala mediante un hiperparámetro de tasa de aprendizaje o contraimiento (denotado como **lambda** o *learning rate*).

El modelo combinado resultante no reemplaza las estructuras previas por un árbol de mayor profundidad, sino que acumula sus predicciones secuencialmente:

y_hat_combinado = y_hat_arbol1 + lambda * Prediccion_arbol2(residuos)

A continuación, el proceso se repite de forma cíclica:

- Se recalculan los nuevos residuos restando la predicción combinada actual del valor real y.
- Se entrena un tercer árbol de tocón sobre estos nuevos residuos, encontrando la división óptima en **x = 3.1**.
- Se añade la contribución escalada del tercer árbol al ensamble existente: **y_hat_nuevo = y_hat_anterior + lambda * Prediccion_arbol3**.

4. Algoritmo General de Gradient Boosting para Regresión

El procedimiento secuencial descrito se sintetiza en la siguiente estructura algorítmica iterativa:

Paso / Fase	Descripción Operativa del Algoritmo

1. Inicialización	Se construye un primer modelo base f_0(x) (por ejemplo, la media de y o un árbol inicial simple sobre los datos originales).
2. Cálculo de Residuos	Para cada iteración m = 1, 2, ..., M, se calculan los residuos actuales: r_i,m = y_i - f_{m-1}(x_i).
3. Ajuste del Aprendiz Débil	Se entrena un nuevo árbol simple h_m(x) utilizando las variables predictoras X y los residuos r_i,m como variable objetivo.
4. Actualización del Ensamble	Se actualiza el modelo acumulado añadiendo el nuevo árbol escalado: f_m(x) = f_{m-1}(x) + lambda * h_m(x), hasta alcanzar el criterio de parada.

5. Tabla Comparativa: Random Forest vs. Gradient Boosting

La siguiente matriz sintetiza las diferencias estructurales clave entre ambos enfoques de ensamblado:

Dimensión	Random Forest (Bagging)	Gradient Boosting
Construcción de Árboles	Independiente y en paralelo.	Iterativa y secuencial.
Complejidad del Árbol Base	Árboles profundos (Bajo sesgo, alta varianza).	Árboles superficiales / Stumps (Alto sesgo, baja varianza).
Mecanismo de Corrección	Reducción de varianza mediante promediado / votación.	Reducción de sesgo mediante ajuste sobre residuos.
Efecto de la Tasa (lambda)	No aplica (se promedian los árboles por igual).	Escala la contribución de cada árbol para evitar sobreajuste.

TEMA 5.2: ALGORITMOS DE BOOSTING

5.2.1 Recapitulación del Boosting: De Aprendices Débiles a un Modelo Fuerte

1. La Pregunta Teórica Fundamental del Boosting

El marco teórico del Boosting surgió formalmente en la teoría del aprendizaje computacional como respuesta a un interrogante fundamental planteado a finales de la década de 1980. Esta formulación buscaba determinar si era posible mejorar la capacidad predictiva de clasificadores simples mediante un procedimiento formal de combinación.

> ***«¿Puede un conjunto de aprendices débiles crear un único aprendiz fuerte?»***
>
> **— Leslie Gabriel Valiant**
>
> *Profesor T. Jefferson Coolidge de Ciencias de la Computación y Matemáticas Aplicadas en la Universidad de Harvard.*

Para comprender el alcance de esta formulación, es preciso diferenciar conceptualmente ambos tipos de modelos:

- **Aprendiz Débil (Weak Learner):** Modelo hipotético o clasificador simple cuyo rendimiento predictivo es solo ligeramente superior a una adivinación aleatoria (precisión > 50% en clasificación binaria). En la práctica, suele tratarse de un árbol de decisión muy superficial (como un muñón de decisión o *decision stump*), caracterizado por un alto sesgo y una baja varianza.
- **Aprendiz Fuerte (Strong Learner):** Modelo con una elevada capacidad predictiva, capaz de lograr una precisión arbitrariamente alta sobre los datos y exhibir un bajo error de generalización.

2. Intuición Clave: Agregación Aditiva de Modelos Simples

La intuición central que articula todos los algoritmos de boosting establece que es posible tomar una familia de modelos individuales simples **(T_h, h ∈ H)** y combinarlos aditivamente para construir un único modelo ensamble significativamente más expresivo y complejo.

En etapas introductorias, esta estrategia se ilustra mediante la combinación de simples «reglas empíricas» (*rules of thumb*). Aunque cada modelo individual **T_h** por separado sea limitado e incurra en un subajuste alto (alto sesgo) al evaluar la estructura de los datos, la **combinación lineal** de todo el conjunto de aprendices proporciona la flexibilidad necesaria para capturar patrones no lineales altamente complejos.

3. Formulación Matemática de la Combinación Lineal

Matemáticamente, la función de predicción global del modelo ensamble **T** se define como una suma ponderada sobre la familia de aprendices débiles seleccionados:

$$T = \sum \lambda_h \cdot T_h \quad (\text{para } h \in H)$$

Desglose técnico de los componentes de la ecuación:

- **T:** Modelo fuerte o ensamble final consolidado, cuyo comportamiento resulta de la superposición de las contribuciones de todos los aprendices.
- **T_h:** Cada aprendiz débil individual perteneciente al espacio de hipótesis H (por ejemplo, un árbol de decisión básico ajustado en una etapa específica).
- **λ_h (Lambda):** Coeficiente numérico o peso asignado al modelo T_h. Cuantifica el grado de confianza, ponderación o importancia de la regla h dentro de la predicción global.
- **∑_h:** Operador de suma aditiva que integra secuencialmente las respuestas de todos los modelos simples incorporados al sistema.

4. Cuadro Comparativo: Aprendiz Débil vs. Ensamble Fuerte

Dimension Propiedad	Aprendiz Débil (T_h)	Ensamble Boosting (T)
Complejidad del Modelo	Baja (ej. tocón o árbol superficial)	Alta (combinación aditiva flexible)
Estructura de Sesgo	Alto Sesgo (subajusta los datos)	Bajo Sesgo (alta expresividad)
Mecanismo de Acción	Regla simple individual	Suma ponderada de aprendices (∑ λ_h T_h)
Rendimiento Predictivo	Ligeramente superior al azar (>50%)	Arbitrariamente alto (precisión elevada)

5. Material Gráfico de Referencia

A continuación se presenta la lámina original de la lección, donde se resume la cita histórica de Leslie Valiant y la expresión matemática fundamental de la combinación aditiva en Boosting:

5.2.1 Recapitulación del Boosting

Resumen: Boosting

> ¿Puede un conjunto de aprendices débiles crear un único aprendiz fuerte?
>
> *--Leslie Gabriel Valiant, Profesor T. Jefferson Coolidge de Ciencias de la Computación y Matemáticas Aplicadas en la Universidad de Harvard.*

La intuición clave detrás del boosting es que se puede tomar un conjunto de modelos simples ($T_h, h \in \mathbb{H}$) y combinarlos aditivamente en un único modelo, más complejo.

Anteriormente combinamos simples "reglas empíricas" como esas. Cada modelo T_h podría ajustarse mal a los datos, pero una **combinación lineal** del conjunto podría ser expresiva/flexible:

$$T = \sum_h \lambda_h T_h$$

Figura 5.2.1: Lámina explicativa sobre la recapitulación de Boosting, la cita de Leslie Valiant y la formulación aditiva T = ∑ λ_h T_h.

Apuntes de Estudio: Potenciación de Gradiente (Gradient Boosting paso a paso)

Sección 5.2.2 | Construcción Iterativa de Modelos Complejos mediante el Ajuste Secuencial de Residuos

Tabla de Contenidos del Documento

1. Principios Clave y Filosofía del Gradient Boosting

El algoritmo de **Potenciación de Gradiente (Gradient Boosting)** constituye una de las técnicas de aprendizaje supervisado más potentes y utilizadas para datos tabulares en ciencia de datos. A diferencia de las metodologías de ensamble paralelo como Bagging o Random Forest, el Gradient Boosting construye el modelo predictivo de manera estrictamente **secuencial e iterativa**.

> **DOS IDEAS FUNDAMENTALES DEL GRADIENT BOOSTING**
>
> 1. Proceso Iterativo y Secuencial: Cada nuevo modelo simple añadido al ensamble depende de forma directa de los resultados y errores producidos por los modelos anteriores.
> 2. Compensación de Debilidades: El objetivo de cada nueva iteración es entrenar un aprendiz débil (usualmente un árbol de decisión de profundidad 1 o 'stump') que se enfoque específicamente en corregir los residuos (errores) del ensamble acumulado.
> 3. Ensamble Aditivo de Complejidad Progresiva: La predicción final no se obtiene mediante promediado independiente, sino como una suma ponderada de modelos simples: T = T^(1) + λ T^(2) + λ T^(3) + ...

2. Ejecución Algorítmica Paso a Paso (Ejemplo Numérico Práctico)

Para comprender con precisión el funcionamiento interno del Gradient Boosting, se analiza el desarrollo de un problema de regresión continuo utilizando como aprendices débiles árboles de decisión de profundidad unívoca, conocidos técnicamente como **tocones de decisión (stumps)**.

Paso 1: Ajuste del Primer Aprendiz Débil (T^(1))

Se inicia el proceso entrenando un modelo inicial simple **T^(1)** sobre los datos de entrenamiento originales {(x_1, y_1), ..., (x_N, y_N)}. El algoritmo selecciona la división óptima en el punto **x = 6.8** para minimizar el Error Cuadrático Medio (MSE):

- Si **x > 6.8**: El árbol predice el valor medio de las observaciones en esa región, obteniendo **$\bar{y}$ = 7**.
- Si **x ≤ 6.8**: El árbol predice el valor medio del subgrupo correspondiente, resultando en **$\bar{y}$ = 0**.

Paso 2: Cálculo de los Primeros Residuos (r_1, ..., r_N)

Para lograr que el modelo aprenda de sus propios errores, se expresan formalmente las deficiencias del modelo a través de los **residuos (residuals)**. El residuo de cada observación i se define como la diferencia exacta entre la variable objetivo real y la predicción del primer árbol:

$$r_i = y_i - T^{(1)}(x_i)$$

Se establece el modelo acumulado inicial como **T ← T^(1)** y se pasa a la siguiente etapa utilizando los residuos como la nueva variable objetivo a predecir.

Paso 3: Entrenamiento del Segundo Tocón sobre Residuos (T^(2))

Se construye un segundo árbol de profundidad 1 (**T^(2)**) ajustado sobre los residuos {r_1, ..., r_N} obtenidos en el paso 2. La mejor división que minimiza el MSE ponderado de los residuos se localiza en **x = 3.5**:

- Si **x > 3.5**: El modelo predice un residuo medio de **r̄ = -2**.
- Si **x ≤ 3.5**: El modelo predice un residuo medio de **r̄ = 2**.

Paso 4: Combinación Ponderada de Modelos (T ← T + λ T^(2))

Para refinar la predicción global, se combina el primer árbol con el segundo escalado por un factor de contracción o **tasa de aprendizaje (learning rate, λ)**. Asumiendo un valor ilustrativo de **λ = 0.5**, la actualización sigue la fórmula **T ← T + λ T^(2)**. Las nuevas predicciones por regiones quedan configuradas como sigue:

Región de la Variable (x)	Cálculo Combinado de la Predicción	Predicción Final Resultante
x > 6.8 (y x > 3.5)	7 + 0.5 × (-2)	6.0
3.5 < x ≤ 6.8	0 + 0.5 × (-2)	-1.0
x ≤ 3.5	0 + 0.5 × (2)	1.0

Nota Técnica: Aunque estas reglas agrupadas podrían estructurarse gráficamente como un único árbol más profundo, en la práctica el algoritmo no reestructura la arquitectura del árbol, sino que simplemente continúa sumando secuencialmente las predicciones numéricas a las existentes.

Pasos 5, 6 y 7: Iteración Continua sobre los Nuevos Residuos

El proceso se repite sistemáticamente evaluando los nuevos errores del modelo combinado T:

- **Paso 5:** Se recalculan los residuos r_1, ..., r_N comparando los valores reales y_i contra las predicciones del modelo combinado T (valores 6.0, -1.0 y 1.0).
- **Paso 6:** Se ajusta un tercer tocón T^(3) sobre los nuevos residuos. La división óptima se encuentra en x = 3.1, prediciendo -1 para x > 3.1 y +1 para x ≤ 3.1.

- **Paso 7:** Se actualiza nuevamente el ensamble sumando el tercer árbol escalado: $T \leftarrow T + \lambda T^{(3)}$. Este ciclo iterativo se repite sucesivamente hasta cumplir con los criterios de detención predefinidos.

3. Formulación Matemática del Algoritmo General

A continuación se presenta la estructura algorítmica formal de Gradient Boosting para problemas de regresión:

Fase del Algoritmo	Ecuación / Procedimiento Formal
1. Inicialización	Ajustar un árbol base inicial $T^{(1)}(x)$ sobre los datos de entrenamiento $\{(x_i, y_i)\}$ y establecer $T(x) = T^{(1)}(x)$.
2. Cálculo de Residuos	Para $m = 2$ hasta M iteraciones: Calcular los residuos $r_{i, m} = y_i - T(x_i)$ para $i = 1, ..., N$.
3. Ajuste y Actualización	Ajustar un aprendiz débil $T^{(m)}(x)$ sobre $\{(x_i, r_{i, m})\}$. Actualizar el ensamble acumulado: $T(x) \leftarrow T(x) + \lambda T^{(m)}(x)$.

4. Cuadro Síntesis de Propiedades Metodológicas

Criterio de Evaluación	Bagging	Random Forest	Gradient Boosting
Construcción de Árboles	Paralela e Independiente	Paralela e Independiente	Secuencial e Iterativa
Arquitectura de Aprendices	Árboles profundos (alto varianza)	Árboles profundos (descorrelacionados)	Árboles superficiales / Tocones (alto sesgo)
Mecanismo Principal	Reducción de Varianza	Reducción de Varianza	Reducción de Sesgo (Ajuste a Residuos)

5. Galería de Láminas y Esquemas del Capítulo

Se integran a continuación las láminas visuales correspondientes a la secuencia algorítmica expuesta en la lección:

5.2.2 Potenciación de Gradiente

Refuerzo de gradiente

Dos ideas principales del refuerzo de gradiente:

1. Se realiza **iterativamente**, a diferencia de bagging y random forest. Cada nuevo modelo añadido dependería del resultado de los modelos anteriores.

2. Cada nuevo modelo simple añadido al conjunto compensa las **debilidades** del **actual** conjunto.

El refuerzo de gradiente es un método para **iterativamente** construir un *complejo* modelo T por *añadiendo modelos simples*.

Gradient Boosting with an example

Consider the dataset in the graph below:

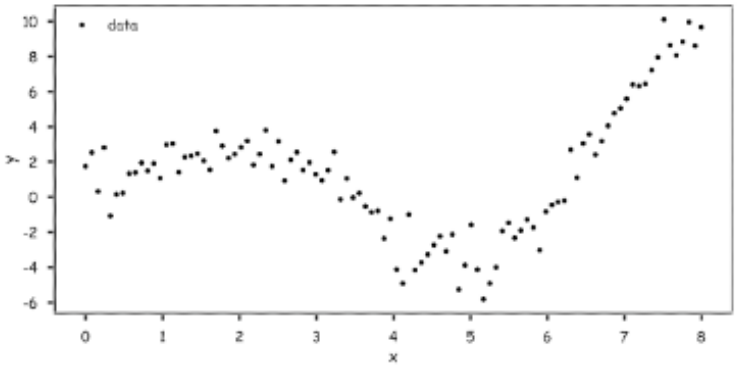
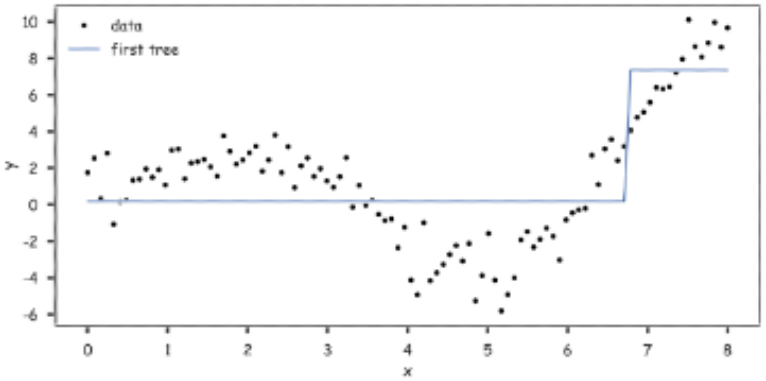


Figura 1: Dos ideas principales del refuerzo de gradiente e introducción del dataset inicial.

Let us use a very weak learner, a regression tree with only one split: the tree of depth one, aka a **"stump"**.

Step 1

Fit a simple model $T^{(0)}$ on the training data $\{(x_1, y_1), \ldots, (x_N, y_N)\}$. The model $T^{(0)}$ has only one split at x = 6.8. If a point has x that is above x = 6.8 then the tree predicts the mean of the y values of the points that are above x = 6.8, which in this case is $\hat{y} = 7$. If a point has x that is below x = 6.8, then the tree predicts the mean of the points that are below x = 6.8, which would be $\hat{y} = 0$ in this example.



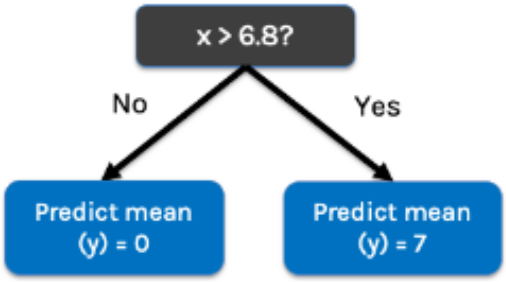
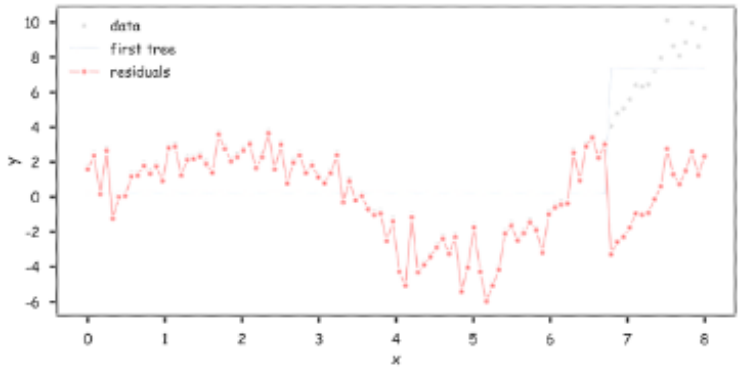


Figura 2: Paso 1 - Ajuste del primer aprendiz débil T^(1) sobre la división x = 6.8.

Step 2

We want to learn from our mistakes. How could we express the mistakes of the algorithm here?

We could look at the residuals! Compute the residuals $r_1, \ldots, r_N$ for $T^{(0)}$, and set $T \leftarrow T^{(0)}$.



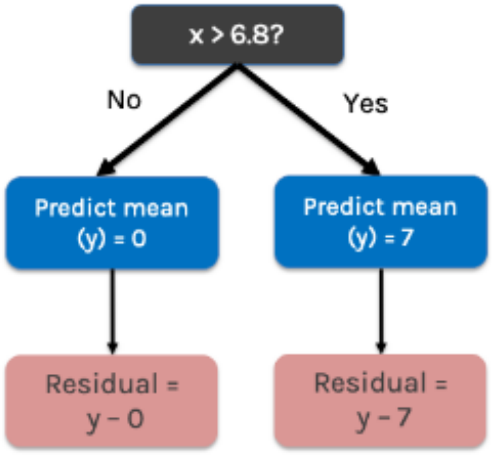
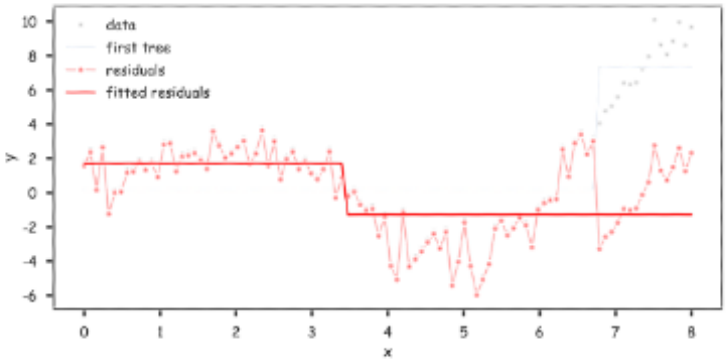


Figura 3: Paso 2 - Expresión formal de errores mediante el cálculo de residuos r_i.

Step 3

Now, we will build a tree of the same depth to predict the residuals. It will be trained on the residuals that we got in Step 2, instead of the original data. Based on the data, x = 3.5 is the best split we can perform to minimize the weighted MSE of the residuals.

Fit another model $T^{(1)}$ on $r_1, \ldots, r_N$.



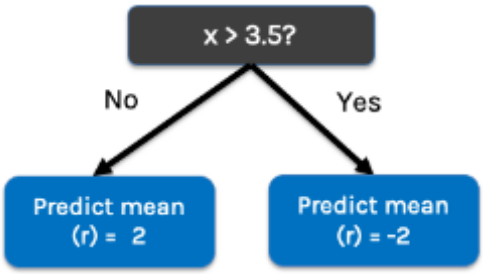
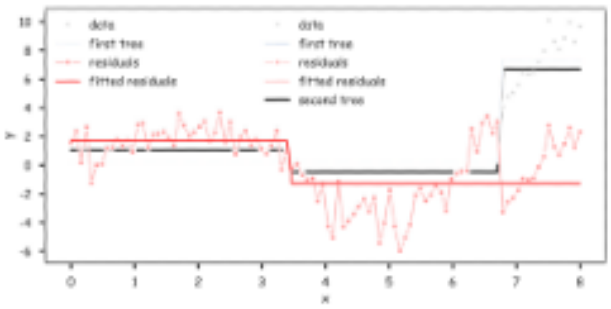


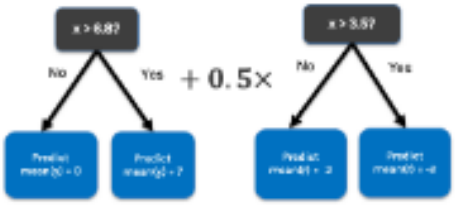
Figura 4: Paso 3 - Ajuste del segundo árbol T^(2) sobre los residuos en la división x = 3.5.

Step 4

If we combine the first tree (from Step 1) and the second tree (from Step 3) we could get better predictions. We can create a third tree that will be a combination of the predictions of the first and second trees.

Combine the two trees in Steps 1 and 3 by setting $T \leftarrow T + \lambda T^{(1)}$. Assume $\lambda = 0.5$. We will explain later how to set λ.





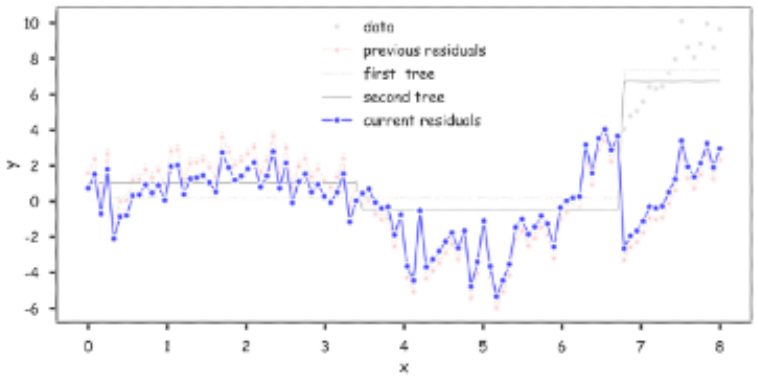
The third tree, which is a combination of the first two trees, predicts: - if an observation has x > 6.8, the predicted value will be 7 + 0.5 • (-2) = 6. - if an observation has x > 6.8 but x is not larger than 6.8, then the predicted value is 0 + 0.5 • -2 = -1 - if the observation's x is not larger than 3.5, the predicted value will be 0.5 • 2 = 1

The predictions could be represented as a deeper tree; however, in practice we do not create the deeper tree, we just keep adding the predictions to the existing ones.

Figura 5: Paso 4 - Combinación T^(1) + λ T^(2) con factor de aprendizaje λ = 0.5.

Step 5

Repeat Step 2 on the new model by calculating the residuals $r_1, \ldots, r_N$ for current model, our third tree. This is a combination of the first tree fitted to the data and the second tree fitted to the calculated residuals of the first tree.



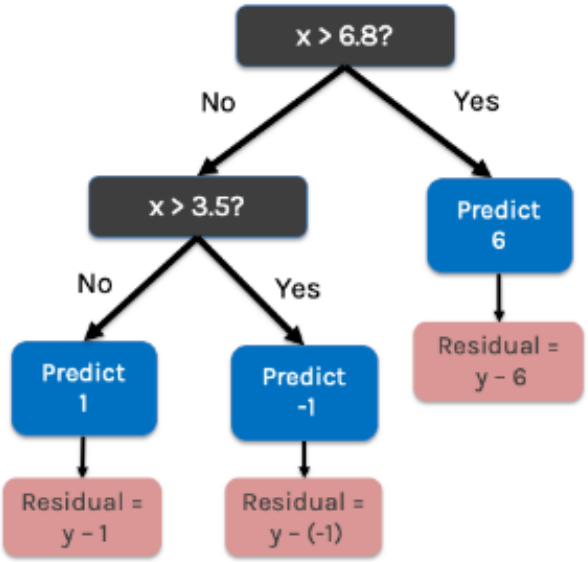
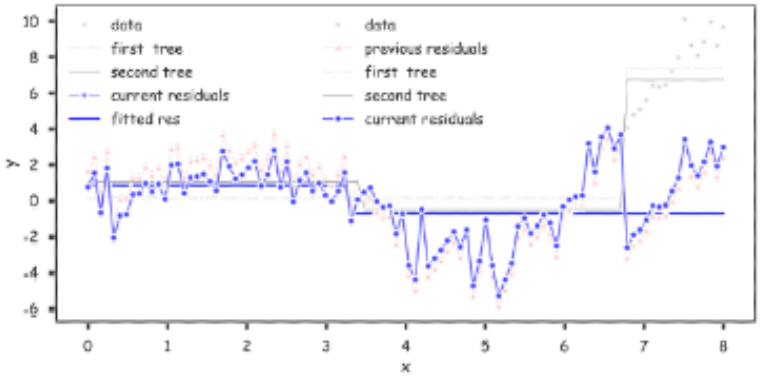


Figura 6: Paso 5 - Cálculo de nuevos residuos sobre las predicciones refinadas del modelo combinado.

Step 6

Repeat Step 3 and fit another simple tree model of the same depth of one $T^{(2)}$ on the new residuals calculated in Step 5: $(x_1, r_1), \ldots, (x_N, r_N)$

After fitting a new stump on the new residuals, we see the decision rule for the stump tree as follows: if x > 3.1 the model predicts -1, else it predicts 1.



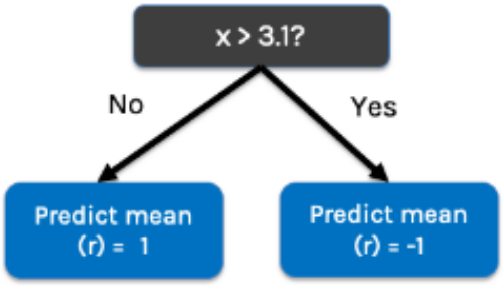
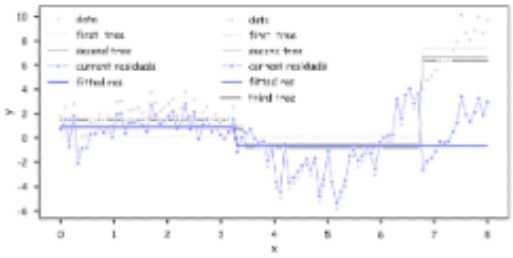
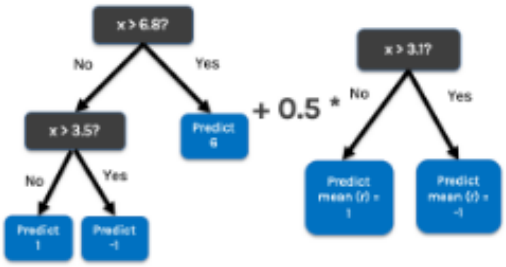


Figura 7: Paso 6 - Ajuste del tercer tocón T^(3) sobre la división x = 3.1.

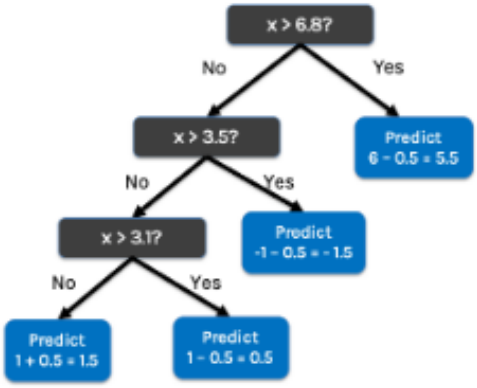
Step 7

As before, we combine the third tree from Step 4 (our first combined tree) with the new stump model fitted to the residuals in Step 6 using some λ. Combine the two trees in Steps 4 and 6 by setting $T \leftarrow T + \lambda T^{(2)}$. Again we assume $\lambda = 0.5$.





We can represent the combination of two trees from Step 7 as one large tree as follows. (For better visualization, in reality, we are not building complex trees, we are just adding predictions).



We repeat the steps again and again until some stopping criteria are met, which we will introduce in a moment.

Figura 8: Paso

APUNTES DE ESTUDIO TÉCNICO | APRENDIZAJE POR ENSAMBLE

5.2.3 Resumen del Algoritmo de Gradient Boosting

Módulo de Modelos Basados en Árboles y Boosting | Documento Consolidado

---

Índice de Contenidos

(La tabla de contenidos se generará automáticamente al abrir este documento en Microsoft Word o Google Docs).

1. Fundamentos y Mecanismo General del Gradient Boosting

El algoritmo de **Gradient Boosting** es una técnica de aprendizaje supervisado por ensamble que construye un modelo predictivo complejo mediante la adición secuencial de modelos base simples (conocidos como **aprendices débiles** o *weak learners*, habitualmente árboles de decisión de baja profundidad o tocones).

A diferencia de las metodologías de ensamble paralelo como *Bagging* y *Random Forest*, donde cada árbol se entrena de forma independiente para reducir la varianza de modelos complejos, el Gradient Boosting opera de manera estrictamente iterativa. Cada nuevo modelo introducido en la secuencia se entrena explícitamente para predecir los **residuos de error** acumulados por el ensamble en los pasos anteriores, reduciendo sistemáticamente el sesgo del modelo.

2. Formulación Algorítmica Paso a Paso

El algoritmo formal presentado en la lección sintetiza la secuencia matemática requerida para entrenar un ensamble de Gradient Boosting sobre un conjunto de datos de entrenamiento $\{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$:

> **Estructura Formal del Algoritmo de Gradient Boosting**
>
> **Paso 1 (Inicialización):** Ajustar un modelo simple T^(1) sobre los datos de entrenamiento originales {(x_1, y_1), ..., (x_N, y_N)}. Establecer el ensamble inicial como T ← T^(1).
>
> **Paso 2 (Residuos Iniciales):** Calcular los residuos iniciales r_1, ..., r_N para el modelo T, donde cada residuo r_n representa la diferencia r_n = y_n - T(x_n).
>
> **Paso 3 (Bucle Iterativo):** Para i = 1 hasta que se cumpla la condición de parada predefinida (número máximo de árboles M o tolerancia de error):
>
> • **Paso 3a (Entrenamiento en Residuos):** Ajustar un nuevo modelo simple T^(i) sobre los residuos actuales, entrenando con el dataset {(x_1, r_1), (x_2, r_2), ..., (x_N, r_N)}.

> • **Paso 3b (Actualización del Ensamble):** Actualizar la función predictora del ensamble acumulado agregando el nuevo aprendiz escalado por la tasa de aprendizaje λ: T ← T + λ T^(i).
>
> • **Paso 3c (Actualización de Residuos):** Recomputar los residuos de cada muestra individual mediante la regla r_n ← r_n - λ T^(i)(x_n) para n = 1, ..., N.

3. Componentes Clave y Análisis Operativo

Para comprender la solidez del algoritmo, es esencial descomponer la función de cada uno de los parámetros y variables matemáticos involucrados en la iteración:

Símbolo / Término	Denominación Técnica	Función en el Algoritmo
T^(1)	Modelo Base / Inicializador	Primer árbol simple ajustado directamente sobre la variable objetivo real y.
r_n	Residuo de la Muestra n	Diferencia entre el valor real y_n y la predicción acumulada del ensamble T(x_n).
T^(i)	Aprendiz Débil Iterativo	Árbol de profundidad limitada (ej. tocón) entrenado para aproximar los residuos r_n.
λ (Lambda)	Tasa de Aprendizaje (Learning Rate)	Constante de contracción que escala la contribución de cada nuevo árbol para evitar el sobreajuste.

4. El Rol de la Tasa de Aprendizaje (λ) y Contracción (Shrinkage)

La constante **λ (lambda)** cumple un papel regulador crítico denominado **hiperparámetro de contracción** (o *learning rate*). Al multiplicar las predicciones del aprendiz débil $T^{(i)}$ por $\lambda \in (0, 1]$, se garantiza que ningún árbol individual altere drásticamente las fronteras de decisión del modelo global.

Valores más pequeños de λ (por ejemplo, $\lambda = 0.01$ o $\lambda = 0.1$) requieren un número mayor de iteraciones i para reducir el error de entrenamiento, pero mejoran sustancialmente la capacidad de **generalización** del ensamble sobre datos no vistos, evitando el memorismo de patrones de ruido.

5. Material Gráfico Integrado de la Lección

A continuación se presenta la lámina original de la lección que sintetiza la estructura del algoritmo de Gradient Boosting:

5.2.3 Resumen del algoritmo de Gradient Boosting

Resumen del algoritmo de Gradient Boosting:

Paso 1

Ajustar un modelo simple $T^{(0)}$ en los datos de entrenamiento $(x_1, y_1), \ldots, (x_N, y_N)$. Establecer $T \leftarrow T^{(0)}$.

Paso 2

Calcular los residuos $r_1, \ldots, r_N$ para T.

Para $i = 1$ hasta que se cumpla la condición de parada:

Paso 3

Ajustar un modelo simple, $T^{(i)}$, en los actuales **residuos**, es decir, entrenar el modelo utilizando $\{(x_1, r_1), \ldots, (x_N, r_N)\}$.

Paso 4

Establecer el modelo actual T a $T \leftarrow T + \lambda T^{(i)}$.

Paso 5

Calcular residuos, establecer $r_n \leftarrow r_n - \lambda T^{(i)}(x_n)$, $n = 1, \ldots, N$ donde λ es una constante llamada la **tasa de aprendizaje**.

Figura 5.2.3: Lámina explicativa del Resumen del Algoritmo de Gradient Boosting.

Sección 5.2.4: Criterios de Terminación y Convergencia en Gradient Boosting

Análisis de las Estrategias de Parada, Convergencia del Descenso del Gradiente y el Impacto de la Tasa de Aprendizaje (λ)

Índice de Contenidos

(El índice de contenidos se actualizará automáticamente al abrir el documento en Microsoft Word o Google Docs).

1. Dinámica de Convergencia en el Descenso del Gradiente

En condiciones ideales, el algoritmo de Descenso del Gradiente (Gradient Boosting) funciona como un proceso de optimización iterativo que aproxima progresivamente la función de pérdida hacia su óptimo global o local. En cada iteración, se incorpora un nuevo aprendiz débil orientado a corregir los residuos residuales generados por la combinación de todos los modelos anteriores.

A pesar de su fundamentación matemática sólida, la implementación práctica plantea un dilema metodológico crucial: **¿Cuándo debemos detener el proceso iterativo de construcción de árboles?**

2. Estrategia 1: Límite Fijo del Número Máximo de Iteraciones

La técnica más directa para finalizar el entrenamiento consiste en definir a priori un número máximo de iteraciones o árboles (por ejemplo, M iteraciones fijas).

> **Limitación Crítica del Límite Arbitrario de Iteraciones:**
>
> Al fijar de manera arbitraria el número máximo de iteraciones, es imposible garantizar que el modelo se encuentre lo suficientemente cerca del punto óptimo de convergencia al finalizar. Si el número asignado es insuficiente, el algoritmo sufrirá de subajuste (underfitting); si el número es excesivo, se incurrirá en un desperdicio computacional e incrementará el riesgo de sobreajuste (overfitting).

3. Estrategia 2: Parada Basada en la Magnitud de Actualización

Una alternativa dinámica consiste en evaluar la magnitud de las mejoras en cada paso y detener el proceso cuando las actualizaciones se vuelven lo suficientemente pequeñas (por ejemplo, cuando los residuos predichos por el nuevo árbol T' caen por debajo de un umbral de tolerancia ε).

Peligro del Criterio de Residuos Mínimos:

Si el algoritmo intenta detenerse únicamente cuando las actualizaciones sean infinitesimalmente pequeñas, nos enfrentamos a un nuevo problema severo: ¡el algoritmo puede no terminar nunca! Debido a la disminución asintótica de los residuos, la tasa de cambio puede mantenerse en rangos mínimos sin cruzar el umbral absoluto, atrapando al modelo en un bucle infinito de iteraciones.

4. El Rol Regulador de la Tasa de Aprendizaje (λ)

Tanto el problema de la convergencia prematura en límites fijos como el riesgo de no terminación en umbrales dinámicos están estrechamente vinculados a la magnitud de la tasa de aprendizaje (λ, o shrinkage factor).

- **Tasa de aprendizaje reducida (λ pequeña):** Proporciona una regularización robusta que previene el sobreajuste, pero requiere un número significativamente mayor de iteraciones. Si el número máximo de árboles fijado es arbitrariamente bajo, el modelo quedará lejos del óptimo.
- **Tasa de aprendizaje elevada (λ grande):** Acelera los primeros pasos de convergencia, pero puede provocar oscilaciones agresivas alrededor del mínimo de la pérdida, impidiendo que la magnitud de los residuos disminuya limpiamente y haciendo imposible alcanzar el criterio de parada dinámico.

Matriz Comparativa de Estrategias de Terminación

Criterio de Parada	Mecanismo Principal	Riesgo / Desventaja	Interacción con λ
Límite Fijo de Iteraciones	Fijar un número máximo M de árboles en el descenso.	Riesgo de subajuste si M es pequeño o sobreajuste si M es excesivo.	Si λ es pequeño, M debe ser lo suficientemente grande para converger.
Magnitud de Actualización	Detener cuando los residuos de T' sean < ε.	¡El algoritmo puede no terminar nunca si las mejoras son asintóticas!	Si λ es alto, las oscilaciones pueden impedir cruzar el umbral ε.

5. Material Gráfico Integrado de la Lección

5.2.4 Terminación

Terminación

Bajo condiciones ideales, el Descenso del Gradiente aproxima iterativamente y converge al óptimo.

¿Cuándo **terminamos** el Descenso del Gradiente?

- Podemos **limitar el número de iteraciones** en el descenso. Pero para una elección arbitraria del número máximo de iteraciones, no podemos garantizar que estamos lo suficientemente cerca del óptimo al final.
- Si el descenso se detiene cuando las **actualizaciones son lo suficientemente pequeñas** (por ejemplo, los residuos de T son pequeños), nos enfrentamos a un nuevo problema: ¡el algoritmo puede nunca terminar!

Ambos problemas tienen que ver con la magnitud de la tasa de aprendizaje, λ.

Figura 5.2.4: Diapositiva original sobre los dilemas de terminación en Descenso del Gradiente y el rol de la tasa de aprendizaje (λ).

5.2.5 ¿Por qué Funciona el Gradient Boosting?

Formulación Matemática como Descenso del Gradiente en el Espacio de Predicción y Rol de la Tasa de Aprendizaje (λ)

Tabla de Contenidos del Documento

(Nota: La tabla de contenidos se actualizará automáticamente al abrir el archivo en Microsoft Word o Google Docs).

1. Introducción y la Pregunta Fundamental de Convergencia

En las lecciones anteriores se demostró empíricamente cómo el algoritmo de **Gradient Boosting** construye un modelo predictivo complejo mediante la adición secuencial de modelos simples o aprendices débiles (como árboles de regresión o de decisión de baja profundidad, conocidos como *stumps*). Sin embargo, surge una interrogante matemática fundamental: **¿Existe una garantía teórica de que este procedimiento iterativo reduzca progresivamente el error y converja hacia una solución óptima?**

> **Dilema Teórico Central**
> Añadir continuamente modelos ajustados a los errores anteriores plantea dos preguntas críticas:
> 1. ¿Por qué la suma ponderada de modelos sobre los residuos garantiza que los residuos totales disminuyan?
> 2. ¿Qué papel cumple exactamente el hiperparámetro λ (tasa de aprendizaje) y cómo se relaciona con los métodos de optimización clásica?

Demostración Matemática Básica de la Reducción de Residuos

Para entender la intuición de la reducción del error, consideremos el flujo desde su inicio formal:

- **1. Modelo Inicial:** Se inicia con un modelo base inicial T_0. Al realizar una predicción para una observación x, denotamos la predicción como T_0(x).
- **2. Ajuste de Residuos:** Se entrena un segundo modelo T_1 cuya meta es aproximar los residuos del primer modelo, e_0 = y - T_0(x), de modo que T_1(x) ≈ e_0.
- **3. Ensamble Ponderado:** Combinamos el modelo inicial y el nuevo modelo mediante un factor de ponderación λ, obteniendo el modelo ensemble T_nuevo(x) = T_0(x) + λ · T_1(x).

Al calcular el nuevo residuo e_nuevo tras la actualización, obtenemos la siguiente relación algebraica:

$$e_{nuevo} = y - T_{nuevo}(x) = y - [T_0(x) + \lambda \cdot T_1(x)] = [y - T_0(x)] - \lambda \cdot T_1(x) = e_{anterior} - \lambda \cdot T_1(x)$$

Dado que T_1(x) se entrena para tener el mismo signo y magnitud aproximada que e_anterior, el producto λ · T_1(x) sustrae una fracción sustancial del residuo previo. En consecuencia, la magnitud

absoluta del nuevo residuo |e_nuevo| es estrictamente menor que |e_anterior|, demostrando que **los residuos decrecen de forma monótona iteración a iteración**.

2. Formulación del Boosting como Descenso del Gradiente (Gradient Descent)

Para conectar la intuición de los residuos con la teoría de optimización, debemos reformular el **Gradient Boosting** dentro del marco formal del **Descenso del Gradiente (Gradient Descent)**. En el aprendizaje automático, la meta de cualquier modelo es encontrar una función o conjunto de parámetros que minimice una **función de pérdida (Loss Function, L)**.

Principios Fundamentales del Descenso del Gradiente Clásico

En la optimización paramétrica estándar (como en la regresión lineal o logística), se busca un vector de pesos w que minimice L(w):

- **Funciones Convexas:** En el caso ideal donde L(w) es una función convexa, existe un único mínimo global que se puede resolver analíticamente o mediante gradientes.
- **Funciones No Convexas:** Cuando la función de pérdida no es convexa, pueden existir múltiples mínimos locales. Se utiliza el descenso de gradiente iterativo inicializando w aleatoriamente y actualizándolo en la dirección opuesta al gradiente.

La regla de actualización iterativa en el descenso de gradiente paramétrico está dada por:

$$w_{nuevo} = w_{anterior} - \lambda \cdot \nabla_w L(w)$$

El gradiente ∇_w L(w) indica la dirección de mayor incremento de la pérdida (ascenso más pronunciado). Por lo tanto, moverse en la dirección negativa del gradiente (-∇_w L) garantiza avanzar hacia la dirección de *descenso más pronunciado (steepest descent)*, conduciendo al algoritmo hacia el mínimo local o global.

3. El Dilema del Espacio Paramétrico vs. Espacio de Predicción

Aquí es donde surge la principal dificultad técnica en los modelos basados en árboles de decisión: **¿En qué espacio estamos aplicando el descenso del gradiente?**

Modelos Paramétricos vs. No Paramétricos

En los modelos paramétricos tradicionales, el espacio de búsqueda es el **espacio de parámetros (pesos w)**, el cual tiene una dimensión fija. Sin embargo, los árboles de decisión son modelos **no paramétricos**. Aunque un árbol tiene parámetros internos (puntos de corte, profundidad, valores en las hojas), el número total de parámetros no es fijo ni preconcebido, sino que depende de la estructura del árbol y los datos de entrenamiento.

La Solución Trivial en el Espacio de Predicción

Supongamos que intentamos aplicar descenso de gradiente directamente sobre las predicciones de las muestras de entrenamiento y_hat = [y_hat_1, y_hat_2, ..., y_hat_N]. Si consideramos la función de pérdida de Error Cuadrático Medio (MSE):

$$L(y, \hat{y}) = (1 / N) \cdot \Sigma_i (y_i - \hat{y}_i)^2$$

Si tomamos la derivada parcial de la pérdida respecto a cada predicción y_hat_i e igualamos a cero para encontrar el mínimo:

$$\partial L / \partial \hat{y}_i = -2 \cdot (y_i - \hat{y}_i) = 0 \implies \hat{y}_i = y_i$$

> **El Problema de la Solución Trivial**
> Si actualizamos directamente las predicciones y_hat_i hasta que y_hat_i = y_i, el modelo simplemente memorizará cada punto del conjunto de entrenamiento.
>
> Esto NO nos ayuda, porque no hemos aprendido una función generalizable f(x) que conecte las variables de entrada x con las salidas y para nuevos datos no observados. Ir directamente al espacio de predicción sin restricciones conduce a sobreajuste puro y falta total de capacidad predictiva.

4. Optimización en el Espacio de Funciones y Aproximación del Gradiente (r_hat)

Para superar la trampa de la solución trivial, el Gradient Boosting realiza el descenso de gradiente dentro del **Espacio de Funciones (Function Space)**. Este espacio representa el conjunto infinito de todos los modelos o árboles posibles que pueden conectar las entradas x con las respuestas y.

El Gradiente Negativo como Residuo Verdadero

Para la función de pérdida cuadrática MSE, el gradiente negativo de la pérdida con respecto a la predicción del modelo actual T(x_i) es exactamente el **residuo verdadero (r_i*)**:

$$-\partial L(y_i, T(x_i)) / \partial T(x_i) = 2 \cdot (y_i - T(x_i)) \propto r_i^* = y_i - T(x_i)$$

Aproximación del Gradiente Mediante Aprendices Débiles (r_hat)

Como el espacio de todas las funciones posibles es infinito e inabarcable, no podemos movernos directamente en la dirección del gradiente negativo perfecto r_i* sin perder generalización. En su lugar, aplicamos una estrategia de aproximación:

- **1. Entrenamiento del Modelo Débil:** En lugar de saltar directamente al residuo verdadero r_i*, entrenamos un árbol de decisión simple T_h(x) sobre el conjunto de datos (x_i, r_i*).
- **2. Dirección Aproximada (r_hat):** El árbol resultante T_h(x) genera una predicción r_hat_i que actúa como una aproximación suavizada y generalizable del gradiente negativo verdadero.

- **3. Paso en el Espacio Funcional:** Nos desplazamos en el espacio de funciones un pequeño paso λ en la dirección de la función aproximada T_h(x).

> **Interpretación Geométrica en el Espacio Funcional**
> Imaginemos un punto de partida T_0 en el espacio de funciones. El vector que apunta directamente al mínimo de pérdida es el residuo verdadero (estrella r*).
>
> En lugar de dar un salto directo hacia r*, entrenamos un subárbol que encuentra una dirección cercana (vector r_hat). Al avanzar una distancia controlada λ · r_hat, alcanzamos un nuevo punto funcional (punto verde) que está provadamente más cerca del mínimo de la función de pérdida. Repitiendo este proceso, el algoritmo navega por el espacio de funciones hasta alcanzar la convergencia.

5. El Rol Regulador de la Tasa de Aprendizaje (λ)

La constante **λ (lambda)** cumple exactamente el mismo papel que la **tasa de aprendizaje (learning rate)** en el descenso de gradiente estándar. Controla la magnitud del paso que damos en la dirección del aprendiz débil ajustado.

Efecto del Valor de λ en la Dinámica del Aprendizaje

Configuración de λ	Comportamiento del Algoritmo	Consecuencia en el Modelo
λ Elevado (e.g., λ = 1.0)	Da pasos muy grandes en el espacio funcional. El algoritmo salta agresivamente sobre el mínimo.	Riesgo de oscilación, sobreajuste prematuro (overfitting) e inestabilidad en la generalización.
λ Reducido (e.g., λ = 0.01 - 0.1)	Avanza en pasos pequeños y cautelosos. Exige un mayor número de árboles (iteraciones).	Convergencia suave, alta precisión y excelente capacidad de generalización en datos no vistos.

Nota Técnica: En variantes avanzadas como XGBoost o LightGBM, se incorporan derivadas de segundo orden (matriz Hessiana) para adaptar dinámicamente el tamaño del paso, optimizando aún más la trayectoria en el espacio funcional.

6. Cuadro Síntesis Metodológico: Descenso de Gradiente vs. Gradient Boosting

La siguiente tabla resume las equivalencias directas entre el Descenso de Gradiente Tradicional y el Gradient Boosting Funcional:

Dimensión / Componente	Descenso de Gradiente Clásico	Gradient Boosting Funcional
Espacio de Búsqueda	Espacio Paramétrico (pesos w de	Espacio de Funciones (árboles de decisión

	dimensión fija)	no paramétricos)
Objetivo de Optimización	Minimizar L(w) ajustando los vectores de pesos	Minimizar L(y, T(x)) sumando funciones o árboles
Dirección de Actualización	Gradiente negativo exacto: -∇_w L(w)	Aproximación del gradiente mediante un árbol: r_hat ≈ -∂L / ∂T(x)
Mecanismo de Paso	w_nuevo = w - λ · ∇_w L	T_nuevo(x) = T_anterior(x) + λ · T_h(x)
Parámetro de Ponderación (λ)	Tasa de aprendizaje para escalar el vector de pesos	Factor de contracción / Learning Rate para escalar el árbol de residuos

Conclusión del Capítulo

Gradient Boosting no es simplemente una técnica heurística de suma de árboles; es una formulación rigurosa de **descenso de gradiente en el espacio de funciones**. Al sustituir el gradiente negativo exacto por un aprendiz débil (r_hat) y escalar su aportación mediante λ, el algoritmo logra una convergencia suave hacia el mínimo de la pérdida, combinando la flexibilidad de los modelos no paramétricos con la estabilidad teórica de la optimización convexa y n

Capítulo 5: Ensamble de Modelos y Boosting

Sección 5.3.1: ¿Por qué funciona el Gradient Boosting? y Revisión del Descenso del Gradiente

Tabla de Contenidos (actualizable al abrir el documento)

1. Demostración Matemática de la Reducción Iterativa de Residuos

El algoritmo de **Gradient Boosting** construye de manera iterativa un ensamble aditivo de modelos simples (árboles de decisión débiles, $T^{(h)}$). La premisa teórica fundamental es que cada nuevo árbol que se incorpora al ensamble se entrena específicamente para modelar los residuos producidos por la combinación de los árboles anteriores. De este modo, en cada iteración del algoritmo, los residuos totales del modelo global disminuyen cuantitativamente, garantizando la mejora progresiva de la precisión predictiva.

Desglose Formal del Proceso de Disminución de Residuos:

- **Paso 1 (Modelo Inicial):** Se construye el primer árbol simple $T^{(0)}$, el cual genera un vector de predicciones $T^{(0)}_{pred} = T^{(0)}(x_n)$.
- **Paso 2 (Definición de Residuos Iniciales):** Se calcula la discrepancia entre las observaciones reales y y las predicciones iniciales, definiendo los 'residuos cero' (r_0): $r_0 = y - T^{(0)}_{pred}$.
- **Paso 3 (Ajuste del Segundo Árbol sobre Residuos):** Se entrena un nuevo modelo débil $T^{(1)}$ utilizando los residuos r_0 como variable objetivo. Por consiguiente, $T^{(1)}$ aproxima directamente el residuo actual: $T^{(1)}_{pred} \approx y - T^{(0)}_{pred}$.
- **Paso 4 (Combinación Aditiva Ponderada):** Se combina el modelo base con el nuevo árbol escalado por el parámetro de tasa de aprendizaje λ: $T' = T^{(0)} + \lambda T^{(1)}$.
- **Paso 5 (Recálculo de Residuos en el Nuevo Ensamble):** Los nuevos residuos del ensamble combinado T' se expresan como: $y - T'_{pred} = y - T^{(0)}_{pred} - \lambda T^{(1)}_{pred} = r_0 - \lambda T^{(1)}_{pred}$.
- **Conclusión Monótona:** A medida que el algoritmo acumula árboles ponderados, la actualización recursiva $r_n \leftarrow r_n - \lambda T^{(h)}(x_n)$ reduce sistemáticamente la magnitud de los residuos en cada iteración.

5.3.1 ¿Por qué funciona el Gradient Boosting?

Why does Gradient Boosting work?

Intuitively, it makes sense that we are adding corrections to the model. Each simple model $T^{(i)}$ that we add to our ensemble T, models the residuals of T.

Thus, with each addition of $T^{(i)}$, the residuals are reduced: $r_n \leftarrow r_n - \lambda T^{(i)}(x_n)$

The claim is that the residuals will continue to reduce with every new iteration of the algorithm. Therefore, the model will improve. Can we prove it?

We started by modeling $T^{(0)}$ that gave us some predictions $T^{(0)}_{pred}$:

$$T^{(0)}_{pred} = T^{(0)}(x)$$

When we train a new tree $T^{(1)}$ on residuals of the previous tree $T^{(0)}$, the new tree $T^{(1)}$ will **approximate** the residuals:

$$T^{(1)}_{pred} \approx y - T^{(0)}_{pred}$$

We will call $y - T^{(0)}_{pred}$ "residuals zero" $r^{(0)}$.

Finally, we combine $T^{(0)}$ and $T^{(1)}$ with some lambda:

$$T \leftarrow T^{(0)} + \lambda T^{(1)}$$

and the residuals of this model T would be:

$$y - T_{pred} = y - T^{(0)}_{pred} - \lambda T^{(1)}_{pred}$$

$$y - T_{pred} = r^{(0)} - \lambda T^{(1)}_{pred}$$

Therefore, as we combine the models, the residuals keep decreasing:

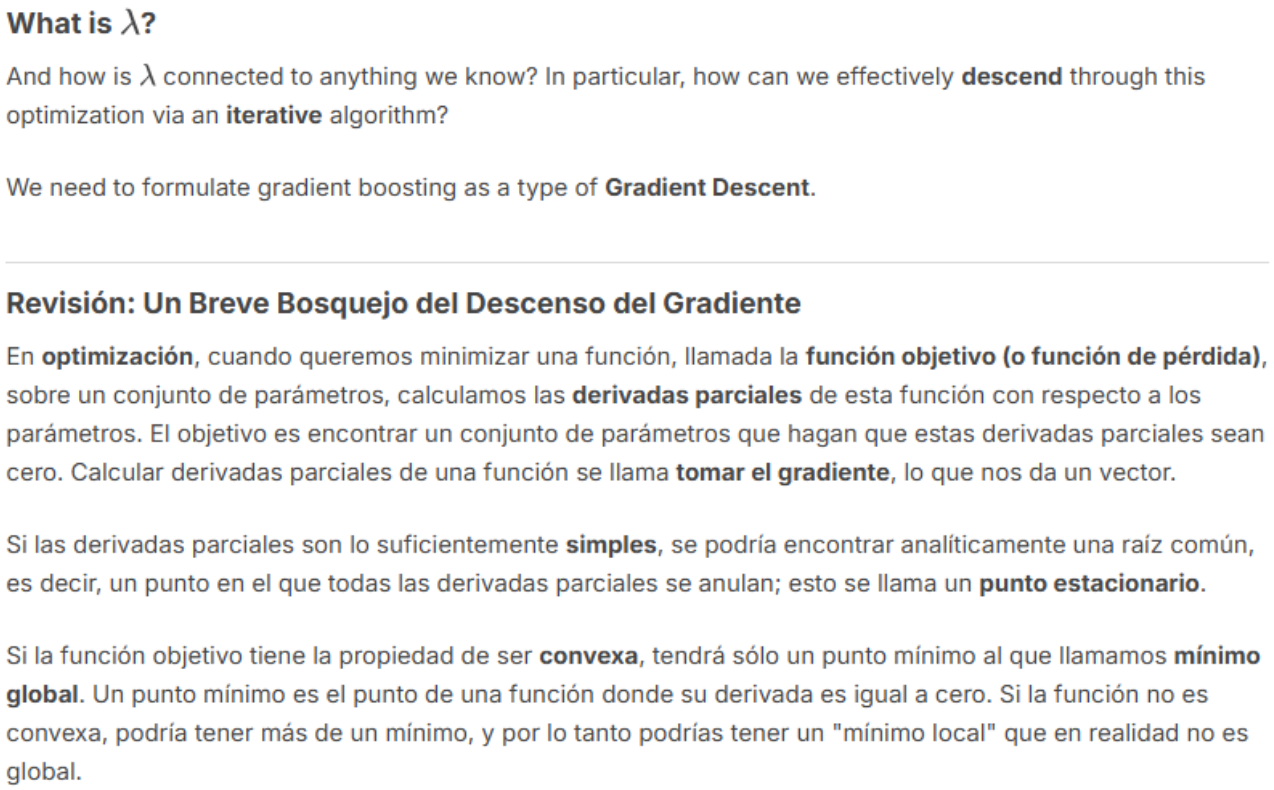
$$r_n \leftarrow r_n - \lambda T^{(i)}(x_n)$$

Figura 5.3.1a: Formulación inductiva de la reducción progresiva de residuos mediante la suma ponderada de aprendices débiles.

2. Vinculación entre Boosting y el Algoritmo de Descenso del Gradiente

Para fundamentar rigurosamente la función del parámetro de ponderación **λ** y entender por qué la corrección aditiva no oscila caóticamente, es necesario formular el Gradient Boosting dentro de la estructura general de la optimización por **Descenso del Gradiente**. En optimización matemática, el

descenso del gradiente es un método iterativo diseñado para encontrar el mínimo de una función de pérdida u objetivo calculando la dirección de mayor descenso.



What is λ?

And how is λ connected to anything we know? In particular, how can we effectively **descend** through this optimization via an **iterative** algorithm?

We need to formulate gradient boosting as a type of **Gradient Descent**.

Revisión: Un Breve Bosquejo del Descenso del Gradiente

En **optimización**, cuando queremos minimizar una función, llamada la **función objetivo (o función de pérdida)**, sobre un conjunto de parámetros, calculamos las **derivadas parciales** de esta función con respecto a los parámetros. El objetivo es encontrar un conjunto de parámetros que hagan que estas derivadas parciales sean cero. Calcular derivadas parciales de una función se llama **tomar el gradiente**, lo que nos da un vector.

Si las derivadas parciales son lo suficientemente **simples**, se podría encontrar analíticamente una raíz común, es decir, un punto en el que todas las derivadas parciales se anulan; esto se llama un **punto estacionario**.

Si la función objetivo tiene la propiedad de ser **convexa**, tendrá sólo un punto mínimo al que llamamos **mínimo global**. Un punto mínimo es el punto de una función donde su derivada es igual a cero. Si la función no es convexa, podría tener más de un mínimo, y por lo tanto podrías tener un "mínimo local" que en realidad no es global.

Figura 5.3.1b: Conexión teórica entre el parámetro lambda y el algoritmo clásico de Descenso del Gradiente.

2.1 Principios de Optimización en Funciones Convexas y No Convexas

En un problema de optimización sobre un conjunto de parámetros $w = [w_1, w_2, \dots, w_M]$, el objetivo principal es minimizar una función de pérdida $f(w)$. El cálculo del vector de derivadas parciales (el gradiente $\nabla f(w)$) identifica la pendiente de la función en el punto actual:

Tipo de Función	Punto Estacionario	Comportamiento del Gradiente
Función Convexa	Posee un único punto mínimo global ($ \nabla f(w) = 0$).	Cualquier método de descenso local garantiza la convergencia directa al óptimo absoluto.
Función No Convexa	Puede presentar múltiples mínimos locales y puntos de silla.	El gradiente permite descender iterativamente, pero el punto final depende de la inicialización y el tamaño de paso.

2.2 Algoritmo Iterativo del Descenso del Gradiente Tradicional

Cuando la solución analítica $\nabla f(w) = 0$ resulta intratable debido a la complejidad de la función de pérdida, el descenso por gradiente procede numéricamente a través de tres fases secuenciales:

• **1. Inicialización de Parámetros:** Se asignan valores iniciales arbitrarios al vector de parámetros $w = [w_1, w_2, \dots, w_M]$. Por ejemplo, en regresión logística, w corresponde a los coeficientes de regresión (betas β).

• **2. Evaluación del Vector Gradiente:** Se evalúa la derivada parcial de la función objetivo f en el estado actual de los parámetros: $\nabla_w f(w) = \left[\frac{\partial f}{\partial w_1}, \frac{\partial f}{\partial w_2}, \dots, \frac{\partial f}{\partial w_M} \right]$.

• **3. Actualización Negativa del Vector:** Se ajusta la posición del vector moviéndose en la dirección opuesta al gradiente (dirección de máximo descenso): $w \leftarrow w - \lambda \nabla_w f(w)$, donde el escalar λ representa la tasa de aprendizaje (*learning rate*).

En la práctica, si nuestras funciones objetivo son complicadas, encontrar analíticamente un punto estacionario es intratable. En este caso, usamos un método iterativo llamado **Descenso del Gradiente**.

1. Inicializa los parámetros de la función objetivo/de pérdida a cualquier valor.

   Los w valores a continuación son los parámetros sobre los que estamos minimizando. Por ejemplo, en la regresión logística los parámetros serían las betas (β).

$$w = [w_1, w_2, \dots, w_j]$$

2. Toma el gradiente de la función objetivo, f, en los valores actuales de los parámetros.

$$\nabla_w f(w) = \left[\frac{\partial f}{\partial w_1}(w), \frac{\partial f}{\partial w_2}(w), \dots, \frac{\partial f}{\partial w_j}(w) \right]$$

3. Ajusta los parámetros por algún múltiplo negativo del gradiente. Al hacerlo, nos movemos paso a paso en la dirección opuesta al gradiente.

$$w \leftarrow w - \lambda \nabla f_w(w)$$

donde λ es la tasa de aprendizaje.

El descenso por gradiente puede ser un tema bastante complejo. Si encuentras que necesitas más información, hay buenas referencias en IBM o Cornell, o La Biblioteca de Aprendizaje Abierto del MIT.

Figura 5.3.1c: Pasos algorítmicos del Descenso del Gradiente e iteración de actualización de parámetros.

3. Síntesis Teórica y Fuentes Bibliográficas

La equivalencia conceptual entre el descenso del gradiente tradicional y el Gradient Boosting reside en que, mientras el primero actualiza parámetros continuos (w) restando el gradiente de la pérdida, el segundo actualiza funciones predictivas (T) añadiendo subárboles que simulan el gradiente negativo (residuos). En ambos casos, el hiperparámetro **λ** regula la magnitud de la actualización para impedir la divergencia del algoritmo y asegurar una convergencia suave hacia el óptimo global.

Referencias de Ampliación Académica:

- IBM Learning Resource: Gradient Descent Optimization Concepts.
- Cornell University CS Department: Numerical Optimization in Machine Learning.
- MIT Open Learning Library: Optimization Methods and Loss Minimization.

5.3.2 ¿Por qué Funciona el Descenso del Gradiente?

Fundamentación matemática, interpretación vectorial de derivadas parciales y convergencia en superficies convexas

Índice de Contenidos

(El índice de contenidos se actualizará automáticamente al abrir el archivo en Microsoft Word o Google Docs).

1. Teorema de Convergencia en Funciones Convexas

El algoritmo del Descenso del Gradiente es un método de optimización iterativo diseñado para encontrar el mínimo de una función objetivo mediante correcciones sucesivas en la posición de los parámetros. La garantía fundamental de este procedimiento establece que, si la función objetivo es **convexa**, el método moverá sistemáticamente la estimación **x** hasta ubicarla en la vecindad inmediata del mínimo global, siempre que se elija una tasa de aprendizaje (**λ**) apropiada.

> **Afirmación Central de Convergencia:** Si la función de pérdida f(x) es estrictamente convexa, el proceso iterativo x^(k+1) = x^(k) - λ ∇f(x^(k)) garantiza que la secuencia de puntos converge hacia el mínimo global x* a medida que k tiende a infinito, siempre que la tasa de aprendizaje λ satisfaga las condiciones de estabilidad de Lipschitz.

La elección de la tasa de aprendizaje **λ** resulta crítica para garantizar la convergencia. Un valor demasiado grande de **λ** puede provocar oscilaciones que sobrepasen el mínimo (divergencia), mientras que un valor excesivamente pequeño ralentiza drásticamente la tasa de convergencia del modelo.

2. Interpretación Vectorial del Gradiente y Regla de Actualización

Para comprender geométricamente por qué funciona el descenso de gradiente, es necesario analizar el significado geométrico del gradiente como vector. En cualquier punto del dominio de una función diferenciable, el vector gradiente —compuesto por las derivadas parciales de la función con respecto a cada variable— señala la **dirección de la tasa máxima de aumento posible** (el ascenso más pronunciado).

- **Dirección de Ascenso Máximo:** El vector de derivadas parciales ∇f(x) apunta hacia la trayectoria donde la función f crece de manera más acelerada.
- **Dirección de Descenso Máximo (-∇f):** Al tomar el opuesto del gradiente (-∇f(x)), obtenemos de forma ortogonal e instantánea la dirección donde la función decrece a la máxima velocidad posible.

- **Regla de Actualización con Paso λ:** Al restar un múltiplo λ del gradiente a la posición actual x (según la ecuación x_nuevo = x_actual - λ ∇f(x)), desplazamos la posición x exactamente en la dirección del descenso más pronunciado mediante un paso escalado por λ.

3. Visualización Geométrica en Superficies Parabólicas Tridimensionales

Al aplicar esta técnica a una función de pérdida convexa cuadrática o parabólica en tres dimensiones **z = f(x, y)**, el proceso de optimización se traduce geométricamente en dar pasos sucesivos descendiendo por las paredes de la cuenca parabólica desde cualquier punto inicial hasta alcanzar el punto de mínima altura (mínimo global).

5.3.2 ¿Por qué funciona el descenso de gradiente?

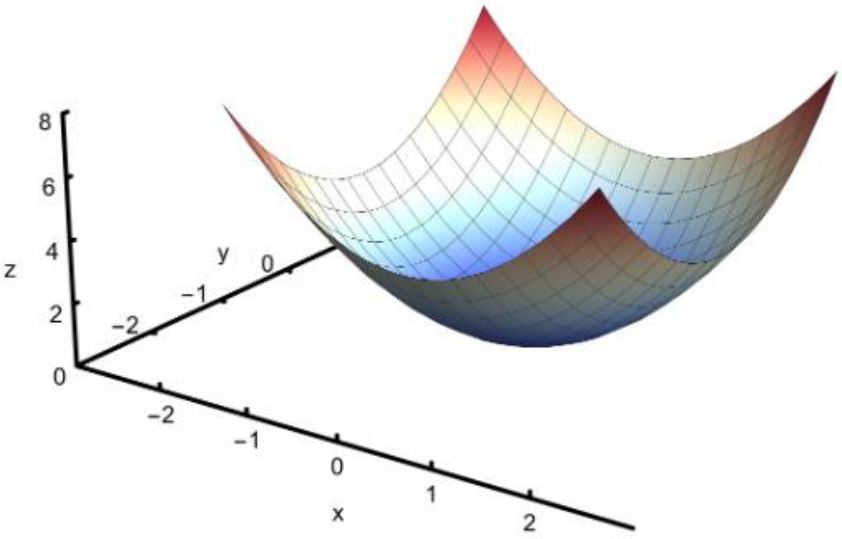
¿Por qué funciona el descenso de gradiente?

Afirmación: Si la función es convexa, este método iterativo eventualmente moverá x lo suficientemente cerca del mínimo, para una elección apropiada de λ.

¿Por qué funciona? Recall que, como vector, el gradiente en un punto proporciona la dirección para la tasa máxima de aumento posible. En otras palabras, en cualquier momento, las derivadas parciales nos muestran la dirección del ascenso más pronunciado. El negativo de las derivadas parciales nos dará la dirección del descenso más pronunciado.

Restando un λ múltiplo del gradiente de x, mueve x en la dirección opuesta al gradiente (por lo tanto, hacia el descenso más pronunciado) mediante un paso de tamaño λ.

Así que si comenzamos con una función parabólica como esta:



Entonces podremos usar esta técnica para dar pasos descendiendo por la parábola desde cualquier punto, descendiendo el gráfico de f y eventualmente alcanzando el mínimo.

Figura 5.3.2a: Exposición conceptual del Descenso del Gradiente sobre una superficie parabólica convexa 3D z = f(x, y).

En la representación visual wireframe en tres dimensiones, se observa cómo la secuencia discreta de iteraciones genera una trayectoria descendente (marcada por los puntos de paso de actualización). Cada paso desplaza las coordenadas del modelo hacia niveles de menor pérdida **z** hasta estabilizarse en el centro de la parábola donde las derivadas parciales se reducen a cero ($\nabla f = 0$).

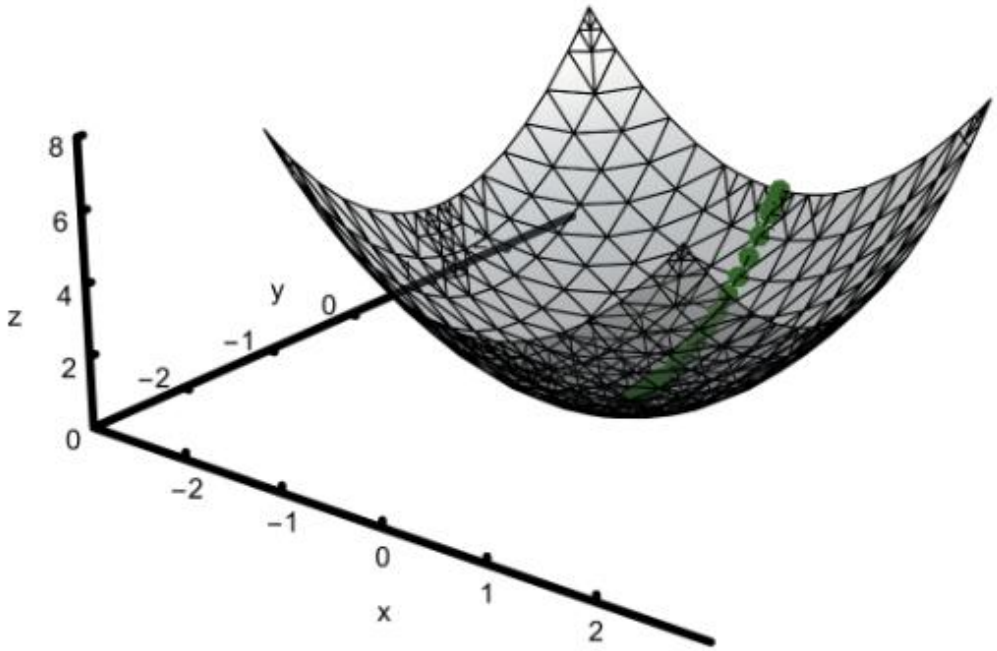


Figura 5.3.2b: Trayectoria de pasos iterativos (puntos verdes) descendiendo por la superficie de la función hasta alcanzar el mínimo global.

4. Cuadro Síntesis de Componentes Mecánicos del Descenso del Gradiente

La siguiente tabla resume la función matemática, interpretación física y relevancia práctica de cada elemento en la regla de actualización del Descenso del Gradiente:

Componente Matemático	Interpretación Física / Geométrica	Impacto en la Optimización
Vector Gradiente ∇f(x)	Vector de derivadas parciales que indica la dirección de máxima tasa de crecimiento de la pérdida.	Proporciona la brújula direccional exacta en cada punto del espacio de parámetros.
Signo Negativo (-∇f)	Orientación opuesta al gradiente para pasar del ascenso al descenso más pronunciado.	Garantiza que la actualización reduzca la pérdida en lugar de incrementarla.
Tasa de Aprendizaje (λ)	Escalar que controla el tamaño o longitud del paso en cada iteración.	Regula el compromiso entre velocidad de convergencia y estabilidad de la trayectoria.
Convexidad de f(x)	Propiedad geométrica donde cualquier segmento entre dos puntos queda por encima de la gráfica.	Asegura la ausencia de mínimos locales atrapantes, garantizando el mínimo global x*.

Sección 5.3.3: Gradient Boosting como Descenso de Gradiente

Análisis Teórico: Optimización en Espacios Funcionales, Solución Trivial en el Espacio de Predicción y Ajuste de la Tasa de Aprendizaje

Nota: La tabla de contenidos se actualizará automáticamente al abrir este documento en Microsoft Word o Google Docs.

---

1. La Optimización en el Espacio de Predicción y el Dilema de la Solución Trivial

Para comprender la verdadera naturaleza matemática del Gradient Boosting, se debe examinar qué ocurre cuando formulamos el problema de aprendizaje supervisado directamente sobre el espacio de predicciones. Imaginemos que disponemos de un conjunto de datos de entrenamiento simplificado compuesto por N observaciones (x_1, y_1), (x_2, y_2), ..., (x_N, y_N). En este espacio, cada punto representa un vector ordenado de valores predichos y_hat = (y_hat_1, y_hat_2, ..., y_hat_N).

Si nuestro objetivo es minimizar la función de pérdida del error cuadrático medio (MSE, por sus siglas en inglés):

> **Formulación del Gradiente del MSE en el Espacio de Predicciones**
> MSE = (1 / N) * Σ_{i=1}^N (y_i - y_hat_i)^2
>
> Derivadas parciales con respecto a cada predicción y_hat_i:
> ∂MSE / ∂y_hat_i = - (2 / N) * (y_i - y_hat_i)
>
> Igualando el gradiente a cero para encontrar los puntos estacionarios:
> - (2 / N) * (y_1 - y_hat_1) = 0 => y_hat_1 = y_1
> - (2 / N) * (y_2 - y_hat_2) = 0 => y_hat_2 = y_2
> - (2 / N) * (y_N - y_hat_N) = 0 => y_hat_N = y_N

El Problema de Memorización Directa (Ausencia de Mapeo x -> y)

Dado que la función de pérdida MSE es estrictamente convexa, existe un único minimizador global perfecto y* = (y_1, y_2, ..., y_N). Al aplicar el descenso de gradiente estándar sobre las predicciones, el

algoritmo convergerá directamente a asignar y_hat_i = y_i para cada observación de entrenamiento. Sin embargo, este resultado constituye una solución trivial e inservible:

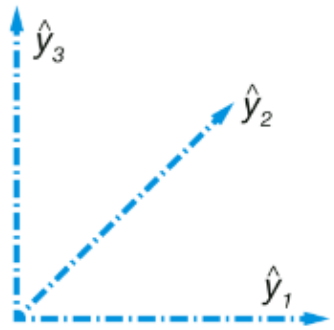
- **Memorización vacía:** El modelo no ha aprendido ninguna relación, patrón o regla que vincule las características de entrada (x) con las salidas objetivos (y).
- **Incapacidad de generalización:** Las secuencias de predicciones puras y_hat_i^(1), y_hat_i^(2), ..., y_hat_i^(M) no generan una función reutilizable, imposibilitando realizar inferencia sobre observaciones no vistas o de prueba (test data).

5.3.3 Gradient Boosting como descenso de gradiente

Gradient Boosting as Gradient Descent

Prediction space

Assume for now that we have 3 training examples $(x_1, y_1), (x_2, y_2), (x_3, y_3)$. Let us look at the space of predictions.



In this space, each point represents our set of predictions $y = (\hat{y}_1, \hat{y}_2, \hat{y}_3)$ for all data points. Note that here we have not assumed any relation between x and y whatsoever; we are just optimizing over the space of predictions.

The goal is to find the predictions that will minimize our loss function. The loss for which we are optimizing is the MSE:

$$L(y) = \frac{1}{3}[(y_1 - \hat{y}_1)^2 + (y_2 - \hat{y}_2)^2 + (y_3 - \hat{y}_3)^2]$$

As we discussed earlier, if we take the partial derivative and equate it to zero, we will find the minimum of the loss. If we change our prediction in some direction, what should $\hat{y}_1$, $\hat{y}_2$, and $\hat{y}_3$ be?

$$\frac{\partial L}{\partial \hat{y}_1} = -\frac{2}{3}(y_1 - \hat{y}_1)$$

\frac{\partial L}{\partial \hat{y}_2} = -\frac{2}{3} (y_1 - \hat{y}_2)

\frac{\partial L}{\partial \hat{y}_3} = -\frac{2}{3} (y_1 - \hat{y}_3)

Figura 1: Representación del espacio de predicción para N=3 puntos y cálculo de derivadas parciales del MSE.

The stationary point should be at:

-\frac{2}{3} (y_1 - \hat{y}_1) = 0

-\frac{2}{3} (y_2 - \hat{y}_2) = 0

-\frac{2}{3} (y_3 - \hat{y}_3) = 0

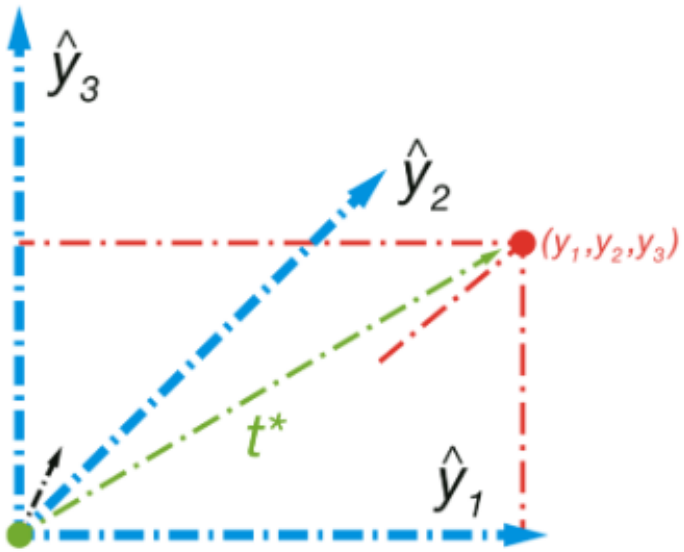
What will be the values \hat{y}_1, \hat{y}_2, and \hat{y}_3 that minimize the equations above?

It will be y_1, y_2, and y_3 themselves. Here we have a very trivial solution, which is that we can set our prediction to be the true y values, in this case our loss is zero, and we are done. :)

However, is there any problem with that? What has the model learned?

- Nothing!!

Here, as the loss function is convex, we already know the unique minimizer y^{*} = (y_1, y_2, y_3). So, if we start at the origin (or anywhere else), we will end at y^{*} when we optimize using gradient descent. That means that if we walk into a prediction space that has a very trivial solution, where the function is very convex, we do not even need gradient descent, we can just go straight for y_1, y_2, y_3, and that is a problem.



More generally, we have:

MSE = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2

Figura 2: Solución trivial en el espacio de predicción donde los valores predichos igualan exactamente a los valores reales y_i.

2. Comparativa de Régimenes: Espacio Paramétrico vs. Espacio de Funciones

Para superar la trampa de la solución trivial, es imprescindible diferenciar la arquitectura de optimización empleada según el tipo de algoritmo de aprendizaje:

Criterio / Dimensión	Regimen Paramétrico (ej. Redes Neuronales, Regresión)	Regimen No Paramétrico (ej. Árboles de Decisión, Ensembles)
Objeto de Optimización	Espacio de Parámetros / Pesos w = [w_1, w_2, ..., w_p].	Espacio de Funciones Abstractas / Espacio de Predicciones.
Estructura del Modelo	Estructura fija dependiente de un vector estático de pesos w.	Estructura flexible/dinámica sin un número fijo de parámetros (árboles de profundidad variable).
Estrategia de Descenso	Descenso de gradiente estándar ajustando los pesos: w <= w - λ * ∇_w L(w).	Ajuste de aprendices débiles sobre el gradiente negativo (residuos): y_hat <= y_hat + λ * r_hat(x).

En los modelos paramétricos tradicionales, como la regresión logística o las redes neuronales profundas, la función estimada f_hat(x) está parametrizada explícitamente por un vector de pesos w. El descenso por gradiente se ejecuta en este espacio de parámetros w, garantizando que el modelo mantenga su estructura funcional fija.

Por el contrario, los árboles de decisión son modelos no paramétricos: no poseen un conjunto prefijado de pesos, ya que el número de divisiones (splits) y hojas cambia según los datos. Intentar optimizar sobre el espacio abstracto de todas las funciones posibles f: X -> Y es inviable por ser un espacio infinito e intratable.

Treating this as an optimization problem, we can try to directly minimize the MSE with respect to the predictions:

\begin{aligned} \nabla_{\hat{y}} MSE &= [\frac{\partial MSE}{\partial \hat{y}_1}, ... , \frac{\partial MSE}{\partial \hat{y}_N}] \\ \nabla_{\hat{y}} MSE &= -2[y_1 - \hat{y}_1, ... , y_N - \hat{y}_N] \\ \nabla_{\hat{y}} MSE &= -2[r_1, ... , r_N] \end{aligned}

The updated step for the gradient descent would look like:

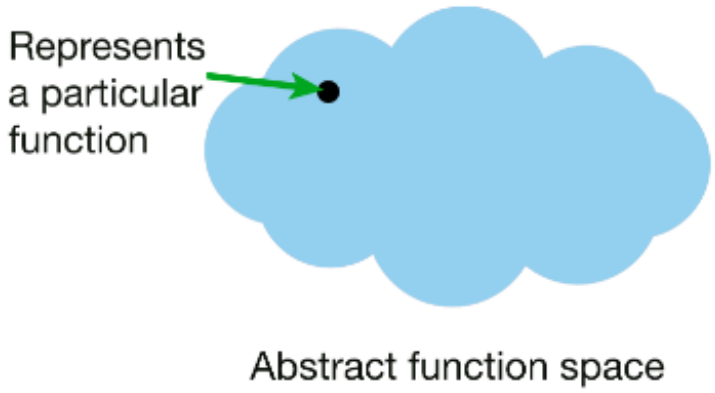
\hat{y}_n \leftarrow \hat{y}_n + \lambda r_n, \quad n=1, ... , N

There are two reasons why minimizing the MSE with respect to \hat{y}_n is not interesting:

- We know where the minimum MSE occurs: \hat{y}_n = y_n, for every n.
- Learning sequences of predictions, \{\hat{y}_n^{(1)} ... \hat{y}_n^{(i)} ...\} does not produce a model. This is because we have by no means learned to map predictors to predictions. (In \hat{y}_n^{(i)}, i is the iteration, `num(epoch)`).

When trying to view Gradient Boosting as a form of Gradient Descent, we need to look at the **function space**.

Imagine a space where each point represents a function f:X \rightarrow y, then ideally we want to optimize over this space to find the best estimator. A point on the picture below represents one function, and in our world a function will be a tree (it could be a logistic regression, but we will talk about trees).



This space represents all possible trees that we can imagine. The point is to find the tree in that space that will give us the best results, and that is impossible. Because this space is huge, it is infinite!

Figura 3: Contraste entre la minimización directa del MSE y la necesidad de buscar en un espacio abstracto de funciones.

What do we do in this case?

- We go into what is called **parametric space**.

In case of **parametric models**, we optimize our model in the weight space (parameters of our model).

\hat{f} := \left[w_1, w_2, ..., w_N\right]

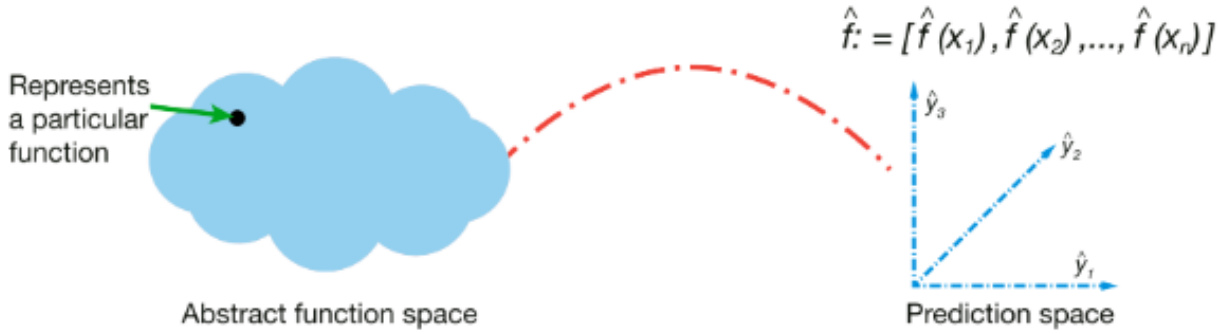
We read := as "f-hat is represented by".

When we go to parametric space, everything will become much easier since f will now depend on some w-s, and the big scary cloud above becomes something much simpler - it becomes the parameter space. And now we probably can manage the task in that space, like we did when working with linear regression and logistic regression.

What about non-parametric models? We have a problem, we cannot do an optimization in a non-parametric space as such.

Trees are non-parametric models, because we are working in a non-fixed parametric space, we do not know how many splits and how many leaves the tree will have.

In non-parametric models, like trees, we do not have parameter space, we only have prediction space.



We do not have parameters to optimize on, and if we optimize over the predictions, we get into the trivial solution which was mentioned before.

Figura 4: Transición desde el espacio de parámetros en modelos paramétricos al espacio de predicción en modelos no paramétricos.

3. Gradient Boosting: Descenso de Gradiente Restringido en el Espacio Funcional

Para resolver el dilema entre el espacio infinito e intratable de funciones y la solución trivial del espacio de predicciones, Gradient Boosting aplica un enfoque elegante y riguroso: realiza un descenso por gradiente restringido en el espacio funcional.

Sea L(y, T(x)) la función de pérdida acumulada. En el espacio de predicciones, el vector de dirección óptimo para dar un paso descendente corresponde exactamente al gradiente negativo de la pérdida:

> **Derivación Formal de la Dirección de Descenso Óptima**
> Vector de dirección óptimo ideal (Gradiente Negativo):
> t* = - [∂L / ∂y_hat_1, ∂L / ∂y_hat_2, ..., ∂L / ∂y_hat_N]
>
> Para el caso específico del error cuadrático medio (MSE):
> ∂L / ∂y_hat_n = - 2 * (y_n - y_hat_n)
> t* = [(y_1 - y_hat_1), (y_2 - y_hat_2), ..., (y_N - y_hat_N)] = [r_1, r_2, ..., r_N]
>
> Por lo tanto, la dirección de descenso más pronunciado en el espacio de predicción está alineada con los RESIDUOS VERDADEROS (r_n).

Aproximación mediante una Familia Simple de Modelos (H)

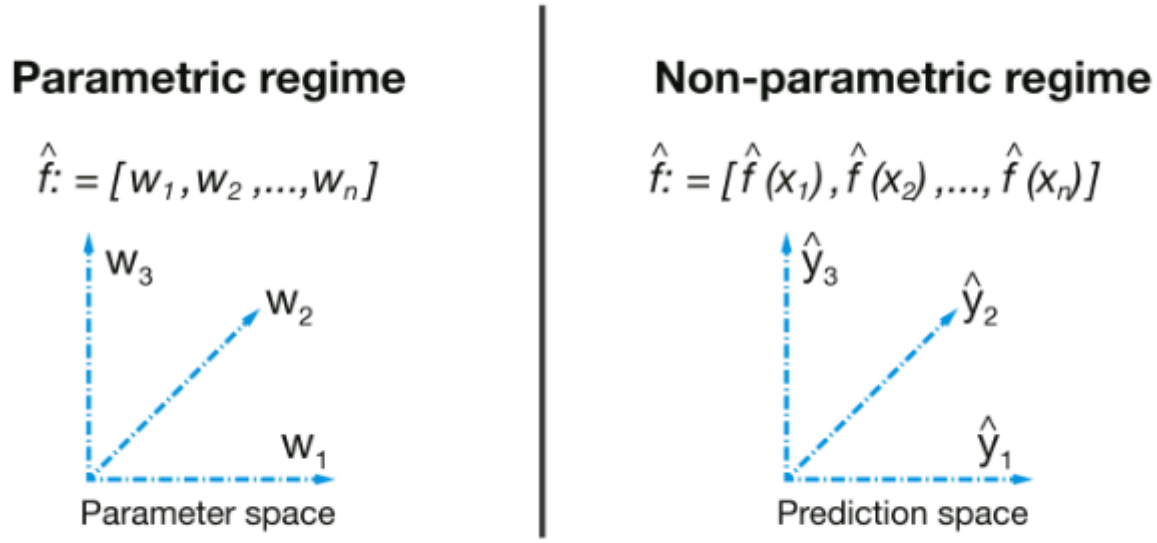
Como no se puede tomar un paso directo en la dirección exacta de t* sin caer en la memorización trivial, restringimos la dirección del paso a una clase o familia limitada de modelos base H (por ejemplo, árboles de decisión con profundidad máxima restringida, max_depth = 5).

El objetivo en cada iteración es seleccionar un aprendiz débil t_h(x) ∈ H tal que el vector de sus predicciones t = [t_h(x_1), t_h(x_2), ..., t_h(x_N)] sea lo más cercano posible al vector de residuos verdaderos t* = [r_1, r_2, ..., r_N]. Esto se resuelve entrenando sencillamente el aprendiz débil sobre el nuevo conjunto de datos formado por los pares (x_n, r_n).

De este modo, la regla de actualización recursiva de las predicciones toma la forma:

> **Regla de Actualización del Gradient Boosting**
> y_hat_n <= y_hat_n + λ * r_hat_n(x_n), para n = 1, ..., N
>
> donde r_hat_n(x_n) representa la predicción emitida por el modelo base entrenado para aproximar los residuos (el gradiente negativo), y λ ∈ (0, 1] es la tasa de aprendizaje.

In summary, for parametric models (say neural nets), gradient descent is done in the parameter space. Boosting can be thought of as doing gradient descent in the prediction space.



How do we deal with working in prediction space?

Let our current estimate be T^{(n) (x)}. In the prediction space this function is just a vector of the prediction values. In this case, the optimal direction to take a step using Gradient Descent is along the **negative gradient**.

t^{*} = - \frac{\partial \mathcal{L}}{\partial T^{(n)} (x)}

t^{*} = - [\frac{\partial \mathcal{L}}{\partial \hat{y}_1}, \frac{\partial \mathcal{L}}{\partial \hat{y}_2}, ... , \frac{\partial \mathcal{L}}{\partial \hat{y}_N}]

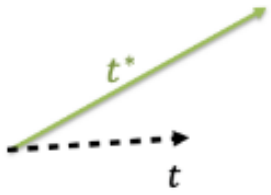
t^{*} = - [(\hat{y}_1 - y_1), ... , (\hat{y}_N - y_N)]

t^{*} = 2[r_1, r_2, ... , r_N]

Hence, the optimal direction to step in along the **residuals**.

Figura 5: Esquema comparativo de los regímenes paramétrico y no paramétrico y la formulación del gradiente negativo como el vector de residuos.

If we are restricting to a simple family of functions, we might not be able to always take a step in this optimal direction t*. But we want to take as close a step to this direction as possible. Let \mathbb{H} be a simple family of models (say Decision Trees of `max_depth = 5`).



Then we need to find some t \in \mathbb{H} such that t = [t(x_1), t(x_2), ... , t(x_N)] is as close to t^{*} = [r_1, r_2, ... , r_N] as possible. Well, this is quite easy to do if we train another model in \mathbb{H} on the training set {(x_1, r_1), (x_2, r_2), ... , (x_N, r_N)}.

So, we are changing our prediction in a way that we go to as close as possible to the true (optimal) y in small steps, and in that way it allows us to connect the input with the output. We start with t^{(*)} and find a model that approximates that direction (t).

t^{(*)} is our true residual. Our tree is approximating the true residual. We end up going in a similar, but not exactly the same direction as t^{(*)}.

In summary, we model the gradients and we follow those gradients in order to get to the minimum, and that is how we connect the input to the output.

Hence instead of using the gradient (the residuals) we use an **approximation** of the gradient that depends on the predictors:

\hat{y}_n \leftarrow \hat{y}_n + \lambda \hat{r}_n (x_n), \quad n=1, 2, ... , N

In Gradient Boosting, we use a simple model \hat{r}_n (x_n), to approximate the residuals r_n (x_n) in each iteration (which is the negative gradient of the loss in the prediction space).

> **QNOTA TÉCNICA PARA EL ESTUDIANTE**
>
> **Nota:** El boosting de gradiente desciende en un espacio de modelos o funciones que mapean x_n a y_n !

Figura 6: Aproximación de la dirección óptima t mediante un aprendiz débil restringido t en la familia H.*

4. Selección y Regulación de la Tasa de Aprendizaje (λ)

Al formalizar Gradient Boosting como un algoritmo de descenso por gradiente, podemos extender de forma directa las metodologías analíticas avanzadas para la selección del hiperparámetro de la tasa de aprendizaje λ (learning rate).

Comportamiento bajo Tasa de Aprendizaje Constante

- **Tasa de aprendizaje demasiado pequeña (λ « 0.01):** Si la tasa de aprendizaje es excesivamente pequeña, el modelo realiza pasos infinitesimales en el espacio funcional. Se requieren cientos o miles de iteraciones para aproximar el mínimo, incrementando drásticamente el costo computacional.
- **Tasa de aprendizaje demasiado grande (λ » 0.5):** El algoritmo sobrepasa constantemente la posición del mínimo funcional, provocando oscilaciones violentas ('rebotando' alrededor del óptimo) e impidiendo la convergencia del ensamble.
- **Ajuste empírico recomendatorio:** Requiere una búsqueda en malla o ajuste óptimo mediante validación cruzada (Cross-Validation) en el conjunto de validación.

Estrategia Dinámica / Variable basada en la Magnitud del Gradiente

Una aproximación superior consiste en adaptar dinámicamente el valor de λ en función de la magnitud del vector gradiente acumulado, expresado mediante la norma $||\nabla f(x)||$:

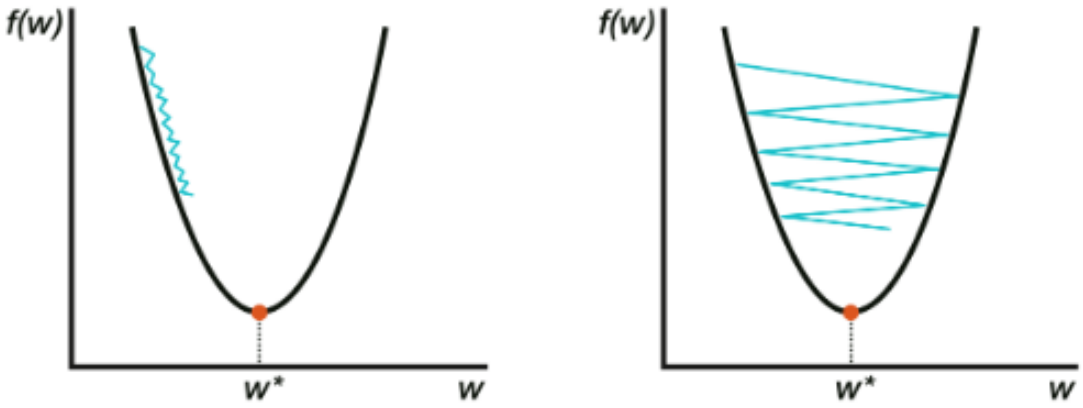
> **Ajuste Dinámico de la Tasa de Aprendizaje**
> $\lambda = h(|| \nabla f(x) ||)$
>
> • Lejos del óptimo ($||\nabla f(x)||$ elevado): Se asigna una tasa de aprendizaje mayor λ para acelerar el desplazamiento inicial hacia la cuenca del mínimo.
> • Cerca del óptimo ($||\nabla f(x)||$ reducido): Se reduce automáticamente el valor de λ para realizar pasos finos y asegurar una convergencia suave y precisa.

¿Por qué queremos conectar el gradient boosting con el descenso por gradiente?

- Al establecer esta conexión, podemos importar la gran cantidad de técnicas para estudiar el descenso por gradiente y analizar el gradient boosting.

Por ejemplo, podemos razonar fácilmente sobre cómo elegir la tasa de aprendizaje \lambda en el gradient boosting.

Para una tasa de aprendizaje constante \lambda, si \lambda es demasiado pequeño (imagen de la izquierda), se requieren demasiadas iteraciones. Si \lambda es demasiado grande, el algoritmo puede "rebotar" alrededor del óptimo y nunca acercarse lo suficiente.



Existen formas de controlar \lambda que se discutirán con más detalle en los siguientes cursos.

En resumen, elegir \lambda: - Si \lambda es una constante, entonces se debe ajustar mediante validación cruzada. - Para obtener mejores resultados, use variable \lambda. Es decir, permita que el valor de \lambda dependa del gradiente:

\lambda = h(\lvert \lvert \nabla f(x) \rvert \rvert),

donde \nabla f(x) es la magnitud del gradiente. Así: - alrededor del óptimo, cuando el gradiente es pequeño \lambda debe ser pequeño. - lejos del óptimo, cuando el gradiente es grande, \lambda debe ser mayor.

Figura 7: Impacto del tamaño de paso λ en la convergencia del descenso de gradiente y formulación de la tasa de aprendizaje variable dependiente de la magnitud del gradiente.

5. Resumen Ejecutivo de la Nota Técnica

Gradient Boosting desciende formalmente en un espacio de funciones o modelos que conectan las variables de entrada (x_n) con las respuestas objetivas (y_n). A diferencia del descenso de gradiente convencional en el espacio paramétrico (donde se ajustan pesos estáticos) o en el espacio de predicciones puro (donde se cae en la memorización trivial), Gradient Boosting proyecta el gradiente

negativo (residuos) sobre una familia restringida de aprendices débiles H. Esta aproximación iterativa permite construir un modelo global expresivo, de bajo sesgo y altamente generalizable.

Descenso de Gradiente (Gradient Descent)

Sección 5 Suplementario — Optimización Iterativa de Funciones de Pérdida

Índice de Contenidos

1. La Función de Pérdida y el Objetivo de Optimización

En la Ciencia de Datos y el Aprendizaje Automático, la construcción de modelos eficientes se fundamenta en la definición y minimización de una **función de pérdida (loss function)**. La función de pérdida representa la agregación matemática de todos los errores individuales que comete el modelo al realizar predicciones sobre el conjunto de datos de entrenamiento. Por lo tanto, el objetivo primordial de la fase de entrenamiento consiste en encontrar la combinación de parámetros que minimice dicho error acumulado.

La función de pérdida evalúa las predicciones emitidas por el modelo y genera una métrica cuantitativa sobre su desempeño. Sin embargo, no basta con saber si el modelo funciona bien o mal; para optimizarlo, se requiere identificar la dirección exacta en la que se deben ajustar sus parámetros internos. La función de pérdida depende directamente de los parámetros del modelo, denotados genéricamente como *w_i* (equivalentes a los coeficientes *β_i* en modelos de regresión lineal y logística). Formales como:

L(w_0, w_1, w_2, ...) --> Para un solo parámetro: L(w_1)

5 Suplementario: Descenso de Gradiente

Descenso del gradiente

En las tareas de Ciencia de Datos y Aprendizaje Automático trabajamos con funciones de pérdida de modelos. Normalmente nuestro objetivo es minimizar una función de pérdida. La función de pérdida es la agregación de todos los individuales *errores* de un modelo que tenemos en el conjunto de entrenamiento. Al minimizar la función de pérdida nos esforzamos por encontrar el modelo que tenga los menores errores.

La función de pérdida de un modelo toma las predicciones de un modelo y nos indica qué tan mal o bien está funcionando basado en esas predicciones. Sin embargo, no basta con saber si nuestro modelo está funcionando bien o mal, necesitamos saber cómo podría mejorar, es decir, cómo debería ajustar sus parámetros para rendir mejor. Ya que lo único que podemos hacer en el modelo, además de cambiar el modelo por completo, es cambiar sus parámetros.

La función de pérdida depende de los parámetros del modelo. Llamémoslos w_i (en la regresión lineal y logística los llamábamos betas: β_i. No importa cómo los llamemos, lo importante es recordar que estos son algunos de los parámetros del modelo de los que depende la función de pérdida.). Podemos representar la función de pérdida como una función de los parámetros de la siguiente manera:

$$\mathcal{L}\left(w_0, w_1, w_2, \ldots, w_i\right)$$

Para simplificar, consideremos únicamente un parámetro por el momento:

$$\mathcal{L}\left(w_1\right)$$

Figura: Representación de la función de pérdida L(w_1) en función del parámetro del modelo.

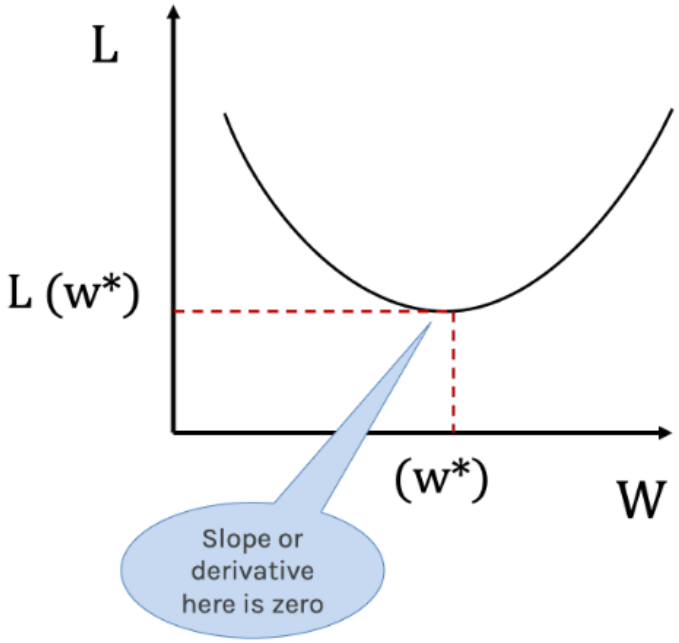
> **PRINCIPIO FUNDAMENTAL:** Ajustar los parámetros internos w_i es la única vía directa para mejorar el desempeño de un algoritmo sin cambiar completamente la arquitectura del modelo. La función de pérdida actúa como el mapa topográfico que guía esa búsqueda.

2. Condición de Mínimo y Métodos de Solución: Analítica vs. Iterativa

El punto ideal de entrenamiento corresponde al parámetro óptimo **w*** que genera el valor mínimo absoluto de la pérdida **L(w*)**. Desde la perspectiva del cálculo diferencial, un mínimo local o global ocurre en el punto donde la pendiente de la recta tangente a la curva es exactamente igual a cero. Matemáticamente, la condición de primer orden se expresa mediante la derivada de la función de pérdida respecto al parámetro:

dL(w) / dw = 0

Idealmente, queremos conocer el valor de w que nos dé la **mínima** $\mathcal{L}(w)$, lo que significa que la pérdida está en su **mínimo**.



Para encontrar el punto óptimo (el mínimo) de una función $\mathcal{L}(w)$, tomamos la derivada con respecto al parámetro:

$$\frac{d\mathcal{L}(w)}{dw} = 0$$

Figura: Condición de óptimo L(w): la pendiente o derivada en el punto mínimo es igual a cero.*

Para encontrar la posición del óptimo existen dos aproximaciones metodológicas principales:

- **Solución en forma cerrada (Analítica):** Consiste en despejar directamente la variable w en la ecuación diferencial. Este procedimiento es factible en algoritmos como la Regresión Lineal mediante la ecuación de mínimos cuadrados ordinarios (MCO).
- **Métodos Iterativos (Descenso de Gradiente):** En la mayoría de los modelos complejos (como Regresión Logística, Redes Neuronales o Máquinas de Vector de Soporte), no existe una solución cerrada analítica. En estos casos, se debe recurrir a un algoritmo iterativo que aproxime gradualmente el mínimo partiendo de una suposición inicial w_0.

Normalmente, cuando comenzamos con algunos parámetros del modelo, es muy probable que no estemos en el mínimo, por lo que necesitamos indicarle al modelo cómo cambiar para mejorar, y para ser el mejor posible debe estar en el mínimo de la función de pérdida. El modelo no puede hacerlo mejor que eso.

El objetivo es encontrar dónde la pendiente (la derivada) de la función de pérdida es igual a cero. Existen varias formas de abordar esta tarea. Una forma sería resolver la ecuación anterior, es decir, encontrar la w de la ecuación. (Esto podría hacerse en la regresión lineal.) Sin embargo, en muchos casos la solución en forma cerrada de la ecuación no existe (por ejemplo, en la regresión logística). En ese caso, tenemos que recurrir a un enfoque diferente para resolver el problema:

Paso 1

Comience con algún valor aleatorio asignado a nuestro parámetro del modelo. w_0 es nuestra primera suposición de cuál debería ser el valor del parámetro del modelo.

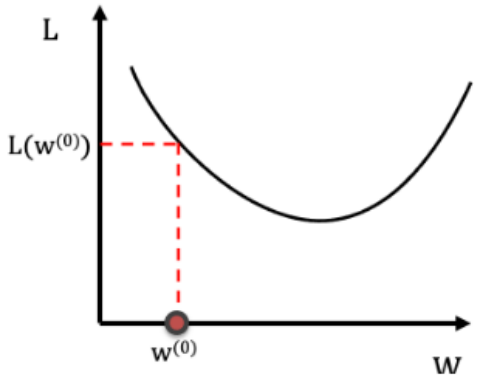


Figura: Inicialización del parámetro w_0 cuando no existe solución cerrada directa.

3. Algoritmo Paso a Paso del Descenso de Gradiente

El algoritmo de Descenso de Gradiente opera mediante una secuencia estructurada de actualizaciones que ajustan el parámetro en la dirección opuesta a la pendiente de la función de pérdida:

Paso 1: Inicialización de Parámetros

Se asigna un valor inicial arbitrario o aleatorio al parámetro del modelo, denotado como **w_0**. Este valor representa la estimación inicial del algoritmo sobre la posición de los parámetros óptimos.

Paso 2: Cálculo de la Pendiente (Derivada Local)

Se evalúa la derivada de la función de pérdida en el punto actual **w_0**. La derivada representa la pendiente de la recta tangente a la curva de pérdida en dicho punto, indicando la razón de cambio instantánea de la pérdida.

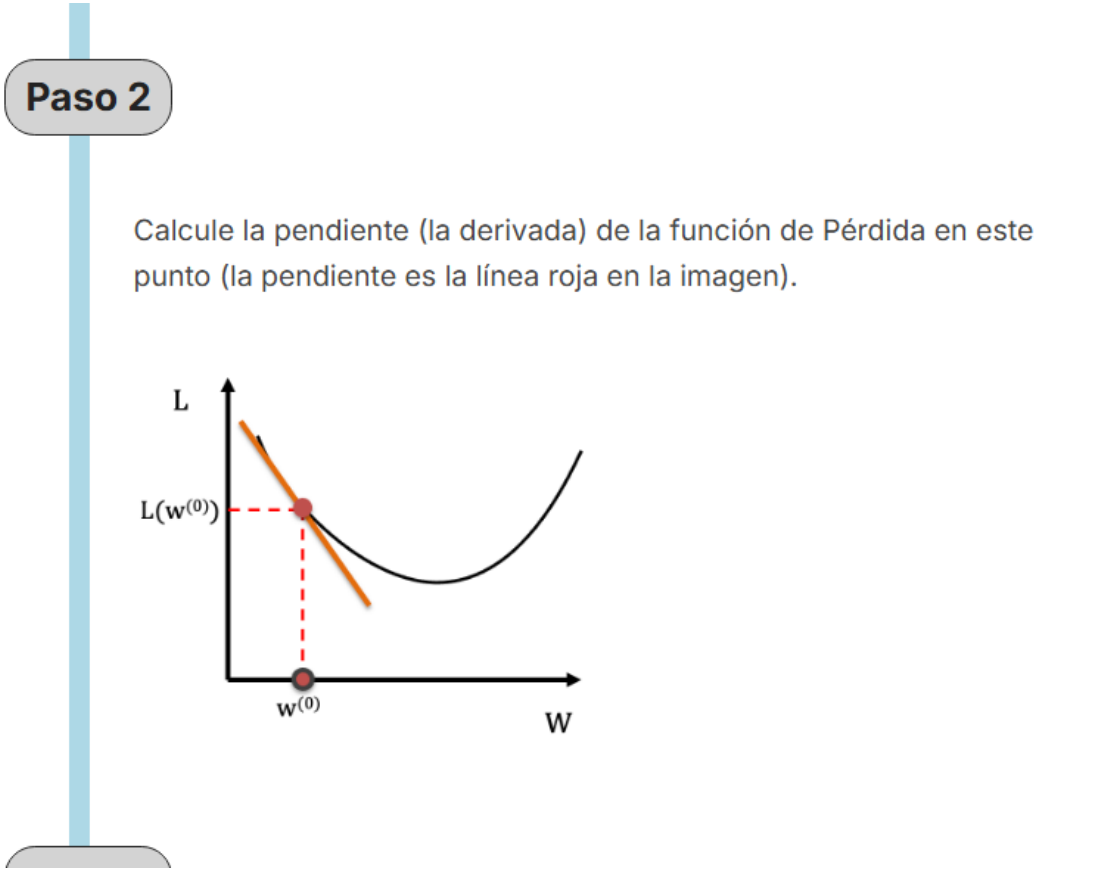


Figura: Cálculo de la pendiente (línea tangente roja) de la función de pérdida en el punto w_0.

Paso 3: Regla de Dirección del Paso

Se efectúa un desplazamiento en la **dirección opuesta al signo de la derivada** para asegurar que la pérdida disminuya en la siguiente iteración. La regla de movimiento establece:

- **Derivada Negativa (dL/dw < 0):** La pendiente disminuye hacia la derecha. Por lo tanto, el parámetro debe incrementarse (desplazamiento a la derecha) para avanzar hacia el mínimo.
- **Derivada Positiva (dL/dw > 0):** La pendiente aumenta hacia la derecha. Por lo tanto, el parámetro debe reducirse (desplazamiento a la izquierda) para aproximarse al óptimo.

Paso 3

Da un paso en la dirección opuesta a la derivada. Si la derivada es negativa, nos movemos hacia la derecha; si la derivada es positiva, nos movemos hacia la izquierda. En la imagen de abajo la pendiente es negativa, por lo que nos movemos a la derecha.

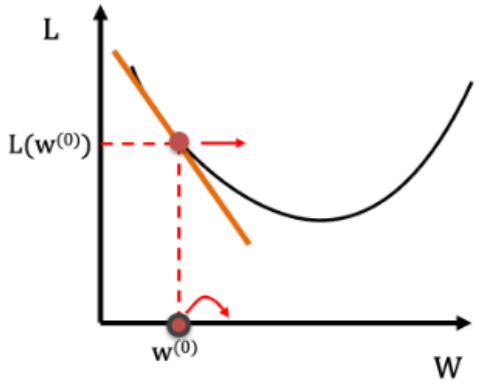


Figura: Desplazamiento a la derecha cuando la pendiente evaluada es negativa.

Paso 4: Actualización y Reevaluación en w_1

Al realizar el paso desde **w_0**, se obtiene una nueva posición de parámetro **w_1**. En esta nueva ubicación se recalcula la derivada. Si la pendiente sigue siendo negativa, se realiza un paso adicional hacia la derecha hacia **w_2**.

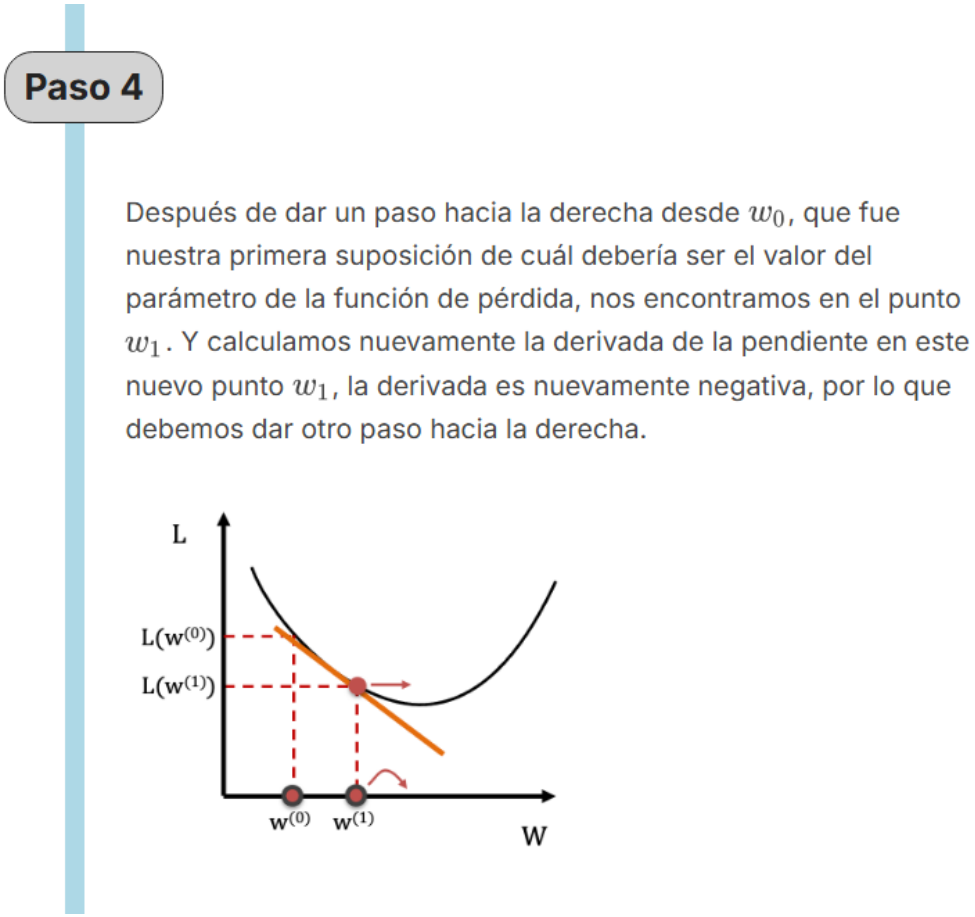


Figura: Reevaluación de la derivada en el nuevo punto w_1 y continuación del descenso.

Paso 5: Repetición Iterativa del Proceso

El procedimiento se repite cíclicamente (recalculando la derivada y avanzando en sentido opuesto) descendiendo paulatinamente por la curva de pérdida hasta aproximarse a la cuenca de atracción del mínimo.

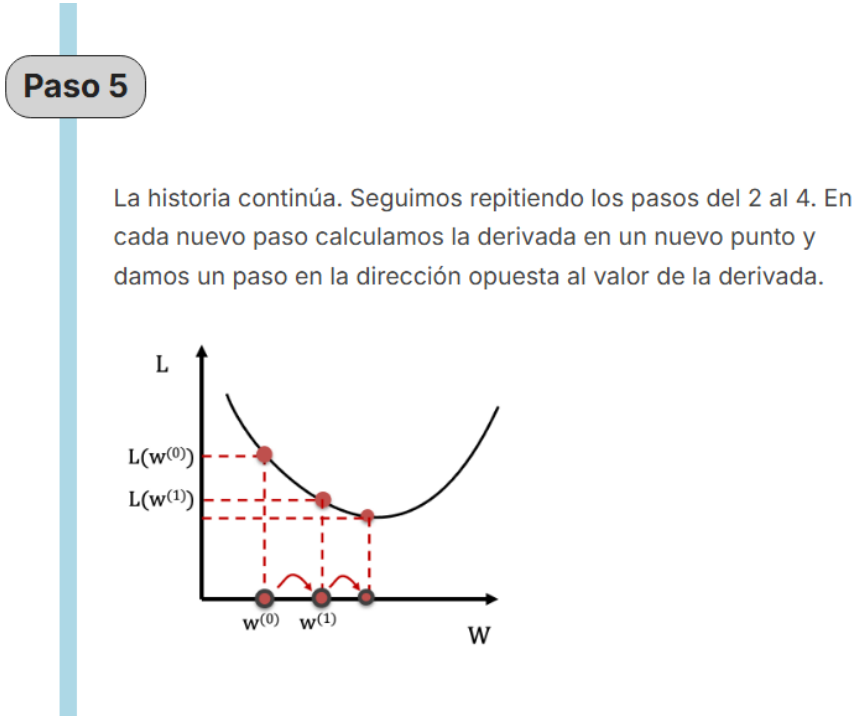


Figura: Proceso iterativo de avance continuo hacia el mínimo de la función de pérdida.

4. Criterios de Parada e Implementación Computacional

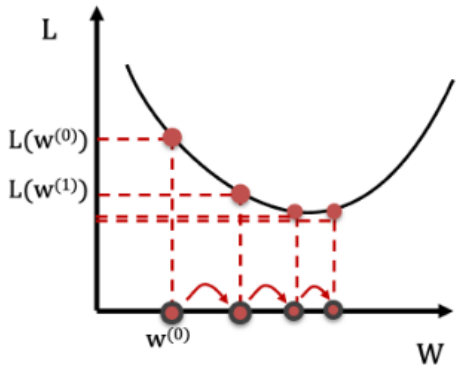
Teóricamente, el algoritmo debería detenerse en el instante exacto en que la derivada alcanza un valor nulo (**dL/dw = 0**). Sin embargo, en entornos computacionales reales, obtener exactamente cero es numéricamente inviable debido a los límites de precisión en aritmética de punto flotante.

En la práctica, se aplican dos criterios principales de parada:

- **Umbral de Tolerancia en el Cambio de Pérdida (ε):** Se establece un umbral numérico pequeño ε. El algoritmo finaliza si la reducción en la pérdida entre dos iteraciones consecutivas es inferior a ε, expresado como |L(w_{k+1}) - L(w_k)| < ε.
- **Límite Máximo de Iteraciones:** Se define un número máximo de pasos permitidos para garantizar la terminación del algoritmo y prevenir bucles infinitos en superficies complejas.

Paso 6

Deténgase cuando no haya más mejoras o después de cierto número de iteraciones.



Sería conveniente detener el algoritmo cuando la derivada sea cero. En la práctica, obtener cero en la computadora es difícil. En cambio, podemos establecer un umbral para la derivada o, equivalentemente, podríamos decir "Si no mejoramos en cierta cantidad, debemos detenernos".

Figura: Criterio de detención al aproximarse al mínimo de la función de pérdida.

5. Resumen del Algoritmo de Descenso de Gradiente

La siguiente tabla condensa las fases constitutivas del algoritmo de Descenso de Gradiente, contrastando sus fundamentos matemáticos con su propósito práctico:

Fase / Paso	Expresión / Regla	Interpretación y Propósito
1. Inicialización	w = w_0 (aleatorio)	Establece el punto de partida del parámetro en la superficie de pérdida.
2. Evaluación	dL(w) / dw	Mide la pendiente y tasa de cambio instantánea en la posición actual.
3. Dirección	Si dL/dw < 0 -> Derecha Si dL/dw > 0 -> Izquierda	Asegura el avance en sentido opuesto a la pendiente para reducir la pérdida.

4. Actualización	w_{k+1} = w_k - step	Genera el nuevo valor de parámetro trasladando la posición sobre la curva.
5. Terminación	|L(w_{k+1}) - L(w_k)| < ε o k >= MaxIter	Finaliza la ejecución cuando las mejoras resultan despreciables o se alcanza el límite.

SÍNTESIS METODOLÓGICA: El Descenso de Gradiente es la columna vertebral de la optimización en Machine Learning. Al transformar la búsqueda de parámetros óptimos en un proceso iterativo guiado por derivadas, permite entrenar modelos complejos en los que no existe una solución analítica directa.

Preguntas Frecuentes sobre el Descenso del Gradiente

Generalización Multidimensional, Magnitud Dinámica del Paso y Tasa de Aprendizaje (η)

Índice de Contenidos

(El índice de contenidos se actualizará automáticamente al abrir el archivo en Microsoft Word o Google Docs).

1. Generalización Multidimensional mediante el Vector Gradiente

La generalización del descenso del gradiente a funciones de pérdida con múltiples parámetros se logra calculando las derivadas parciales individuales respecto a cada variable. En la práctica, los modelos de aprendizaje automático no dependen de un único peso, sino de vectores de parámetros de alta dimensión ($w = [w_1, w_2, ..., w_p]$). Para optimizar simultáneamente todos los parámetros, la función de pérdida $L(w)$ se evalúa derivando parcialmente respecto a cada parámetro por separado mientras se mantienen constantes los demás.

El conjunto ordenado de todas estas derivadas parciales se denomina **vector gradiente** (denotado como ∇L o $\nabla_w L$). Matemáticamente, el gradiente se expresa de la siguiente manera:

> **Definición Formal del Vector Gradiente:**
>
> $$\nabla_w L(w) = [\ \partial L / \partial w_1\ ,\ \partial L / \partial w_2\ ,\ \dots\ ,\ \partial L / \partial w_p\ ]$$
>
> **Donde cada componente $\partial L / \partial w_j$ representa la tasa de cambio instantánea de la pérdida respecto al j-ésimo parámetro del modelo.**

Para encontrar un punto estacionario en un espacio multidimensional, el algoritmo busca igualar el vector gradiente completo a cero ($\nabla L = 0$), lo que equivale a resolver simultáneamente el sistema de p ecuaciones diferenciales de primer orden.

5 Suplementario: Preguntas frecuentes

Pregunta:

¿Cómo generalizamos este algoritmo si tenemos más de un parámetro del cual depende la función de pérdida?

▼ Mostrar Respuesta

- Tomamos las derivadas parciales (derivadas separadas de la función de pérdida, donde cada derivada se calcula con respecto a un parámetro a la vez) de la función de pérdida y las igualamos a cero. En matemáticas esto también se llama tomar **el gradiente**:

$$\nabla_W L(W) = \left[\frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial W_2}, \cdots, \frac{\partial L}{\partial W_p}\right]$$

Figura 1: Formulación del vector gradiente mediante derivadas parciales individuales (Fuente original de la lección).

2. Determinación del Tamaño del Paso y la Tasa de Aprendizaje (η)

El tamaño del paso en cada iteración del algoritmo se ajusta de forma directamente proporcional a la magnitud de la pendiente de la función de pérdida. Esta propiedad garantiza un comportamiento autoregulado de optimización: cuando los parámetros están lejanos al mínimo y la pendiente es muy pronunciada, el algoritmo toma pasos amplios para acelerar la convergencia; en cambio, cuando se aproxima al mínimo y la pendiente disminuye progresivamente hacia cero, los pasos se reducen automáticamente para evitar sobrepasar el punto óptimo.

La deducción matemática de la regla de actualización se construye a partir de los siguientes tres pasos fundamentales:

- **1. Esquema general de actualización:** Cualquier algoritmo iterativo actualiza la estimación del parámetro mediante el operador w ← w + paso.
- **2. Definición proporcional de dirección opuesta:** Para descender por la superficie de error, el paso debe dirigirse en sentido inverso a la derivada y escalarse por una constante: paso = -η · (dL / dw).
- **3. Ecuación de actualización unidimensional:** Sustituyendo la expresión del paso se obtiene w ← w - η · (dL / dw).
- **4. Regla de actualización vectorial convencional:** Ampliando al caso multidimensional en el paso de iteración discreto t a t+1: $w^{(t+1)} \leftarrow w^{(t)} - \eta \cdot \nabla_w L(w^{(t)})$.

> **Interpretación de los Componentes de la Regla de Actualización:**
>
> • **Signo Negativo (-):** Garantiza el movimiento hacia el descenso más pronunciado (dirección opuesta al gradiente ascendente).
>
> • **Constante Eta (η):** Representa la constante de proporcionalidad, formalmente denominada Tasa de Aprendizaje (Learning Rate). Controla qué porcentaje de la magnitud del gradiente se traduce en

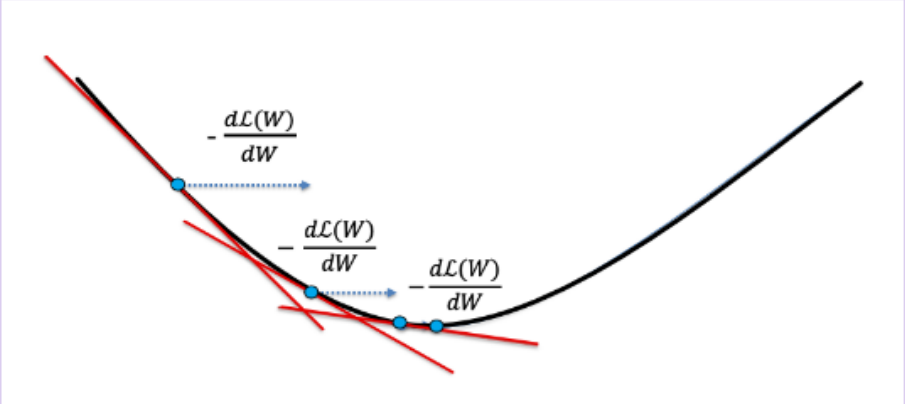
movimiento efectivo.

PREGUNTA:

¿Cuál debería ser el tamaño del paso?

▼ Mostrar respuesta

Podemos hacer que el tamaño del paso sea directamente proporcional a la pendiente. Cuanto mayor es la pendiente, mayor es el tamaño del paso; cuando la pendiente disminuye, también lo hace el tamaño del paso.



Matemáticamente, dar un paso significa:

$$w^{new} = w^{old} + paso$$

Dar un paso en la dirección opuesta a la derivada y proporcional a la derivada significa:

$$w^{new} = w^{old} - \eta \frac{dL}{dw}$$

El signo menos está presente porque estamos dando un paso en la dirección opuesta. La letra griega "eta" η es la constante de proporcionalidad, también conocida como **la tasa de aprendizaje**.

Cambiando a una notación más convencional:

$$x^{(i+1)} = w^{(i)} - \eta \frac{dL}{dw}$$

Figura 2: Derivación matemática del tamaño de paso proporcional a la derivada y la constante η (Fuente original de la lección).

3. Matriz Síntesis de Conceptos y Fórmulas Clave

El siguiente cuadro resume de forma estructurada los componentes matemáticos y la función de cada elemento dentro de la dinámica de optimización del descenso del gradiente:

Concepto Clave	Expresión Matemática	Función en la Optimización

Derivada Parcial	$\partial L / \partial w_j$	Mide la sensibilidad instantánea de la pérdida respecto al parámetro j-ésimo manteniendo fijos los demás.
Vector Gradiente	$\nabla_w L = [\partial L/\partial w_1, \ldots, \partial L/\partial w_p]$	Agrupa las tasas de cambio de todos los parámetros; indica la dirección del ascenso más pronunciado.
Tamaño de Paso Proporcional	$paso = -\eta \cdot (dL / dw)$	Ajusta la magnitud del desplazamiento de acuerdo con la inclinación local de la curva de error.
Regla de Actualización	$w^{(t+1)} \leftarrow w^{(t)} - \eta \cdot \nabla L(w^{(t)})$	Ecuación iterativa fundamental que desplaza los parámetros en la dirección opuesta al gradiente.

4. Consideraciones Prácticas sobre la Elección de la Tasa de Aprendizaje (η)

La constante de proporcionalidad η (learning rate) condiciona de forma crítica la convergencia y la estabilidad del algoritmo. Si el valor de η es excesivamente grande, los pasos resultantes pueden rebasar el mínimo local o global, provocando oscilaciones inestables o incluso la divergencia del algoritmo. Por el contrario, si η es demasiado pequeño, el tamaño del paso se vuelve insignificante, lo que genera un proceso de optimización extremadamente lento que puede quedar atrapado en zonas planas o requerir un número prohibitivo de iteraciones.

Por este motivo, en la práctica moderna de machine learning se utilizan técnicas complementarias como el decrecimiento del learning rate (learning rate decay) o algoritmos adaptativos (como Adam, RMSprop o AdaGrad) que ajustan dinámicamente η en función del historial del gradiente.

Sección 5 Suplementario: Resumen del Descenso del Gradiente

Síntesis conceptual sobre optimización de primer orden, naturaleza iterativa y control mediante la tasa de aprendizaje

Índice de Contenidos

(El índice de contenidos se actualizará automáticamente al abrir el archivo en Microsoft Word o Google Docs).

1. Definición y Naturaleza del Descenso del Gradiente como Algoritmo de Primer Orden

El **descenso del gradiente** constituye el algoritmo de optimización fundamental en Ciencia de Datos y Aprendizaje Automático para encontrar el mínimo de una función de pérdida *L(w)*. Se clasifica formalmente como un algoritmo de **primer orden** debido a que evalúa únicamente la primera derivada parcial de la función respecto a los parámetros del modelo durante el proceso de optimización, omitiendo información de derivadas de segundo orden (como la matriz Hessiana).

- **Optimizador de Primer Orden:** Utiliza exclusivamente la información del vector de primeras derivadas (gradiente) para determinar la dirección de actualización de los parámetros.
- **Objetivo de Minimización:** Busca localizar el punto del espacio paramétrico w* que minimice el error agregado de las predicciones sobre el conjunto de entrenamiento.

2. Proceso Iterativo y Criterios de Parada

El algoritmo ópera de manera estrictamente **iterativa**, lo que implica que la actualización de los parámetros se repite sistemáticamente ciclo a ciclo hasta que se satisface una condición de detención predefinida.

- **Naturaleza Secuencial:** A partir de una estimación inicial w_0, las predicciones y los gradientes se reevalúan en cada paso t para generar el nuevo valor w_{t+1}.
- **Condición de Parada:** El bucle iterativo finaliza cuando la magnitud de la derivada cae por debajo de un umbral de tolerancia ε, o bien cuando se alcanza un número máximo prefijado de iteraciones.

3. Dirección de Descenso Máximo de la Función de Pérdida

Desde la perspectiva geométrica, la función de pérdida **L** disminuye con la máxima rapidez posible cuando nos desplazamos en la dirección del **gradiente negativo (-∇L)**. Dado que el gradiente apunta

hacia la dirección de máximo incremento de la función, su opuesto indica la trayectoria de descenso más pronunciado hacia el mínimo local o global.

4. Control del Paso Mediante la Tasa de Aprendizaje (η)

La magnitud del ajuste en cada iteración viene regulada directamente por la **tasa de aprendizaje (η o λ)**, que actúa como la constante de proporcionalidad que escala el paso en la dirección del gradiente negativo según la regla $w_{t+1} = w_t - \eta \cdot (dL / dw)$.

- **Tasa Reducida (η muy pequeña):** Garantiza un descenso estable, pero requiere un elevado número de iteraciones para aproximarse al óptimo.
- **Tasa Elevada (η muy grande):** Acelera los primeros pasos, pero introduce el riesgo de oscilar de forma inestable alrededor del mínimo o provocar divergencia.

5. Tabla Resumen de los Principios del Descenso del Gradiente

La siguiente tabla sintetiza las propiedades fundamentales expuestas en la lección de resumen:

Propiedad Clave	Definición Técnica	Impacto Operativo
Optimización de Primer Orden	Utiliza únicamente la primera derivada parcial (dL/dw) para guiar el modelo.	Bajo costo computacional por iteración en comparación con métodos de segundo orden.
Método Iterativo	Actualización secuencial de parámetros repetida hasta cumplir una condición de parada.	Permite aproximar numéricamente el mínimo sin requerir solución analítica cerrada.
Dirección del Gradiente Negativo	L disminuye a la máxima velocidad en la dirección opuesta a la derivada.	Asegura la trayectoria de descenso más eficaz hacia la minimización del error.
Tasa de Aprendizaje (η)	Constante de proporcionalidad que modula el tamaño del paso de actualización.	Regula el compromiso entre rapidez de convergencia y estabilidad numérica.

6. Lámina de Referencia de la Lección

A continuación se adjunta la captura de pantalla oficial correspondiente a la Sección 5 Suplementario: Resumen:

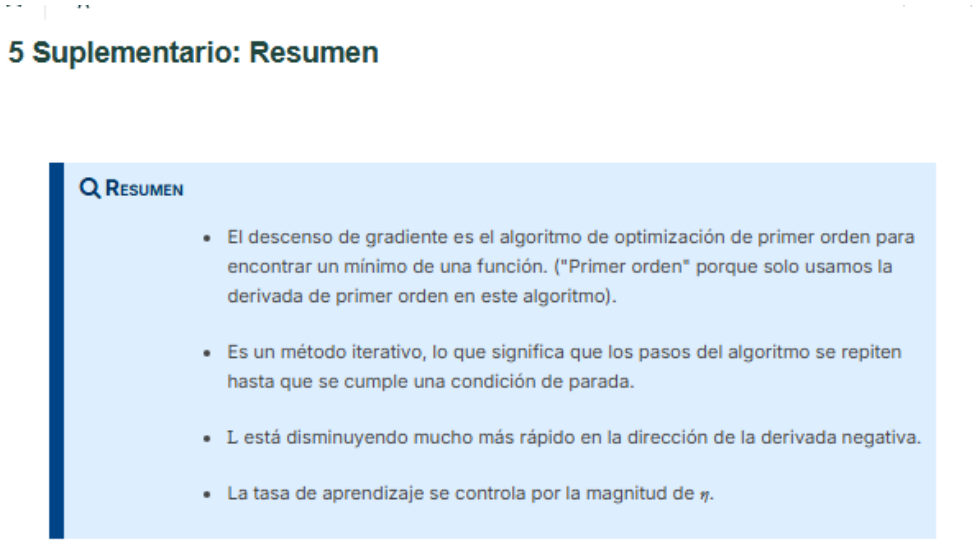


Figura 1: Lámina explicativa de la Sección 5 Suplementario - Resumen del Descenso del Gradiente.

Consideraciones Avanzadas sobre el Descenso del Gradiente

Cálculo de Derivadas, Configuración de la Tasa de Aprendizaje y Manejo de Mínimos Locales vs. Globales

Tabla de Contenidos

1. Cálculo de Derivadas Mediante la Regla de la Cadena

En los algoritmos de optimización basados en el Descenso del Gradiente, es imprescindible calcular las derivadas parciales de la función de pérdida respecto a cada uno de los parámetros del modelo. La diferenciación de funciones arbitrarias complejas o compuestas se realiza sistemáticamente mediante la **Regla de la Cadena**.

Cuando la salida de un modelo está estructurada como una composición de funciones, la derivada de la variable final respecto a la variable de entrada original se calcula multiplicando las derivadas parciales intermedias. Específicamente, si se define una variable intermedia **y = g(x)** y la función objetivo final **z = f(y) = f(g(x))**, la derivada de **z** con respecto a **x** se expresa formalmente como:

$$\partial z / \partial x = (\partial z / \partial y) \cdot (\partial y / \partial x)$$

2. Configuración y Efecto de la Tasa de Aprendizaje (η)

La elección del hiperparámetro de la tasa de aprendizaje (representado por **η**) determina directamente la magnitud de los pasos que el algoritmo efectúa sobre la superficie de pérdida en cada iteración. Una comprensión clara de la analogía geométrica del descenso por una colina permite diagnosticar y corregir comportamientos anómalos durante el entrenamiento.

- **Tasa de Aprendizaje Demasiado Pequeña (η ≪ óptimo):** El algoritmo realiza avances extremadamente pequeños en cada paso. Aunque eventualmente alcanzará el fondo de la superficie de pérdida, requerirá un número excesivo de iteraciones y un tiempo de cómputo ineficiente.
- **Tasa de Aprendizaje Demasiado Grande (η ≫ óptimo):** El algoritmo sobrepasa el punto mínimo en cada paso, generando oscilaciones erráticas y rebotando de un lado a otro de la superficie de pérdida sin lograr estabilizarse, lo que conduce a la divergencia.
- **Tasa de Aprendizaje Apropiada:** Permite una convergencia suave y rápida hacia el mínimo. A medida que el parámetro se aproxima al fondo de la colina, la magnitud del gradiente disminuye gradualmente, provocando que los pasos se reduzcan de forma natural.

> 📌 **REGLA GENERAL PRÁCTICA:** Como regla empírica general en Ciencia de Datos y Aprendizaje Automático, la tasa de aprendizaje se configura dentro del rango estándar de **10^{-1} a 10^{-4} (0.1 a 0.0001)**. Seleccionar valores dentro de esta ventana proporciona un equilibrio adecuado entre velocidad de convergencia y estabilidad numérica.

Tabla 1: Comparativa de Comportamiento según el Valor de la Tasa de Aprendizaje (η)

Configuración de η	Comportamiento del Algoritmo	Resultado y Convergencia
η Demasiado Pequeño ($< 10^{-4}$)	Pasos diminutos en la dirección del gradiente.	Convergencia extremadamente lenta; alto costo de cómputo.
η Demasiado Grande ($> 10^{-1}$)	Paso sobredimensionado que sobrepasa el mínimo.	Oscilaciones erráticas y divergencia de la función de pérdida.
η Apropiado (10^{-1} a 10^{-4})	Avanza hacia el mínimo reduciendo el paso automáticamente.	Convergencia exitosa y estable al punto de mínima pérdida.

3. Mínimos Locales vs. Mínimos Globales

La garantía de que el algoritmo de Descenso del Gradiente encuentre el óptimo absoluto depende exclusivamente de la geometría matemática de la función de pérdida:

- **Funciones de Pérdida Convexas:** Poseen un único punto de mínimo global en todo su dominio. Modelos clásicos como la **Regresión Lineal, Regresión Polinómica y Regresión Logística** cuentan con funciones de pérdida convexas, garantizando matemáticamente que cualquier mínimo encontrado corresponda al mínimo global.
- **Funciones de Pérdida No Convexas:** Presentan múltiples valles y depresiones en el espacio de parámetros. En estos escenarios existen diversos mínimos locales (puntos donde la derivada se anula localmente) además del mínimo global, lo que puede provocar que el gradiente se detenga en una solución subóptima.

4. Estrategias Prácticas ante Funciones No Convexas

Debido a que el Descenso del Gradiente estándar puede quedar atrapado en un mínimo local al anularse la derivada en dicho punto, la práctica moderna del Aprendizaje Automático emplea diversas heurísticas para mitigar esta limitación:

- **Múltiples Inicializaciones Aleatorias:** Ejecutar el algoritmo desde diferentes posiciones iniciales aleatorias y seleccionar la solución que registre el valor mínimo de pérdida entre todas las ejecuciones.
- **Descenso de Gradiente Estocástico (SGD):** Utilizar subconjuntos aleatorios (minibatches) para estimar el gradiente, introduciendo una estocasticidad que permite 'escapar' de valles locales poco profundos.
- **Inyección de Ruido en la Función de Pérdida:** Perturbar ligeramente los gradientes durante la optimización para alterar las cuencas de atracción locales.
- **Variación Dinámica de Tamaños de Paso:** Probar esquemas con programación de tasas de aprendizaje o impulsos (momentum).

Límite Teórico: Lamentablemente, ninguna de las estrategias heurísticas anteriores garantiza al 100% la localización del mínimo global en superficies no convexas. La única técnica con garantía matemática absoluta es la ejecución del algoritmo de Monte Carlo por Cadenas de Markov (MCMC) un número infinito de veces. En la práctica, se acepta que el Descenso del Gradiente busca soluciones locales de alta calidad.

5. Esquemáticos y Gráficos Explicativos Oficiales

5 Suplementario: Consideraciones sobre el descenso de gradiente

Consideraciones sobre el descenso de gradiente

Todavía necesitamos calcular las derivadas.

¿Podemos calcular la derivada de *cualquier* función de pérdida?

Sí, podemos hacerlo usando la **Regla de la cadena**.

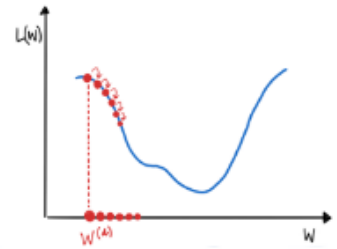
Si tenemos una función y tal como $y = g(x)$, y z es una función de y, $z = f(y) = f(g(x))$, por lo tanto z es una función de una función de x. La Regla de la cadena nos dice cómo encontrar una derivada parcial de z con respecto a x :

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y}\frac{\partial y}{\partial x}$$

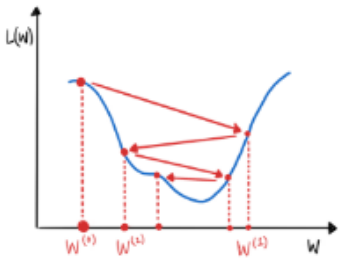
Configurar la tasa de aprendizaje

Nuestra elección de la tasa de aprendizaje tiene un impacto significativo en el rendimiento del Descenso del Gradiente.

Podemos pensar en encontrar un mínimo de una función como si fuera una colina que descendemos. Cuando η es demasiado pequeño, el algoritmo avanza muy poco. Eventualmente llegaremos al fondo de la "colina", pero puede tardar demasiado.

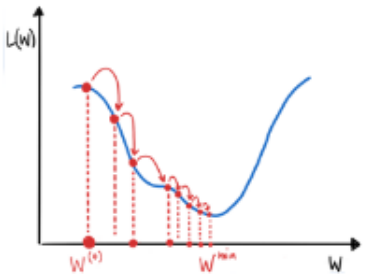


Cuando η es demasiado grande, el algoritmo puede sobrepasar el mínimo y tener oscilaciones erráticas. Parecería que saltamos de un lado de la colina al otro.



Cuando η tiene un valor apropiado, el algoritmo encontrará el mínimo. Esto significa que el algoritmo **converge**! Nota: A medida que descendemos, el paso se hace más pequeño porque el gradiente disminuye. La regla general es usar la tasa de aprendizaje dentro de 10^{-3} a 10^{-2} el rango.

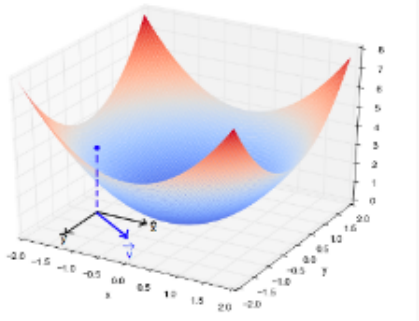
Figura 1: Consideraciones sobre la Regla de la Cadena y la configuración del rango de la tasa de aprendizaje (η).



Mínimos Locales vs Globales

¿Cómo sabemos que encontramos el mínimo global?

Depende de una función de pérdida. Si es una función convexa, lo que significa que tiene un solo mínimo, tenemos la garantía de encontrar el mínimo global.



Algunos modelos, como las regresiones lineal, polinómica y logística, tienen funciones de pérdida convexas. Sin embargo, algunas otras funciones de pérdida no necesariamente serán convexas. Estas funciones pueden tener varios mínimos locales además del mínimo global, como se muestra en el gráfico a continuación.

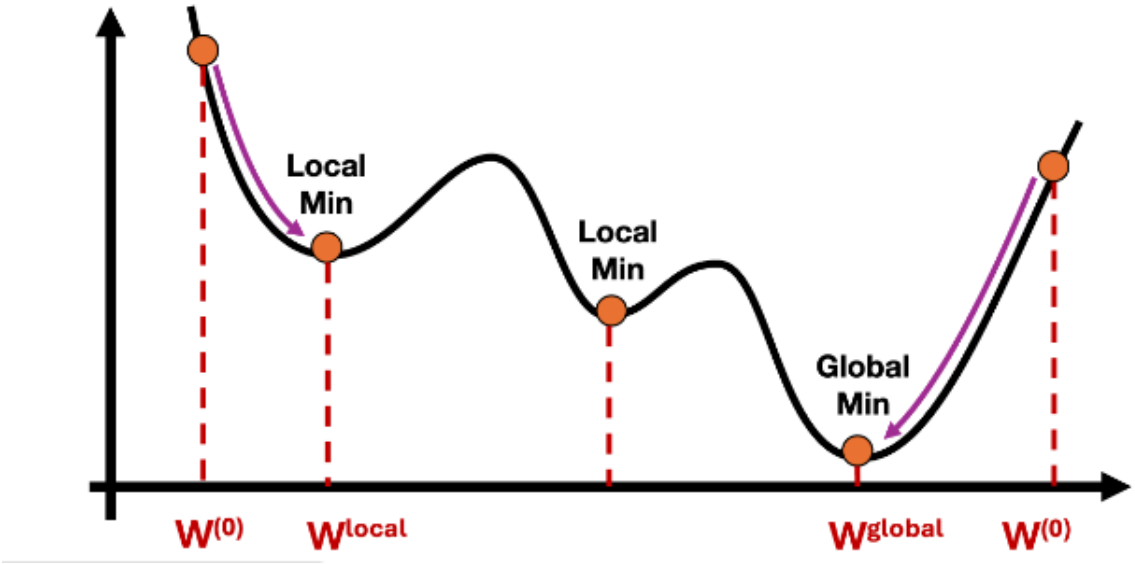


Figura 2: Comparativa gráfica entre funciones de pérdida convexas (mínimo global único) y no convexas (mínimos locales).

A partir de la descripción del descenso del gradiente se podría concluir que puede encontrar un mínimo local y detenerse allí. ¿Cómo lo manejamos? ¿Cómo sabemos si encontramos un mínimo local o no?

Algunas estrategias para abordarlo:

- Ejecuta el algoritmo de descenso del gradiente varias veces desde una ubicación aleatoria y encuentra el mínimo de los mínimos.
- Utiliza el descenso de gradiente estocástico (discutido en cursos posteriores).
- Agrega ruido a la función de pérdida.
- Prueba diferentes tamaños de paso.

Ninguna de las ideas anteriores garantizará encontrar mínimos globales, lamentablemente. El único enfoque que lo garantizará es ejecutar MCMC una cantidad infinita de veces, pero está fuera del alcance de este curso (¡Hablaremos de ello en los próximos cursos, estén atentos!). Por ahora, tenemos que acostumbrarnos a la idea de que el Descenso del Gradiente no garantiza encontrar el mínimo global de una función.

Figura 3: Estrategias de mitigación frente a mínimos

AdaBoost (Adaptive Boosting): Fundamentos, Algoritmo y Optimización

Apuntes Académicos de Clasificación Ensamblada y Descenso de Gradiente con Pérdida Exponencial

---

1. Principios Fundamentales e Ideas Clave de AdaBoost

El algoritmo AdaBoost (Adaptive Boosting) es uno de los métodos de ensamble más influyentes y ampliamente utilizados para tareas de clasificación binaria. A diferencia de las técnicas de promediado independiente como Bagging o Random Forest, AdaBoost construye un modelo complejo de forma secuencial e iterativa, adaptando continuamente el foco de atención hacia los errores cometidos en las etapas previas.

> **Dos Ideas Pilares en AdaBoost:**
> **1. Construcción Aditiva Secuencial:** Combina iterativamente un conjunto de aprendices débiles (weak learners) para generar un clasificador fuerte final T. En árboles de decisión, un aprendiz débil es un árbol de profundidad 1 (1 nodo de corte y 2 hojas), denominado 'tocón' o 'tallo' (stump).
> **2. Aprendizaje de Errores por Reponderación Dinámica:** Cada nuevo tocón agregado al ensamble aprende directamente de los fallos del modelo actual. Esto se logra asignando pesos progresivamente más altos a aquellas observaciones que el modelo actual clasifica de manera errónea.

Esquema Conceptual de Introducción a AdaBoost



Figura 1: Representación conceptual de AdaBoost y la combinación iterativa de tocones (stumps).

Codificación Matemática de Etiquetas

A diferencia de la codificación habitual {0, 1} empleada en regresión logística, AdaBoost requiere definir formalmente las etiquetas de clase como $y_n \in \{-1, +1\}$. Esta convención no es arbitraria; permite simplificar algebraicamente la verificación de clasificaciones correctas e incorrectas:

- **Ventaja Álgebraica:** Permite expresar la coincidencia entre la verdad fundamental y la predicción como el producto $y_n \cdot T(x_n)$.
- **Acierto:** Si la predicción es correcta, y_n y $T(x_n)$ tienen el mismo signo, resultando en un producto positivo (+1).
- **Fallo:** Si la predicción es incorrecta, los signos difieren, produciendo un producto negativo (-1).

2. Desarrollo Algorítmico Paso a Paso con Ejemplo Ilustrativo

Considere un conjunto de datos bidimensional con observaciones pertenecientes a dos clases: Círculos Naranjas (Clase +1) y Triángulos Azules (Clase -1). El proceso paso a paso para construir el ensamble se compone de las siguientes fases:

Paso 1: Ajuste del Tocón Inicial (S_0)

Se entrena un primer tocón de decisión S_0 buscando la regla que maximice la pureza general (mediante el Índice de Gini ponderado). Por ejemplo, al establecer una frontera de decisión en $x_1 = 4.6$, la región derecha contiene 2 círculos naranjas y 4 triángulos azules (predicción azul/clase -1), mientras que la región izquierda queda dominada por círculos naranjas (predicción naranja/clase +1).

Paso 2: Asignación de Pesos Uniformes y Cálculo del Error Ponderado (ε_0)

Inicialmente, cada una de las N observaciones recibe un peso uniforme w_n^{(0)} = 1/N. Para un conjunto de N = 8 puntos, cada punto comienza con peso 1/8. El error total ε_0 del tocón se evalúa sumando los pesos de las observaciones mal clasificadas mediante la función indicadora I(·):

> **Fórmula del Error Ponderado (ε_i):**
> **ε_i = Σ_{n=1}^N w_n^{(i)} · I(S_i(x_n) ≠ y_n)**
> **Donde la función indicadora I takes el valor 1 si la predicción no coincide con la verdad fundamental. En el ejemplo, si existen 2 errores en la región derecha y 0 en la izquierda, ε_0 = 1/8 + 1/8 = 2/8 = 0.25.**

Paso 3 y 4: Factor de Escala (λ_i) y Actualización del Ensamble

Cada tocón contribuye al ensamble global ponderado por un factor de escala o tasa de confianza λ_i. A los tocones con menor tasa de error ε_i se les otorga un valor λ_i más elevado (mayor peso en la decisión final), mientras que a aquellos con peor desempeño se les asigna menor importancia.

La combinación aditiva del modelo ensamble T_i en la iteración i se expresa mediante:

- **Modelo Inicial:** T_0 = λ_0 · S_0 (Ensamble inicial tras el primer tocón)
- **Fórmula General:** T_i = T_{i-1} + λ_i · S_i (Actualización recursiva acumulativa)

Paso 5: Reponderación Dinámica de Muestras y Normalización

Para obligar a que el siguiente tocón S_{i+1} priorice las muestras donde el modelo actual falló, se ajustan los pesos de cada observación de acuerdo con la fórmula de reponderación exponencial:

> **Fórmula de Actualización de Pesos (w_{n, i+1}):**
> **w_{n, i+1} = (1 / Z) · w_{n, i} · exp(- λ_i · y_n · T_{i-1}(x_n))**
> **Donde Z es la constante de normalización calculada como Z = Σ w_{n, i+1}' para garantizar que la suma de los nuevos pesos sea igual a 1 (Σ w_{n, i+1} = 1).**

El comportamiento de esta regla se fundamenta en los signos del exponente:

- **Si la clasificación es correcta:** El producto y_n · T(x_n) > 0, por lo que el peso se multiplica por exp(-λ_i) < 1. Como resultado, el peso de la muestra disminuye.
- **Si la clasificación es incorrecta:** El producto y_n · T(x_n) < 0, por lo que el peso se multiplica por exp(+λ_i) > 1. Como resultado, el peso de la muestra aumenta significativamente.

Estrategias para la Creación del Nuevo Tocón Ponderado

Una vez actualizados los pesos normalizados w_n, existen dos alternativas metodológicas equivalentes para entrenar el siguiente tocón S_{i+1}:

- **Opción A (Remuestreo Bootstrap Ponderado):** Generar un nuevo conjunto de datos mediante muestreo aleatorio con reemplazo de tamaño N, utilizando los pesos w_n como probabilidades de selección. Las muestras mal clasificadas aparecerán duplicadas múltiples veces.
- **Opción B (Índice de Gini Ponderado):** Modificar la función de costo del árbol de decisión para que el cálculo de la impureza de Gini incorpore directamente los pesos w_n de cada observación.

3. Fundamentación Matemática: Pérdida Exponencial y Tasa Óptima

Mientras que en Gradient Boosting para regresión se optimiza la pérdida del error cuadrático medio (MSE), en AdaBoost se minimiza de forma estricta la función de pérdida exponencial:

> **Función de Pérdida Exponencial (L_exp):**
> **L_exp(y, ŷ) = (1 / N) · Σ_{n=1}^N exp(- y_n · ŷ_n)**
> **Donde y_n ∈ {-1, +1} representa la verdad fundamental y ŷ_n corresponde a la predicción emitida por el ensamble acumulado.**

Derivación Analítica del Factor de Escala Óptimo (λ_i)

Una ventaja clave de AdaBoost es que el factor de escala óptimo λ_i no requiere una búsqueda heurística por hiperparámetros, sino que se calcula de manera analítica optimizando la pérdida exponencial con respecto a λ_i:

> **Tasa de Aprendizaje Óptima (λ_i):**
> **λ_i = (1 / 2) · ln((1 - ε_i) / ε_i)**
> **Propiedades numéricas:**
> **• Si ε_i = 0.5 (rendimiento equivalente a un clasificador aleatorio), ln(1) = 0, por lo que λ_i = 0 (el tocón no aporta información y se descarta).**
> **• Si ε_i < 0.5 (buen desempeño), (1 - ε_i)/ε_i > 1, resultando en un λ_i positivo y creciente conforme ε_i tiende a 0.**
> **• Si ε_i > 0.5, λ_i se vuelve negativo, invirtiendo la regla de clasificación del tocón.**

AdaBoost como Descenso del Gradiente

Desde la perspectiva de la optimización en el espacio de predicciones, AdaBoost ejecuta un descenso de gradiente sobre la pérdida exponencial:

- **1. Definición del Gradiente:** El gradiente de la pérdida exponencial con respecto a las predicciones establece que w_n = exp(-y_n · ŷ_n).

- **2. Re-ponderación de Muestras:** Cuando $y_n = \hat{y}_n$ (clasificación correcta), el peso w_n es pequeño. Cuando $y_n \neq \hat{y}_n$, el peso w_n es grande.
- **3. Paso de Actualización:** El paso de actualización en AdaBoost se formula como $\hat{y}_n \leftarrow \hat{y}_n + \lambda \cdot w_n \cdot y_n$, aproximado mediante un modelo simple S_i entrenado sobre los datos reponderados.

4. Matriz Comparativa: Gradient Boosting vs. AdaBoost

A pesar de compartir el principio aditivo de combinación de aprendices débiles, Gradient Boosting y AdaBoost difieren sustancialmente en sus funciones de pérdida y mecanismos de actualización:

Criterio / Componente	Gradient Boosting	AdaBoost
Función de Pérdida Principal	Error Cuadrático Medio (MSE) en Regresión	Pérdida Exponencial $L_{exp} = (1/N) \Sigma \exp(-y \cdot \hat{y})$
Estrategia de Corrección	Ajuste directo de los residuos ($r_n = y_n - \hat{y}_n$)	Reponderación de observaciones según $\exp(-\lambda y \hat{y})$
Tipo de Aprendiz Débil	Árboles de regresión de profundidad fija (ej. max_depth=3)	Tocones / Tallos (Stumps: depth=1, 2 hojas)
Tasa de Aprendizaje (λ)	Constante fija de contracción (ej. $\lambda = 0.1$ o 0.01)	Calculada analíticamente: $\lambda_i = 1/2 \ln((1-\varepsilon_i)/\varepsilon_i)$

5. Criterios de Parada, Hiperparámetros y Librerías Comerciales

Criterios de Terminación y Riesgo de Sobreajuste

A diferencia de Bagging y Random Forest (donde agregar más árboles reduce monótonamente la varianza sin provocar sobreajuste), los métodos de Boosting como AdaBoost se sobreajustarán si se ejecutan durante un número excesivo de iteraciones. Por lo tanto, es indispensable configurar adecuadamente los criterios de detención:

- **Número Máximo de Estimadores (n_estimators):** Límite fijado para el número total de tocones o iteraciones creadas.
- **Mejora Mínima en la Pérdida:** Detención cuando el cambio en la pérdida exponencial cae por debajo de un umbral predefinido.
- **Paciencia / Parada Temprana (Early Stopping):** Número de iteraciones consecutivas sin reducción en el error de validación.

Ecosistema de Librerías Comerciales de Boosting

En el ámbito profesional y en competiciones de ciencia de datos (Kaggle), los algoritmos de boosting son implementados mediante librerías de alto rendimiento altamente optimizadas:

- **XGBoost (Extreme Gradient Boosting):** Implementación extremadamente eficiente del Gradient Boosting basada en divisiones por histogramas y paralelizaciones avanzadas.
- **LightGBM (Light Gradient Boosting Machine):** Desarrollada por Microsoft, compite con XGBoost optimizando la velocidad de entrenamiento y el consumo de memoria mediante el crecimiento de árboles por hojas (leaf-wise).
- **CatBoost:** Librería optimizada que ofrece un tratamiento nativo y automático de características categóricas sin necesidad de One-Hot Encoding previo.

Sección 6.1.1: Algoritmo AdaBoost

Manual de Apuntes Técnicos y Traza de Ejecución Numérica Paso a Paso

---

Índice de Contenidos

(El índice interactivo anterior se actualizará automáticamente al abrir este documento en Microsoft Word o Google Docs).

1. Configuración Inicial del Problema de Clasificación

El algoritmo AdaBoost (Adaptive Boosting) es un método de aprendizaje por ensamble enfocado en construir de manera iterativa un clasificador fuerte T(x) mediante la combinación ponderada de clasificadores débiles S(x). En el ámbito de los árboles de decisión, un aprendiz débil se define de forma estándar como un **tocón de decisión (stump)**, es decir, un árbol de decisión superficial con una única división que posee un solo nodo raíz y dos hojas.

Consideremos un conjunto de datos sintético bidimensional compuesto por N = 8 observaciones con dos características continuas o predictores, x1 y x2. Cada punto de datos pertenece a una de las dos clases objetivo codificadas numéricamente como:

- **Clase 1 (y_n = 1):** Representada gráficamente por círculos de color naranja.
- **Clase 2 (y_n = -1):** Representada gráficamente por triángulos de color azul.

> **NOTA TÉCNICA: CODIFICACIÓN SIMBÓLICA DE CLASES**
>
> La codificación de etiquetas en AdaBoost utiliza $y_n \in \{-1, +1\}$ en lugar del esquema binario tradicional {0, 1}. Esta elección matemática permite simplificar la formulación de los factores de reponderación exponencial y verificar la coincidencia directa de clasificaciones mediante el producto $y_n \cdot T(x_n)$.
>
> Si $y_n \cdot T(x_n) > 0$, la predicción del modelo y la clase real coinciden (clasificación correcta); si $y_n \cdot T(x_n) < 0$, existe un error de clasificación.

6.1.1 Algoritmo AdaBoost

Algoritmo AdaBoost

Considere el siguiente conjunto de datos, con 2 predictores x_1 y x_2, donde todos los puntos de datos se clasifican en la Clase 1 $y_n = 1$ (representado por círculos naranjas) o la Clase 2 $y_n = -1$ (representado por triángulos azules)

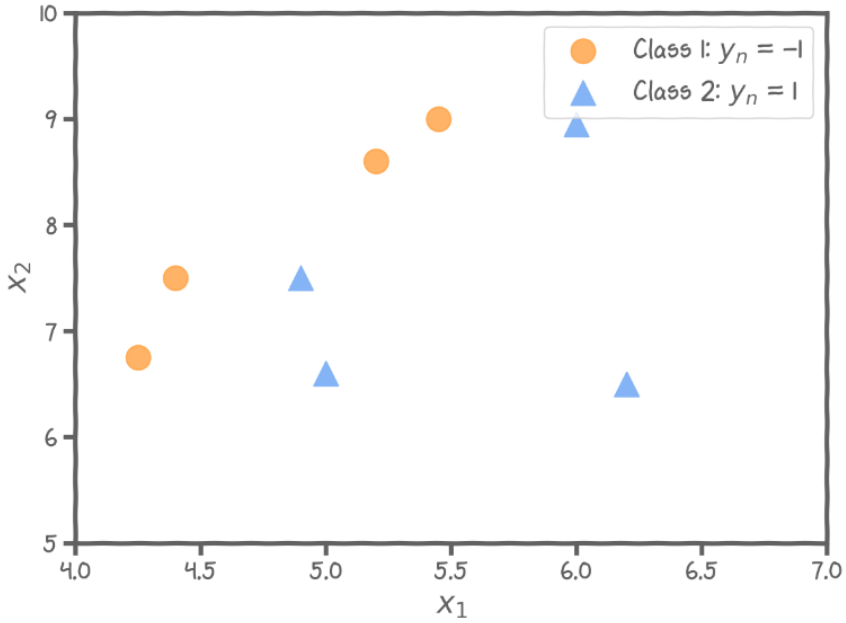


Figura 1: Distribución del conjunto de datos inicial con 8 puntos en el espacio bidimensional (x1 vs x2).

2. Primera Iteración: Ajuste del Tocón Inicial $S^{(o)}$

El proceso algorítmico de AdaBoost comienza seleccionando el primer clasificador débil $S^{(o)}$ y evaluando su desempeño sobre el conjunto de entrenamiento bajo una distribución de pesos inicialmente uniforme.

Paso 1: Ajuste del primer tocón $S^{(o)}$

Se ajusta un tocón de decisión $S^{(o)}$ sobre el espacio de características evaluando las divisiones posibles para maximizar la pureza general. El punto de corte óptimo se establece en la característica x1 = 4.6:

- **Región izquierda (x1 ≤ 4.6):** Contiene 2 círculos naranjas (y = 1) y 0 triángulos azules. La clase mayoritaria es Naranja (y = 1).
- **Región derecha (x1 > 4.6):** Contiene 2 círculos naranjas (y = 1) y 4 triángulos azules (y = -1). La clase mayoritaria es Azul (y = -1).

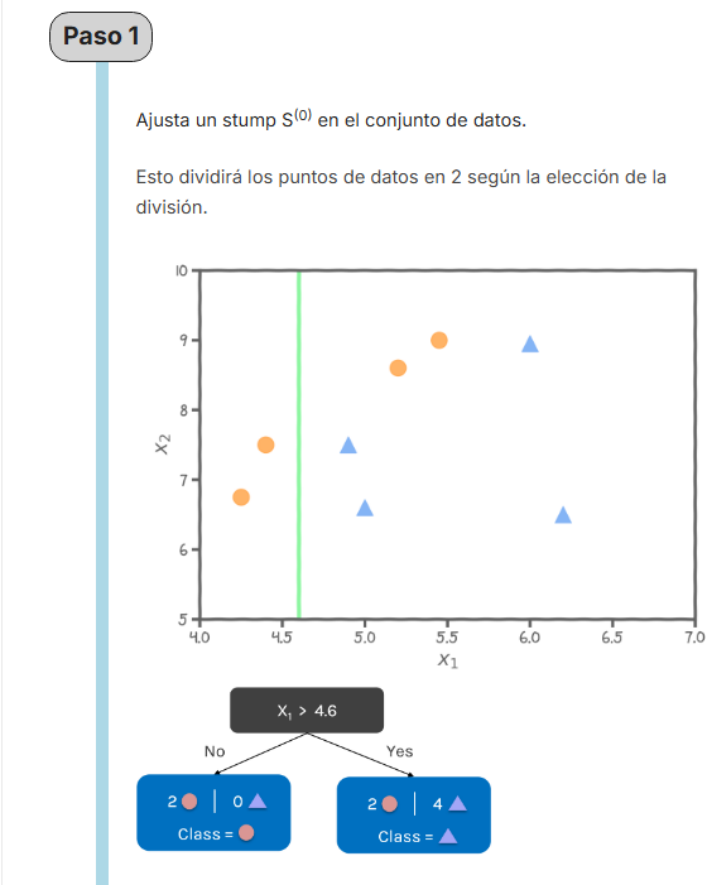


Figura 2: División inicial del espacio de características mediante el tocón $S^{(0)}$ en x1 = 4.6.

Paso 2: Inicialización de pesos de muestras y cálculo del error ε_o

Se asigna inicialmente a cada observación un peso uniforme $w_n^{(0)} = 1/N = 1/8 = 0.125$. A continuación, se calcula el error ponderado total del tocón ε_o evaluando la función indicadora de clasificación incorrecta:

$$\varepsilon_o = \sum_{n=1}^{N} w_n^{(o)} \cdot I(y_n \neq S^{(o)}(x_n))$$

Al clasificar con x1 > 4.6, los 2 círculos naranjas situados a la derecha quedan mal clasificados (predichos como Azul y = -1 cuando su clase real es y = 1). Como cada punto tiene un peso de 1/8, el error total acumulado es:

$$\text{Error}(\varepsilon_o) = 1/8 + 1/8 = 2/8 = 0.25$$

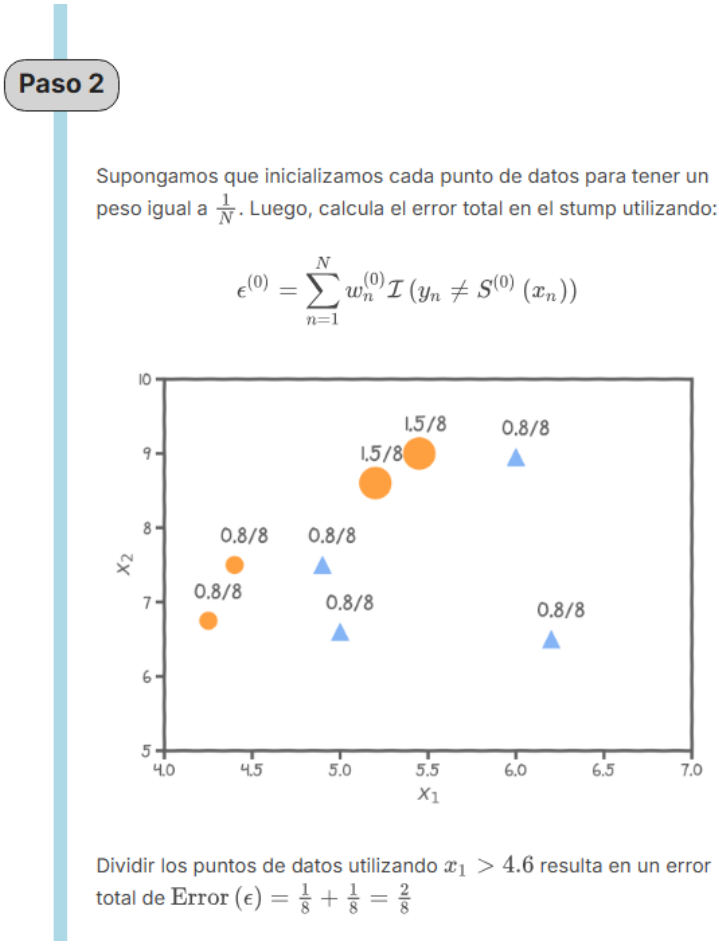


Figura 3: Evaluación de la tasa de error ponderada $\varepsilon_0 = 2/8 = 0.25$ para el tocón $S^{(0)}$.

Paso 3 y Paso 4: Asignación de $\lambda^{(0)}$ y Construcción del Ensamble $T^{(0)}$

Para cuantificar la contribución del primer clasificador débil al modelo global, se asigna un factor de escalado $\lambda^{(0)}$. A aquellos tocones con menor tasa de error se les otorga un coeficiente mayor de importancia. Para esta primera iteración, se fija $\lambda^{(0)} = 0.25$.

El modelo de ensamble acumulado $T^{(i)}(x)$ se define según la ecuación aditiva general:

$$\mathbf{T^{(i)}(x) = \lambda^{(0)}\, S^{(0)}(x) \;\; (para\ i = 0), \;\; T^{(i-1)}(x) + \lambda^{(i)}\, S^{(i)}(x) \;\; (para\ i \geq 1)}$$

Sustituyendo los valores iniciales para i = 0, obtenemos la primera etapa del modelo combinado: $\mathbf{T^{(0)}(x) = 0.25 \cdot S^{(0)}(x)}$.

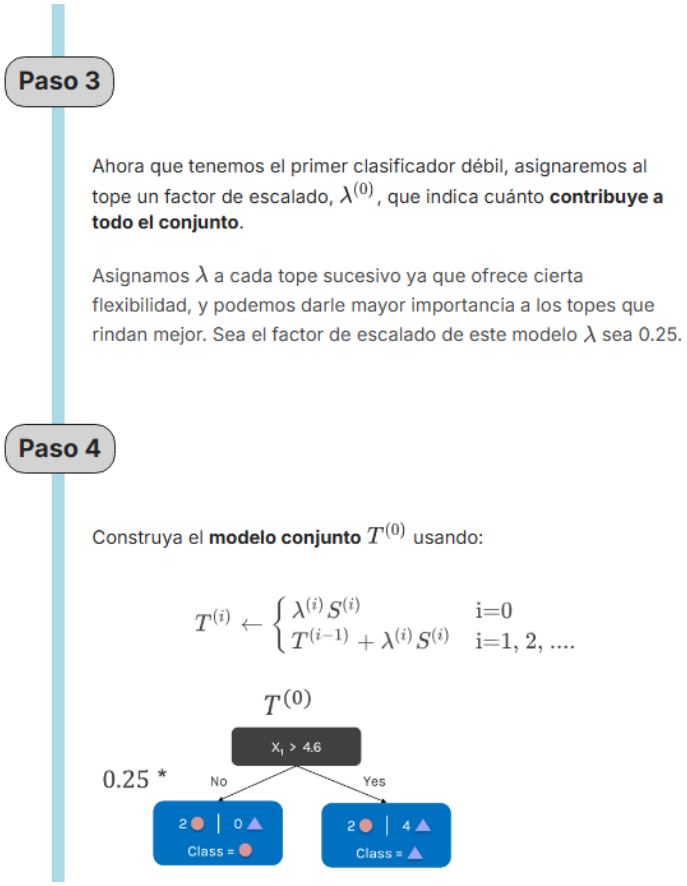


Figura 4: Construcción del modelo combinado $T^{(o)}$ escalado por $\lambda^{(o)} = 0.25$.

3. Segunda Iteración: Reponderación de Muestras y Ajuste de $S^{(1)}$

Paso 5: Ajuste exponencial y normalización de pesos $w_n^{(1)}$

Para garantizar que el próximo clasificador débil se enfoque prioritariamente en las observaciones mal clasificadas por el modelo actual $T^{(o)}$, se actualiza la distribución de pesos $w_n^{(1)}$ mediante la función exponencial de reponderación:

$$w_n^{(1)} = [\ w_n^{(o)} \cdot \exp(\ -\lambda^{(o)} \cdot y_n \cdot T^{(o)}(x_n)\ )\ ] / Z$$

Donde Z representa la constante de normalización requerida para que la suma total de los pesos se mantenga igual a 1 (∑_{n=1}^N $w_n^{(1)}$ = 1). La dinámica de esta actualización se desglosa así:

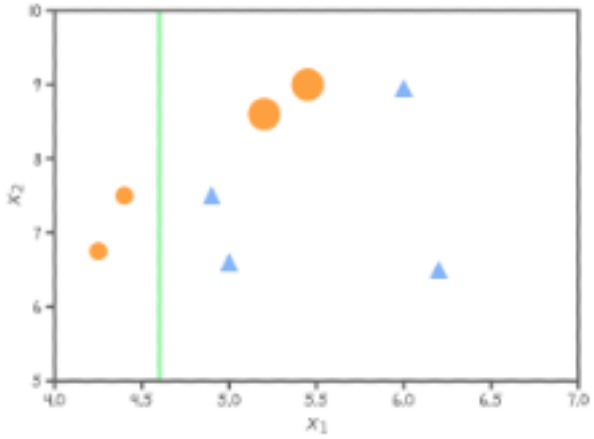
- **Muestras clasificadas correctamente ($y_n \cdot T^{(o)}(x_n) > 0$):** El exponente es negativo, lo que reduce el peso individual de estas 6 observaciones de 1/8 a aproximadamente 0.8/8.
- **Muestras mal clasificadas ($y_n \cdot T^{(o)}(x_n) < 0$):** El exponente se vuelve positivo, incrementando el peso individual de las 2 muestras erróneas de 1/8 a 1.5/8.

Paso 5

Ajuste los pesos asignados a cada punto de datos para asegurar que el siguiente tope se enfoque en los puntos **mal clasificados por el conjunto** utilizando la siguiente ecuación:

$$w_n^{(1)} \leftarrow \frac{w_n^{(0)} e^{-\alpha_1 y_n T^{(0)}(x_n)}}{z} = \frac{w_n^{(1')}}{z}$$

Esto incrementa los pesos de las muestras mal clasificadas y disminuye los pesos de las muestras correctamente clasificadas.



A partir de la ecuación anterior, tenemos $w_n^{(i)}$ como los nuevos pesos que se calculan a partir de los pesos antiguos, $w_n^{(0)} \cdot y_n$ es la verdad de referencia que es ya sea -1 o 1. La predicción del conjunto anterior se representa por $T^{(0)}(x_n)$.

El nuevo peso se obtiene después de la normalización por un factor z.

Así, para nuestro ejemplo tenemos;

$$w^{(1)} \leftarrow \frac{w^{(1')}}{z} = \frac{\frac{1.3}{8}}{\frac{1.3}{8} \cdot 2 + \frac{0.7}{8} \cdot 6} \approx \frac{1.5}{8}$$

$$w^{(1)} \leftarrow \frac{w^{(1')}}{z} = \frac{\frac{0.7}{8}}{\frac{1.3}{8} \cdot 2 + \frac{0.7}{8} \cdot 6} \approx \frac{0.8}{8}$$

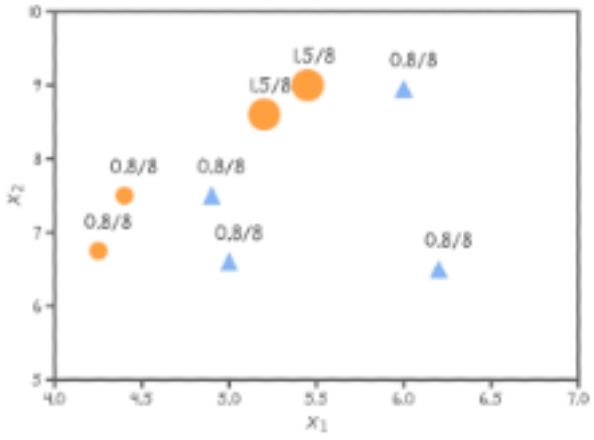


Figura 5: Reponderación de muestras y normalización por Z para priorizar errores previos.

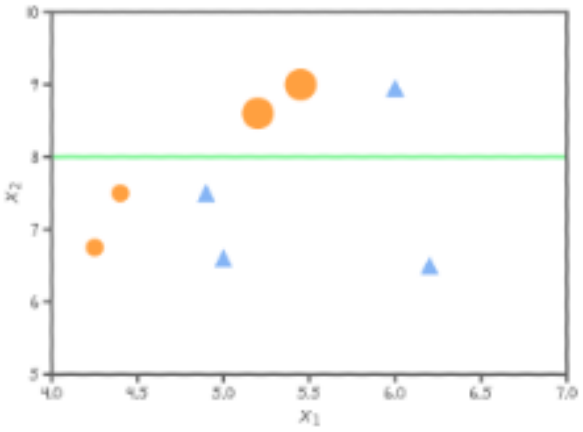
Paso 6: Entrenamiento del segundo tocón $S^{(1)}$ y cálculo de ε_1

Se entrena el segundo tocón de decisión $S^{(1)}$ sobre la distribución de datos reponderada. La nueva regla de división óptima se sitúa en la característica x2 = 8.0. Es crucial observar que el nuevo clasificador débil logra clasificar correctamente los dos puntos de datos (círculos naranjas) que el modelo $S^{(0)}$ había fallado previamente.

Evaluando la tasa de error ponderado sobre la nueva distribución de pesos $w_n^{(1)}$, se obtiene un error total de **Error(ε_1) = 0.20**. Con base en este rendimiento, se asigna al segundo tocón un factor de importancia de **$\lambda^{(1)}$ = 0.20**.

Paso 6

Crea otro árbol débil $S^{[1]}$ en los datos con nuevos pesos.

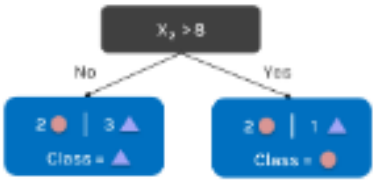


Es importante notar que el nuevo árbol débil ha clasificado correctamente los puntos de datos que el anterior clasificó erróneamente.

Paso 7

Con los nuevos pesos, calcula el error total del árbol mediante:

$$\epsilon^{(1)} = \sum_{n=1}^{N} w_n^{(1)} \mathcal{I}\left(y_n \neq S^{(1)}\left(x_n\right)\right)$$



Dividiendo los puntos de datos usando $x_2 > 8$ resulta en un error total de $\text{Error}\left(\epsilon\right) = \frac{0.6}{8} + \frac{0.6}{8} + \frac{0.6}{8} = 0.3$

Paso 8

Asigna al árbol débil una escala, $\lambda^{(1)}$, que indica cuánto **contribuye al conjunto completo.**

Sea este peso del modelo $\lambda^{(1)}$ ser 0.20.

Figura 6: Entrenamiento de $S^{(1)}$ con corte en x2 = 8.0 y cálculo del error ponderado $\varepsilon_1 = 0.20$.

4. Integración del Ensamble Secuencial y Tercera Iteración

Paso 7 y Paso 8: Combinación aditiva del modelo $T^{(2)}$

Incorporate el segundo tocón al modelo acumulado estableciendo $T^{(1)}(x) = T^{(0)}(x) + \lambda^{(1)} S^{(1)}(x) = 0.25\, S^{(0)}(x) + 0.20\, S^{(1)}(x)$.

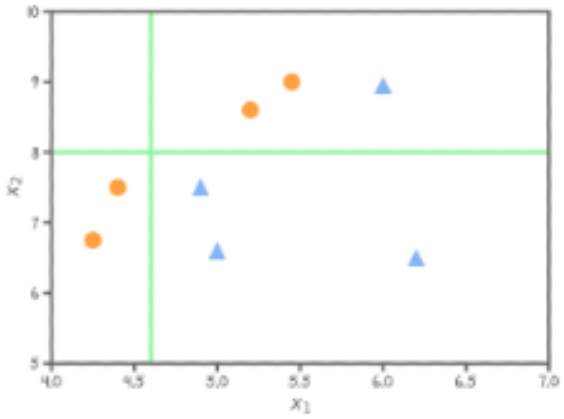
Repitiendo exactamente la misma secuencia operacional de cálculo de errores, reponderación exponencial de muestras y ajuste de un tercer tocón $S^{(2)}$ con factor de importancia asignado de $\lambda^{(2)} = 0.42$, construimos el modelo de ensamble de tres etapas:

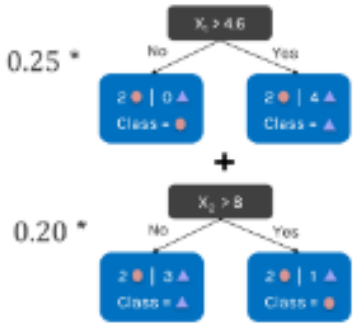
$$\mathbf{T^{(2)}(x) = 0.25 \cdot S^{(0)}(x) + 0.20 \cdot S^{(1)}(x) + 0.42 \cdot S^{(2)}(x)}$$

Paso 9

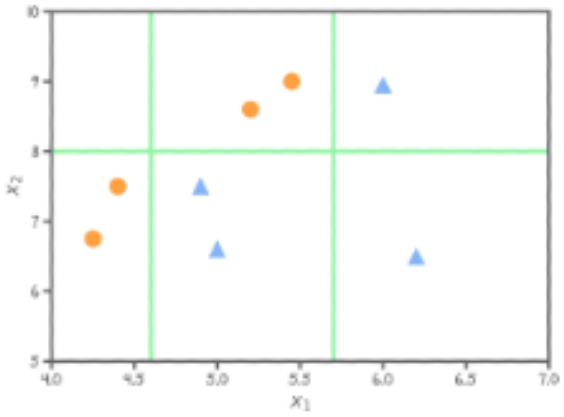
Construir el **modelo de conjunto** $T^{(1)}$ usando:

$$T^{(i)} \leftarrow \begin{cases} \lambda^{(i)} S^{(i)} & \text{i=0} \\ T^{(i-1)} + \lambda^{(i)} S^{(i)} & \text{i=1, 2, ...} \end{cases}$$





Repitiendo el mismo proceso nuevamente, obtenemos:



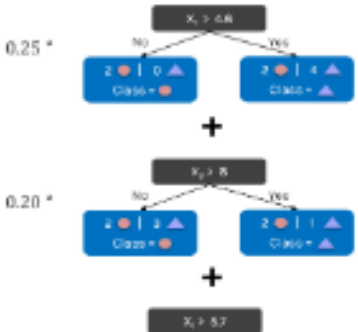


Figura 7: Combinación aditiva final del ensamble $T^{(2)}$ compuesto por tres tocones ponderados.

REFLEXIÓN Y ESCALABILIDAD EN ADABOOST

En este conjunto pedagógico de solo 8 puntos, extender el algoritmo hasta cientos de iteraciones no es necesario. Sin embargo, en problemas reales con miles de observaciones y regiones complejas no lineales, la acumulación aditiva de múltiples tocones de decisión simples permite construir fronteras de decisión altamente expresivas y precisas sin incurrir en el sesgo típico de modelos no flexibles.

A diferencia de Bagging o Random Forest, las iteraciones consecutivas en AdaBoost dependen directamente de los errores cometidos en los pasos anteriores.

5. Matriz Resumen de Pasos y Ecuaciones del Algoritmo AdaBoost

La siguiente tabla condensa la secuencia lógica, las fórmulas matemáticas aplicadas y los valores concretos calculados a lo largo de las iteraciones del algoritmo:

Paso	Fase Algorítmica	Ecuación / Expresión	Valor en el Ejemplo
Paso 1	Ajuste del Tocón Inicial	Corte en feature óptimo x1	División x1 = 4.6
Paso 2	Error Ponderado ε_o	$\varepsilon = \sum w_n\, I(y_n \neq S(x_n))$	$\varepsilon_o = 2/8 = 0.25$
Paso 3	Factor de Escala $\lambda^{(o)}$	Importancia del clasificador débil	$\lambda^{(o)} = 0.25$
Paso 4	Ensamble Acumulado $T^{(o)}$	$T^{(o)}(x) = \lambda^{(o)}\, S^{(o)}(x)$	$T^{(o)} = 0.25\, S^{(o)}$
Paso 5	Reponderación Exponencial	$w_n^{(1)} = [w_n^{(o)} \exp(-\lambda\, y_n\, T(x_n))] / Z$	Errores: 1/8 → 1.5/8
Paso 6	Entrenamiento del Tocón $S^{(1)}$	Corte en feature x2 con pesos $w_n^{(1)}$	x2 = 8.0, $\varepsilon_1 = 0.20$
Paso 7	Actualización del Ensamble $T^{(1)}$	$T^{(1)} = T^{(o)} + \lambda^{(1)}\, S^{(1)}$	$\lambda^{(1)} = 0.20$
Paso 8	Etapa $T^{(2)}$ y Modelo Final	$T^{(i)} = T^{(i-1)} + \lambda^{(i)}\, S^{(i)}$	$0.25\, S^{(o)} + 0.20\, S^{(1)} + 0.42\, S^{(2)}$

Capítulo 6: Algoritmos de Boosting para Clasificación

Sección 6.1.2 Resumen del Algoritmo AdaBoost y Formulación Matemática Completa

---

Índice de Contenidos

(El índice de contenidos anterior se actualizará automáticamente al abrir el documento en Microsoft Word o Google Docs).

1. Resumen Algorítmico General de AdaBoost

El algoritmo de Boosting Adaptativo (AdaBoost) construye de manera secuencial un clasificador fuerte $T^{(\text{final})}(x)$ mediante la combinación aditiva ponderada de múltiples aprendices débiles $S^{(i)}(x)$. A diferencia de los métodos de ensamblado paralelo como Bagging o Random Forest, AdaBoost ajusta adaptativamente la importancia de cada observación en la muestra de entrenamiento en cada iteración, concentrando el esfuerzo computacional del siguiente aprendiz débil en aquellas observaciones que fueron clasificadas de forma errónea por el ensamble acumulado en la etapa anterior.

Dado un conjunto de datos de entrenamiento compuesto por N observaciones numeradas $n = 1, 2, \dots, N$, expresado formalmente como $\{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$, donde x_n representa el vector de características predictoras y $y_n \in \{-1, +1\}$ es la etiqueta de clase binaria real, la secuencia operativa paso a paso se estructura de la siguiente manera:

- **Paso 0 Inicialización de la Distribución de Pesos:** Se asigna inicialmente un peso idéntico e uniforme a cada observación del conjunto de entrenamiento: $w_n^{(0)} = \frac{1}{N}$ para $n = 1, \dots, N$, garantizando una ponderación equitativa en el punto de partida.
- **Paso 1 Entrenamiento del Aprendiz Débil $S^{(i)}$:** Para cada iteración $i = 0, 1, \dots, T$, se entrena un clasificador débil $S^{(i)}$ (comúnmente un árbol de decisión de un solo nivel o tocón/*stump*) utilizando la distribución de pesos actual $w_n^{(i)}$.
- **Paso 2 Cálculo del Error Ponderado Total ϵ_i:** Se evalúa la tasa de error ponderada del aprendiz débil $S^{(i)}$ sumando los pesos de las muestras clasificadas incorrectamente mediante la función indicadora $\mathbb{I}$:
  $$\epsilon_i = \sum_{n=1}^N w_n^{(i)} \cdot \mathbb{I}\left(y_n \neq S^{(i)}(x_n)\right)$$

- **Paso 3 Cálculo de la Importancia del Modelo $\lambda^{(i)}$:** Se determina el factor de escalado de confianza o tasa de aprendizaje individual del tocón $\lambda^{(i)}$. A los clasificadores con un error ponderado menor se les otorga una mayor ponderación de voto en el ensamble.
- **Paso 4 Actualización del Modelo de Ensamble Acumulado $T^{(i)}$:** Se construye o expande el ensamble agregando aditivamente la contribución del nuevo clasificador débil escalado: $$T^{(i)}(x) = \begin{cases} \lambda^{(0)} S^{(0)}(x) & \text{para } i = 0 \\ T^{(i-1)}(x) + \lambda^{(i)} S^{(i)}(x) & \text{para } i = 1, 2, \dots \end{cases}$$
- **Paso 5 Reequilibrio Exponencial de Pesos de Muestra $w_n^{(i+1)}$:** Se recalculan los pesos de las observaciones para la siguiente iteración $i+1$, incrementando el peso relativo de las muestras mal clasificadas y reduciendo el de las correctas: $$w_n^{(i+1)} = \frac{w_n^{(i)} \cdot \exp\left(-\lambda^{(i)} \cdot y_n \cdot S^{(i)}(x_n)\right)}{Z}$$ donde Z es la constante de normalización requerida para asegurar que la suma de los nuevos pesos sea igual a 1 ($\sum_{n=1}^N w_n^{(i+1)} = 1$).
- **Paso 6 Evaluación del Clasificador Final Ensamble $T^{(\text{final})}$:** Al alcanzar la condición de parada (por ejemplo, completar $T+1$ iteraciones o lograr un error nulo), la predicción de clase binaria para una nueva muestra x se emite aplicando la función signo a la combinación lineal final: $$T^{(\text{final})}(x) = \text{signo}\left(\sum_{i=0}^T \lambda^{(i)} S^{(i)}(x)\right)$$

> **MECANISMO DE VOTO PONDERADO EN ADABOOST:** En la fórmula del paso 6, el signo positivo de la suma ponderada asigna la predicción a la Clase 1 (+1), mientras que un signo negativo la asigna a la Clase 2 (-1).
>
> Si un aprendiz débil $S^{(i)}$ obtiene un desempeño aleatorio (error $\epsilon_i = 0.5$), su coeficiente de confianza $\lambda^{(i)}$ se anula a 0, neutralizando su impacto en la decisión del ensamble.

2. Preguntas Frecuentes y Aspectos Prácticos de Implementación

En la práctica de la Ciencia de Datos, la aplicación de AdaBoost plantea interrogantes clave en cuanto a la parametrización de los clasificadores base y el tratamiento computacional de los pesos de las muestras. A continuación se sintetizan las tres consultas técnicas fundamentales abordadas en el curso:

2.1 Creación de Árboles de Decisión de Un Nivel (Tocones / Stumps)

Un aprendiz débil en AdaBoost suele ser un árbol de decisión extremadamente simple que consta de un único nodo de decisión (raíz) y dos hojas terminales. En librerías de aprendizaje automático como Scikit-Learn, estos tocones se instancian configurando explícitamente el hiperparámetro de profundidad máxima a **max_depth = 1**. Esta restricción estructural garantiza que cada modelo base posea un alto sesgo individual pero una varianza muy baja, actuando como una regla empírica simple (*rule of thumb*).

2.2 Integración de Pesos Normalizados en el Entrenamiento del Tocón

Existen dos métodos metodológicos equivalentes para entrenar un árbol de decisión de un solo nivel utilizando la distribución de pesos de muestras $w_n^{(i)}$:

- **Opción A (Re-muestreo Bootstrap Ponderado):** Generar un nuevo conjunto de datos de entrenamiento del mismo tamaño N que el original mediante muestreo aleatorio con reemplazo, donde la probabilidad de selección de cada punto es exactamente proporcional a su peso recién actualizado $w_n^{(i+1)}$. Las observaciones clasificadas incorrectamente previamente aparecerán múltiples veces duplicadas en la nueva muestra, forzando al tocón a priorizar su corrección.
- **Opción B (Impureza Gini / Entropía Ponderada):** Utilizar directamente el dataset original pasando el vector de pesos $w_n^{(i+1)}$ al algoritmo de división. En este caso, el cálculo del índice de impureza de Gini o entropía en cada posible división de características escala la contribución de cada observación según su peso w_n, evaluando divisiones que maximicen la pureza ponderada.

2.3 Elección de la Pérdida Exponencial frente al Error Cuadrático Medio (MSE)

A diferencia del Gradient Boosting para regresión (que minimiza la pérdida MSE del residuo continuo), AdaBoost está diseñado para problemas de clasificación binaria y optimiza una función objetivo denominada **Pérdida Exponencial**:

$$\text{Pérdida Exponencial} = \frac{1}{N} \sum_{n=1}^N \exp\left(-y_n \hat{y}_n\right)$$

Donde $y_n \in \{-1, +1\}$ representa la clase real y $\hat{y}_n = T(x_n)$ es la predicción continua emitida por el ensamble. La pérdida exponencial es una función suave, estrictamente convexa y diferenciable que actúa como una cota superior acotada (*upper bound*) de la pérdida de clasificación 0-1 no diferenciable, penalizando fuertemente los errores de margen ($y_n \hat{y}_n < 0$).

3. Deducción Analítica de la Tasa de Aprendizaje Óptima ($\lambda^{(i)}$)

Una de las propiedades matemáticas más destacables de AdaBoost es que, a diferencia de Gradient Boosting (donde la tasa de aprendizaje λ es un hiperparámetro fijo o ajustado mediante validación cruzada), en AdaBoost la tasa de aprendizaje o factor de escala óptimo $\lambda^{(i)}$ para cada tocón se puede calcular **de forma analítica y exacta** en cada iteración.

Al agregar el i-ésimo aprendiz débil $S^{(i)}(x)$ con factor de escala λ, la pérdida exponencial del nuevo ensamble acumulado $T^{(i)}(x) = T^{(i-1)}(x) + \lambda S^{(i)}(x)$ se minimiza resolviendo la siguiente ecuación de optimización:

$$\lambda^{(i)} = \arg\min_{\lambda} \frac{1}{N} \sum_{n=1}^N \exp\left(-y_n \left[T^{(i-1)}(x_n) + \lambda S^{(i)}(x_n)\right]\right)$$

Diferenciando esta expresión con respecto a λ e igualando la derivada a cero, se obtiene la fórmula analítica en función del error ponderado del tocón ϵ_i:

$$\lambda^{(i)} = \frac{1}{2} \ln\left(\frac{1 - \epsilon_i}{\epsilon_i}\right)$$

Donde $\epsilon_i = \sum_{n=1}^N w_n^{(i)} \cdot \mathbb{I}(y_n \neq S^{(i)}(x_n))$ representa la tasa de error ponderada en la iteración i. El análisis cualitativo de esta fórmula revela tres comportamientos clave:

- **Si $\epsilon_i \to 0$ (clasificador casi perfecto):** El cociente $\frac{1 - \epsilon_i}{\epsilon_i} \to \infty$, provocando que $\lambda^{(i)} \to +\infty$. Al tocón se le asigna un peso de voto masivo en el ensamble.
- **Si $\epsilon_i = 0.5$ (clasificador equivalente al azar):** El cociente $\frac{0.5}{0.5} = 1$, por lo que $\ln(1) = 0$ y $\lambda^{(i)} = 0$. El modelo no aporta información útil y su voto queda completamente anulado.
- **Si $\epsilon_i > 0.5$ (desempeño peor que el azar):** El cociente es menor que 1, produciendo un valor negativo $\lambda^{(i)} < 0$. AdaBoost invierte automáticamente las predicciones del tocón para aprovechar su error sistemático.

4. Interpretación de AdaBoost como Descenso de Gradiente

Existe una profunda conexión teórica entre el algoritmo AdaBoost y el marco general de Descenso de Gradiente en el Espacio de Funciones/Predicciones. Evaluar AdaBoost desde esta perspectiva permite justificar rigurosamente por qué la regla de reponderación exponencial coincide exactamente con el vector gradiente negativo de la función de pérdida.

Calculando la derivada parcial de la pérdida exponencial $\mathcal{L}_{\text{exp}} = \frac{1}{N} \sum_{n=1}^N \exp(-y_n \hat{y}_n)$ con respecto a las predicciones continuas del ensamble $\hat{y} = [\hat{y}_1, \hat{y}_2, \dots, \hat{y}_N]$, obtenemos el vector gradiente:

$$\nabla_{\hat{y}} \mathcal{L}_{\text{exp}} = \left[-y_1 e^{-y_1 \hat{y}_1}, -y_2 e^{-y_2 \hat{y}_2}, \dots, -y_N e^{-y_N \hat{y}_N} \right]$$

Definiendo el peso no normalizado de la muestra n como $w_n = \exp(-y_n \hat{y}_n)$, observamos las siguientes propiedades:

- **Cuando $y_n = \hat{y}_n$ (clasificación correcta):** El producto $y_n \hat{y}_n > 0$, de modo que $w_n = e^{-\text{positivo}}$ es un número pequeño.
- **Cuando $y_n \neq \hat{y}_n$ (clasificación errónea):** El producto $y_n \hat{y}_n < 0$, de modo que $w_n = e^{+\text{positivo}}$ crece exponencialmente a un valor grande.

Sustituyendo esta definición, el vector gradiente de la pérdida exponencial se simplifica directamente a:

$$\nabla_{\hat{y}} \mathcal{L}_{\text{exp}} = \left[-y_1 w_1, -y_2 w_2, \dots, -y_N w_N \right]$$

El paso de actualización estándar en el descenso de gradiente para ajustar las predicciones adopta la forma:

$$\hat{y}_n \leftarrow \hat{y}_n + \lambda w_n y_n, \quad n = 1, \dots, N$$

Al igual que en Gradient Boosting para regresión, en AdaBoost no es posible tomar un paso directo en la dirección exacta del gradiente negativo sin un mapa funcional predictor. Por lo tanto, se aproxima el término de gradiente reponderado $\lambda w_n y_n$ mediante un aprendiz débil $S^{(i)}(x_n)$, entrenándolo sobre el conjunto objetivo reponderado $\{(x_1, w_1 y_1), \dots, (x_N, w_N y_N)\}$. En conclusión, **el descenso de gradiente con pérdida exponencial es matemáticamente equivalente a**

entrenar iterativamente aprendices débiles enfocados en las observaciones clasificadas incorrectamente por el modelo anterior.

5. Tabla Comparativa de Fórmulas y Componentes de AdaBoost

La siguiente tabla resume los componentes matemáticos, ecuaciones y roles funcionales del algoritmo AdaBoost descritos en la lección:

Componente / Variable	Expresión Matemática	Significado y Rol Práctico
Distribución Inicial de Pesos	$w_n^{(0)} = \frac{1}{N}$	Ponderación equitativa inicial para todas las N muestras de entrenamiento.
Error Ponderado del Tocón	$\epsilon_i = \sum_{n=1}^N w_n^{(i)} \mathbb{I}(y_n \neq S^{(i)}(x_n))$	Suma de pesos de las observaciones clasificadas de forma errónea por $S^{(i)}$.
Factor de Escala / Tasa de Aprendizaje	$\lambda^{(i)} = \frac{1}{2} \ln\left(\frac{1 - \epsilon_i}{\epsilon_i}\right)$	Importancia analítica asignada al voto del tocón $S^{(i)}$ en el ensamble final.
Actualización Exponencial de Pesos	$w_n^{(i+1)} = \frac{w_n^{(i)} \exp\left(-\lambda^{(i)} y_n S^{(i)}(x_n)\right)}{Z}$	Aumenta el peso de errores y reduce el de aciertos para la iteración $i+1$.
Función de Pérdida Exponencial	$\mathcal{L}_{\text{exp}} = \frac{1}{N} \sum_{n=1}^N \exp\left(-y_n \hat{y}_n\right)$	Función objetivo convexa y diferenciable acotadora del error de clasificación.
Modelo de Ensamble Final	$T^{(\text{final})}(x) = \text{signo}\left(\sum_{i=0}^T \lambda^{(i)} S^{(i)}(x)\right)$	Clasificación binaria final emitida por combinación lineal ponderada de tocones.

6. Galería de Ilustraciones y Diagramas Originales

A continuación se incluyen las seis capturas de pantalla originales de la lección para referencia gráfica directa del contenido expuesto:

6.1.2 Resumen - Algoritmo AdaBoost

Veamos esto de manera más general. Los pasos del algoritmo AdaBoost son:

Paso 1

Dado el conjunto de datos de entrenamiento $(x_1, y_1), \ldots, (x_N, y_N)$, elija una distribución inicial $w_n^{(0)} = \frac{1}{N}$,
Para i = 0 . . . hasta que se cumpla la condición de parada.

Paso 2

Entrenar un aprendiz débil $S^{(i)}$ utilizando los pesos $w_n^{(i)}$

Paso 3

Calcular el error total del aprendiz débil utilizando:

$$\epsilon^{(i)} = \sum_{n=1}^{N} w_n^{(i)} \mathcal{I}\left(y_n \neq S^{(i)}\left(x_n\right)\right)$$

Figura 6.1: Pasos iniciales del Algoritmo AdaBoost (Paso 0 a Paso 2).

Paso 4

Calcular la importancia de cada modelo $\lambda^{(i)}$

Paso 5

Construir el modelo conjunto utilizando:

$$T^{(i)} \leftarrow \begin{cases} \lambda^{(i)} S^{(i)} & \text{i=0} \\ T^{(i-1)} + \lambda^{(i)} S^{(i)} & \text{i=1, 2, ...} \end{cases}$$

Paso 6

Ajustar los pesos asignados a cada punto de datos para asegurar que el siguiente clasificador se enfoque en los puntos mal clasificados por el clasificador anterior

$$w_n^{(i+1)} \leftarrow \frac{w_n^{(i)} e^{-\lambda^{(i)} y_n S^{(i)}(x_n)}}{Z}$$

Figura 6.2: Construcción aditiva del ensamble y fórmula de actualización de pesos (Paso 3 a Paso 5).

Modelo Final

$$T^{(i)}(x) = \text{signo}\left[\sum_{i=1}^{T} \lambda^{(i)} S^{(i)} x\right]$$

Figura 6.3: Formulación del Clasificador Ensamble Final mediante la función signo.

Preguntas comunes sobre AdaBoost:

¿Cómo creo un árbol de decisión de un nivel?

En AdaBoost, los árboles de decisión de un nivel se crean usando un árbol de decisión simple con max_depth = 1

¿Cómo utilizar los pesos normalizados para crear un árbol de decisión de un nivel?

Crea un nuevo conjunto de datos del mismo tamaño que el conjunto de datos original con repeticiones basadas en el peso de muestra recién actualizado.

¿Cómo calculo el factor de escala $\lambda^{(i)}$ de cada árbol de decisión de un nivel?

En el gradient boosting para regresión, minimizamos la pérdida de MSE. En AdaBoost, la función que elegimos se llama **pérdida exponencial:**

$$\text{Pérdida Exp} = \frac{1}{N}\sum_{n=1}^{N} e^{(-y_n \hat{y_n})}$$

$$\text{donde } y_n \in -1, 1$$

La pérdida exponencial es diferenciable con respecto a $\hat{y_n}$ y es una cota superior del error.

Figura 6.4: Preguntas frecuentes sobre AdaBoost, tocones de decisión y pérdida exponencial.

Elegir la tasa de aprendizaje

A diferencia del caso del gradient boosting para regresión, podemos resolver analíticamente la tasa de aprendizaje óptima para AdaBoost, optimizando:

$$\underset{\lambda}{\text{argmin}} \frac{1}{N}\sum_{n=1}^{N} e^{(-y_n(T+\lambda^{(i)}S^{(i)}(x_n)))}$$

Haciendo esto, obtenemos:

$$\lambda^{(i)} = \frac{1}{2} ln\left(\frac{1-\epsilon}{\epsilon}\right)$$

donde

$$\epsilon = \sum_{n=1}^{N} w_n^{(i)} \mathcal{I}\left(y_n \neq T^{(i)}\left(x_n\right)\right)$$

$$w_n^{(i+1)} \leftarrow \frac{w_n^{(i)} e^{-\lambda^{(i)}}}{Z}$$

Formulación Matemática - AdaBoost

Descenso del Gradiente con Pérdida Exponencial

La pérdida exponencial de AdaBoost se da por;

$$\text{Pérdida Exponencial} = \frac{1}{N}\sum_{n=1}^{N} e^{(-y_n \hat{y_n})}$$

Figura 6.5: Optimización analítica de la Tasa de Aprendizaje y Pérdida Exponencial.

Similar a lo que hicimos para el gradient boosting, calcula el gradiente de la pérdida con respecto a las predicciones:

\[\nabla_{\hat{y}} \text{Pérdida Exponencial} = \left[-y_1e^{(-y_1\hat{y_1}), ... , -y_ne^{(-y_n \hat{y_n}) } \right]
\]

Establecer $w_n = exp\left(-y_n \hat{y_n}\right)$

Nótese que cuando $y_n = \hat{y_n}$, el peso w_n es pequeño; cuando $y_n \neq \hat{y_n}$, el peso es mayor.

$$\nabla_{\hat{y}} \text{Pérdida Exponencial} = \left[-y_1 w_1, \ldots, -y_n w_n\right]$$

De esta manera, vemos que el gradiente es simplemente una reponderación aplicada a los valores objetivo.

El paso de actualización en el descenso del gradiente es:

$$\hat{y_n} \leftarrow \hat{y_n} + \lambda w_n y_n, n = 1, \ldots, N$$

Al igual que en el gradient boosting, aproximamos el gradiente, $\lambda w_n y_n$, con un modelo simple, $S^{(i)}$, que depende de x_n.

Esto significa entrenar $S^{(i)}$ on un conjunto de valores objetivo reponderados:

$$\left(x_1, w_1 y_1\right), \ldots, \left(x_N, w_N y_N\right)$$

Es decir, el descenso de gradiente con pérdida exponencial significa entrenar iterativamente modelos simples que **se enfoca en los puntos mal clasificados por el modelo previo.**

Figura 6.6: Formulación de AdaBoost como Descenso de Gradiente con Pérdida Exponencial.

APUNTES DE MACHINE LEARNING — CAPÍTULO 6: ALGORITMOS DE BOOSTING

Sección 6.1.3 Comparación: Gradient Boosting y AdaBoost

Índice de Contenidos

(El índice de contenidos anterior se actualizará automáticamente al abrir el documento en Microsoft Word o Google Docs).

1. Visión General de la Comparativa

Gradient Boosting y AdaBoost constituyen las dos arquitecturas fundamentales dentro de la familia de algoritmos de Boosting en aprendizaje automático. Aunque ambos métodos comparten la filosofía central de construir iterativamente un ensamble fuerte mediante la combinación aditiva de aprendices débiles (weak learners), difieren de manera sustancial en la función de pérdida que optimizan, el mecanismo matemático para calcular el gradiente y la estrategia empleada para actualizar las predicciones en cada iteración.

> **Principio de Unificación Teórica**
>
> Tanto Gradient Boosting como AdaBoost pueden formularse unificadamente dentro del marco de Descenso del Gradiente en el Espacio de Predicciones. La distinción matemática clave reside en que Gradient Boosting opera con pérdidas diferenciables continuas como el Error Cuadrático Medio (MSE) en problemas de regresión, mientras que AdaBoost minimiza la Pérdida Exponencial (Exponential Loss) sobre clasificaciones binarias con etiquetas en {-1, +1}.

2. Análisis Comparativo por Dimensiones Matemáticas

Función de Pérdida Objetivo a Minimizar

En **Gradient Boosting** (orientado típicamente a regresión), el objetivo optimizado es el Error Cuadrático Medio (MSE), formulado como:

$$\text{MSE} = \frac{1}{N} \sum_{n=1}^N \left(y_n - \hat{y}_n\right)^2$$

Por su parte, en **AdaBoost** (diseñado para clasificación binaria), se minimiza la Pérdida Exponencial (Exp Loss), expresada como:

$$\text{Exp Loss} = \frac{1}{N} \sum_{n=1}^N \exp\left(-y_n \hat{y}_n\right)$$

Cálculo del Gradiente respecto a las Predicciones

Para **Gradient Boosting**, el gradiente de la función MSE respecto al vector de predicciones $\hat{y}$ proporciona directamente los residuos reales de las observaciones:

$$\nabla_{\hat{y}} \text{MSE} = -2 \left[r_1, r_2, \dots, r_N \right] \quad \text{donde } r_n = y_n - \hat{y}_n$$

En **AdaBoost**, el gradiente de la Pérdida Exponencial equivale a aplicar un esquema de reponderación exponencial a los valores objetivo y_n:

$$\nabla_{\hat{y}} \text{Exp Loss} = \left[-y_1 w_1, -y_2 w_2, \dots, -y_N w_N \right] \quad \text{donde } w_n = \exp\left(-y_n \hat{y}_n\right)$$

Regla de Actualización de Predicciones (Update Step)

En **Gradient Boosting**, la actualización de cada observación $n = 1, \dots, N$ suma la predicción estimada del nuevo aprendiz débil $\hat{r}_n(x_n)$ ajustado sobre los residuos:

$$\hat{y}_n \leftarrow \hat{y}_n + \lambda \hat{r}_n(x_n), \quad n = 1, \dots, N$$

En **AdaBoost**, la actualización de las predicciones añade la contribución ponderada del nuevo tocón $S^{(i)}(x_n)$ sobre la muestra reponderada:

$$\hat{y}_n \leftarrow \hat{y}_n + \lambda w_n y_n, \quad n = 1, \dots, N$$

3. Cuadro Comparativo Consolidado

La siguiente tabla sintetiza lado a lado las propiedades operativas y las formulaciones matemáticas que diferencian a Gradient Boosting de AdaBoost:

Criterio / Dimensión	Gradient Boosting	AdaBoost
1. Función Objetivo (Loss Function)	Minimiza el Error Cuadrático Medio (MSE): $$\text{MSE} = \frac{1}{N} \sum_{n=1}^N (y_n - \hat{y}_n)^2$$	Minimiza la Pérdida Exponencial (Exponential Loss): $$\text{Exp Loss} = \frac{1}{N} \sum_{n=1}^N \exp(-y_n \hat{y}_n)$$
2. Gradiente de la Pérdida (∇_y Loss)	Calcula el gradiente respecto a las predicciones: $$\nabla_{\hat{y}} \text{MSE} = -2 [r_1, \dots, r_N]$$ (Vectores de residuos reales directos).	Calcula el gradiente ponderado: $$\nabla_{\hat{y}} \text{Exp Loss} = [-y_1 w_1, \dots, -y_N w_N]$$ Estableciendo $w_n = \exp(-y_n \hat{y}_n)$.
3. Paso de Actualización (Update Step)	Actualización aditiva sobre residuos: $$\hat{y}_n \leftarrow \hat{y}_n +	Actualización sobre objetivos reponderados:

	\lambda \hat{r}_n(x_n)$$ para $n = 1, \dots, N$.	$$\hat{y}_n \leftarrow \hat{y}_n + \lambda w_n y_n$$ para $n = 1, \dots, N$.

4. Captura de Pantalla Original de Referencia

A continuación se inserta la lámina explicativa oficial de la Sección 6.1.3 correspondiente al material lectivo del curso:

6.1.3 Comparación: Gradient Boosting y Adaboost

Comparación: Gradient Boosting y AdaBoost

Aquí hay una comparación entre los procedimientos de Gradient Boosting y AdaBoost. Puedes encontrar la misma información en una tabla más abajo en la página si prefieres verlo de esa manera.

- En Gradient Boosting minimizamos el MSE dado por; $MSE = \frac{1}{N}\sum_{n=1}^{N}(y_n - \hat{y_n})^2$, mientras que en AdaBoost minimizamos la pérdida exponencial dada por $\text{Exp Loss} = \frac{1}{N}\sum_{n=1}^{N} e^{(-y_n \hat{y_n})}$
- Gradient Boosting calcula el gradiente de la pérdida con respecto a las predicciones $\nabla_{\hat{y}} \text{MSE} = -2\left[r_1, \dots, r_n\right]$, mientras que AdaBoost calcula el gradiente $\nabla_{\hat{y}} \text{Exp Loss} = \left[-y_1 w_1, \dots, -y_n w_n\right]$ estableciendo $w_n = exp\left(-y_n \hat{y_n}\right)$
- El paso de actualización en Gadient Boosting se da por $\hat{y_n} \leftarrow \hat{y_n} + \lambda \hat{r_n} n = 1, \dots, N$. El paso de actualización en el caso de AdaBoost se da por $\hat{y_n} \leftarrow \hat{y_n} + \lambda w_n y_n, n = 1, \dots, N$

Gradient Boost	AdaBoost
Minimiza el MSE: $$MSE = \frac{1}{N}\sum_{n=1}^{N}(y_n - \hat{y_n})^2$$	Minimiza la pérdida exponencial: $$\text{Exp Loss} = \frac{1}{N}\sum_{n=1}^{N} e^{(-y_n \hat{y_n})}$$
Calcula el gradiente de la pérdida con respecto a las predicciones: $$\nabla_{\hat{y}} \text{MSE} = -2\left[r_1, \dots, r_n\right]$$	Calcula el gradiente: $$\nabla_{\hat{y}} \text{Pérdida exponencial} = \left[-y_1 w_1, \dots, -y_n w_n\right]$$ estableciendo $w_n = exp\left(-y_n \hat{y_n}\right)$
Paso de actualización: $$\hat{y_n} \leftarrow \hat{y_n} + \lambda \hat{r_n} n = 1, \dots, N$$	Paso de actualización: $$\hat{y_n} \leftarrow \hat{y_n} + \lambda w_n y_n, n = 1, \dots, N$$

Figura 6.1: Lámina oficial de la Sección 6.1.3 — Comparación entre Gradient Boosting y AdaBoost.

> **Conclusión del Estudio Comparativo**
>
> En resumen, mientras que Gradient Boosting desciende en el espacio funcional ajustando directamente

> aprendices débiles sobre los residuos numéricos continuos (con la pérdida MSE), AdaBoost realiza un descenso de gradiente ajustando aprendices débiles (tocones) sobre observaciones objetivo reponderadas según el error previo (con la pérdida exponencial). Ambas técnicas ejemplifican la potencia del aprendizaje secuencial aditivo.

Sección 6.1.4: Reflexiones Finales sobre Boosting

Ecosistema de Librerías de Alto Rendimiento: XGBoost, LightGBM y CatBoost

ÍNDICE SINTÉTICO DE CONTENIDOS

1. Prevalencia del Boosting en Datos Estructurados y Tabulares
2. XGBoost (Extreme Gradient Boosting): Rendimiento Paralelepipédico y Escala Distribuida
3. LightGBM (LGBM): Eficiencia por División de Hojas y Dominio Competitivo
4. CatBoost: Algoritmo Nativo para Características Categóricas e Industria de Gran Escala
5. Matriz Comparativa Técnica entre Frameworks de Boosting
6. Material de Referencia Original Adjunto

1. Prevalencia del Boosting en Datos Estructurados y Tabulares

Los algoritmos basados en Gradient Boosting Decision Trees (GBDT) constituyen el estándar de la industria para el aprendizaje automático sobre datos tabulares y estructurados. A diferencia de las redes neuronales, que destacan en el procesamiento de imágenes, audio y texto no estructurado, las librerías modernas de boosting optimizan de manera iterativa el sesgo y la varianza, logrando el máximo rendimiento predictivo en hojas de cálculo y bases de datos relacionales.

2. XGBoost (Extreme Gradient Boosting): Rendimiento Paralelepipédico y Escala Distribuida

XGBoost es una biblioteca optimizada de gradient boosting distribuido diseñada para ser altamente eficiente, flexible y portátil. Implementa algoritmos de aprendizaje automático bajo el marco general de Gradient Boosting, proporcionando una construcción paralela de árboles (GBDT, GBM) que resuelve problemas complejos de ciencia de datos con alta velocidad y precisión analítica.

> **Características Clave de XGBoost:**• Procesamiento Distribuido Masivo: Se ejecuta nativamente en entornos a gran escala como Kubernetes, Hadoop, SGE, Dask, Spark y PySpark.
>
> • Escalabilidad a Gran Escala: Capaz de procesar conjuntos de datos que involucran miles de millones de ejemplos mediante paralelización a nivel de nodo y columna.
>
> • Regularización Integrada: Controla el sobreajuste mediante términos de penalización L1 y L2 sobre la estructura de los árboles.

3. LightGBM (LGBM): Eficiencia por División de Hojas y Dominio Competitivo

LightGBM (Máquinas de Gradiente Potenciado Ligeras) es un marco de gradient boosting desarrollado por Microsoft que utiliza algoritmos de aprendizaje basados en árboles con crecimiento por hojas (leaf-wise). Está diseñado específicamente para ofrecer máxima eficiencia computacional, menor consumo de memoria RAM y una velocidad de entrenamiento significativamente superior en grandes volúmenes de datos.

> **Atributos Principales de LightGBM (LGBM):**• Estrategia Leaf-wise: Elige las hojas con mayor pérdida de gradiente para dividir, logrando una menor pérdida total que el método tradicional level-wise.
>
> • Éxito en Competiciones: Se ha consolidado como la herramienta preferida en soluciones ganadoras dentro de Kaggle y competiciones de ciencia de datos.
>
> • Optimización de Memoria: Emplea algoritmos basados en histogramas para agrupar valores continuos en bins discretos.

4. CatBoost: Algoritmo Nativo para Características Categóricas e Industria de Gran Escala

CatBoost es un algoritmo de gradient boosting sobre árboles de decisión desarrollado por investigadores e ingenieros de Yandex. Destaca por su tratamiento nativo y automático de variables categóricas, eliminando la necesidad de preprocesamientos manuales complejos como One-Hot Encoding o Target Encoding extensivos.

> **Aplicaciones de Producción e Industria de CatBoost:**• Despliegue en Yandex: Utilizado internamente para motores de búsqueda, sistemas de recomendación, asistentes personales y vehículos autónomos.
>
> • Adopción Global en la Industria: Implementado operacionalmente en instituciones científicas como el CERN y empresas tecnológicas como Cloudflare y Careem Taxi.
>
> • Manejo Categórico Robusto: Mitiga el cambio objetivo (target leakage) mediante esquemas de codificación ordenada en tiempo de entrenamiento.

5. Matriz Comparativa Técnica entre Frameworks de Boosting

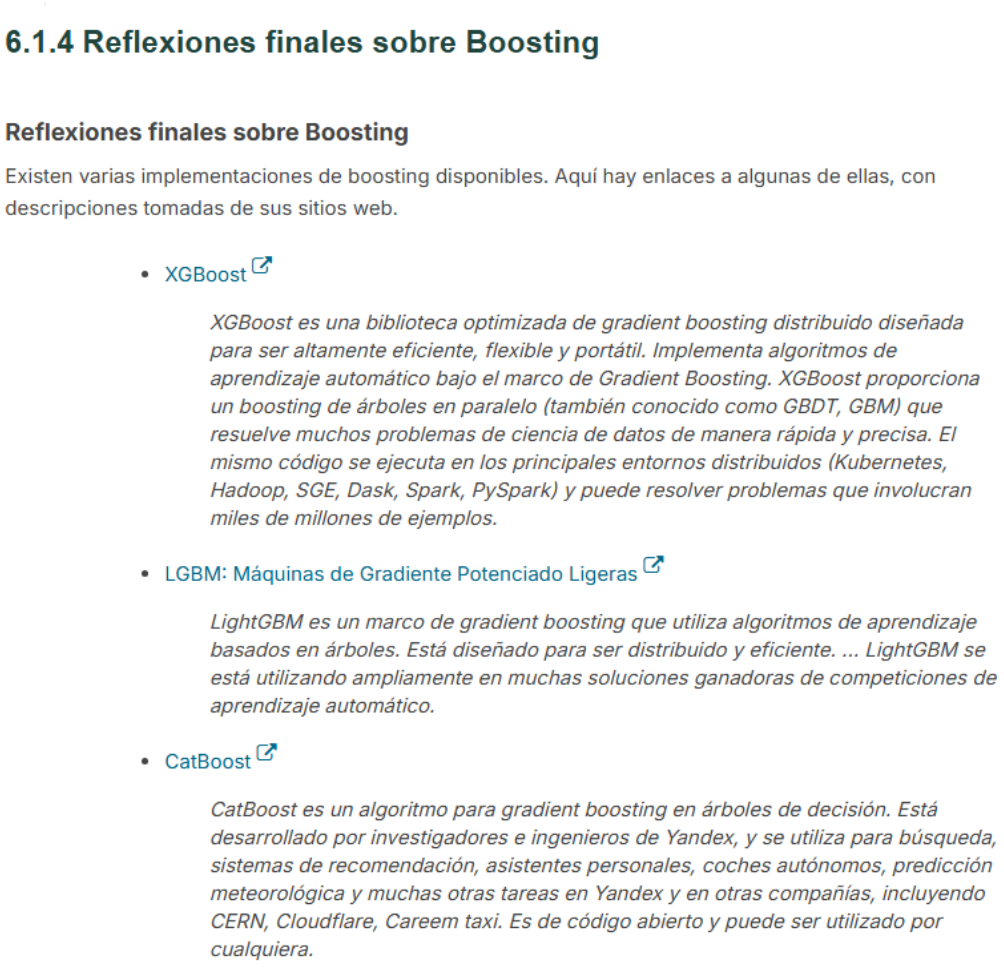
La siguiente tabla sintetiza las fortalezas relativas, desarrolladores y escenarios de aplicación óptimos para los tres marcos de trabajo:

Librería	Desarrollador / Origen	Ventaja Competitiva Principal	Casos de Uso Destacados

XGBoost	Comunidad Open Source / Tianqi Chen	Cómputo distribuido masivo y paralelización a gran escala (GBDT)	Infraestructuras Spark, Dask, Kubernetes y conjuntos masivos de datos
LightGBM	Microsoft	Velocidad extrema, crecimiento leaf-wise y bajo consumo de memoria RAM	Competiciones de ciencia de datos (Kaggle) y filtrado rápido
CatBoost	Yandex	Tratamiento nativo y automático de características categóricas sin fuga de datos	Búsqueda web, motores de recomendación, vehículos autónomos y CERN

6. Material de Referencia Original Adjunto

A continuación se adjunta la imagen oficial original correspondiente a esta lección de sintesis:



6.1.4 Reflexiones finales sobre Boosting

Reflexiones finales sobre Boosting

Existen varias implementaciones de boosting disponibles. Aquí hay enlaces a algunas de ellas, con descripciones tomadas de sus sitios web.

- XGBoost

  XGBoost es una biblioteca optimizada de gradient boosting distribuido diseñada para ser altamente eficiente, flexible y portátil. Implementa algoritmos de aprendizaje automático bajo el marco de Gradient Boosting. XGBoost proporciona un boosting de árboles en paralelo (también conocido como GBDT, GBM) que resuelve muchos problemas de ciencia de datos de manera rápida y precisa. El mismo código se ejecuta en los principales entornos distribuidos (Kubernetes, Hadoop, SGE, Dask, Spark, PySpark) y puede resolver problemas que involucran miles de millones de ejemplos.

- LGBM: Máquinas de Gradiente Potenciado Ligeras

  LightGBM es un marco de gradient boosting que utiliza algoritmos de aprendizaje basados en árboles. Está diseñado para ser distribuido y eficiente. ... LightGBM se está utilizando ampliamente en muchas soluciones ganadoras de competiciones de aprendizaje automático.

- CatBoost

  CatBoost es un algoritmo para gradient boosting en árboles de decisión. Está desarrollado por investigadores e ingenieros de Yandex, y se utiliza para búsqueda, sistemas de recomendación, asistentes personales, coches autónomos, predicción meteorológica y muchas otras tareas en Yandex y en otras compañías, incluyendo CERN, Cloudflare, Careem taxi. Es de código abierto y puede ser utilizado por cualquiera.

Figura 1: Diapositiva original sobre las implementaciones de Boosting (XGBoost, LightGBM y CatBoost).

Tema Especial: IA en la Creación del Curso

Análisis de transparencia sobre el uso selectivo, supervisión humana y límites técnicos de la IA

1. Declaración de Principios: Uso Selectivo y Supervisado de IA

Al finalizar la formación, el equipo docente del curso realiza un ejercicio de transparencia sobre la metodología de producción utilizada. Se confirma que **sí se emplearon herramientas de Inteligencia Artificial**, pero bajo un enfoque extremadamente selectivo, acotado a tareas técnicas repetitivas y siempre con estricta revisión humana.

> **Principio de Gobernanza Pedagógica:** La IA actuó exclusivamente como un acelerador de tareas mecánicas de maquetación. Ningún contenido conceptual, argumento explicativo ni material didáctico clave fue generado o modificado por modelos automáticos sin el control directo de los profesores y diseñadores instruccionales.

2. Áreas de Aplicación Técnica Efectiva de la IA

El uso de herramientas de IA generativa se limitó a dos flujos mecánicos donde demostraron eficiencia técnica:

- **Conversión de Tablas a Código HTML (Gemini / ChatGPT):** Se utilizaron modelos de visión por computador para transformar imágenes de tablas de datos en estructuración HTML. Aunque el proceso resultó ágil, requirió auditoría minuciosa, ya que en ocasiones los modelos inventaban columnas innecesarias (por ejemplo, añadir un 'Número de Paciente' no existente en tablas de presión arterial). Pese a estos fallos, la generación y posterior corrección resultó más rápida que la maquetación manual desde cero.
- **Asistencia de Código en Entorno de Desarrollo (GitHub Copilot):** En Visual Studio Code, Copilot asistió en la repetición de estructuras sintácticas en HTML, tales como la conversión de listas de texto sin formato en etiquetas de lista ordenada/desordenada de forma automatizada.

3. Componentes y Áreas donde se Descartó el Uso de IA

Debido a deficiencias en precisión, contexto o calidad pedagógica, se prohibió explícitamente el uso de IA en los siguientes apartados del curso:

Componente del Curso	Herramienta Evaluada	Motivo de Descarte / Criterio Humano Aplica
Redacción y Edición de Texto	Copilot / ChatGPT	Incompatibilidad de tono pedagógico y riesgo de imprecisiones conceptuales. Todo el texto fue redactado 100% por los docentes.

Texto Alternativo (Alt Text)	Modelos Generativos	Incapacidad de la IA para capturar el contexto instruccional específico requerido para la accesibilidad educativa de alta calidad.
Generación de Imágenes	DALL-E / Midjourney	Se prefirió el rigor técnico de diagramas creados por diseñadores gráficos o generados mediante código (Python, R, Mathematica, PowerPoint).
Sintetización de Voz / Audio	Redes Neuronales de Voz	Se intentó clonar la voz del Dr. Protopapas para correcciones; la falta de naturalidad obligó a regrabar locuciones en estudio o usar Final Cut.
Edición de Vídeo	Herramientas de Vídeo IA	Toda la producción y postproducción audiovisual se realizó con software de edición tradicional y edición humana directa.

4. Conclusión: Eficiencia sin Pérdida de Juicio Humano

La experiencia docente concluye que la Inteligencia Artificial actúa como un excelente catalizador de productividad para profesionales experimentados. Las herramientas de IA aceleraron los flujos de trabajo técnicos, pero no simplificaron la toma de decisiones pedagógicas. El rol de los tecnólogos instruccionales y profesores sigue siendo el pilar insustituible en la creación de contenidos educativos de excelencia.

Tema especial: IA en la creación de este curso

Tema Especial: IA en la creación de este curso

Ahora que casi has terminado el curso, puede que te interese saber cómo lo hicimos. Específicamente, ¿utilizamos alguna IA en la creación de este curso?

¡Sí! Pero fuimos muy selectivos acerca de exactamente qué usamos y dónde.

- Algunas de las tablas se convirtieron de imágenes a HTML mediante IA, utilizando Gemini y/o ChatGPT. Encontramos ese proceso relativamente fiable. Tuvimos que revisar cuidadosamente los resultados. A veces, la IA añadía una columna extra, como agregar un Número de Paciente a la tabla con Presión Arterial y Frecuencia Cardíaca. (No es *irrazonable* tener un Número de Paciente, pero era *innecesario* y no formaba parte de la tabla original.) Aun con los errores, nos resultó más rápido que la IA generara la tabla HTML a partir de la imagen y luego eliminaramos las columnas que no se necesitaban.
- Parte del HTML fue modificado utilizando Copilot en Visual Studio Code. Hizo un buen trabajo repitiendo bloques de código con pequeños cambios en cada bloque; por ejemplo, si le proporcionábamos una lista sin formato de cadenas, era capaz de crear una lista HTML de forma fiable.
- Ningún texto del curso fue escrito o editado por IA. A veces, Copilot sugería oraciones o párrafos que podrían encajar después de nuestro texto, pero no aceptamos esas sugerencias. La redacción que Copilot y otras IAs produjeron simplemente no encajaba con el resto del curso, y tampoco siempre era correcta.
- No escribimos ninguno de los texto alternativo con IA. Muchos modelos de IA generativa eran *a veces* capaces de describir imágenes con precisión, y a veces no. Sin embargo, *nunca* proporcionó el contexto que da lugar a un texto alternativo de alta calidad.
- No creamos ninguna de las imágenes con IA. Algunas de ellas fueron creadas por un diseñador gráfico. Otras fueron hechas por el equipo del curso utilizando Python, R, Mathematica o PowerPoint.
- No usamos IA en los vídeos. Intentamos entrenar una red neuronal con el discurso del Dr. Protopapas para ver si podíamos usarla para corregir algunos errores iniciales en los vídeos, pero nunca sonó exactamente como él. Simplemente volvimos a grabar, o usamos edición de audio tradicional en Final Cut.

En general, descubrimos que la IA *aceleró* el proceso, pero no lo hizo *más fácil* para crear el curso.

Cuando nosotros *hizo* use herramientas de IA, las personas que normalmente harían el trabajo de todos modos eran las que usaban las herramientas. Por ejemplo, nuestros tecnólogos instruccionales normalmente crearían las tablas HTML, y ellos fueron los que usaron IA para hacer las tablas HTML más rápidas.

Figura 1: Captura de pantalla original sobre la sección 'Tema Especial: IA en la creación de este curso'.